

Architecture Decision Records

Vollständige ADR-Historie inklusive Revalidierungen

Sämtliche Architekturentscheidungen der Plattform in chronologischer Form: aktive ADRs, aufgelöste ADRs und die zugehörigen Revalidierungs-Nachweise. Jede Entscheidung nennt Kontext, Alternativen und Konsequenzen und eignet sich als Erzählgrundlage für die Entwicklungsgeschichte des Systems.

Repository: [SvenKulesa/Finance](#)

Stand: [claude/pdf-dokumente-notebook-llm-l6kcxq @ 4eadf9d](#)

Erzeugt am: 16.08.2026

Enthaltene Dokumente: 88

Umfang: ca. 65.782 Wörter

Dieses PDF ist eine Quelle für NotebookLM. Jedes eingebettete Dokument beginnt auf einer neuen Seite und nennt seinen Repository-Pfad, damit erzeugte Podcasts, Video-Skripte und Zusammenfassungen ihre Aussagen exakt auf die Originaldatei zurückführen können.

Inhalt dieses Quellpakets

1. ADR-0003.5: Owner-IAM, Passkey/2FA-Absicherung & zugriffsbeschränkte Systemzonen	8
<code>docs/adr/ADR-0003_5-identity-access-management.md</code>	
2. ADR 0004 branding and panel removal	12
<code>docs/adr/ADR-0004-branding-and-panel-removal.md</code>	
3. ADR 0005 frontend module integration	14
<code>docs/adr/ADR-0005-frontend-module-integration.md</code>	
4. ADR 0006 platform director connection	16
<code>docs/adr/ADR-0006-platform-director-connection.md</code>	
5. ADR 0007 compliance value chain	19
<code>docs/adr/ADR-0007-compliance-value-chain.md</code>	
6. ADR-0010: Erweiterung des Enterprise Standards um ESS-0001-CONTRACTS Chapter 11-20, ESS-0002, ESS-0003 und die ESS Registry	22
<code>docs/adr/ADR-0010-enterprise-standard-extension.md</code>	
7. ADR-0011: Bestandsschutz und Zielstruktur für die Root-Abweichungen <code>`server/`</code> , <code>`sql/`</code> und <code>`server.ts`</code>	28
<code>docs/adr/ADR-0011-bestandsschutz-root-abweichungen.md</code>	
8. ADR-0013: Konsolidierung der ESS-Dokumentationsverantwortung, Vergabe von ESS-0010 und ESS-0011	36
<code>docs/adr/ADR-0013-ess-documentation-responsibility-consolidation.md</code>	
9. ADR-0014: Documentation Governance Validator als Kernkomponente der Documentary Engine	42
<code>docs/adr/ADR-0014-documentation-governance-validator.md</code>	
10. ADR-0015: Einführung der Traceability-Komponente als Plattformmodul	48
<code>docs/adr/ADR-0015-enterprise-traceability-component.md</code>	
11. ADR-0016: Nachvergabe der reservierten ESS-Nummern 0004 bis 0009 als Komponentenspezifikationen	54
<code>docs/adr/ADR-0016-ess-component-specifications.md</code>	
12. ADR-0017: Produktions-Audit-Nachbesserungen — Quota-Durchsetzung, Supabase-Härtung, Stripe-Jahresrabatt, Compliance-Backend	59
<code>docs/adr/ADR-0017-audit-nachbesserungen-quota-supabase-stripe-compliance.md</code>	
13. ADR-0019: Fork-Divergenz zwischen Entwicklungs- und Produktivstand, Konsolidierung des ADR- und ESS-Nummernraums	65
<code>docs/adr/ADR-0019-fork-divergenz-und-nummernraum-konsolidierung.md</code>	
14. ADR-0020: Multi-Provider Market Data Routing & Evidence-Preserving Failover	70
<code>docs/adr/ADR-0020-multi-provider-market-data-routing.md</code>	
15. ADR-0021: Aktivierung zusätzlicher externer Market-Data-Provider	73

[docs/adr/ADR-0021-external-market-data-provider-activation.md](#)

16. ADR-0022 — Evidence-gated Bond Scoring Architecture 76

[docs/adr/ADR-0022-bond-scoring-evidence-architecture.md](#)

17. ADR-0023 — FRED & ECB Macro/Rate Evidence Sources 79

[docs/adr/ADR-0023-macro-rate-evidence-sources.md](#)

18. ADR-0024 — Cross-Asset Risk Context Contract 82

[docs/adr/ADR-0024-cross-asset-risk-context.md](#)

19. ADR-0025 — Verified Traditional Quotes & Screening Eligibility 84

[docs/adr/ADR-0025-verified-traditional-quotes-and-screening-eligibility.md](#)

20. Architectural Decision Record (ADR-0026) 86

[docs/adr/ADR-0026-social-media-direct-publishing-real-integration.md](#)

21. Architectural Decision Record (ADR-0027) 88

[docs/adr/ADR-0027-social-media-access-restriction-owner-founder.md](#)

22. ADR-0028 — Screening SLO, empirical confidence and horizon-exact validation 89

[docs/adr/ADR-0028-screening-slo-confidence-and-horizon-validation.md](#)

23. ADR-0029 — Bond Scoring Weights Proposal (ADR-0022 Follow-up) 91

[docs/adr/ADR-0029-bond-scoring-weights-proposal.md](#)

24. ADR-0031 — Supabase Free-Tier Auth Security Compensating Controls 93

[docs/adr/ADR-0031-supabase-free-tier-auth-security-compensating-controls.md](#)

25. ADR-0034 — Central Subscription Entitlements and Warren Buffett Access 98

[docs/adr/ADR-0034-central-subscription-entitlements-and-buffett-access.md](#)

26. Architectural Decision Record (ADR-0035) 102

[docs/adr/ADR-0035-protected-google-marketing-integration-strict-csp.md](#)

27. ADR-0036 — Server Composition and Modularization 108

[docs/adr/ADR-0036-server-composition-and-modularization.md](#)

28. Architectural Decision Record (ADR-0037) 110

[docs/adr/ADR-0037-render-production-configuration-governance.md](#)

29. ADR-0038 — Binance Spot als Datenquelle der öffentlichen AI-Kurzanalyse 123

[docs/adr/ADR-0038-binance-landing-ai-quick-analysis.md](#)

30. ADR-0039 — Human-Authorized Pull Request Creation and Advisory Process Governance 126

[docs/adr/ADR-0039-human-authorized-pr-creation-and-advisory-governance.md](#)

31. ADR-0040 — CSP Runtime Remediation and Safe Strict-Policy Rollout 129

[docs/adr/ADR-0040-csp-runtime-remediation-safe-rollout.md](#)

32. Architectural Decision Record (ADR-0041) 135

[docs/adr/ADR-0041-enterprise-market-data-provider-and-mcp-architecture.md](#)

33. Architectural Decision Record (ADR-0042) 145

[docs/adr/ADR-0042-basic-consent-mode-v2-google-tag-gating.md](#)

34. ADR-0043 — Supabase Privilege Separation & Render Production Hardening	151
docs/adr/ADR-0043-supabase-privilege-separation-render-production-hardening.md	
35. ADR-0044 — Production Runtime Artifact Immutability	155
docs/adr/ADR-0044-production-runtime-artifact-immutability.md	
36. ADR-0045 — Stripe Event Ownership & Durable Inbox	160
docs/adr/ADR-0045-stripe-event-ownership-durable-inbox.md	
37. ADR-0046 — Modern Supabase Key Contracts and AI Admin Control Plane	166
docs/adr/ADR-0046-modern-supabase-key-contracts-and-ai-admin-control-plane.md	
38. ADR-0047 — GitHub Actions as authoritative Render production deploy gate	169
docs/adr/ADR-0047-github-authoritative-pre-merge-ci-gate.md	
39. ADR-0048 — Render Docker Image Hardening	172
docs/adr/ADR-0048-render-docker-image-hardening.md	
40. ADR-0049 — Docker Runtime Filesystem Hardening	175
docs/adr/ADR-0049-docker-runtime-filesystem-hardening.md	
41. ADR-0050 — Agent Tool & Capability IAM Foundation (Supabase, Phase 1)	177
docs/adr/ADR-0050-agent-tool-capability-iam-foundation.md	
42. ADR-0051 — Capability/Grant IAM, Approval-Artefakt & governter Admin-Agent (ESS-0018 Phase 2)	180
docs/adr/ADR-0051-capability-grant-approval-workflow.md	
43. ADR-0052 — Transactional, Idempotent PDF-Credit Ledger	183
docs/adr/ADR-0052-pdf-credit-ledger.md	
44. ADR-0053 — Node.js 24 LTS & Git 2.55 Toolchain Baseline	188
docs/adr/ADR-0053-node24-lts-git255-toolchain.md	
45. ADR-0054 — Durable Worker/Outbox/Lease	191
docs/adr/ADR-0054-durable-worker-outbox-lease.md	
46. ADR-0055 — Event-Driven CAPITAL-AI Value Chain Governance Integration	197
docs/adr/ADR-0055-event-driven-value-chain-governance-integration.md	
47. ADR-0056 — Observability & Telemetry Baseline	199
docs/adr/ADR-0056-observability-telemetry-baseline.md	
48. ADR-0057 — Provider-Neutral Agent Control Plane and Plane Separation	202
docs/adr/ADR-0057-provider-neutral-agent-control-plane-and-plane-separation.md	
49. ADR-0058 — Agent Identity, Capability and Risk Authorization	204
docs/adr/ADR-0058-agent-identity-capability-and-risk-authorization.md	
50. ADR-0059 — Agent Execution Audit and OpenTelemetry Correlation	207
docs/adr/ADR-0059-agent-execution-audit-and-otel-correlation.md	
51. ADR-0060 — Software Supply Chain Provenance and Attestation	212
docs/adr/ADR-0060-software-supply-chain-provenance-and-attestation.md	

52. ADR-0061 — Deployment Identity and Production Mutation	213
<code>docs/adr/ADR-0061-deployment-identity-and-production-mutation.md</code>	
53. ADR-0062 — Agent Cutover and Provider Profile Contract	214
<code>docs/adr/ADR-0062-agent-cutover-and-provider-profile-contract.md</code>	
54. ADR-0063 — Agent Assurance, Incident Response and Break-Glass	215
<code>docs/adr/ADR-0063-agent-assurance-incident-break-glass.md</code>	
55. ADR-0064 — Supabase Native MFA / AAL2 Hardening	216
<code>docs/adr/ADR-0064-supabase-native-mfa-aal2-hardening.md</code>	
56. ADR-0065 — Systemadmin Roadmap Execution Mandate	221
<code>docs/adr/ADR-0065-systemadmin-roadmap-execution-mandate.md</code>	
57. ADR-0066 — Passkey-only Human/Owner PR Authorization	226
<code>docs/adr/ADR-0066-passkey-only-owner-pr-authorization.md</code>	
58. ADR-0067 — S1 Security Hardening Interlock	232
<code>docs/adr/ADR-0067-s1-security-hardening-interlock.md</code>	
59. ADR-0068 — First Bounded Autonomous Work Package	236
<code>docs/adr/ADR-0068-first-bounded-autonomous-work-package.md</code>	
60. ADR-0069 — Human/Owner PR Gate ohne selbstreferenziellen Bootstrap	240
<code>docs/adr/ADR-0069-human-owner-comment-gate-and-dispatched-pr-ci.md</code>	
61. ADR-0070 — GitGuardian und Snyk als externe GitHub-Sicherheitsdienste	243
<code>docs/adr/ADR-0070-gitguardian-snyk-external-app-integration.md</code>	
62. ADR-0071 — Konsolidierte Roadmap-Authority und angefragte Systemadministrator-Mutationen	246
<code>docs/adr/ADR-0071-consolidated-roadmap-and-requested-systemadmin-mutations.md</code>	
63. ADR-0072: Alpaca Shadow-Integration und Stilllegung von Gemini	249
<code>docs/adr/ADR-0072-alpaca-shadow-and-gemini-retirement.md</code>	
64. ADR-0073 — Konsolidierung der technischen PR-CI auf build-and-test	251
<code>docs/adr/ADR-0073-ci-consolidation-build-and-test.md</code>	
65. ADR-0074 — Generalized Systemadmin Work-Package Catalog	254
<code>docs/adr/ADR-0074-generalized-systemadmin-work-package-catalog.md</code>	
66. ADR-0075 Phase 3.4.7 — Market Data Runtime Facade	259
<code>docs/adr/ADR-0075-phase-3-4-7-runtime-facade.md</code>	
67. ADR-0076: Naming & Repository Convention Validator als Capability des Enterprise Version Managers	261
<code>docs/adr/ADR-0076-version-manager-repository-convention-validator.md</code>	
68. ADR-0077 — Canonical Naming, Bilingual Documentation & Vocabulary Governance	262
<code>docs/adr/ADR-0077-canonical-naming-bilingual-documentation-vocabulary-governance.md</code>	
69. ADR-0078 — Vocabulary Governance Authority and Namespace	264

[docs/adr/ADR-0078-vocabulary-governance-authority-and-namespace.md](#)

70. ADR-0079 — Systemadmin GitHub Actions Execution Host Binding 265

[docs/adr/ADR-0079-systemadmin-github-actions-execution-host.md](#)

71. ADR-0080 — Marketing Roadmap Executor and Controlled Content Automation Boundary 266

[docs/adr/ADR-0080-marketing-roadmap-executor-and-content-automation-boundary.md](#)

72. ADR-0081: ADR-Nummernraum-Bereinigung, Shadow-Metadaten-Validator und wöchentlicher Governance-Score 267

[docs/adr/ADR-0081-adr-namespace-cleanup-shadow-validator-and-weekly-score.md](#)

73. ADR-0082 — SeoEngine Platform Module (S1 / WP-S1) 269

[docs/adr/ADR-0082-seo-engine-platform-module.md](#)

74. ADR-0083 — Server Runtime Architecture Consolidation 271

[docs/adr/ADR-0083-server-runtime-architecture-consolidation.md](#)

75. ADR-DRAFT: Prerender / SSG for public SPA routes 274

[docs/adr/ADR-DRAFT-prerender-public-routes.md](#)

76. SUPERSEDED — SeoEngine Platform Module Draft 276

[docs/adr/ADR-DRAFT-seo-engine-platform-module.md](#)

77. anforderungskatalog 277

[docs/adr/anforderungskatalog.md](#)

78. ADR-0017 Revalidation Evidence — 2026-08-02 281

[docs/adr/evidence/ADR-0017-REVALIDATION-2026-08-02.md](#)

79. ADR-0032 Revalidation — R-001 Crypto/Meme Provenance Boundary 284

[docs/adr/evidence/ADR-0032-REVALIDATION-2026-08-08-R001.md](#)

80. IAM Implementation Log — ADR-0003.5 / ADR-0008 286

[docs/adr/IAM_IMPLEMENTATION_LOG.md](#)

81. ADR-Ablage: Konvention für aktive vs. abgeschlossene Entscheidungen 290

[docs/adr/README.md](#)

82. ADR 0008 document hygiene lifecycle fix 294

[docs/adr/resolved/ADR-0008-document-hygiene-lifecycle-fix.md](#)

83. ADR-0009: CORS Hardening für Entwicklungs- und Produktionsumgebung 296

[docs/adr/resolved/ADR-0009-cors-hardening.md](#)

84. ADR-0012: SecurityComplianceAuditor Integration 301

[docs/adr/resolved/ADR-0012-SecurityComplianceAuditor.md](#)

85. ADR-0018: Einführung der Enterprise Event Mesh als Plattformmodul 307

[docs/adr/resolved/ADR-0018-enterprise-event-mesh.md](#)

86. ADR-0030 — Platform Version and Release Lifecycle 313

[docs/adr/resolved/ADR-0030-platform-version-release-lifecycle.md](#)

87. ADR-0032 — Asset Catalog and Market Evidence Separation 323

[docs/adr/resolved/ADR-0032-asset-catalog-market-evidence-separation.md](#)

88. ADR-0033 — Index Provider Mapping and Evidence-Gated Commodity / Sovereign Benchmark Scoring	328
docs/adr/resolved/ADR-0033-index-provider-mapping-commodity-sovereign-evidence-scoring.md	

ADR-0003.5: Owner-IAM, Passkey/2FA-Absicherung & zugriffsbeschränkte Systemzonen

Quelle: docs/adr/ADR-0003_5-identity-access-management.md

ADR-0003.5: Owner-IAM, Passkey/2FA-Absicherung & zugriffsbeschränkte Systemzonen

- **Status:** ACCEPTED
- **Implementation-Status:** IN PROGRESS — M5A REVALIDATION (2026-08-12)
- **Datum:** 2026-07-10
- **Reaktiviert:** 2026-08-12 gemäß docs/adr/README.md, da eine Regression/Assurance-Lücke im als resolved geführten TOTP-Scope nachgewiesen wurde.
- **M5A Authority:** ESS-0020 + ADR-0064
- **Sicherheits-Bereich:** Capital-AI Sicherheitsmanagement & Capital-AI Compliance

M5A Revalidation — 2026-08-12

Die ursprüngliche IAM-Härtung bleibt in wesentlichen Teilen gültig: JWT-only Admin-Auth, serverseitige Rollenprüfung, Break-Glass-Prinzip, Rate-Limiting, Audit/Access-Logging und purpose-bound Step-Up sind weiterhin Architekturbestandteile.

Der frühere COMPLETE-Status ist für den TOTP-/Owner-MFA-Scope jedoch nicht mehr haltbar:

- beide Owner-Profile haben `totp_enabled=true` im CAPITAL-AI-Legacy-Modell;
- Supabase Auth besitzt produktiv **0 native MFA-Faktoren**;
- produktiv bestehen **0 AAL2-Sessions**;
- der eigene TOTP-Flow in `server/stepUp.ts` erzeugt einen CAPITAL-AI-Step-Up-Token, aber keinen Supabase-AAL2-Nachweis;
- `checkAdminAccess()` erzwingt derzeit keinen `aal2`-Claim;
- die Login-Step-Up-Logik kann bei Status-/Netzwerkfehlern fail-open werden;
- Passkey kann den Login-Step-Up alleine erfüllen, obwohl die ursprüngliche Entscheidung TOTP für Owner unabhängig vom Passkey vorsieht.

Deshalb wird dieser ADR aus `resolved/` zurück in den aktiven ADR-Bestand verschoben. ESS-0020 und ADR-0064 definieren die M5A-Remediation. Nach erfolgreicher Native-MFA-/AAL2-Verifikation kann ADR-0003.5 erneut als vollständig verifiziert klassifiziert werden.

🏠 CAPITAL-AI CORE — SECURITY MEMORANDUM

Kontext

Die ursprüngliche Autorisierungsprüfung für Admin-/Owner-Funktionen basierte auf einem Vergleich einer im Request mitgeschickten E-Mail-Adresse gegen eine hartcodierte Liste. Damit bestand faktisch keine belastbare Authentifizierung. ADR-0003.5 ersetzte dieses Modell durch Supabase-Session-IAM, Step-Up, Recovery und Auditierung.

Da die Compliance-Wertschöpfungskette einen unveränderlichen Audit-Trail sowie eine klare Governance-Instanz voraussetzt, muss die Zugriffskontrolle auf einem echten IAM-Fundament bleiben. Gleichzeitig soll der Owner auch im Kompromittierungs-/Recovery-Fall nicht durch einen unsicheren Bypass wieder freigeschaltet werden.

Entscheidung

1. Owner-Authentifizierung

- Owner-Accounts werden über **Supabase Auth** verwaltet.
- Passkey/WebAuthn bleibt ein starker primärer bzw. zusätzlicher Authentifizierungsmechanismus.
- **TOTP ist für Owner als zweiter Faktor verpflichtend.** Nach M5A gilt hierfür ausschließlich der in ESS-0020/ADR-0064 definierte Supabase-Native-MFA-/AAL2-Nachweis als Authority.
- Owner-E-Mail-Adressen werden nicht als Autorisierungsmerkmal im Client oder Request verwendet.
- Eine gültige IAM-Rolle ohne erforderliches AAL2 reicht für privilegierte Owner/Admin-Aktionen nicht aus.

2. Rollenmodell

Rolle	Rechte
owner	Vollzugriff inkl. Break-Glass, Rollenverwaltung und kritische Freigaben — jeweils unter den erforderlichen Step-Up/AAL2-Gates
admin	Admin-Panel und definierte Admin-Funktionen; privilegierte Aktionen benötigen AAL2 gemäß ESS-0020
supervisor	Zugriff auf Documentary/Master-Supervisor-Funktionen gemäß Capability-/Policy-Regeln
user	Standardzugriff, kein Admin-Bereich

Rollenänderungen bleiben auditpflichtig.

3. Session-Token und Step-Up

- `/api/admin/*` und andere privilegierte Routen verwenden verifizierte Supabase-Bearer-Tokens, nicht E-Mail-Queryparameter.

- Kritische Owner-Aktionen verlangen zusätzliche frische, zweckgebundene Step-Up-Evidence.
- M5A stellt klar: ein CAPITAL-AI-eigener `x-step-up-token` ist nur **Defense-in-Depth über einer gültigen AAL2-Session**. Er darf AAL2 nicht ersetzen oder simulieren.
- Ungültige, abgelaufene, AAL1- oder stale Sessions führen fail-closed zur Ablehnung.

4. Zugriffsbeschränkte Systemzonen

- Sensible Endpunkte werden serverseitig per IAM + erforderlichem AAL2 geschützt.
- Frontend-Sichtbarkeit ist keine Security Boundary.
- Zugriffe bleiben in `IAM_ACCESS_LOG` bzw. den aktuellen Audit-Authorities nachvollziehbar.

5. Break-Glass

- Break-Glass bleibt ausschließlich Owner-Recovery.
- Recovery darf niemals Rollen erhöhen oder einen künstlichen AAL2-Zustand erzeugen.
- Native MFA-Faktor-Reset wird nach M5A ausschließlich über einen Owner-kontrollierten, auditierten Serverpfad und unterstützte Supabase-Admin-MFA-Operationen ausgeführt.
- Unabhängige Passkeys dürfen nicht mehr automatisch als Nebeneffekt eines TOTP-Resets gelöscht werden, sofern dies nicht separat freigegeben ist.
- Bestehende Legacy-Recovery-Codes bleiben nur als Übergangsmechanismus erhalten, bis Native-MFA-Recovery verifiziert wurde.

M5A Konsequenzen

Beibehalten

- Supabase als Identitäts-/Session-Authority;
- IAM-Rollenmodell;
- Rate-Limiting;
- Audit/Access Logging;
- purpose-bound, single-use Step-Up als zusätzliche Kontrolle;
- Owner-kontrolliertes Break-Glass-Prinzip.

Zu ersetzen/härten

- eigener TOTP als Autorisierungs-Authority → Supabase Native MFA;
- `profiles.totp_enabled` als MFA-Status → native Faktoren/AAL;
- fail-open Login-Step-Up für privilegierte Identitäten → fail-closed;
- Rolle-only Admin-Zugriff → Rolle + AAL2;
- Passkey-alone als generischer Owner-Step-Up → tatsächliche AAL-Evidence gemäß ESS-0020.

Verification Gate

ADR-0003.5 darf erst wieder nach `docs/adr/resolved/` verschoben und als `COMPLETE` markiert werden, wenn M5A folgende Evidence liefert:

1. Native TOTP enroll → challenge → verify PASS;
2. beide Owner-Identitäten verfügen über verifizierte Native-MFA-Faktoren;
3. privilegierte AAL1-/stale-/error-Pfade DENY;
4. serverseitige AAL2-Enforcement ist zentral implementiert;
5. Recovery/Backup-Verfahren PASS;
6. Security Advisor nach Mutation erneut geprüft;
7. CI und M5A Evidence `VERIFIED PASS` ;
8. Roadmap/Traceability auf finalen Merge-/Mutation-Stand synchronisiert.

ADR 0004 branding and panel removal

Quelle: docs/adr/ADR-0004-branding-and-panel-removal.md



CAPITAL-AI DOCUMENTARY

AUTONOMOUS AI DOCUMENT HYGIENIST • VERSION 0.5.4

System-Metadaten	Spezifikation
Plattform-Identität	Capital-AI Documentary (V0.5.4)
Gründer & Inhaber	Sven Kulesa
Zentrale E-Mail	sven.kulesa@capital-ai.online
Echtheits-Emblem	 CAPITAL-AI CORE
Status	Revisionssicher verifiziert & bereinigt

ADR-0004: Entfernung des "Core"-Schriftzugs und des aufklappbaren Profil-Panels aus der Kopfzeile

- **Status:** ACCEPTED
- **Datum:** 2026-07-10
- **Autor:** Sven Kulesa

Kontext

Im ursprünglichen Header-Layout der Benutzeroberfläche befand sich unter dem prominenten "Capital-AI"-Schriftzug das statische Schlagwort "CORE". Zudem enthielt die Header-Leiste ein ausklappbares Profil-Dropdown-Panel (gekennzeichnet durch ein Abwärtspfeil-Symbol), das durch Klick auf das Logo-Modul geöffnet wurde. Im Zuge der Bereinigung der Nutzeroberfläche und zur Vermeidung redundanter Navigationselemente sollte die Versionsnummer direkt auf der Landingpage angezeigt, das Wort "Core" entfernt und das aufklappbare Panel vollständig deaktiviert werden.

Entscheidung

1. **Entfernung der CORE-Branding-Bezeichnung:** Der statische Schriftzug "CORE" unter dem Markenlogo wurde auf allen Hauptebenen (Landingpage sowie Hauptdash-

board und Navigationsschublade) entfernt und durch die aktuelle, einheitliche Beta-Kennzeichnung `VERSION 0.5.4` ersetzt.

2. **Deaktivierung des Header-Dropdown-Panels:** Das Profil-Dropdown-Panel in der oberen Menüleiste (das sich hinter der globalen Suche befand) wurde mitsamt seinem Ausklapp-Pfeil (ChevronDown) entfernt. Das Logo- und Branding-Modul fungiert nun als saubere, nicht-interaktive Statusanzeige.
3. **Zentralisierung der Navigation:** Alle Profil-, Abonnement-, Datenschutz- und Administrationsoptionen verbleiben exklusiv in der ausfahrbaren Seitenleiste (Drawer-Menü), um ein aufgeräumtes und ergonomisches Bedienkonzept zu gewährleisten.

Konsequenzen

- Erhöhte visuelle Ruhe im Header-Bereich.
- Keine überlappenden UI-Komponenten oder redundanten Dropdowns hinter der Suchleiste.
- Vollständig dokumentierter und revisionssicher nachvollziehbarer Änderungsstand unter `docs/`.

ADR 0005 frontend module integration

Quelle: docs/adr/ADR-0005-frontend-module-integration.md



⊞ CAPITAL-AI DOCUMENTARY

AUTONOMOUS AI DOCUMENT HYGIENIST • VERSION 0.5.4

System-Metadaten	Spezifikation
Plattform-Identität	Capital-AI Documentary (V0.5.4)
Gründer & Inhaber	Sven Kulesa
Zentrale E-Mail	sven.kulesa@capital-ai.online
Echtheits-Emblem	⊞ CAPITAL-AI CORE
Status	Revisionssicher verifiziert & bereinigt

ADR-0005: Einbindung eines Capital-AI Front-End Moduls im Capital-AI FinTech Ökosystem

- **Status:** ACCEPTED
- **Datum:** 2026-07-10
- **Autor:** Sven Kulesa

Kontext

Das Capital-AI FinTech Ökosystem wächst zu einer modularen, verteilten Systemlandschaft heran. Um die Benutzeroberfläche (Front-End) der quantitativen Analysetools, Sentiment-Radar-Panels und Risikomodelle nahtlos und performant in dieses Ökosystem zu integrieren, bedarf es einer einheitlichen, standardisierten Integrationsarchitektur. Das Ziel ist es, das in React und Vite entwickelte Front-End-Modul als autarken Baustein bereitzustellen, der sowohl als eigenständige Applikation (SPA) als auch als eingebettetes Modul innerhalb größerer FinTech-Portale und Partner-Plattformen agieren kann.

Entscheidung

Wir legen die folgenden architektonischen Standards für die Einbindung des Front-End-Moduls im FinTech-Ökosystem fest:

1. **Micro-Frontend-Föderation (Webpack/Vite Module Federation):** - Das Front-End wird als "Remote"-Modul konfiguriert. Bestimmte Kern-Views (wie der Graham-DCF-Rechner, die Monte-Carlo-Simulation und das Krypto-Sentiment-Screener-Panel) werden als exportierbare Komponenten deklariert. - Dies ermöglicht es Host-Anwendungen des Ökosystems, diese Panels dynamisch und ohne vollständigen Seiten-Reload zur Laufzeit zu laden und zu rendern.
2. **Sicheres Cross-Origin Messaging (PostMessage-API):** - Für Szenarien, in denen das Modul in einem isolierten `iframe` betrieben wird, wird eine standardisierte, bidirektionale PostMessage-Schnittstelle implementiert. - Ein dedizierter Event-Bus im Front-End fängt Host-Befehle (z.B. Wechsel des ausgewählten Assets, Filteränderungen oder Theme-Umschaltungen) ab und sendet Ergebnisdaten (Score-Alarme, Export-Trigger) zurück an den Host.
3. **Zentralisiertes SSO & Token-Propagation:** - Die Authentifizierung wird über einen übergeordneten Identity Provider (Supabase Auth / Firebase Auth) geteilt. - Das Front-End-Modul akzeptiert JWT-Bearer-Token über URL-Parameter (verschlüsselt), HTTP-Header oder geteilte LocalStorage-Bereiche der Haupt-FinTech-Domain, um eine nahtlose Anmeldung (Single Sign-On) ohne erneute Passworteingabe zu gewährleisten.
4. **Styles-Kapselung & CSS-Präfixe:** - Um Style-Kollisionen im Ökosystem zu vermeiden, wird Tailwind CSS so konfiguriert, dass ein eindeutiger Namespace/Prefix (z.B. `aif-`) für alle CSS-Klassen verwendet wird, falls das Modul nicht im isolierten Shadow DOM eingebunden wird.

Konsequenzen

• Vorteile:

- Maximale Flexibilität bei der Bereitstellung und Integration bei Partnern.
- Unabhängige Update-Zyklen des Front-End-Moduls ohne Beeinträchtigung des restlichen FinTech-Backbones.
- Reibungslose und konsistente Benutzerführung durch automatisches Single-Sign-On.

• Nachteile/Risiken:

- Erhöhte Komplexität beim CORS- und Content-Security-Policy (CSP) Management.
- Notwendigkeit einer akkuraten API-Versionierung zur Vermeidung von Breaking Changes zwischen Host und Remote.

ADR 0006 platform director connection

Quelle: docs/adr/ADR-0006-platform-director-connection.md



⊞ CAPITAL-AI DOCUMENTARY

AUTONOMOUS AI DOCUMENT HYGIENIST • VERSION 0.5.4

System-Metadaten	Spezifikation
Plattform-Identität	Capital-AI Documentary (V0.5.4)
Gründer & Inhaber	Sven Kulesa
Zentrale E-Mail	sven.kulesa@capital-ai.online
Echtheits-Emblem	⊞ CAPITAL-AI CORE
Status	Revisionssicher verifiziert & bereinigt

ADR-0006: Nächste architektonische Schritte zur Anbindung eines Plattform-Direktors innerhalb der Plattform

- **Status:** ACCEPTED
- **Datum:** 2026-07-10
- **Autor:** Sven Kulesa

Kontext

Im Zuge der Skalierung der autonomen quantitative Analyseagenten (Graham-DCF, Monte-Carlo, Sentiment-Radar etc.) bedarf es einer zentralen, übergeordneten Governance-Instanz innerhalb des Backends—dem **Plattform-Direktor (Platform Director)**. Der Plattform-Direktor fungiert als zentraler System-Supervisor, der Ressourcen verwaltet, Agenten-Workflows steuert, Sicherheitsrichtlinien dynamisch anwendet, das System vor Überlastung (Rate Limiting) schützt und ein verlässliches Failover-Routing etabliert. Dieses Dokument skizziert die konkreten nächsten architektonischen Meilensteine zur Anbindung dieses Direktors.

Entscheidung

Wir legen die folgenden vier Kernschritte zur schrittweisen Implementierung und Anbindung des Plattform-Direktors fest:

Schritt 1: Definition des Platform Director API-Contracts & Zustandsmodells

- **Maßnahme:** Entwurf eines robusten Zustandsmodells im Backend zur permanenten Erfassung aller aktiven Systemkomponenten (Scanners, LLM-Knoten, API-Pipelines).
- **Schnittstellen:**
 - `GET /api/director/health` : Ermittelt die aktuelle Auslastung und den Latenzstatus aller angebundenen Agenten-Subsysteme.
 - `POST /api/director/route` : Nimmt Anfragen entgegen und entscheidet über das optimale Ziel-Routing basierend auf DSGVO-Vorgaben und Benutzerrechten (z.B. Zuweisung von Premium-Anfragen an High-Performance-Modelle).

Schritt 2: Implementierung des dynamischen Load-Balancing & Failover-Managers

- **Maßnahme:** Einbettung einer intelligenten Routing-Logik im Express-Server. Wenn eine Schnittstelle (z.B. Google Search Grounding für Sentiment) verzögert antwortet oder ein API-Limit erreicht ist, leitet der Plattform-Direktor die Abfrage automatisch an ein redundantes Ausweich-Modell (z.B. Wechsel von Gemini 1.5 Flash auf Gemini 2.0 / alternative Scanners) weiter.
- **Vorteil:** Erhöhte Ausfallsicherheit der Applikation im Live-Betrieb bei hoher Last.

Schritt 3: Einbau des Governance- & Rate-Limit-Wächters (DSGVO-Compliance)

- **Maßnahme:** Der Direktor überwacht die Einhaltung der Datenschutzrichtlinien. Er maskiert PII (Personally Identifiable Information) in Logfiles, bevor Anfragen an externe LLM-Schnittstellen gesendet werden.
- **Sicherheits-Bypass:** Definition einer granularen Access-Control-List (ACL) für Sven Kulesa (`sven.kulesa@gmx.net` und `sven.kulesa@gmail.com`), um administrativen System-Eingriff und Telemetrie-Einsicht in Echtzeit zu gestatten.

Schritt 4: Implementierung des Director Control Centers im Admin-Panel

- **Maßnahme:** Erweiterung des Admin-Panels in der Benutzeroberfläche um eine interaktive Dashboard-Ansicht für den Plattform-Direktor.
- **Funktionen:**
 - Echtzeit-Statusanzeige aller Agenten-Auslastungen.
 - Manuelle Override-Schalter, um bestimmte Daten-Pipelines temporär zu stoppen, zu drosseln oder umzurouten (z.B. bei API-Wartungen).
 - Live-Ansicht anonymisierter System-Audit-Logs zur Performance-Analyse.

Konsequenzen

- **Vorteile:**
 - Maximale Skalierbarkeit durch Entkopplung von API-Client und Server-Routing.
 - Gewährleistung strengster Datenschutzstandards durch automatische Vorprüfung der Payloads durch den Plattform-Direktor.

- Vereinfachte Diagnose und Wartung des Gesamtsystems durch die Admin-Bedien-schnittstelle.
- **Nachteile/Risiken:**
- Der Plattform-Direktor stellt einen potenziellen Single Point of Failure (SPOF) dar; daher muss dieser hochverfügbar und schlank implementiert werden.
- Minimaler Latenz-Overhead durch die zwischengeschaltete Routing- und Validierungs-ebene.

ADR 0007 compliance value chain

Quelle: docs/adr/ADR-0007-compliance-value-chain.md



⊞ CAPITAL-AI DOCUMENTARY

AUTONOMOUS AI DOCUMENT HYGIENIST • VERSION 0.5.4

System-Metadaten	Spezifikation
Plattform-Identität	Capital-AI Documentary (V0.5.4)
Gründer & Inhaber	Sven Kulesa
Zentrale E-Mail	sven.kulesa@capital-ai.online
Echtheits-Emblem	⊞ CAPITAL-AI CORE
Status	Revisionssicher verifiziert & bereinigt

ADR-0007: Bereitstellung einer vollen Capital-AI Compliance Wertschöpfungskette

- **Status:** ACCEPTED
- **Datum:** 2026-07-10
- **Autor:** Sven Kulesa

Kontext

Im modernen FinTech- und Kapitalmarktregulierungs-Umfeld (z.B. MiFID II, BaFin, DSGVO) ist die bloße Anzeige von statischen Berichten unzureichend. Um eine revisionssichere, automatisierte und lückenlose Compliance-Prüfung zu garantieren, muss die gesamte Wertschöpfungskette der Datenströme, Score-Berechnungen, Audit-Loggings und Export-Zertifizierungen digital abgebildet werden. Dieses Dokument definiert die Schritte zur Bereitstellung einer durchgängigen **Capital-AI Compliance Wertschöpfungskette**.

Entscheidung

Wir etablieren eine 5-stufige, integrierte Compliance-Wertschöpfungskette (Compliance Value Chain) innerhalb der Plattform-Architektur:

Stufe 1: Daten-Ingestion & Validierungs-Gateway (Datenintegrität)

- **Maßnahme:** Jede in das System einfließende Markt- oder Asset-Information wird über ein strikt typisiertes Gateway validiert.
- **Zweck:** Ausschluss von manipulierten Daten oder Fake-Werten (Einhaltung der "No-Mock-Data-Policy"). Unvollständige Abfragen werden sofort isoliert und im Audit-Log vermerkt.

Stufe 2: Anonymisierung & PII-Maskierung (Datenschutz-by-Design)

- **Maßnahme:** Alle einfließenden Abfragen von Nutzern werden vor der Weiterleitung an externe Analysedienste (z.B. Google GenAI / LLM API-Schnittstellen) bereinigt.
- **Zweck:** Vollständiger Schutz personenbezogener Daten (Art. 32 DSGVO). Personenbezogene Verbindungsdaten werden im RAM-Bypassing-Verfahren gefiltert, sodass IP-Adressen oder Nutzeridentitäten niemals externe Server erreichen.

Stufe 3: Deterministische Berechnungs-Engines & Sentiment-Verifizierung

- **Maßnahme:** Berechnungsformeln (wie Graham-DCF oder Risikokoeffizienten) laufen deterministisch auf dem Server. KI-gestützte Sentiment-Analysen müssen über nachvollziehbares Grounding (wie Google Search Grounding) mit genauer Quellenangabe belegt werden.
- **Zweck:** Einhaltung der finanzrechtlichen Offenlegungspflichten und Gewährleistung einer lückenlos nachvollziehbaren Berechnungs-Historie.

Stufe 4: Unveränderbares Event-Logging (Immutable Audit Trail)

- **Maßnahme:** Jede kritische Transaktion, jeder PDF-Export und jeder Administrator-Eingriff wird in einem manipulationssicheren, strukturierten Protokoll mit Zeitstempel aufgezeichnet (z.B. in der `AuditLogs` Tabelle/Komponente).
- **Zweck:** Erbringung rechtssicherer Nachweise gegenüber Regulierungsbehörden (BaFin, DSGVO-Aufsicht).

Stufe 5: Automatisierte Konformitäts-Zertifizierung & PDF-Export

- **Maßnahme:** Die Export-Engine generiert interaktive Compliance-Zertifikate (PDFs) mit einem eindeutigen kryptografischen Hash-Präfix, das die Echtheit des Berichts fälschungssicher bestätigt.
- **Zweck:** Bereitstellung direkt verwertbarer Konformitätsberichte für Anleiheemittenten, Fondsmanager und institutionelle Mandanten.

Konsequenzen

- **Vorteile:**
 - Nahtloser Übergang von quantitativer Rohdatenanalyse zu einer auditierten, rechtlich defensiblen Berichterstattung.
 - Deutlich reduziertes Haftungsrisiko für den Plattformbetreiber durch präventive Offenlegungsausschlüsse.

- Steigerung des Kundenvertrauens im institutionellen Enterprise-Segment durch verifizierte, quellengestützte KI-Auswertungen.
- **Herausforderungen:**
- Erhöhte Anforderungen an die Performance der PDF-Generierungs- und Logging-Engines.
- Kontinuierliche Abstimmung der Systemgrenzen auf die sich wandelnden europäischen Regulierungsentwürfe (z.B. EU AI Act).

ADR-0010: Erweiterung des Enterprise Standards um ESS-0001-CONTRACTS Chapter 11-20, ESS-0002, ESS-0003 und die ESS Registry

Quelle: docs/adr/ADR-0010-enterprise-standard-extension.md

ADR-0010: Erweiterung des Enterprise Standards um ESS-0001-CONTRACTS Chapter 11-20, ESS-0002, ESS-0003 und die ESS Registry

Status

Accepted

Implementation-Status

IN PROGRESS — Die Standarderweiterung ist vollständig verabschiedet und im Repository abgelegt. Die maschinelle Durchsetzung der neuen Kapitel (Validatoren, Contract Tests, Registry-Generierung) ist Gegenstand der nachgelagerten Umsetzungsstufen und noch nicht implementiert. Siehe [docs/architecture/REPOSITORY_STRUCTURE_ANALYSIS.md](#), Stufen 1 bis 11.

Datum

2026-07-30

Verantwortlich

Platform Director

Betroffene Dokumente

- [.ai/skills/ESS-0001-Contracts.md](#) (Version 1.0.0 → 1.1.0)
- [.ai/skills/ESS-0002-Supervisor-Architect.md](#) (neu)
- [.ai/skills/ESS-0003-Platform-Director.md](#) (neu)
- [.ai/registry/ess-registry.json](#) (neu)
- [docs/architecture/*](#) (fünf Enterprise Reports, neu)

Kontext

Der CAPITAL-AI Core besitzt mit ESS-0001 und ESS-0001-CONTRACTS eine vollständige Enterprise-Architektur auf konzeptioneller Ebene.

Eine vollständige Konsistenzanalyse sämtlicher Enterprise-Artefakte (dokumentiert in docs/architecture/ARCHITECTURE_GAP_REPORT.md) hat 29 Befunde ergeben, davon 9 der Stufe Critical und 10 der Stufe High.

Drei Befunde betreffen unmittelbar die Vollständigkeit und Widerspruchsfreiheit des Standards selbst:

- 1. **Abgebrochene Kapitelkette.** ESS-0001-CONTRACTS Chapter 10 verweist im Abschnitt Integration ausdrücklich auf Chapter 11, 12 und 13. Diese Kapitel existieren nicht. Das Dokument endet ohne Version History und ohne Approval-Abschnitt.
- 2. **Zehn Regelbereiche ohne technischen Vertrag.** Für Security, Compliance, Qualität, Erweiterungen, Migration, Lebenszyklus, Knowledge Graph, Repository Governance, AI Orchestration, Digital Twin, Automatisierung und den Standardabschluss existiert keine verbindliche Regelung.
- 3. **Zwei Regelkonflikte innerhalb der bestehenden Dokumente.** ESS-0001 Chapter 8 und ESS-0001-CONTRACTS Chapter 8 verwenden unterschiedliche Event-Namenskonventionen. Chapter 2 listet Core nicht als Pflichtmodul, während Chapter 3 und Chapter 6 Core als Fundament der Layer-Hierarchie definieren.

Ohne Auflösung dieser Punkte ist der in Chapter 1 geforderte Determinismus KI-gestützter Entwicklung nicht erreichbar: zwei KI-Systeme mit identischer Aufgabenstellung würden unterschiedliche Event-Namen, unterschiedliche Modullisten und unterschiedliche Governance-Auslegungen erzeugen.

Entscheidung

1. ESS-0001-CONTRACTS wird um die Kapitel 11 bis 20 erweitert

Kapitel	Gegenstand
11	Enterprise Security & Compliance Contracts
12	Enterprise Validation & Quality Contracts
13	Enterprise Plugin & Extension Contracts
14	Enterprise Migration & Lifecycle Contracts
15	Enterprise Knowledge Graph Contracts
16	Enterprise Repository Governance
17	Enterprise AI Orchestration Contracts
18	Enterprise Digital Twin Contracts
19	Enterprise Automation Contracts
20	Official Enterprise Standard

Die Dokumentversion steigt von 1.0.0 auf **1.1.0**. Es handelt sich um eine additive Erweiterung ohne Breaking Change: keine bestehende Regel wurde geändert, entfernt oder um-nummeriert.

2. Bestehende Regelkonflikte werden durch Klarstellung aufgelöst, nicht durch Änderung

Chapter 20 enthält den Abschnitt Normative Clarifications. Die Auflösung folgt ausschließlich der bereits in Chapter 1 festgelegten Rangfolge (ADR → ESS Contracts → ESS → Projektdokumentation → Implementierung):

- **Event Naming:** Verbindlich gilt das Suffix `Event` gemäß Contracts Chapter 8. Die Bezeichnungen in ESS-0001 Chapter 8 sind als Kategorienamen zu lesen. ESS-0001 bleibt unverändert.
- **Core:** `src/platform/Core` ist gemäß Chapter 3 und Chapter 6 verbindlicher Bestandteil der Plattform und wird als technische Basisschicht geführt, nicht als fachliches Modul.
- **Querschnittsmodule:** Die 13 Pflichtmodule ohne eigene Layer-Ebene erhalten in Chapter 16 eine verbindliche Abhängigkeitsregel. Die Hierarchie aus Chapter 6 bleibt unverändert.
- **Lifecycle vs. Version Category:** Beide Vokabulare bleiben unverändert bestehen. Chapter 14 definiert die eindeutige Zuordnung.

3. Der ESS-Nummernraum wird verbindlich registriert

ESS-0001 reserviert in den Abschnitten Integration Points und Related Enterprise Specifications die Nummern ESS-0002 bis ESS-0009 mit festen Titeln. Diese Reservierung ist bindend und wird unverändert übernommen. Eine abweichende Belegung — etwa „ESS-0004 Documentary Engine“ — wäre eine Verletzung von ESS-0001 und zugleich ein Duplikat, da die Documentary Engine bereits vollständig durch ESS-0001 spezifiziert ist.

Die Registry wird als `.ai/registry/ess-registry.json` geführt. Der freie Nummernraum beginnt bei ESS-0010.

4. Es werden ausschließlich zwei neue ESS-Dokumente angelegt

Dokument	Begründung
ESS-0002 Supervisor Architect	Der Supervisor ist Stufe 4 der Wertschöpfungskette und in Chapter 6 als eigener Layer geführt, besitzt jedoch keine Spezifikation. Echte Regelungslücke.
ESS-0003 Platform Director	Der Platform Director ist oberste Governance-Instanz und Freigabestufe, besitzt jedoch keine Spezifikation. Ergänzt ADR-0006, ersetzt es nicht.

Für die reservierten Nummern ESS-0004 bis ESS-0007 sowie ESS-0009 wird **kein** Dokument angelegt: die jeweiligen Regelbereiche sind durch die Kapitel 9, 11, 12, 15 und 19 vollständig geregelt. Eigenständige Dokumente wären Duplikate und würden gegen das Verbot redundanter Dokumente (ESS-0001, Zero Duplication) verstoßen. Die Begründung je Nummer ist in der ESS Registry hinterlegt.

ESS-0008 (AI Agent Framework) bleibt reserviert und wird als offener Regelbedarf geführt: die fachlichen Domänen-Agenten unter `src/agents` besitzen keine Spezifikation, und Chapter 17 regelt ausschließlich die Orchestrierung der KI-Systeme, nicht das Agentenmodell der Fachdomänen.

ESS-0010 (Repository Governance) wird ausdrücklich **nicht** angelegt. Der Regelbereich ist durch Chapter 16 vollständig und normativ abgedeckt; ein eigenes Dokument wäre ein Duplikat. Die Nummer bleibt frei.

5. Zwei geprüfte Kapitelkandidaten werden nicht als eigene Kapitel geführt

Vor Abschluss des Standards wurde geprüft, ob über Chapter 20 hinaus weitere Kapitel notwendig sind. Zwei Bereiche wurden untersucht und bewusst integriert statt separiert:

- **Registry & Discovery** — geregelt in Chapter 7 (Enterprise Registry), Chapter 15 (Registry Relationship) und Chapter 16 (Repository Governance).
- **Observability & Telemetry** — geregelt in Chapter 12 (Observability Contract), Chapter 18 (Twin States) und Chapter 19 (Execution Contract).

Ein eigenständiges Kapitel hätte in beiden Fällen bestehende Regeln wiederholt. Die Prüfung ist in Chapter 20, Abschnitt Completeness Assessment, dokumentiert.

6. Fünf Enterprise Reports werden unter `docs/architecture/` geführt

Report	Gegenstand
<code>ARCHITECTURE_GAP_REPORT.md</code>	29 Befunde der Konsistenzanalyse
<code>REPOSITORY_STRUCTURE_ANALYSIS.md</code>	Strukturvergleich und priorisierter Umsetzungsplan (Stufen 0-11)
<code>COMPONENT_CONSISTENCY_REPORT.md</code>	Konformitätsprüfung sämtlicher 22 Komponenten
<code>AI_VALUE_CHAIN_VALIDATION.md</code>	Validierung der achtstufigen Wertschöpfungskette
<code>ENTERPRISE_MATURITY_REPORT.md</code>	Reifegradbewertung in 13 Kategorien

Diese Reports sind Analyseartefakte, keine normativen Dokumente. Normative Wirkung besitzen ausschließlich ESS-Dokumente und ADRs (Chapter 20, Standard Definition).

Alternativen

A) Keine Erweiterung, Regelung ad hoc bei Bedarf. Verworfen. Chapter 10 verweist bereits auf nicht existierende Kapitel; der Standard wäre dauerhaft gebrochen. Ohne Verträge für Security, Qualität und Migration bleibt jede KI-gestützte Änderung eine Einzelfallauslegung.

B) Bestehende Dokumente ändern, um Konflikte zu beseitigen. Verworfen. Eine Änderung an ESS-0001 (etwa der Event-Namen) wäre ein Breaking Change an der Enterprise Baseline und hätte sämtliche darauf verweisenden Artefakte entwertet. Die Rangfolge aus Chapter 1 löst den Konflikt bereits ohne Änderung.

C) Eigene ESS-Nummerierung nach thematischer Logik vergeben. Verworfen. Die Reservierung in ESS-0001 ist bindend. Eine abweichende Vergabe hätte zwei widersprüchliche Dokumentregistrierungen und ein Duplikat der Documentary-Spezifikation erzeugt.

D) Für jede reservierte Nummer ein Dokument anlegen. Verworfen. Acht zusätzliche Dokumente hätten überwiegend bestehende Kapitelinhalte wiederholt und gegen Zero Duplication verstoßen.

Konsequenzen

Positiv

- Die Kapitelkette ist geschlossen; Chapter 10 verweist nicht mehr ins Leere.
- Zehn zuvor ungeregelte Bereiche besitzen einen verbindlichen technischen Vertrag.
- Beide Regelkonflikte sind aufgelöst, ohne eine bestehende Regel zu verändern.
- Der ESS-Nummernraum ist eindeutig; künftige Erweiterungen sind deterministisch.
- Der Digital Twin besitzt erstmals messbare Kriterien (Twin States, Drift-Toleranz null).
- Die AI-Wertschöpfungskette ist erstmals normativ definiert, inklusive Auslöser, Übergabeartefakten und Abbruchbedingungen.
- Die Definition-Dimension des Reifegrads steigt in acht Kategorien deutlich an (siehe `ENTERPRISE_MATURITY_REPORT.md`).

Negativ / Aufwand

- Die Zahl der maschinell durchzusetzenden Regeln steigt erheblich: Chapter 12 fordert 16 Pflichtvalidatoren, die Kapitel 11 bis 19 definieren über 80 Enterprise Events. Keine dieser Regeln ist derzeit implementiert.
- Bei vollständiger Durchsetzung von Chapter 18 (Drift-Toleranz null, nur `Synchronized` ist freigabefähig) wäre gegenwärtig keine Produktionsfreigabe zulässig, da der Twin-Zustand sämtlicher Komponenten `Unknown` lautet. Die Durchsetzung tritt erst mit der Implementierung der Validatoren ein; bis dahin gilt der Bestandsbetrieb unverändert weiter.
- Chapter 16 macht die bestehenden Root-Abweichungen (`server/` , `sql/` , `server.ts`) formal sichtbar. Sie sind über einen gesonderten ADR als Ausnahme zu registrieren.

Neutral

- Es wurde kein produktiver Code geändert. Diese Entscheidung betrifft ausschließlich Standard- und Analysedokumente.
- Bestehende produktionsreife Komponenten (`server/documentHygiene.ts` , `server/systemEvents.ts` , `server/versionManager.ts` , `server/decisionEngine.ts` , `server/iam/`) bleiben unverändert. Chapter 14 schreibt ausdrücklich Registrierung und Kapselung statt Neuentwicklung vor.

Folgeentscheidungen

Die folgenden Punkte erfordern eigene ADRs und sind mit dieser Entscheidung **nicht** getroffen:

1. Bestandsschutz und Zielstruktur für die Root-Verzeichnisse `server/` und `sql/` sowie für `server.ts` (Chapter 2, Chapter 16).
2. Auflösung der vier widersprüchlichen Versionsstände und Zuweisung der Versionshoheit (GAP-019, Chapter 9).
3. Überführung der Domänenlogik aus `src/agents`, `src/orchestrator`, `src/services` nach `src/features/<domain>` (Chapter 2, Chapter 14).
4. Eröffnung von ESS-0008 (AI Agent Framework) bei nachgewiesenem Regelbedarf.
5. Nachdokumentation von ADR-0001 bis ADR-0003 als Dokumente sowie Ergänzung von ADR-0009 in `adr_history.json` (GAP-026).

Referenzen

- ESS-0001 — Documentary & Code Intelligence Architect
- ESS-0001-CONTRACTS — Enterprise Technical Contracts, Version 1.1.0
- ESS-0002 — Supervisor Architect
- ESS-0003 — Platform Director
- ADR-0006 — Plattform-Direktor (ergänzt, nicht ersetzt)
- ADR-0007 — Compliance-Wertschöpfungskette
- `docs/architecture/ARCHITECTURE_GAP_REPORT.md`
- `docs/architecture/REPOSITORY_STRUCTURE_ANALYSIS.md`
- `docs/architecture/COMPONENT_CONSISTENCY_REPORT.md`
- `docs/architecture/AI_VALUE_CHAIN_VALIDATION.md`
- `docs/architecture/ENTERPRISE_MATURITY_REPORT.md`

ADR-0011: Bestandsschutz und Zielstruktur für die Root-Abweichungen ``server/``, ``sql/`` und ``server.ts``

Quelle: docs/adr/ADR-0011-bestandsschutz-root-abweichungen.md

ADR-0011: Bestandsschutz und Zielstruktur für die Root-Abweichungen `server/`, `sql/` und `server.ts`

Status

Accepted

Implementation-Status

IN PROGRESS — Die Ausnahmen sind registriert und damit vertragskonform getragen (`ai/registry/exception-registry.json`).

Vier von acht Ausnahmen sind aufgelöst:

ID	Pfad	Aufgelöst am	Nachweis
EXC-0003	<code>sql/</code>	2026-07-30	<code>docs/migration/MI-GRATI-ON_EXC-0003_sql_to_supabase.md</code>
EXC-0005	<code>ORCHESTRA-TORS_AND_SCORING_ENGINES.md</code>	2026-07-31	<code>docs/migration/MI-GRATI-ON_EXC-0005_EXC-0007_root_docs.md</code>
EXC-0007	<code>favicon.svg</code>	2026-07-31	<code>docs/migration/MI-GRATI-ON_EXC-0005_EXC-0007_root_docs.md</code>

EXC-0003: byteidentische Überführung nach `supabase/migrations/20260730000000_user_quota.sql`, Verzeichnis entfernt. Dabei wurde FND-EXC-0003-01 festgestellt — die Migration ist im aktuellen Codebestand verwaist (`src/lib/freeTierLimits.ts` existiert nicht und hat nie existiert). Klärung, ob `public.user_quota` in der Zielumgebung Daten enthält, steht aus.

EXC-0005: byteidentische Überführung nach `docs/architecture/`. Die in ADR-0011 zusätzlich vorgesehene Ergänzung von ESS-/ADR-Referenzen wurde bewusst nicht miterledigt, um die Prüfsumme als Nachweis der reinen Strukturverlagerung zu erhalten (bleibt unter GAP-029 offen).

EXC-0007: byteidentische Überführung nach `public/`. `index.html` war unverändert lassbar, da die Referenz bereits root-absolut (`/favicon.svg`) war. Dabei wurde FND-EXC-0007-01 aufgedeckt: da `public/` zuvor leer war und Vite ausschließlich `publicDir` nach `dist/` ko-

piert, gelangte das Favicon vermutlich nie in den Produktions-Build. Diese Migration behebt den Fehler als Nebeneffekt der Strukturkonformität.

Drei Ausnahmen bleiben mit Permanent-Status bestehen (EXC-0004, EXC-0006, EXC-0008) — das ist der beabsichtigte Endzustand, keine offene Arbeit.

EXC-0001 und EXC-0002 bestehen unverändert fort. Ihre Überführung ist Gegenstand der Umsetzungsstufen 3, 9 und 11 aus `docs/architecture/REPOSITORY_STRUCTURE_ANALYSIS.md` (Enterprise Event Bus, Security & Compliance, Legacy Migration) und wurde bewusst nicht begonnen — siehe Abschnitt Bewertung von EXC-0001/EXC-0002 unten. Die maschinelle Überwachung sämtlicher Ausnahmen setzt zudem den Structure Validator aus Chapter 12 voraus, der noch nicht implementiert ist.

Datum

2026-07-30

Verantwortlich

Platform Director

Betroffene Pfade

`server/`, `server.ts`, `sql/`, `AGENTS.md`, `ORCHESTRATORS_AND_SCORING_ENGINES.md`, `index.html`, `favicon.svg`, Toolchain-Konfiguration im Root

Kontext

ESS-0001-CONTRACTS Chapter 2 definiert die zulässigen Root-Verzeichnisse abschließend:

```
.ai/ docs/ scripts/ src/ supabase/ tests/ public/ dist/
```

Als Root-Dokumente sind ausschließlich `README.md`, `CHANGELOG.md`, `LICENSE`, `CODE_OF_CONDUCT.md` und `CONTRIBUTING.md` zugelassen. Chapter 2 legt zusätzlich fest, dass `src/` den gesamten Quellcode der Plattform enthält.

Das Repository weicht davon an mehreren Stellen ab. Die Abweichungen bestanden bereits vor Einführung des Enterprise Standards. Mit ADR-0010 wurde Chapter 16 (Repository Governance) verabschiedet, wodurch diese Abweichungen erstmals formal sichtbar werden:

Nicht dokumentierte Abweichungen sind Befunde der Stufe Critical. — ESS-0001-CONTRACTS Chapter 16, Root Governance

Betroffen sind acht Positionen, dokumentiert als GAP-006 in `docs/architecture/ARCHITECTURE_GAP_REPORT.md`.

Die schwerwiegendste Position ist `server/`: dort liegt der vollständige produktive Backend-Code, darunter vier Komponenten, die ESS-0001 Chapter 2 (Existing Documentary Detection) ausdrücklich vor Neuentwicklung schützt — `documentHygiene.ts`, `systemEvents.ts`, `versionManager.ts` und `decisionEngine.ts`.

Damit besteht ein Zielkonflikt zwischen zwei verbindlichen Regeln:

- Chapter 2 fordert sämtlichen Quellcode unter `src/`.
- ESS-0001 Chapter 2 untersagt die Neuentwicklung produktionsreifer Bestandskomponenten.

Beide Regeln bleiben gültig. Chapter 14 (Legacy Contract, Adapter Contract) löst den Konflikt auf, setzt jedoch eine registrierte Ausnahme als Zwischenzustand voraus.

Entscheidung

1. Sämtliche acht Root-Abweichungen werden als Ausnahmen registriert

Die Registrierung erfolgt in `.ai/registry/exception-registry.json` gemäß Exception Contract aus Chapter 16 mit den dort geforderten Feldern: Exception ID, betroffener Pfad, verletzter Contract, Begründung, Risiko, Zielzustand, Verantwortlicher, ADR-Referenz und Status.

ID	Pfad	Status	Frist	Risiko
EXC-0001	<code>server/</code>	Time Limited	2027-01-31	Medium
EXC-0002	<code>server.ts</code>	Time Limited	2027-01-31	High
EXC-0003	<code>sql/</code>	Time Limited	2026-09-30	Low
EXC-0004	<code>AGENTS.md</code>	Permanent	—	Low
EXC-0005	<code>ORCHESTRA-TORS_AND_SCORING_ENGINES.md</code>	Time Limited	2026-09-30	Low
EXC-0006	<code>index.html</code>	Permanent	—	Low
EXC-0007	<code>favicon.svg</code>	Time Limited	2026-09-30	Low
EXC-0008	Toolchain-Konfiguration (10 Dateien)	Permanent	—	Low

Damit wandelt sich GAP-006 von einem nicht registrierten Critical-Befund in ein bewusst getragenes, überwachtes und mit Zielzustand versehenes Risiko. Die Abweichung besteht unverändert fort — sie ist ab sofort lediglich vertragskonform dokumentiert.

2. Kein produktiver Code wird im Rahmen dieser Entscheidung verschoben

Diese Entscheidung verschiebt keine einzige Datei. Sie registriert den Ist-Zustand und legt den Zielzustand fest. Jede tatsächliche Verlagerung erfolgt ausschließlich als Migration gemäß Chapter 14 — mit Impact Analyse, Reihenfolge, Validierung und Rollback-Plan.

Begründung: `server.ts` wird unmittelbar von `package.json` (`dev`, `build`) und dem `Dockerfile` referenziert. Eine Verlagerung ohne gleichzeitige Anpassung von Build, Container und Deployment unterbricht den Wirkbetrieb. Chapter 14 verbietet Migrationen ohne Rollback-Strategie ausdrücklich.

3. Drei Ausnahmen erhalten Permanent-Status

`AGENTS.md`, `index.html` und die Toolchain-Konfiguration verbleiben dauerhaft im Root.

Der Grund ist in allen drei Fällen derselbe: Werkzeugbindung. Die Dateien werden von ihrer jeweiligen Werkzeugkette ausschließlich im Repository-Root gelesen. Eine Verlagerung würde ihre Wirkung aufheben, ohne einen Vorteil zu erzeugen.

Für die Toolchain-Konfiguration kommt hinzu, dass ESS-0001 Chapter 2 `package.json`, `package-lock.json`, `tsconfig.json`, `vite.config.*`, `Dockerfile`, `.env.example` und `render.yaml` ausdrücklich als im Root zu erkennende Konfigurationsdateien führt und ihre dortige Lage damit voraussetzt. Die Root-Regel in Contracts Chapter 2 zielt auf Dokumente und Quellcode, nicht auf Werkzeugkonfiguration.

Permanent-Status entbindet nicht von der Überwachung. Die Positionen bleiben in der Registry geführt und werden vom Structure Validator geprüft.

4. `sql/` wird als Duplikat behandelt, nicht nur als Strukturabweichung

`sql/` enthält genau eine Datei: `001_user_quota.sql`. Für denselben Artefakttyp existiert bereits `supabase/migrations/` mit `20260711000000_iam.sql`.

Damit liegt zusätzlich zu Chapter 2 eine Verletzung von Chapter 2 (File Placement Rules) und Chapter 3 (eine Verantwortung, genau ein Verzeichnis) vor: Migrationen sind auf zwei Orte verteilt. Eine Migration kann dadurch übersehen oder doppelt ausgeführt werden.

Diese Position besitzt die kürzeste Frist und das beste Aufwand-Nutzen-Verhältnis aller acht Ausnahmen — eine Datei, ein Zielort, geringes Risiko.

5. Zielstruktur für `server/`

Die Zuordnung folgt ausschließlich den in Chapter 3 definierten Verzeichnisverantwortlichkeiten. Sie legt fest, wohin eine Komponente gehört — nicht, wann sie dorthin überführt wird.

Bestand	Zielkomponente	Contract-Grundlage
<code>documentHygiene.ts</code>	<code>src/platform/Documentary/</code>	Chapter 3 — Documentary Directory
<code>documentSanitizer.ts</code>	<code>src/platform/Documentary/</code>	Chapter 3
<code>fileWatcher.ts</code>	<code>src/platform/Documentary/Discovery/</code>	ESS-0001 Chapter 2
<code>decisionEngine.ts</code>	<code>src/platform/Documentary/Engine/</code>	Chapter 3
<code>systemEvents.ts</code>		Chapter 8, Chapter 12

Bestand	Zielkomponente	Contract-Grundlage
	src/platform/Events/ + src/platform/Telemetry/	
versionManager.ts	src/platform/VersionManager/	Chapter 9
iam/ (5 Dateien)	src/platform/Security/	Chapter 11
stepUp.ts	src/platform/Security/	Chapter 11
stripe.ts	src/features/billing/	Chapter 2 — Feature Isolation
mailer.ts	src/features/notifications/	Chapter 2
ai.ts	src/features/ai/ bzw. AI Gateway	Chapter 2, Chapter 10
orchestrator.ts	src/features/<domain>/	Chapter 2
db.ts	src/platform/Shared/	Chapter 3 — Shared Layer
env.ts, ownerConfig_server.ts	src/config/	Chapter 2 — Configuration

Bis zur Überführung erfolgt die Anbindung ausschließlich über Adapter gemäß Chapter 14: Der Adapter kapselt die Legacy-Implementierung vollständig, veröffentlicht ausschließlich Enterprise Interfaces, erzeugt Enterprise Events — und verändert die Legacy-Implementierung nicht.

6. Reihenfolge der Auflösung

Reihenfolge	Ausnahme	Voraussetzung
1	EXC-0003 sql/	keine — sofort umsetzbar
2	EXC-0005, EXC-0007	keine — reine Verlagerung
3	EXC-0001 server/	Event Bus (Stufe 3), Adapter-Fähigkeit (Stufe 11)
4	EXC-0002 server.ts	EXC-0001 überwiegend abgeschlossen

server.ts wird zuletzt aufgelöst, weil sich sein Umfang mit jeder aus server/ überführten Komponente verringert. Eine frühe Verlagerung würde denselben Aufwand mehrfach erzeugen.

Alternativen

A) Abweichungen unregistriert lassen. Verworfen. Chapter 16 stuft nicht dokumentierte Abweichungen als Critical ein. Der Zustand wäre dauerhaft nicht konform, ohne dass ein Weg zur Auflösung existierte.

B) Chapter 2 ändern und server/ sowie sql/ als zulässige Root-Verzeichnisse aufnehmen. Verworfen. Das wäre eine Änderung am Enterprise Standard zur Anpassung

an einen Ist-Zustand. Chapter 2 trifft eine bewusste Architekturaussage — sämtlicher Quellcode unter `src/`. Eine Aufweichung würde die Feature Isolation und die Layer-Zuordnung gleichermaßen entwerten und für jede künftige Abweichung als Präzedenzfall dienen.

C) Sofortige Verlagerung des gesamten Backends. Verworfen. Verletzt Chapter 14 (keine Migration ohne Impact Analyse und Rollback) und gefährdet den Wirkbetrieb. `server.ts` allein umfasst 1.934 Zeilen und ist direkt an Build, Container und Deployment gebunden.

D) Sämtliche Ausnahmen als Permanent führen. Verworfen. Permanent-Status ist ausschließlich bei Werkzeugbindung gerechtfertigt. Für `server/`, `server.ts` und `sql/` besteht keine solche Bindung; ein Permanent-Status wäre lediglich ein dauerhafter Verzicht auf die Zielarchitektur.

Konsequenzen

Positiv

- GAP-006 ist aufgelöst: aus einem nicht registrierten Critical-Befund wird ein registriertes, befristetes und mit Zielzustand versehenes Risiko.
- Der Zielkonflikt zwischen Chapter 2 und ESS-0001 Chapter 2 ist entschieden, ohne eine der beiden Regeln zu verändern.
- Für jede der 15 Dateien unter `server/` existiert erstmals eine verbindliche Zielkomponente.
- Die Auflösungsreihenfolge ist festgelegt; `sql/` ist sofort umsetzbar.
- Die Exception Registry ist angelegt und kann vom Structure Validator unmittelbar ausgewertet werden, sobald dieser existiert.

Negativ / Aufwand

- Vier Ausnahmen tragen Fristen. Läuft eine Frist ohne Auflösung ab, entsteht gemäß Chapter 16 automatisch ein Befund. Die Fristen sind damit eine bewusst gesetzte Verpflichtung.
- Die Überwachung der Registry setzt den Structure Validator (Chapter 12) voraus. Bis dahin erfolgt die Prüfung manuell — die Registry ist bis dahin eine Zusage, keine Durchsetzung.
- Die Zielstruktur für `server/` bindet 14 Überführungen an Chapter 14 und erzeugt damit Migrationsaufwand, der zuvor nicht ausgewiesen war.

Neutral

- Kein produktiver Code wurde geändert oder verschoben. Der Wirkbetrieb ist unberührt.
- Die vier durch ESS-0001 geschützten Documentary-Komponenten bleiben unverändert und werden gekapselt, nicht neu entwickelt.

Bewertung von EXC-0001/EXC-0002 zum Stand 2026-07-31

Bei der Umsetzung der übrigen sechs Ausnahmen wurde geprüft, ob EXC-0001 (`server/``) und EXC-0002 (`server.ts``) im selben Arbeitsgang aufgelöst werden können. Das Ergebnis ist Nein — und zwar aus genau den Gründen, die dieses ADR für sie selbst bereits festgehalten hat.

Befund

`server/`` enthält 15 Dateien, deren Zielkomponenten sich über sechs verschiedene Bereiche verteilen (`Documentary``, `Events``, `Telemetry``, `VersionManager``, `Security``, `src/features/*``, `src/config/``). `server.ts`` (1.934 Zeilen) verdrahtet sämtliche dieser Module zu einer laufenden Express-Anwendung und wird direkt von `package.json`` (`dev``, `build``) und dem `Dockerfile`` referenziert. Darunter befindet sich sicherheitskritischer Code — `server/iam/`` (Authentifizierung, Rate Limiting, Secret-Verschlüsselung, TOTP) und `server/stripe.ts`` (Zahlungsverkehr).

Eine Verlagerung dieser Art ist keine Structural Migration wie bei EXC-0003, EXC-0005 und EXC-0007. Dort genügte ein reiner Git-Rename mit Prüfsummenvergleich, weil jeweils genau eine Datei ohne Code-Abhängigkeiten betroffen war. Hier stehen dem gegenüber:

- Import-Pfade in `server.ts`` und in sämtlichen 15 verschobenen Dateien müssten gleichzeitig umgeschrieben werden.
- Der Enterprise Event Bus, über den die Zielkomponenten laut Chapter 8 und Chapter 17 kommunizieren sollen, existiert nicht — die Module könnten am neuen Ort nicht in die vertraglich vorgesehene Architektur eingebettet werden, sondern nur erneut als loses Dateikonvolut.
- Der Adapter-Mechanismus aus Chapter 14, der die Legacy-Implementierung kapseln soll, ohne sie zu verändern, setzt Enterprise Interfaces voraus, die ebenfalls nicht existieren.
- Ein Fehler in Authentifizierung oder Zahlungsverkehr wäre, anders als bei den bisherigen vier Ausnahmen, keine Dokumentations- oder Struktur-Regression, sondern ein Sicherheits- oder Compliance-Vorfall.

Entscheidung

EXC-0001 und EXC-0002 werden mit diesem ADR **nicht** aufgelöst. Sie werden im Ausnahmestatus `Time Limited`` bis 2027-01-31 belassen, wie bereits festgelegt.

Diese Entscheidung ist keine Abweichung von ADR-0011 — sie ist seine Bestätigung: ADR-0011 hat die Reihenfolge selbst vorgegeben (Abschnitt Reihenfolge der Auflösung) und EXC-0001 sowie EXC-0002 ausdrücklich als von Stufe 3 (Event Bus) und Stufe 11 (Legacy Migration) abhängig markiert. Diese Stufen sind nicht Gegenstand dieses ADR und wurden im Rahmen dieser Arbeit nicht umgesetzt.

Nächster zulässiger Schritt

Sobald der Enterprise Event Bus (Stufe 3) und mindestens ein funktionierender Adapter-Mechanismus (Stufe 11) vorliegen, kann für `server/`` ein eigener Migrationsplan mit Impact

Analyse, Reihenfolge je Datei und Rollback-Strategie gemäß Chapter 14 erstellt werden. Bis dahin bleibt `server/` unverändert im Wirkbetrieb.

Folgeentscheidungen

Nicht Gegenstand dieser Entscheidung und weiterhin offen:

1. Versionshoheit und Auflösung der vier widersprüchlichen Versionsstände, einschließlich der abweichenden Versionsangabe in `metadata.json` (GAP-019).
 2. Zielstruktur der Domänenlogik aus `src/agents`, `src/orchestrator`, `src/services` nach `src/features/<domain>` (GAP-009). Betrifft `src/`, nicht den Root, und wird gesondert entschieden.
 3. Konkrete Migrationspläne je Ausnahme mit Impact Analyse und Rollback gemäß Chapter 14.
-

Referenzen

- ESS-0001 — Chapter 2 (Existing Documentary Detection, Konfigurationsdateien)
- ESS-0001-CONTRACTS — Chapter 2, Chapter 3, Chapter 11, Chapter 14, Chapter 16
- ADR-0010 — Enterprise Standard Extension (Einführung Chapter 16)
- ADR-0003.5 — Owner-IAM (betrifft `server/iam/`, Implementation-Status COMPLETE)
- `.ai/registry/exception-registry.json`
- `docs/architecture/ARCHITECTURE_GAP_REPORT.md` — GAP-006, GAP-009, GAP-020, GAP-029
- `docs/architecture/REPOSITORY_STRUCTURE_ANALYSIS.md` — Umsetzungsstufen 3, 9, 11

ADR-0013: Konsolidierung der ESS-Dokumentationsverantwortung, Vergabe von ESS-0010 und ESS-0011

Quelle: docs/adr/ADR-0013-ess-documentation-responsibility-consolidation.md

ADR-0013: Konsolidierung der ESS-Dokumentationsverantwortung, Vergabe von ESS-0010 und ESS-0011

Status

Accepted

Implementation-Status

COMPLETE (verifiziert 2026-07-31) — Sämtliche sieben ESS-Dokumente tragen eine Dokumentklasse und vollständige Cross-References. ESS-0010, ESS-0011 und ESS-0011-CONTRACTS sind angelegt und in der Registry geführt. Die Responsibility Matrix ist als docs/architecture/ESS_RESPONSIBILITY_MATRIX.md verbindlich abgelegt.

Offen bleibt ausschließlich die **maschinelle** Durchsetzung: der DocumentResponsibilityValidator setzt Umsetzungsstufe 4 aus docs/architecture/REPOSITORY_STRUCTURE_ANALYSIS.md voraus und ist nicht Gegenstand dieser Entscheidung.

Datum

2026-07-31

Verantwortlich

Platform Director

Hinweis zur Nummernvergabe

Dieser ADR wurde ursprünglich als **ADR-0012** erstellt. Während der Bearbeitung wurde auf main unabhängig ein anderer **ADR-0012 — SecurityComplianceAuditor Integration** gemergt (Commits 04de42a , d1348c7).

ESS-0001-CONTRACTS Chapter 16 legt fest: „Jede ESS-Nummer wird genau einmal vergeben“ — für ADR-Nummern gilt dieselbe Regel über „ADR-Nummern sind vierstellig und fortlaufend“.

Die auf main bereits gemergte Vergabe besitzt Vorrang. Dieser ADR wurde daher auf **ADR-0013** umnummeriert, der zugehörige Governance-ADR von 0013 auf **ADR-0014**. Sämtliche Referenzen in 18 Dateien wurden entsprechend nachgezogen.

Die Kollision selbst ist ein Anwendungsfall der Regel `GOV-ADR-004` aus ESS-0012-CONTRACTS — sie wäre bei aktivem Governance Validator automatisch erkannt worden.

Betroffene Dokumente

`.ai/skills/ESS-0001-Documentary-Architect.md` (ergänzt)

`.ai/skills/ESS-0001-Contracts.md` (ergänzt)

`.ai/skills/ESS-0002-Supervisor-Architect.md` (ergänzt)

`.ai/skills/ESS-0003-Platform-Director.md` (ergänzt)

`.ai/skills/ESS-0010-Documentary-Engine.md` (neu)

`.ai/skills/ESS-0011-Enterprise-Traceability.md` (neu)

`.ai/skills/ESS-0011-Contracts.md` (neu)

`.ai/registry/ess-registry.json` (fortgeschrieben)

`docs/architecture/ESS_RESPONSIBILITY_MATRIX.md` (neu)

`docs/architecture/ESS_CONSOLIDATION_REPORT.md` (neu)

Kontext

Eine vollständige Analyse des ESS-Dokumentenbestands (`ARCH-CONSOL-0001`) hat zehn Befunde ergeben. Drei davon erforderten eine Architekturentscheidung.

K-10 — Die Documentary Engine besaß keine eigene Spezifikation. Sie war über vier Dokumente verteilt beschrieben: 26 Nennungen in ESS-0001 (Vision), 42 in ESS-0001-CONTRACTS (Contracts), je eine Handvoll in ESS-0002 und ESS-0003. Als einzige Kernkomponente mit 19 Unterverzeichnissen besaß sie kein Spezifikationsdokument, während Supervisor (ESS-0002) und Platform Director (ESS-0003) je eines haben.

K-05 — ESS-0001 vermischt Vision und technische Spezifikation. Acht der neun Kapitel enthalten technische Inhalte (AST-Analyse, JSON-Modellnamen, Event-Modelle, Versionsmatrizen). Ohne Abgrenzung entsteht bei Anlage einer technischen Spezifikation eine konkurrierende Zuständigkeit.

K-02 — Nummernkonflikt. Die auslösende Aufgabenstellung wies die Documentary Engine der Nummer ESS-0004 zu. ESS-0004 ist jedoch über ESS-0001 (Related Enterprise Specifications) und die gemergte `ess-registry.json` verbindlich als Enterprise Version Manager reserviert. ESS-0001-CONTRACTS Chapter 16 legt fest: „Reservierte Nummern werden niemals abweichend belegt.“

Zusätzlich waren die Cross-References unvollständig (K-08): nur zwei von vier Dokumenten führten maschinell auswertbare Referenzen, und diese ohne die geforderte Untergliederung.

Entscheidung

1. Nummernvergabe aus dem freien Nummernraum

Nummer	Dokument
ESS-0010	Documentary Engine — technische Spezifikation
ESS-0011	Enterprise Traceability — ETM-Spezifikation
ESS-0011-CONTRACTS	ETM Contracts (Vertragsteil, belegt keine eigene Nummer)

Die Reservierung ESS-0004 = Enterprise Version Manager bleibt **unverändert bestehen**.

ESS-0010 war frei: Repository Governance war ursprünglich dafür vorgesehen, wurde jedoch in der Registry als `not_required` eingestuft, da ESS-0001-CONTRACTS Chapter 16 den Bereich vollständig und normativ abdeckt. Die Nummer wurde damit regulär weitervergeben. Der freie Nummernraum beginnt nun bei ESS-0012.

Diese Vergabe verletzt keine bestehende Regel und erfordert keine Aufhebung einer Reservierung.

2. ESS-0001 wird als Foundational Architecture Document gekennzeichnet

ESS-0001 erhält im Frontmatter `classification.type: Foundational Architecture Document` sowie im Dokumentkopf einen Abschnitt Dokumentklassifizierung mit folgender Vorrangregel:

Die Kapitel 2 bis 9 beschreiben den ursprünglichen technischen Architekturentwurf. Für die verbindliche technische Spezifikation gilt ausschließlich ESS-0010. Bei Abweichungen besitzt ESS-0010 Vorrang.

Kein Inhalt wurde entfernt. Die technischen Kapitel bleiben vollständig erhalten. Die Umwidmung ist ausschließlich deklaratorisch — sie ordnet die Autorität zu, ohne den Text zu verändern. Diese Systematik entspricht der bereits in Chapter 20 etablierten Normative Clarification.

3. Verbindliche Responsibility Matrix

`docs/architecture/ESS_RESPONSIBILITY_MATRIX.md` legt für alle sieben ESS-Dokumente fest: Dokumentklasse, Verantwortung, zulässige und unzulässige Inhalte, Vorrangregeln bei Konflikt sowie eine Prüfliste vor Neuanlage.

Sie definiert **keine** neuen Enterprise-Regeln, sondern ordnet ausschließlich bestehende Verantwortlichkeiten zu.

4. Einheitliche Cross-Reference-Konvention

Jedes ESS-Dokument führt fünf Felder: `dependsOn`, `relatedEss`, `relatedAdr`, `relatedComponents`, `relatedSkills`.

Dokumenttyp	Form
mit YAML-Frontmatter	Schlüssel <code>crossReference:</code>
ohne Frontmatter (*-CONTRACTS)	Markdown-Abschnitt Cross Reference

Die Zweiform ist bewusst: ESS-0001-CONTRACTS besitzt kein Frontmatter, dessen nachträgliche Einführung wäre eine Änderung statt einer Ergänzung. In ESS-0002 und ESS-0003 blieb der ursprüngliche flache `references:`-Block unverändert erhalten; `crossReference:` wurde additiv daneben ergänzt.

5. `docs/ess/` wird nicht eingeführt

Sämtliche ESS-Dokumente verbleiben unter `.ai/skills/`.

Chapter 2 weist `.ai/` ausdrücklich Skills und Contracts zu, Chapter 5 verortet KI-Artefakte dort, und fünf der sieben Dokumente tragen den Frontmatter-Schlüssel `skill:`. Ein neues Root-Unterverzeichnis wäre eine Strukturänderung mit ADR-Pflicht und erforderte eine Migration sämtlicher Dokumente samt Referenzanpassung — ohne erkennbaren Nutzen.

6. Kein neuer Contract für die Documentary Integration

Die geforderte automatische Prüfung auf doppelte Verantwortlichkeiten, widersprüchliche Regeln, konkurrierende Standards und fehlende Referenzen ist bereits vertraglich gedeckt:

Prüfung	Bestehender Contract
doppelte Verantwortlichkeiten	Chapter 3 — eine Verantwortung, genau ein Verzeichnis
widersprüchliche Regeln	Chapter 12, Chapter 20 — Rangfolge
konkurrierende Standards	Chapter 16 — ESS Registry Contract
fehlende Referenzen	Chapter 7 — Validation
Wissenskonflikte	Chapter 15 — <code>KnowledgeConflictDetectedEvent</code>

Erforderlich ist ausschließlich ein `DocumentResponsibilityValidator` als weiterer Validator in der Umsetzung von Chapter 12 — keine neue Regel.

Alternativen

A) ESS-0004 umwidmen, wie in der Aufgabenstellung formuliert. Verworfen. Erfordert die Aufhebung einer in ESS-0001 verankerten und bereits gemergten Reservierung, widerspricht Chapter 16 und der Vorgabe „ohne bestehende Enterprise-Regeln zu verändern“. Der Version Manager verlöre seine Nummer und benötigte eine neue — ein Folgekonflikt ohne Gegenwert.

B) Technische Kapitel aus ESS-0001 nach ESS-0010 verschieben. Verworfen. Verstößt gegen die Vorgabe „Vorhandene Inhalte dürfen nicht gelöscht oder überschrieben

werden." Die deklaratorische Vorrangregel erreicht dieselbe Eindeutigkeit ohne Substanzverlust und erhält das Dokument als historischen Nachweis.

C) Keine Konsolidierung, Verantwortlichkeiten im Einzelfall klären. Verworfen. Die Documentary Engine wäre weiterhin über vier Dokumente verteilt beschrieben. Jede künftige Erweiterung müsste erneut entscheiden, wo sie hingehört — genau die Nichtdeterminiertheit, die Chapter 1 ausschließt.

D) `docs/ess/` einführen und alle Dokumente verschieben. Verworfen. Strukturänderung mit ADR-Pflicht, Migration von sieben Dokumenten, Anpassung von mindestens elf referenzierenden Stellen — ohne Nutzen gegenüber der vertragskonformen Ablage unter `.ai/skills/`.

Konsequenzen

Positiv

- Die Documentary Engine besitzt erstmals eine eigene technische Spezifikation.
- Die Doppelzuständigkeit zwischen ESS-0001 und der technischen Ebene ist durch eine ausdrückliche Vorrangregel beseitigt, ohne Inhalt zu verlieren.
- Sämtliche sieben ESS-Dokumente führen vollständige, maschinell auswertbare Cross-References (zuvor: zwei von vier, unvollständig).
- Die Enterprise Traceability Matrix ist erstmals spezifiziert und besitzt eigene, klar begrenzte Contracts.
- Die Nummernvergabe bleibt lückenlos und regelkonform; keine Reservierung wurde angetastet.
- Künftige ESS-Anlagen haben eine verbindliche Prüfliste.

Negativ / Aufwand

- ESS-0011-CONTRACTS führt sechs zusätzliche Coverage-Schwellwerte und acht Orphan-Klassen ein, die maschinell zu prüfen sind. Ohne den `TraceabilityValidator` ist das eine Zusage, keine Durchsetzung.
- Zwei Cross-Reference-Formen (YAML und Markdown) erhöhen den Parseraufwand geringfügig. Der Alternativweg — nachträgliches Frontmatter in ESS-0001-CONTRACTS — wäre eine Änderung an einem bestehenden Dokument gewesen und wurde deshalb verworfen.
- ESS-0001 enthält weiterhin technische Inhalte, die formal nicht mehr seiner Klasse entsprechen. Das ist eine bewusst getragene Folge des Löschverbots und durch die Vorrangregel entschärft, aber nicht beseitigt.

Neutral

- Kein produktiver Code wurde berührt.
- Kein Dokument wurde gelöscht, verschoben oder inhaltlich überschrieben.

- Sämtliche Änderungen an bestehenden Dokumenten sind reine Ergänzungen im Frontmatter oder im Dokumentkopf.
-

Folgeentscheidungen

Nicht Gegenstand dieser Entscheidung:

1. `DocumentResponsibilityValidator` und `TraceabilityValidator` als ausführbare Validatoren (Umsetzungsstufe 4).
 2. Eröffnung von ESS-0004 (Enterprise Version Manager) bei nachgewiesenem Regelbedarf.
 3. Nachdokumentation von ADR-0001 bis ADR-0003 sowie Ergänzung von ADR-0009 in `adr_history.json` (GAP-026).
 4. Ergänzung von ESS-/ADR-Referenzen in den Bestandsdokumenten unter `docs/` (GAP-029).
-

Referenzen

- ESS-0001-CONTRACTS — Chapter 1, Chapter 2, Chapter 3, Chapter 5, Chapter 7, Chapter 12, Chapter 15, Chapter 16, Chapter 20
- ESS-0001 — Related Enterprise Specifications (Nummernreservierung)
- ADR-0010 — Enterprise Standard Extension
- ADR-0011 — Bestandsschutz Root-Abweichungen
- `docs/architecture/ESS_CONSOLIDATION_REPORT.md` — ARCH-CONSOL-0001, Befunde K-01 bis K-10
- `docs/architecture/ESS_RESPONSIBILITY_MATRIX.md` — ARCH-RESP-0001
- `.ai/registry/ess-registry.json`

ADR-0014: Documentation Governance Validator als Kernkomponente der Documentary Engine

Quelle: docs/adr/ADR-0014-documentation-governance-validator.md

ADR-0014: Documentation Governance Validator als Kernkomponente der Documentary Engine

Status

Accepted

Implementation-Status

IN PROGRESS — Spezifikation, Regelwerk, Verzeichnisstruktur, Metadaten und Skill sind vollständig. Die **Implementierung** ist nicht begonnen und setzt die Umsetzungsstufen 1 bis 4 aus docs/architecture/REPOSITORY_STRUCTURE_ANALYSIS.md voraus (Schemata, Core/Interfaces, Enterprise Event Bus, Validator-Basisklasse).

Bis dahin sind die 57 Regeln aus ESS-0012-CONTRACTS manuell anzuwenden und als manuell gekennzeichnet zu dokumentieren.

Datum

2026-07-31

Verantwortlich

Platform Director

Betroffene Artefakte

src/platform/Documentary/Governance/ (neu, 8 Unterverzeichnisse)

.ai/skills/ESS-0012-Documentation-Governance.md (neu)

.ai/skills/ESS-0012-Contracts.md (neu)

.ai/skills/Documentation-Governance-Validator.md (neu)

.ai/registry/ess-registry.json (fortgeschrieben)

docs/architecture/ESS_RESPONSIBILITY_MATRIX.md (erweitert)

Kontext

Die Plattform beschreibt sich seit ESS-0010 selbst (Documentary Engine) und verknüpft ihre Artefakte seit ESS-0011 bidirektional (Traceability Matrix). Sie **prüft sich jedoch nicht selbst**.

ESS-0001-CONTRACTS Chapter 12 definiert zwar 16 Pflichtvalidatoren, diese prüfen jedoch Struktur und Schema — nicht die semantische Widerspruchsfreiheit der Governance-Dokumente untereinander. Konkret ungeprüft blieben bislang:

- doppelte und widersprüchliche ESS-Regeln über Dokumentengrenzen hinweg
- widersprüchliche, veraltete oder undokumentierte ADRs
- doppelte, widersprüchliche und nicht referenzierte Contracts
- Breaking Changes ohne ADR
- aggregierte Governance- und Repository-Reife

Diese Lücke ist nicht theoretisch. Sämtliche bisherigen Befunde dieser Sitzung — GAP-006 (nicht registrierte Strukturabweichungen), GAP-013 bis GAP-015 (fehlende Metadaten), GAP-019 (vier widersprüchliche Versionsstände), K-02 (Nummernkonflikt), K-08 (fehlende Cross-References) — wurden **manuell** gefunden. Keiner davon wäre durch eine bestehende automatisierte Prüfung aufgefallen.

Zusätzlich wächst der Regelbestand: nach ADR-0010 (Chapter 11–20) und ADR-0013 (ESS-0010, ESS-0011) umfasst der Standard sieben ESS-Dokumente mit 20 Contract-Kapiteln. Ohne maschinelle Prüfung skaliert die manuelle Konsistenzsicherung nicht.

Entscheidung

1. Anlage der Komponente unter `Documentary/Governance/`

Der Governance Validator wird als Unterkomponente der Documentary Engine geführt:

```
src/platform/Documentary/Governance/  
  Contracts/  Validators/  Rules/  Services/  
  Reports/    Events/      Models/ Interfaces/
```

Abweichung von der Aufgabenformulierung. Die auslösende Aufgabe listete `Governance/`, `Contracts/`, `Validators/`, `Reports/`, `Rules/`, `Events/`, `Services/`, `Models/`, `Interfaces/` als Geschwister direkt unter `Documentary/`.

Fünf dieser Verzeichnisse — `Contracts/`, `Validators/`, `Events/`, `Models/`, `Interfaces/` — existieren dort bereits und sind in ESS-0001-CONTRACTS Chapter 2 mit eigener Verantwortung belegt. Eine erneute Anlage auf gleicher Ebene wäre entweder wirkungslos oder würde Chapter 3 verletzen (eine Verantwortung, genau ein Verzeichnis).

Die Unterordnung unter `Governance/` erhält die Absicht der Aufgabe vollständig, vermeidet die Kollision und lässt die bestehende Documentary-Struktur unverändert.

Neu angelegt wurden ausschließlich `Governance/` und dessen acht Unterverzeichnisse. Damit ist dies eine Strukturserweiterung nach Chapter 2 und Chapter 16 — der vorliegende ADR erfüllt die dortige ADR-Pflicht.

2. ESS-0012 und ESS-0012-CONTRACTS

Die Notwendigkeitsprüfung nach der Prüfliste aus `ARCH-RESP-0001` ergab, dass die semantische Governance-Prüfung von keinem bestehenden Dokument abgedeckt ist. Der Nachweis ist in ESS-0012, Abschnitt Notwendigkeitsnachweis, tabellarisch geführt.

Beide Dokumente sind scharf abgegrenzt. Sie enthalten **nicht**:

Inhalt	Verbleibt in
Validator-Basisvertrag, Severity, Quality Gates	Chapter 12
Coverage-Schwellwerte, Orphan-Klassen	ESS-0011-CONTRACTS
Twin-Zustände, Drift-Toleranz	Chapter 18
Knoten- und Beziehungsregeln	Chapter 15
Metadaten-Pflichtfelder	Chapter 7
Event-Namenskonvention	Chapter 8

ESS-0012-CONTRACTS definiert 57 Regeln in neun Bereichen mit stabilen IDs (`G0V-<AREA>-<NNN>`), Nachweispflicht und deterministischem Scoring.

3. ADR-Nummer 0013 statt 0011

Abweichung von der Aufgabenformulierung. Die Aufgabe nannte „ADR-0011 Documentation Governance Validator“. ADR-0011 ist jedoch bereits vergeben (Bestandsschutz und Zielstruktur für die Root-Abweichungen, gemergt über PR #2). ADR-0013 ist ebenfalls vergeben (ESS-Konsolidierung).

ESS-0001-CONTRACTS Chapter 16 legt fest: „Vergebene Nummern werden niemals umbenannt“ und „ADR-Nummern sind vierstellig und fortlaufend“. Die nächste freie Nummer ist **ADR-0014**.

4. Read-Only-Prinzip

Der Validator besitzt **keinen** Schreibzugriff auf Quellcode, Dokumentation, Metadaten, Registry, Knowledge Graph, Digital Twin, ESS- oder ADR-Dokumente. Er schreibt ausschließlich in seinen eigenen Befund- und Berichtsspeicher.

Begründung: Eine Prüfinstanz, die den Prüfgegenstand verändert, kann ihn nicht mehr unabhängig beurteilen. Dieselbe Trennung gilt bereits für den Supervisor (ESS-0002).

5. Rollentrennung

Instanz	Rolle
Governance Validator	stellt fest

Instanz	Rolle
Supervisor	bewertet und eskaliert
Platform Director	entscheidet und empfiehlt
Version Manager	leitet Versionsauswirkung ab

Der Validator blockiert selbst nicht, empfiehlt keine Version und erzeugt keine Handlungsempfehlung. Er liefert ausschließlich Befunde und Kennzahlen.

6. Verbindlichkeit für alle KI-Systeme

Claude Code, Google AI Studio, ChatGPT und künftige Systeme verwenden denselben Validator und dieselben Regeln. Eigene Prüfregeln, Deaktivierung von Regeln, Änderung von Schweregraden und Prüfergebnisse ohne Ausführung sind unzulässig.

Dies konkretisiert Chapter 10 und Chapter 17 für die Dokumentations-Governance, ohne neue AI-Governance-Regeln einzuführen.

Alternativen

A) Governance-Prüfungen in Chapter 12 ergänzen. Verworfen. Chapter 12 ist Teil des Master Enterprise Standard und regelt Validator-Grundlagen global. 57 dokumentspezifische Regeln dort aufzunehmen hätte den globalen Standard mit Detailregeln überfrachtet und der in ARCH-RESP-0001 festgelegten Verantwortungsabgrenzung widersprochen.

B) Prüfungen in ESS-0011 (Traceability) unterbringen. Verworfen. ESS-0011 verknüpft Artefakte; es bewertet ihre inhaltliche Qualität nicht. Die Vermischung hätte zwei Verantwortungen in einem Dokument erzeugt — genau das, was ADR-0013 gerade beseitigt hat.

C) Kein eigenes Dokument, Prüfregeln direkt im Skill. Verworfen. Ein Skill ist eine Handlungsanweisung ohne normative Wirkung. Regeln mit Blockadewirkung für Produktionsfreigaben benötigen Contract-Rang.

D) Sofortige Implementierung in TypeScript. Verworfen. Ohne Core, Interfaces, Models und Event Bus (Stufen 1–3) wäre der Validator ein isoliertes Skript ohne Event-Anbindung, ohne Registry-Eintrag und ohne Twin-Integration — und damit nach seinen eigenen Regeln GOV-REPO-009 und GOV-KG-001 selbst ein Governance-Verstoß.

Konsequenzen

Positiv

- Die semantische Governance-Prüfung ist erstmals spezifiziert und regelbasiert.
- 57 Regeln mit stabilen IDs, Nachweispflicht und deterministischem Scoring.
- Sämtliche bisher manuell gefundenen Befundklassen sind künftig maschinell erfassbar.

- Die Rollentrennung Validator/Supervisor/Director/Version Manager ist eindeutig.
- Alle KI-Systeme erhalten dasselbe Prüfregelwerk.
- Das neue Manifest erfüllt erstmals den vollständigen Metadata Contract aus Chapter 7 (`layer` , `lifecycle` , `health` , `quality` , `security` , `ai` , `repository`) und dient als Referenz für die Nachpflege der 22 Bestandsmanifeste (GAP-015).

Negativ / Aufwand

- 57 zusätzliche Regeln erhöhen den Implementierungsaufwand von Stufe 4 erheblich.
- Die Schwellwerte aus Chapter 3 (Governance Score ≥ 80 , Repository Health ≥ 80) würden beim aktuellen Repository-Zustand **nicht erreicht** — die Regeln beschreiben einen Zielzustand, der zunächst systematisch verletzt wird. Das ist beabsichtigt und macht den Rückstand messbar, erzeugt aber bei Aktivierung zunächst eine große Befundmenge.
- Bis zur Implementierung ist die Prüfung manuell und damit weder vollständig noch reproduzierbar.

Neutral

- Kein produktiver Code wurde verändert.
- Die bestehende Documentary-Struktur (19 Unterverzeichnisse) bleibt unangetastet.
- Kein bestehendes Dokument wurde inhaltlich überschrieben.

Folgeentscheidungen

Nicht Gegenstand dieser Entscheidung:

1. Implementierung des Validators (Umsetzungsstufen 1-4).
2. Zeitpunkt der Aktivierung der Schwellwerte aus ESS-0012-CONTRACTS Chapter 3 — eine Aktivierung vor Behebung der Bestandsbefunde würde jede Produktionsfreigabe blockieren.
3. Nachpflege der 22 Bestandsmanifeste auf den vollständigen Metadata Contract (GAP-015).
4. `component.yaml` und `CHANGELOG.md` je Komponente (GAP-013, GAP-014) — beides ist durch `GOV-REPO-002` und `GOV-REPO-004` künftig prüfbar.

Referenzen

- ESS-0012 — Documentation Governance
- ESS-0012-CONTRACTS — Documentation Governance Contracts, 57 Regeln
- `.ai/skills/Documentation-Governance-Validator.md` — SKILL-GOV-0001
- ESS-0010 — Documentary Engine
- ESS-0011 / ESS-0011-CONTRACTS — Enterprise Traceability

- ESS-0001-CONTRACTS — Chapter 2, 3, 7, 8, 9, 10, 12, 15, 16, 17, 18
- ADR-0010 — Enterprise Standard Extension
- ADR-0013 — ESS Documentation Responsibility Consolidation
- docs/architecture/ESS_RESPONSIBILITY_MATRIX.md — ARCH-RESP-0001
- docs/architecture/REPOSITORY_STRUCTURE_ANALYSIS.md — Umsetzungsstufen 1-4

ADR-0015: Einführung der Traceability-Komponente als Plattformmodul

Quelle: docs/adr/ADR-0015-enterprise-traceability-component.md

ADR-0015: Einführung der Traceability-Komponente als Plattformmodul

Status

Accepted

Implementation-Status

IN PROGRESS — Stufen 1 bis 4 sind inzwischen codebasiert umgesetzt und getestet. Das ursprüngliche ADR beschrieb die Komponente zum Entscheidungszeitpunkt noch als nicht implementiert; dieser Zustand ist überholt.

Verifiziert am 2026-08-02:

1. **Stufe 1-2 umgesetzt:** Discovery, Models, Builder, CoverageAnalyzer, OrphanDetector, Registry und Reporter verarbeiten reale Repository-Quellen und erzeugen die Enterprise Traceability Matrix sowie Coverage-/Orphan-Artefakte.
2. **Stufe 3 teilweise umgesetzt:** sechs reale Traceability-Events werden über die Enterprise Event Mesh veröffentlicht. Die im Manifest deklarierten Consume-Events werden noch nicht abonniert; `CoverageThresholdViolatedEvent` und `OrphanResolvedEvent` werden bewusst nicht synthetisch erzeugt, solange die dafür notwendigen Laufzeitbedingungen fehlen.
3. **Stufe 4 umgesetzt:** generische Validator-Basisklasse und `traceabilityMatrixValidator.ts` sind produktiv im Traceability-Lauf verdrahtet.
4. `npm run traceability:build` ist ausführbar und Bestandteil des Repository-Toolings.
5. TypeScript-/Vitest-/Predeploy-Prüfungen wurden in den zugehörigen Implementierungs-Commits erfolgreich ausgeführt.

Noch offen für COMPLETE:

- die vollständige event-getriebene Consume-Seite der Traceability-Komponente;
- die vorgesehenen Coverage-Threshold-Transitions mit realem Zustandsvergleich;
- `OrphanResolvedEvent` auf Basis zweier persistierter Läufe;
- der weitergehende Knowledge-Graph-/Digital-Twin-Ausbau, soweit er für die vollständige Zielarchitektur aus ESS-0011/ESS-0001-CONTRACTS benötigt wird;
- vollständige Reife-/Acceptance-Verifikation aller in ESS-0011 vorgesehenen Achsen.

Die Komponente bleibt deshalb korrekt im aktiven ADR-Ordner und wird **nicht** nach `resolved/` verschoben.

Datum

2026-07-31

Verantwortlich

Platform Director

Betroffene Artefakte

`src/platform/Traceability/`

`docs/traceability/`

`.ai/registry/ess-registry.json`

`docs/architecture/ESS_RESPONSIBILITY_MATRIX.md`

Kontext

ESS-0011 und ESS-0011-CONTRACTS spezifizieren die Enterprise Traceability Matrix seit ADR-0013 vollständig. Es fehlte zunächst der **Ort** ihrer Umsetzung: Die Spezifikation nennt `src/platform/Registry`, `Knowledge` und `Architecture` als verwandte Komponenten, ohne eine eigene Komponente zu definieren.

Ohne eigene Komponente müsste die Matrixlogik auf drei bestehende Module verteilt werden. Das verstößt gegen ESS-0001-CONTRACTS Chapter 3 (eine Verantwortung, genau ein Verzeichnis) und würde die Zuständigkeit für die Matrix dauerhaft unklar lassen.

Zusätzlich fordert Chapter 2 für jedes neue Plattformmodul einen ADR.

Entscheidung

1. Traceability wird eigenständiges Plattformmodul

```
src/platform/Traceability/  
  Contracts/  Core/      Discovery/  Events/  
  Interfaces/ Models/    Registry/   Reports/  
  Services/   Validators/ Versioning/
```

Die Struktur folgt der in der Aufgabenstellung geforderten Aufteilung. Jedes Unterverzeichnis besitzt genau eine Verantwortung gemäß Chapter 3.

2. Einordnung als Querschnittsmodul mit erweitertem Zugriff

Die ETM benötigt Zugriff auf Core, Shared, Registry, Knowledge, Discovery und Documentary — mehr als Chapter 16 für gewöhnliche Querschnittsmodule zulässt (nur Core und Shared).

Chapter 16 kennt für genau diesen Fall bereits eine Regelung: **Release** besitzt erweiterten Zugriff, „da es deren Ergebnisse zusammenführt“. Die ETM ist derselbe Fall.

Sie wird daher analog eingeordnet. **Die Layer-Hierarchie aus Chapter 6 bleibt unverändert**; es wird keine neue Ebene eingeführt.

Verboten bleiben Abhängigkeiten auf Supervisor und Platform Director — die Kommunikation nach oben erfolgt ausschließlich über Enterprise Events.

3. Vollständiger Metadatensatz als Referenzimplementierung

Die Komponente führte als erste im Repository einen vollständigen Metadatensatz nach dem damals gültigen Governance-Modell ein, einschließlich `manifest.json`, `component.yaml`, `CHANGELOG.md` und belegter README-Dokumentation.

Dieses Muster wurde später auf weitere Plattformkomponenten übertragen und durch zusätzliche Manifest-Integrity-Tests abgesichert.

4. `docs/traceability/` als operative Ebene — bewusst ohne Spezifikationsinhalt

Die Dokumente beschreiben **Betrieb, Anwendung und Nachvollzug**. Sie enthalten ausdrücklich **keine** normative Spezifikation und **keine** neuen Regeln.

Ebene	Ort
Spezifikation	ESS-0011
Regelwerk	ESS-0011-CONTRACTS
operative Doku	<code>docs/traceability/</code>
Implementierung	<code>src/platform/Traceability/</code>

Damit bleibt die Trennung zwischen normativer Spezifikation und operativer Dokumentation erhalten.

5. Kein neues ESS-Dokument

ESS-0011 und ESS-0011-CONTRACTS existieren bereits und decken den Regelbedarf vollständig ab. Eine erneute Anlage wäre ein Duplikat gewesen.

Implementierungsfortschritt nach der Entscheidung

Stufe 1-2 — umgesetzt

Die spätere Implementierung führte reale, repository-basierte Verarbeitung ein:

- Discovery der ESS-Registry, ADR-Historie, Komponenten-Manifeste und Tests;
- Models für Traceability-Entitäten und Beziehungen;
- Builder für die Matrix;
- CoverageAnalyzer;

- OrphanDetector;
- Registry;
- Reporter;
- Ausgabe nach `.ai/knowledge/traceability/` und `docs/traceability/`.

Die Implementierung erzeugt keine erfundenen Relationen, sondern leitet Links ausschließlich aus real vorhandenen Repository-Artefakten ab.

Stufe 3 — teilweise umgesetzt

`Services/runTraceability.ts` veröffentlicht reale Events über die Enterprise Event Mesh, darunter:

- `TraceabilityBuildStartedEvent`;
- `TraceabilityBuildCompletedEvent`;
- `TraceabilityBuildFailedEvent`;
- `CoverageCalculatedEvent`;
- `OrphanDetectedEvent`;
- `TraceabilityReportGeneratedEvent`.

Nicht künstlich veröffentlicht werden:

- `CoverageThresholdViolatedEvent`, solange kein vertraglich wirksamer Threshold-Zustandsübergang berechnet wird;
- `OrphanResolvedEvent`, solange kein belastbarer Vergleich zweier Läufe die Auflösung belegt.

Die im Manifest vorgesehenen Consume-Events sind weiterhin nicht vollständig abonniert. Deshalb ist Stufe 3 noch nicht als vollständige Event-Driven-Integration abgeschlossen.

Stufe 4 — umgesetzt

`Validators/baseValidator.ts` stellt eine generische Validator-Basisklasse bereit; `traceabilityMatrixValidator.ts` ist als konkrete Implementierung in den Traceability-Lauf integriert.

Stufe 5 / Zielarchitektur — offen

Der ursprünglich als harte Voraussetzung formulierte Knowledge-Graph-/Digital-Twin-Blocker war für die ersten vier Ausbaustufen zu streng: die real vorhandenen Registry-, ADR-, Manifest- und Testquellen reichen für eine belastbare erste ETM aus.

Für die **vollständige Zielarchitektur** bleiben Knowledge-Graph-/Digital-Twin-Beziehungen jedoch offen. Dieser Punkt wird nicht als bereits erfüllt dargestellt.

Alternativen

A) Matrixlogik auf Registry, Knowledge und Architecture verteilen.

Verworfen. Verstößt gegen Chapter 3 und würde die Zuständigkeit für die Matrix dauerhaft aufteilen.

B) Traceability als Unterkomponente von Documentary.

Verworfen. Die ETM verknüpft auch Artefakte außerhalb der Documentary Engine — ADRs, Exceptions, Tests, Releases und Komponentenmetadaten.

C) Neue Layer-Ebene für Traceability einführen.

Verworfen. Chapter 6 definiert die Hierarchie abschließend; Chapter 16 deckt den benötigten Querschnittszugriff bereits ab.

D) docs/traceability/ mit Spezifikationsinhalt füllen.

Verworfen. Das wäre ein Duplikat zu ESS-0011 / ESS-0011-CONTRACTS.

Konsequenzen

Positiv

- Die ETM besitzt einen eindeutigen Ort und eine eindeutige Zuständigkeit.
- Matrix-, Coverage- und Orphan-Artefakte werden real aus Repository-Quellen erzeugt.
- Traceability ist an die Enterprise Event Mesh angebunden, ohne nicht belegte Zustandsereignisse zu simulieren.
- Eine generische Validator-Basis ist vorhanden.
- Die Trennung Spezifikation / operative Doku / Implementierung bleibt erhalten.

Negativ / Aufwand

- Die vollständige Event-Consume-Seite ist noch offen.
- Knowledge Graph und Digital Twin sind für die endgültige Enterprise-Zielarchitektur noch nicht produktiv verfügbar.
- Coverage-/Orphan-Zustandsübergänge benötigen persistierte Vergleichsbasis, bevor die dazugehörigen Events ehrlich ausgelöst werden können.

Neutral

- Der aktualisierte ADR-Status ändert keine fachliche Scoring-, Billing-, Auth- oder Datenbanklogik.
 - Die ursprüngliche Architekturentscheidung bleibt bestehen; nur der tatsächliche Implementierungsfortschritt wird nachgezogen.
-

Folgeentscheidungen / verbleibende Arbeit

1. Event-Subscriptions für die in ESS-0011 vorgesehenen Consume-Events implementieren, sofern die jeweiligen realen Producer vorhanden sind.
 2. Persistierten Vergleich zwischen Traceability-Läufen etablieren, bevor `OrphanResolvedEvent` aktiviert wird.
 3. Coverage-Threshold-Policy versionieren und erst danach `CoverageThresholdViolatedEvent` produktiv aktivieren.
 4. Knowledge-Graph-/Digital-Twin-Achsen ergänzen, sobald deren reale Komponenten und Contracts implementiert sind.
 5. Danach vollständige ESS-0011-Acceptance durchführen und erst bei vollständigem PASS nach `docs/adr/resolved/` verschieben.
-

Implementierungs-Evidence

- Commit `964379f81607f4f7686b2891e5d8ffa0101b56ac` — Traceability Stufe 1-2.
 - Commit `665d46f42da4fbd61b0fe818b4026678f90f1c9a` — EventMesh-Anbindung und Validator-Basis.
 - `src/platform/Traceability/manifest.json` — aktueller Implementierungsstand und Event-Grenzen.
 - `tests/unit/traceabilityValidator.test.ts` sowie weitere Traceability-Tests.
 - `npm run traceability:build` — operativer Buildpfad.
-

Referenzen

- ESS-0011 — Enterprise Traceability
- ESS-0011-CONTRACTS — Link Contract, Coverage, Orphans
- ESS-0001-CONTRACTS — Chapter 2, 3, 6, 7, 9, 12, 16, 19
- ESS-0012-CONTRACTS — Governance-/Traceability-Regeln
- ADR-0013 — ESS Documentation Responsibility Consolidation
- ADR-0014 — Documentation Governance Validator
- ADR-0018 — Enterprise Event Mesh
- `docs/traceability/` — operative Dokumentation
- `docs/architecture/REPOSITORY_STRUCTURE_ANALYSIS.md` — Umsetzungsstufen

ADR-0016: Nachvergabe der reservierten ESS-Nummern 0004 bis 0009 als Komponentenspezifikationen

Quelle: docs/adr/ADR-0016-ess-component-specifications.md

ADR-0016: Nachvergabe der reservierten ESS-Nummern 0004 bis 0009 als Komponentenspezifikationen

Status

Accepted

Implementation-Status

COMPLETE (verifiziert 2026-07-31) — Sechs Dokumente angelegt, Registry vollständig, kein reservierter Nummernplatz mehr ohne Dokument. Die beschriebenen Komponenten sind weiterhin nicht implementiert; das ist Gegenstand der Umsetzungsstufen und nicht dieser Entscheidung.

Datum

2026-07-31

Verantwortlich

Platform Director

Betroffene Artefakte

- `.ai/skills/ESS-0004-Enterprise-Version-Manager.md` (neu)
 - `.ai/skills/ESS-0005-Quality-Center.md` (neu)
 - `.ai/skills/ESS-0006-Security-Compliance.md` (neu)
 - `.ai/skills/ESS-0007-Enterprise-Release-Center.md` (neu)
 - `.ai/skills/ESS-0008-AI-Agent-Framework.md` (neu)
 - `.ai/skills/ESS-0009-Enterprise-Knowledge-Platform.md` (neu)
 - `.ai/registry/ess-registry.json` (fortgeschrieben)
 - `docs/architecture/ESS_RESPONSIBILITY_MATRIX.md` (erweitert)
-

Kontext

ESS-0001 reserviert im Abschnitt Related Enterprise Specifications die Nummern ESS-0002 bis ESS-0009 mit festen Titeln. ESS-0002 und ESS-0003 wurden mit ADR-0013 angelegt.

Für **ESS-0004 bis ESS-0009** wurde damals entschieden, **kein** Dokument anzulegen. Die Begründung lautete: Die jeweiligen Regelbereiche seien durch ESS-0001-CONTRACTS Chapter 9, 11, 12, 15 und 19 vollständig abgedeckt, ein eigenes Dokument wäre ein Duplikat und verstieße gegen Zero Duplication.

Diese Einschätzung wurde vom Platform Director überprüft und **verworfen**. Der Befund lautete: Sechs reservierte Nummern ohne Dokument sind eine Lücke, keine Duplikatsvermeidung.

Die Prüfung bestätigt das:

Beobachtung	Bewertung
ESS-0002 und ESS-0003 sind Komponentenspezifikationen, keine Regelwerke	dasselbe Muster ist für 0004-0009 anwendbar
Chapter 9, 11, 12, 15 regeln Verträge , nicht Komponenten	die Komponente bleibt unspezifiziert
<code>src/agents</code> mit acht Agenten besitzt keinerlei Enterprise-Vertrag	echte Regelungslücke, kein Duplikat
Registry führte sechs Einträge mit <code>document: null</code>	für jede Auswertung ein Fehlzustand

Die ursprüngliche Begründung verwechselte **Regelbereich** mit **Komponentenbeschreibung**. Chapter 9 legt fest, wie versioniert wird; es beschreibt nicht, woraus der Version Manager besteht.

Entscheidung

1. Sechs Komponentenspezifikationen unter den reservierten Nummern

Nummer	Titel (unverändert aus ESS-0001)	Spezifizierte Komponente
ESS-0004	Enterprise Version Manager	<code>src/platform/VersionManager</code>
ESS-0005	Quality Center	<code>src/platform/Quality</code>
ESS-0006	Security & Compliance	<code>src/platform/Security</code> , <code>src/platform/Compliance</code>
ESS-0007	Enterprise Release Center	<code>src/platform/Release</code>
ESS-0008	AI Agent Framework	<code>src/agents</code> , <code>src/orchestrator</code>
ESS-0009	Enterprise Knowledge Platform	<code>src/platform/Knowledge</code>

Die Titel entsprechen exakt der Reservierung aus ESS-0001. Keine Umwidmung, keine Ummummerierung.

2. Strikte Abgrenzung gegen die globalen Contracts

Jedes der sechs Dokumente führt im Frontmatter eine `classification.note`, die den Geltungsbereich begrenzt, und einen Abschnitt Abgrenzung mit Verweistabelle.

Dokument	beschreibt	beschreibt nicht
ESS-0004	Komponentenaufbau, Interfaces, Events	Versionsstrategie (Chapter 9)
ESS-0005	Validator-Ausführung, Gate-Runner	Validator-Vertrag, Severity (Chapter 12)
ESS-0006	Security- und Compliance-Komponenten	Sicherheitsregeln (Chapter 11)
ESS-0007	Release-Ausführung, Snapshots	Release-Contracts (Chapter 9, 19)
ESS-0008	Agent-Rahmenwerk	Scoring-Algorithmen, KI-Orchestrierung (Chapter 10, 17)
ESS-0009	Knowledge-Komponente	Wissensmodell (Chapter 4, Chapter 15)

Keine globale Regel wurde wiederholt. Sämtliche Regelbereiche werden referenziert.

3. ESS-0006 führt zwei Komponenten in einem Dokument

Security Center und Compliance Center werden gemeinsam spezifiziert.

Begründung: Beide teilen eine durchgehende Nachweiskette

(`Classification` → Kontrolle → Audit → Evidence → Nachweis). Eine Trennung hätte diese Kette an der Dokumentgrenze zerschnitten. Die Reservierung in ESS-0001 führt beide Bereiche ebenfalls gemeinsam. Die Komponenten bleiben im Repository getrennt.

4. ESS-0008 schließt eine echte Lücke

Chapter 10 und Chapter 17 regeln die **KI-Entwicklungssysteme** — Claude Code, Google AI Studio, ChatGPT. Sie regeln **nicht** die acht produktiven Domänen-Agenten unter `src/agents`.

ESS-0008 führt für diese verbindlich ein: Agent Contract, stabile Agent-Identität, **Determinismus-Deklaration**, Data Integrity Contract, Orchestrator Contract und Registrierungspflicht.

Besonders relevant: Ein nicht deterministischer, LLM-gestützter Agent darf niemals alleinige Quelle einer sicherheits-, abrechnungs- oder compliancerelevanten Entscheidung sein.

ESS-0008 verändert keine Scoring-Algorithmen und keine Bewertungslogik.

5. Aufgabe 2 der Anforderung war bereits erfüllt

Die auslösende Anforderung nannte erneut die Erstellung von Chapter 11 bis 20 in ESS-0001-CONTRACTS. Die Prüfung ergab: **Chapter 1 bis 20 sind lückenlos vorhanden** (ADR-0010, gemergt via PR #1). Es wurde nichts dupliziert.

Alternativen

A) Bei der ursprünglichen Entscheidung bleiben (keine Dokumente). Verworfen. Der Platform Director hat die Einschätzung ausdrücklich überprüft und verworfen. Sechs reservierte Nummern ohne Dokument sind für jede maschinelle Auswertung ein Fehlzustand, und `src/agents` bliebe dauerhaft ohne Vertrag.

B) Titel nach der zuletzt genannten Themenliste vergeben (etwa ESS-0004 = Documentary Engine, ESS-0005 = Knowledge Graph). Verworfen. Das widerspricht der bindenden Reservierung aus ESS-0001 und Chapter 16. Documentary Engine ist bereits ESS-0010, Knowledge Graph ist Chapter 15 und ESS-0009. Eine Neuvergabe hätte drei bestehende Dokumente entwertet.

C) Die Inhalte in ESS-0001-CONTRACTS als weitere Kapitel ergänzen. Verworfen. Komponentenbeschreibungen gehören nicht in den Master Enterprise Standard. Das widerspricht der Responsibility Matrix, die für ESS-0001-CONTRACTS ausdrücklich komponentenspezifische Spezifikationen als unzulässigen Inhalt führt.

D) Ein Sammeldokument für alle sechs Bereiche. Verworfen. Sechs Komponenten mit unterschiedlichen Ownern und Lebenszyklen in einem Dokument hätten dieselbe Vermischung erzeugt, die ADR-0013 gerade beseitigt hat.

Konsequenzen

Positiv

- Kein reservierter Nummernplatz ohne Dokument; die Registry ist erstmals vollständig (15 von 15 Einträgen mit Dokument).
- `src/agents` besitzt erstmals einen Enterprise-Vertrag, einschließlich Determinismus-Deklaration und Data Integrity Contract.
- Jede Kernkomponente der Wertschöpfungskette besitzt nun eine eigene Spezifikation.
- Die Abgrenzung zwischen Vertrag (ESS-0001-CONTRACTS) und Komponente (ESS-000x) ist an sechs weiteren Beispielen durchgezogen.

Negativ / Aufwand

- Sechs weitere Dokumente erhöhen die Pflegelast. Jede Änderung an Chapter 9, 11, 12, 15 oder 19 erfordert künftig eine Prüfung, ob die zugehörige Komponentenspezifikation nachzuziehen ist.

- Die Zahl spezifizierter, aber nicht implementierter Komponenten steigt weiter. Der Enterprise Score wird dadurch nicht steigen — die Definitionsdimension war bereits hoch.
- Das Risiko schleichender Duplikation wächst. Es wird ausschließlich durch die Abgrenzungsabschnitte und die Regel `GOV-ESS-004` beherrscht.

Neutral

- Kein produktiver Code verändert.
 - Kein bestehendes ESS- oder ADR-Dokument inhaltlich verändert.
 - Keine Reservierung umgewidmet.
-

Folgeentscheidungen

1. Umsetzung der spezifizierten Komponenten (Stufen 1 bis 5).
 2. Registrierung der acht Agenten gemäß ESS-0008 Chapter 4.
 3. Nachpflege der Determinismus-Deklaration je Agent.
 4. `DocumentResponsibilityValidator` muss die Abgrenzung der sechs neuen Dokumente prüfen.
-

Referenzen

- ESS-0001 — Related Enterprise Specifications (Nummernreservierung)
- ESS-0001-CONTRACTS — Chapter 4, 6, 7, 8, 9, 10, 11, 12, 14, 15, 16, 17, 19
- ADR-0010 — Enterprise Standard Extension (Chapter 11–20)
- ADR-0013 — ESS Documentation Responsibility Consolidation
- ADR-0015 — Enterprise Traceability Component
- `docs/architecture/ESS_RESPONSIBILITY_MATRIX.md` — ARCH-RESP-0001
- `.ai/registry/ess-registry.json`

ADR-0017: Produktions-Audit-Nachbesserungen — Quota-Durchsetzung, Supabase-Härtung, Stripe-Jahresrabatt, Compliance-Backend

Quelle: docs/adr/ADR-0017-audit-nachbesserungen-quota-supabase-stripe-compliance.md

ADR-0017: Produktions-Audit-Nachbesserungen — Quota-Durchsetzung, Supabase-Härtung, Stripe-Jahresrabatt, Compliance-Backend

Status

Accepted

Implementation-Status

COMPLETE (verifiziert 2026-07-31) — Alle vier Entscheidungen sind umgesetzt und gegen die produktive Supabase-Instanz sowie `tsc --noEmit` / `vite build` verifiziert.

Datum

2026-07-31

Verantwortlich

Platform Director

Betroffene Artefakte

`server/quota.ts` (neu), `server.ts` (Quota-Einbindung, Router-Mount), `supabase/migrations/20260730000000_user_quota.sql` (angepasst), `supabase/migrations/20260731000000_add_missing_service_role_rls_policies.sql` (neu), `supabase/migrations/20260731000100_harden_handle_new_user_search_path.sql` (neu), `supabase/migrations/20260731000200_compliance_runs.sql` (neu), `server/stripe.ts` (Import-Fix), `server/.env.example` (Jahres-Price-ID-Dokumentation), `src/components/Abonnements.tsx` (Hinweistext), `server/compliance/types.ts` (neu), `server/compliance/scanners.ts` (neu), `server/compliance/store.ts` (neu), `server/compliance/router.ts` (neu), `src/components/SecurityComplianceAuditor.tsx` (Import-Fix, `authFetch`), `docs/adr/ADR-0012-SecurityComplianceAuditor.md` (Implementation-Status aktualisiert)

Kontext

`docs/architecture/ENTERPRISE_PRODUCTION_AUDIT.md` (ARCH-AUDIT-0001) benannte vier offene Entscheidungen, die ausdrücklich dem Platform Director vorbehalten waren. Der Platform Director hat alle vier am 2026-07-31 wie folgt entschieden:

1. `user_quota` — per Best-Practice-Abwägung entscheiden.
2. Supabase — Realzustand erneut prüfen und Fehler nur beheben, sofern die Prozessketten Useranmeldung und Abonnement-Synchronisation dadurch nicht beeinträchtigt werden.
3. Der 10%-Jahresrabatt wird über einen Stripe-Produktrabatt gelöst, nicht clientseitig berechnet.
4. Die `/api/compliance/*` Endpunkte aus ADR-0012 werden gemäß der dort beschriebenen Architektur und Prozesskette angebunden.

Entscheidung

1. `user_quota` — Best-Practice-Abwägung

Befund vor der Entscheidung: Die Migration `20260730000000_user_quota.sql` existierte im Repository, war aber **nicht gegen die produktive Datenbank ausgeführt** worden (nicht in `list_migrations` enthalten). Ihr Kommentar verwies auf `src/lib/freeTierLimits.ts` — eine Datei, die **nicht existiert hat**. Die tatsächliche Durchsetzung des Screening-Limits lief ausschließlich über `src/lib/dailyScreeningTracker.ts`, einen reinen `localStorage`-Zähler ohne jede Server-Beteiligung — trivial umgehbar (Audit-Befund S-04) und zugleich die einzige Kontrolle für einen bezahlten Tarif-Unterschied (Starter 5 Screenings/Tag vs. PRO unbegrenzt).

Abwägung: Löschen der Migration hätte den Fehlzustand nur beseitigt, nicht behoben — das zugrunde liegende Problem (Tarifgrenze ohne Serverkontrolle) hätte fortbestanden. Das Tabellendesign selbst ist korrekt (Service-Role-only RLS, saniert). Best Practice für eine monetarisierte Nutzungsgrenze ist serverseitige Durchsetzung.

Entscheidung: Migration aktivieren und mit echter Serverlogik verbinden.

- `server/quota.ts` implementiert `enforceScreeningQuota()`: eingeloggte Nutzer werden per E-Mail in `public.user_quota` gezählt (persistent, geräteübergreifend); Aufrufe ohne verifizierte Session (Free/Gast) fallen auf den bestehenden In-Memory-IP-Limiter aus `server/iam/rateLimiter.ts` zurück, da für sie keine stabile Identität existiert.
- Eingebunden in `GET / POST /api/crypto-scoring/:symbol` in `server.ts` — der bislang völlig ungeschützte Endpunkt, den der client-seitige Zähler zu schützen vorgab.
- Der client-seitige Tracker bleibt unverändert bestehen (sofortiges UI-Feedback ohne Server-Roundtrip); die Serverseite ist jetzt die tatsächliche Durchsetzung, nicht mehr nur die Anzeige.

- Datenbankfehler in diesem Pfad führen bewusst zu `fail-open` (zahlende Kunden dürfen nicht durch einen DB-Ausfall gesperrt werden) — anders als bei `checkAdminAccess()`, das sicherheitskritisch `fail-closed` bleibt.

2. Supabase — Realzustand geprüft, korrigiert wo gefahrlos möglich

Live-Prüfung (2026-07-31): Projekt `ryzywoktpmyhwzxmsty` steht auf `ACTIVE_HEALTHY`, alle 17 vorher dokumentierten Migrationen sind angewendet. Der im Produktionsaudit dokumentierte Zustand `MIGRATIONS_FAILED` betraf einen inzwischen bereinigten Branch-Zustand, nicht das aktuelle Projekt.

Real gefundene und behobene Befunde (Supabase Security Advisor):

Befund	Fix	Risiko für Login/Abo-Sync
6 Tabellen (<code>audit_logs_iam</code> , <code>iam_access_log</code> , <code>step_up_to_kens</code> , <code>break_glass_codes</code> , <code>usage_log</code> , <code>users</code>) mit RLS aktiv, aber ohne Policy	Service-Role-only Policy ergänzt, identisches Muster wie die bestehende <code>user_quota</code> -Policy	Keins — alle sechs Tabellen sind bereits ausschließlich über den Service-Role-Key adressiert
<code>handle_new_user()</code> (Signup-Trigger) ohne festen <code>search_path</code>	<code>SET search_path = public,</code> <code>pg_temp</code> ergänzt, Funktionslogik byte-identisch belassen	Geprüft: additive Härtung, kein Verhaltensunterschied. Läuft bei jeder Neuregistrierung
<code>user_quota</code> -Migration nie angewendet	Angewendet (siehe Punkt 1)	Keins — neue, isolierte Tabelle

Bewusst nicht verändert:

- `stripe.set_updated_at`, `stripe.set_updated_at_metadata`, `stripe.check_rate_limit` (Schema `stripe`) — verwaltet durch die Supabase-Stripe-Sync-Engine, nicht durch dieses Repository. Eingriffe hier würden bei der nächsten Sync-Engine-Aktualisierung überschrieben und riskieren die Abonnement-Synchronisation, die die Vorgabe ausdrücklich schützen sollte.
- `auth_leaked_password_protection` — eine Supabase-Auth-Dashboard-Einstellung, nicht per Migration erreichbar. Verbleibt als offener Punkt für den Platform Director (Supabase Dashboard → Authentication → Policies).

3. Stripe-Jahresrabatt — keine serverseitige Berechnung

Erster Ansatz (verworfen, siehe Alternativen): ein serverseitig automatisch angewendeter Stripe-Coupon (`STRIPE_COUPON_ID_YEARLY`). Der Platform Director hat diesen Ansatz korrigiert: Jahresabonnements benötigen **keine serverseitige Berechnung und somit auch keine Variable und keine Backend-Logik**.

Begründung: `server/stripe.ts` wählt bei `billingPeriod === 'yearly'` bereits seit vorheriger Implementierung eine **eigene** Stripe-Price-ID (`STRIPE_PRICE_ID_STARTER_YEARLY`, `STRIPE_PRICE_ID_PRO_YEARLY`) statt der Monatspreis-ID. Der Jahresrabatt gehört damit strukturell in den Betrag dieses separaten, im Stripe Dashboard gepflegten Price-Objekts — nicht in

eine zusätzliche Coupon-Berechnung zur Laufzeit. Ein Coupon obendrauf hätte den Rabatt dupliziert oder mit dem bereits im Jahrespreis enthaltenen Rabatt kollidieren können.

Entscheidung: Die zunächst eingeführte Coupon-Logik (`effectiveCouponId` , `STRIPE_COUPON_ID_YEARLY` , `stripe.coupons.retrieve()`) wurde vollständig entfernt. `server/stripe.ts` verwendet für `couponId` wieder ausschließlich den vom Client übergebenen Wert — reserviert für vom Kunden eingelöste Promotion-Codes (bestehendes `validate-coupon`-Feature), nicht für den automatischen Jahresrabatt. `server/.env.example` führt `STRIPE_COUPON_ID_YEARLY` nicht mehr. `Abonnements.tsx` zeigt weiterhin eine Vorabschätzung (0.9-Multiplikator auf den Monatspreis) ohne Verweis auf einen serverseitigen Coupon-Mechanismus, da keiner mehr existiert.

4. ADR-0012 Compliance-Endpunkte — Backend implementiert

ADR-0012 dokumentierte den Befund: sieben Frontend-Aufrufe unter `/api/compliance/*`, kein einziger Endpunkt implementiert, Import aus einem nicht existierenden Verzeichnis (`server/compliance/types`). Umgesetzt gemäß der im ADR beschriebenen Architektur (`Admin Portal` → `SecurityComplianceAuditor` → `IAM System / Audit Logging` → `Supabase`):

- `server/compliance/types.ts` — die zuvor fehlende Typdatei, exakt passend zu den bereits im Frontend verwendeten Interfaces.
- `server/compliance/scanners.ts` — 21 Scanner-Module (passend zur im Frontend fest verdrahteten Beschriftung "Scanner-Module (21)", gruppiert in Security (7), Data & Privacy (5), Billing (3), Code Quality (3), Governance (3). Jedes Modul liest den tatsächlichen Repository-/Migrations-/Registry-Zustand zur Laufzeit (Dateisystem-Greps, Migrationsdateien, `ess-registry.json`, ADR-Verzeichnis) — keine festen Demo-Werte.
- **Abweichung zu ADR-0012 korrigiert:** Das ADR nannte `audit_logs_iam/iam_access_log` als Datenquelle für den Auditor. Diese Tabellen sind IAM-Ereignis-Logs (Actor/Target/Action) und für Scan-Ergebnisse strukturell ungeeignet — dieselbe Feststellung, die das ADR selbst bereits als offenen Punkt führte. Stattdessen: eigene Tabellen `compliance_runs` und `compliance_certificates` (Migration `20260731000200_compliance_runs.sql`), Service-Role-only RLS.
- `server/compliance/router.ts` — alle sechs Endpunkte (`dashboard`, `risk`, `certificates`, `run`, `certify`, `report`), jeder über `checkAdminAccess(req, 'compliance:*'`, `ADMIN_ZONE_ROLES`) geschützt — dasselbe Muster wie der bestehende `registry:assets:update`-Zonenschutz in `server.ts`. Der Auditor bleibt damit, wie in ADR-0012 gefordert, auf Leserechte/Analyse/Reporting beschränkt: `run` und `certify` erzeugen ausschließlich Scan-Ergebnisse bzw. Zertifikate, verändern keine IAM-Regeln und vergeben keine Berechtigungen.
- `src/components/SecurityComplianceAuditor.tsx`: kaputter Import korrigiert; alle sieben `fetch()`-Aufrufe auf `authFetch()` umgestellt (bestehende Utility aus `src/lib/authFetch.ts`), da die neuen Endpunkte einen Bearer-Token erfordern — ohne diese Änderung hätte die Admin-Gate-Prüfung jeden Aufruf mit 403 abgelehnt.
- **Verifiziert:** `npx tsc --noEmit` sank von 38 auf 9 Fehler (die verbleibenden 9 sind vorbestehend und nicht Gegenstand dieser Entscheidung — `stepUp.ts`, `systemEvents.ts`, Das-

hboard.tsx). `npx vite build` erfolgreich. Ein bislang unbemerkter, unabhängiger Bug in `server/stripe.ts` (`getServerSupabase` verwendet, aber nie importiert — hätte bei Gast-Checkout-Abschluss eine `ReferenceError` geworfen) wurde beim Bearbeiten derselben Datei mitkorrigiert.

Alternativen

Zu 1) `user_quota` -Migration ersatzlos löschen. Verworfen. Hätte den Fehlzustand entfernt, aber die eigentliche Schwachstelle (S-04) unbehoben gelassen und wäre der einfachere, nicht der bessere Weg gewesen.

Zu 2) Vollständige RLS-Policy-Überarbeitung inkl. differenzierter Nutzerrechte. Verworfen für diesen Schritt. Die sechs betroffenen Tabellen sind ausschließlich server-seitig adressiert; eine Service-Role-only-Policy bildet den tatsächlichen Zugriff exakt ab, ohne die Prozesskette anzufassen. Feingranularere Policies wären eine eigene, hier nicht angeforderte Entscheidung.

Zu 3a) Rabatt weiterhin ausschließlich clientseitig berechnen (ursprünglicher Zustand). Verworfen. Widerspricht der Vorgabe "über Stripe Rabatt in den Produkten gelöst".

Zu 3b) Serverseitiger Stripe-Coupon, automatisch angewendet (erster Umsetzungsversuch). Verworfen und wieder entfernt. Der Rabatt steckt bereits strukturell im Betrag der separaten Jahres-Price-ID; ein zusätzlicher Coupon wäre redundante, potenziell kollidierende Backend-Logik für etwas, das Stripe über die Price-ID-Auswahl bereits abbildet. Explizite Platform-Director-Korrektur: "Alle Jahresabonnements benötigen keine serverseitige Berechnung und somit auch keine Variable und keine Backend-Logik."

Zu 4) `audit_logs_iam` / `iam_access_log` wie im ADR-Text beschrieben zweckentfremden. Verworfen. Das ADR selbst dokumentierte diese Abweichung bereits als offenen Punkt; eigene, korrekt geformte Tabellen lösen den Konflikt, statt ihn fortzuschreiben.

Konsequenzen

Positiv

- S-04 (Quota clientseitig umgehbar) ist geschlossen: Screening-Limit wird jetzt serverseitig durchgesetzt.
- 6 RLS-Advisory-Befunde und 1 Function-Search-Path-Befund gegen die produktive Datenbank behoben, ohne Login- oder Abo-Sync-Verhalten zu verändern.
- Jahresrabatt ist strukturell an die Stripe-Price-ID gebunden, ohne zusätzliche Backend-Logik, Variable oder Fehlerquelle im Server.
- C-01 (kaputter Import, 19 von 38 TypeScript-Fehlern) ist behoben; FND-ADR-0012-01 (fehlende Compliance-Endpunkte) ist geschlossen.

Negativ / Aufwand

- Der Platform Director muss sicherstellen, dass die Beträge der `STRIPE_PRICE_ID_STARTER_YEARLY` / `STRIPE_PRICE_ID_PRO_YEARLY` Price-Objekte im Stripe Dashboard den beworbenen 10%-Rabatt tatsächlich enthalten — das Frontend zeigt weiterhin nur eine Schätzung, keine live abgefragte Zahl.
- Die 21 Compliance-Scanner sind statische Repository-Analysen, kein Ersatz für einen echten SAST/Dependency-Scanner. Sie sind bewusst leichtgewichtig, um ohne externe Abhängigkeiten innerhalb eines Admin-Panel-Requests zu laufen.
- `auth_leaked_password_protection` bleibt als offener, nur per Dashboard lösbarer Punkt bestehen.

Neutral

- Keine bestehende ESS- oder ADR-Entscheidung inhaltlich verändert; ADR-0012 wurde ausschließlich im Implementation-Status-Abschnitt fortgeschrieben.

Folgeentscheidungen

1. Platform Director verifiziert, dass die Beträge der Jahres-Price-IDs im Stripe Dashboard den beworbenen 10%-Rabatt korrekt enthalten.
2. Platform Director aktiviert "Leaked Password Protection" im Supabase-Auth-Dashboard.
3. Prüfung, ob die 21 statischen Compliance-Scanner mittelfristig um einen echten Dependency-/SAST-Scan ergänzt werden sollen.
4. Behebung der verbleibenden 9 vorbestehenden `tsc`-Fehler (`stepUp.ts` , `systemEvents.ts` , `Dashboard.tsx`) — nicht Gegenstand dieser Entscheidung.

Referenzen

- `docs/architecture/ENTERPRISE_PRODUCTION_AUDIT.md` (ARCH-AUDIT-0001) — Befunde S-04, S-01, C-01
- ADR-0012 — SecurityComplianceAuditor Integration
- ADR-0003.5 — Owner-IAM (Rate-Limiting-, Fail-Closed- und Audit-Log-Muster wiederverwendet)
- ESS-0001-CONTRACTS Chapter 11, 12, 14
- `server/quota.ts` , `server/compliance/*` , `supabase/migrations/20260731*`

ADR-0019: Fork-Divergenz zwischen Entwicklungs- und Produktivstand, Konsolidierung des ADR- und ESS-Nummernraums

Quelle: docs/adr/ADR-0019-fork-divergenz-und-nummernraum-konsolidierung.md

ADR-0019: Fork-Divergenz zwischen Entwicklungs- und Produktivstand, Konsolidierung des ADR- und ESS-Nummernraums

Status

Angenommen — 2026-07-31

Kontext

Im Rahmen des Enterprise-FinTech-Architektur-Audits (docs/architecture/ENTERPRISE_FINTECH_ARCHITECTURE_AUDIT.md , Document ID ARCH-AUDIT-0002) wurde der Entwicklungsstand CAPITAL-AI-Dev-main mit dem Produktivstand dieses Repositorys (Commit 3cb1771) vollständig datei- und dokumentenbasiert verglichen.

Der Vergleich hat ergeben, dass es sich **nicht um zwei Stände derselben Codebasis handelt, sondern um zwei divergierte Forks:**

Kennzahl	Wert
Dateien nur im Entwicklungsstand	130
Dateien nur im Produktivstand	155
Gemeinsame Dateien, inhaltlich verschieden	55
Gemeinsame Dateien, identisch	119

Die Divergenz ist nicht auf Anwendungscode beschränkt, sondern betrifft den Governance-Nummernraum selbst. Beide Stände haben unabhängig voneinander ADR- und ESS-Nummern vergeben und belegen dieselben Nummern mit **unterschiedlichen Entscheidungen**.

Kollisionen im ADR-Nummernraum

ADR	Produktivstand (verbindlich)	Entwicklungsstand (abweichend)
ADR-0010	Erweiterung des Enterprise Standards um ESS-0001-CONTRACTS	Enterprise Traceability Matrix
ADR-0011		

ADR	Produktivstand (verbindlich)	Entwicklungsstand (abweichend)
	Bestandsschutz und Zielstruktur für Root-Abweichungen	Documentation Governance Validator
ADR-0012	SecurityComplianceAuditor Integration	Enterprise Architecture Compliance Engine
ADR-0013	Konsolidierung der ESS-Dokumentationsverantwortung	Enterprise Event Mesh

Zusätzlich tragen drei sachlich identische Entscheidungen in beiden Ständen verschiedene Nummern: Enterprise Event Mesh (Produktiv ADR-0018 / Entwicklung ADR-0013), Documentation Governance Validator (ADR-0014 / ADR-0011), Enterprise Traceability (ADR-0015 / ADR-0010).

Kollisionen im ESS-Nummernraum

Zehn ESS-Nummern bezeichnen in beiden Ständen unterschiedliche Komponenten:

ESS	Produktivstand (verbindlich)	Entwicklungsstand (abweichend)
ESS-0002	Supervisor Architect	Platform Governance Standard
ESS-0003	Platform Director	AI Collaboration & Agent Value Chain
ESS-0004	Enterprise Version Manager	Documentary Engine & Digital Twin
ESS-0005	Quality Center	Knowledge Graph Architecture
ESS-0006	Security & Compliance	Version Manager & Lifecycle
ESS-0007	Enterprise Release Center	Master Supervisor
ESS-0008	AI Agent Framework	Platform Director Architecture
ESS-0009	Enterprise Knowledge Platform	Security & IAM Architecture
ESS-0010	Documentary Engine	Repository Topology & Maintenance
ESS-0013	Enterprise Event Mesh	Enterprise Architecture Compliance Engine

ESS-0014 ist im Produktivstand über `.ai/registry/ess-registry.json` (`freeNumberSpaceStartAt: "ESS-0014"`) ausdrücklich als **frei** deklariert, im Entwicklungsstand jedoch mit „Enterprise Event Mesh Specification“ belegt.

Die ESS-Registry des Produktivstands enthält die Regel, dass reservierte Nummern „niemals abweichend belegt werden“ dürfen und „kein KI-System ESS-Nummern eigenständig vergibt“. Der Entwicklungsstand besitzt keine Registry (`.ai/registry/` existiert dort nicht) und kennt das Registry-Verfahren dokumentarisch nicht.

Technisches Merge-Hindernis

Beide Stände enthalten eine vollständige, aber unvereinbare Implementierung der Enterprise Event Mesh unter demselben Verzeichnispfad `src/platform/EventMesh/`. Der Produktivstand verwendet PascalCase-Dateinamen (36 Dateien, ein Typ je Datei), der Entwicklungsstand camelCase (19 Dateien, ein Konzern je Datei). Daraus ergeben sich **neun Kollisionen, die sich ausschließlich in der Groß-/Kleinschreibung unterscheiden**:

Core/EventBus.ts	↔ Core/eventBus.ts
Core/EventDispatcher.ts	↔ Core/eventDispatcher.ts
Core/EventPublisher.ts	↔ Core/eventPublisher.ts
Core/EventRouter.ts	↔ Core/eventRouter.ts
Core/EventSubscriber.ts	↔ Core/eventSubscriber.ts
Registry/ConsumerRegistry.ts	↔ Registry/consumerRegistry.ts
Registry/EventCatalog.ts	↔ Registry/eventCatalog.ts
Registry/EventRegistry.ts	↔ Registry/eventRegistry.ts
Registry/ProducerRegistry.ts	↔ Registry/producerRegistry.ts

Auf case-insensitiven Dateisystemen (macOS-Standard, Windows) sind das jeweils **derselbe Pfad**. Ein unvorbereiteter Merge überschreibt dort Dateien ohne Konflikt und ohne Warnung.

Entscheidung

1. **Der Produktivstand dieses Repositorys ist die führende Umgebung.** Bei jedem Widerspruch zwischen den beiden Ständen — Code, Dokumentation, Nummernvergabe, Namenskonvention — gilt der Produktivstand.
2. **Die ADR- und ESS-Belegung des Produktivstands ist verbindlich.** Die abweichende Belegung des Entwicklungsstands für ADR-0010 bis ADR-0013 sowie ESS-0002 bis ESS-0010, ESS-0013 und ESS-0014 wird als ungültig erklärt. ESS-0014 bleibt frei; die nächste Vergabe erfolgt ausschließlich über `.ai/registry/ess-registry.json`.
3. **Übernahme von Code aus dem Entwicklungsstand erfolgt nur unter Neuvergabe.** Der Entwicklungsstand enthält Implementierungen, die im Produktivstand fehlen — insbesondere `server/compliance/` (Orchestrator, Engines, Evidence Collector, Remediation Planner), `server/documentary/documentaryEngine.ts`, sowie `src/platform/{Compliance,Documentary,Traceability}/` mit zusammen 51 TypeScript-Dateien. Eine Übernahme ist zulässig, jedoch nur, wenn (a) eine neue ESS-/ADR-Nummer über die Registry vergeben wird, (b) die Namenskonvention des Produktivstands (PascalCase je Typ) angewandt wird und (c) die im Audit dokumentierten Sicherheitsmängel des Entwicklungsstands vorher beseitigt sind.
4. **Kein direkter Merge des Verzeichnisses `src/platform/EventMesh/`.** Die neun Case-Kollisionen sind vor jeder Zusammenführung aufzulösen. Die Implementierung des Produktivstands bleibt maßgeblich; die Fassung des Entwicklungsstands wird nicht übernommen.
5. **Sicherheitsmerkmale werden nicht rückportiert.** Der Entwicklungsstand enthält Mechanismen, die im Produktivstand bewusst beseitigt wurden — insbesondere den hartkodierten Administrationstoken in `server/orchestrator.ts` und die Autorisierung

über den vom Client gelieferten E-Mail-Parameter. Diese dürfen unter keinen Umständen in den Produktivstand gelangen. Die entsprechende Umgebungsvariable `ORCHESTRATOR_ADMIN_TOKEN` bleibt in `.env.example` des Produktivstands entfernt (ADR-0003.5).

Begründung

Zwei parallel gepflegte Nummernräume machen jede Querverweisung mehrdeutig: Die Angabe „siehe ESS-0004“ löst je nach gelesenen Stand auf den Version Manager oder auf die Documentary Engine auf. Damit ist die Traceability-Kette `ESS → ADR → Code` nicht mehr eindeutig auflösbar, und die Dokumentation verliert ihre Revisionssicherheit — sie ist die Eigenschaft, die das Governance-Gerüst überhaupt begründet.

Der Produktivstand wurde als führend gewählt, weil er die Governance-Infrastruktur trägt (zwei Registries, 25 Komponenten-Manifeste, dokumentiertes Ausnahmeverfahren, Verantwortungsmatrix) und weil sein Sicherheitsniveau deutlich höher liegt: JWT-basierte, fail-closed ausgelegte Autorisierung mit Rollenmodell, Step-Up-Authentifizierung, serverseitige Quota-Durchsetzung und fünf Datenbank-Migrationen mit Row-Level-Security. Der Entwicklungsstand besitzt zwar mehr Anwendungscode, autorisiert Administrationszugriffe jedoch ohne jedes Credential und ist in dieser Form nicht produktionsfähig.

Die Alternative — beide Stände als gleichrangig zu behandeln und modulweise zusammenzuführen — wurde verworfen: Sie hätte für jede der zehn ESS- und vier ADR-Kollisionen eine Einzelentscheidung erfordert, ohne dass eine der beiden Belegungen einen Vorrang begründen könnte, und hätte die Mehrdeutigkeit für die Dauer der Zusammenführung fortgeschrieben.

Konsequenzen

Positiv

- Der ADR- und ESS-Nummernraum ist wieder eindeutig; Querverweise sind auflösbar.
- Die ESS-Registry bleibt die alleinige Vergabestelle, ihre Regel wird durch diese Entscheidung bestätigt statt umgangen.
- Der Sicherheitsstand des Produktivsystems ist gegen Rückschritte durch Rückportierung geschützt.

Negativ

- Die im Entwicklungsstand vorhandenen Implementierungen (Compliance-Engines, Documentary Engine, Traceability, Meme-Coin-Agenten) stehen dem Produktivstand zunächst nicht zur Verfügung. Ihre Übernahme wird zu eigenständigen Vorhaben mit eigener Nummernvergabe.
- Dokumente des Entwicklungsstands, die auf die dortige Nummernbelegung verweisen, sind nach einer Übernahme jeweils anzupassen.

Offen

- Die im Audit festgestellten Reifegrad-Widersprüche (Entwicklungsstand meldet 76/100 und dreimal 100/100, Produktivstand misst 46/100, Governance 0/100) bleiben bestehen, bis die Kennzahlen des Entwicklungsstands zurückgezogen oder neu berechnet werden. Handgesetzte Reifegrade widersprechen ESS-0001-CONTRACTS Chapter 12, wonach Kennzahlen ausschließlich berechnet und niemals manuell gesetzt werden.

Verweise

- `docs/architecture/ENTERPRISE_FINTECH_ARCHITECTURE_AUDIT.md` (ARCH-AUDIT-0002) — vollständige Befundlage
- `.ai/registry/ess-registry.json` — verbindliche ESS-Nummernvergabe
- `.ai/registry/exception-registry.json` — Ausnahmeverfahren
- `docs/adr/ADR-0003_5-identity-access-management.md` (resolved) — Ablösung des Administrationstokens
- `docs/adr/ADR-0018-enterprise-event-mesh.md` — verbindliche Event-Mesh-Entscheidung

ADR-0020: Multi-Provider Market Data Routing & Evidence-Preserving Failover

Quelle: docs/adr/ADR-0020-multi-provider-market-data-routing.md

ADR-0020: Multi-Provider Market Data Routing & Evidence-Preserving Failover

- **Status:** ACCEPTED
- **Datum:** 2026-08-02
- **Autor:** CAPITAL-AI Architecture

Kontext

CAPITAL-AI besitzt bereits mehrere reale Datenquellen (u. a. CoinGecko, Binance, Kraken, Stooq, Alpha Vantage und FMP). Bisher waren diese Quellen jedoch überwiegend asset- oder service-spezifisch verdrahtet. Ein Ausfall, Rate-Limit oder regionales Netzwerkproblem eines bevorzugten Providers kann dadurch trotz vorhandener Alternativen zu `SOURCE_UNAVAILABLE` führen.

Finanzdaten dürfen gleichzeitig nicht wie austauschbare HTTP-Backends behandelt werden: unterschiedliche Provider haben abweichende Marktdefinitionen, Börsenabdeckung, Zeitstempel, Aggregationsmethoden, Lizenzrechte und Datenqualität. Ein klassisches Round-Robin würde deshalb Datenherkunft verschleiern und kann Scoring-Integrität beschädigen.

Entscheidung

CAPITAL-AI führt eine zentrale, evidence-preserving Market-Data-Routing-Schicht ein.

Die Routing-Entscheidung berücksichtigt:

1. Governance-Basispriorität des Providers
2. Assetklasse und Capability
3. beobachtete Fehlerrate
4. aufeinanderfolgende Fehler
5. Cooldown-/Circuit-Zustand
6. beobachtete Latenz
7. explizite Provider-Provenance im Ergebnis

Die Strategie ist **adaptive failover / load-aware routing**, nicht blindes Round-Robin.

Grundregel

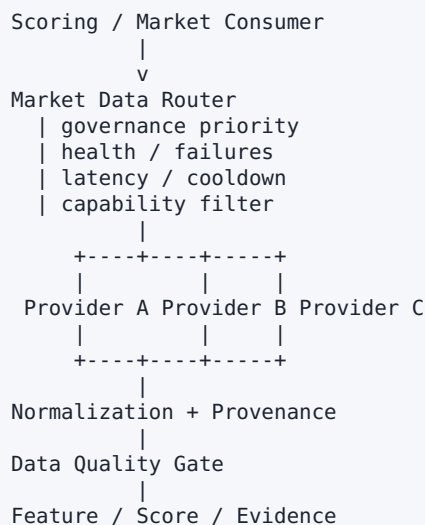
Ein alternativer Provider darf einen Primärprovider nur für eine Capability ersetzen, für die er explizit im Provider Registry zugelassen ist. Provider-ID, Source Path, Observation-/Retrieval-Time und Evidence-ID bleiben bis zum finalen Score erhalten.

Aktivierung neuer Provider

Neue externe Provider werden zunächst als `candidate` registriert und bleiben technisch deaktiviert, bis folgende Punkte erfüllt sind:

- Adapter implementiert und getestet
- API-Key/Secret ausschließlich über Production Environment
- Nutzungs-/Redistributionsrechte geprüft
- Rate-Limit-/Quota-Policy dokumentiert
- Provenance-Contract erfüllt
- Provider Health & Circuit Breaker integriert
- Golden-/Contract-Tests grün
- Production Handoff dokumentiert

Architektur



Initiale Provider-Gruppen

Crypto

- CoinGecko — bevorzugte aggregierte History/Snapshot-Quelle
- Binance — Venue-Fallback für History/Quotes/Orderbook
- Kraken — Venue-Fallback für History/Quotes/Orderbook
- CoinAPI — Kandidat für normalisierte Multi-Exchange-Daten

Stocks / Forex / Indices

- Stooq — bestehende History-Quelle
- Alpha Vantage — bestehende Fundamentals/Market-Data-Quelle
- FMP — bestehende Index-/Traditional-Asset-Quelle
- Twelve Data — Kandidat für globale Multi-Asset-Redundanz
- Finnhub — Kandidat für Fundamentals/Market/Alternative Data
- Massive — Kandidat für US Stocks/Options sowie Multi-Asset-Streaming

Macro / Rates / Bonds

- FRED — Kandidat für makroökonomische Zeitreihen und Zins-/Bond-Kontext
- ECB — offizielle EUR-Referenz- und Euro-Area-Daten; Referenzdaten, keine Execution Prices

Konsequenzen

Positiv

- geringere Single-Provider-Abhängigkeit
- bessere Cold-Start-Resilienz
- providerübergreifende Observability
- kontrollierte Ausweichpfade bei Rate Limits
- vollständige Datenherkunft bleibt erhalten
- neue Provider können ohne Änderung der Scoring-Contracts ergänzt werden

Negativ / Risiken

- mehrere Provider können unterschiedliche Werte für denselben Marktzeitpunkt liefern
- zusätzliche Lizenz- und Kostenkomplexität
- Symbol-/Instrument-Mapping wird komplexer
- Konsens-/Quorum-Logik benötigt eigene Toleranzregeln
- eine niedrige Latenz darf niemals Qualitäts-/Lizenzregeln überstimmen

Nicht entschieden

Dieses ADR aktiviert keine neuen kostenpflichtigen Provider und ändert keine Secrets, Render-, Supabase- oder Billing-Konfiguration. Die Kandidaten sind Architektur-/Integrationsziele; ihre produktive Aktivierung erfordert jeweils einen eigenen Handoff und gegebenenfalls ein Folge-ADR.

ADR-0021: Aktivierung zusätzlicher externer Market-Data-Provider

Quelle: docs/adr/ADR-0021-external-market-data-provider-activation.md

ADR-0021: Aktivierung zusätzlicher externer Market-Data-Provider

- **Status:** ACCEPTED
- **Datum:** 2026-08-02
- **Autor:** CAPITAL-AI Architecture

Kontext

CAPITAL-AI verwendet seit ADR-0020 eine adaptive, provenance-erhaltende Multi-Provider-Routing-Schicht. Die bestehende produktive Provider-Landschaft soll um drei bereits serverseitig konfigurierte Quellen erweitert werden, um Single-Provider-Abhängigkeiten weiter zu reduzieren und die Asset-Abdeckung zu erhöhen.

Verfügbare serverseitige Environment-Variablen:

- `COIN_API_KEY` — CoinAPI
- `EODHD_API_KEY` — EODHD
- `TWELVEDATA_API_KEY` — Twelve Data

Die Schlüssel dürfen ausschließlich im Backend verwendet und niemals an Client-/Browser-Code übertragen werden.

Entscheidung

Die drei Provider werden als aktive, adaptergestützte Datenquellen in die bestehende Market-Data-Routing-Architektur aufgenommen.

CoinAPI

Einsatzschwerpunkt: Crypto.

- OHLCV-Historie über die CoinAPI REST Market Data API
- Referenz-/Spotpreise über Exchange Rates
- Authentifizierung per `X-CoinAPI-Key`
- Provider-/Symbol-ID wird Bestandteil der Provenance
- CoinAPI-Referenzpreise dürfen nicht als venue-spezifischer Execution Price dargestellt werden

Twelve Data

Einsatzschwerpunkt: Multi-Asset.

- `time_series` für Stocks, Forex, Crypto und unterstützte weitere Instrumente
- `/price` für Referenz-/Spotpreise
- Authentifizierung serverseitig über den empfohlenen Authorization-Header
- Exchange-, MIC-, Currency- und Timezone-Metadaten werden soweit verfügbar erhalten
- Null-/Rate-Limit-/Entitlement-Antworten führen fail-closed zu Provider-Fallback

EODHD

Einsatzschwerpunkt: EOD-/Historical-Redundanz.

- End-of-Day Historie für unterstützte Stocks, Forex und Crypto
- `api_token` ausschließlich in serverseitigen Requests
- Symbol-/Exchange-Semantik wird explizit in Provenance erhalten
- EOD-Werte dürfen bei Echtzeit-Quorumprüfungen nicht als zeitgleiche Ausführungspreise behandelt werden

Routing-Prinzip

Die Provider werden nicht per Round-Robin ausgetauscht. Die bestehende Routerlogik berücksichtigt Governance-Priorität, beobachtete Fehler, Cooldowns und Latenz. Freshness, Provenance, Lizenz-/Entitlement-Regeln und Source-Conflict-Gates haben Vorrang vor reiner Verfügbarkeit.

Quorum und Konflikte

Für kritische Preisfelder wird der bestehende `marketDataConsensus`-Contract verwendet. Mehrere unabhängige Quellen dürfen nur dann einen kanonischen Referenzwert erzeugen, wenn:

- genügend unabhängige Provider vorliegen,
- Einheit/Währung übereinstimmt,
- Observation-Time-Skew innerhalb der zulässigen Grenze liegt,
- die Abweichung innerhalb der definierten Basis-Punkte-Toleranz liegt.

Andernfalls wird `SOURCE_CONFLICT` bzw. `INSUFFICIENT_SOURCES` ausgegeben; es wird kein künstlicher Mittelwert erzeugt.

Konsequenzen

Positiv:

- geringere Abhängigkeit von einzelnen Marktquellen,
- breitere Crypto-/Stock-/Forex-Abdeckung,
- zusätzliche unabhängige Evidence für Scoring und Plausibilisierung,

- bessere Runtime-Resilience und Provider-Observability.

Risiken/Controls:

- Quotas und Vertrags-/Redistributionsrechte bleiben provider- und planabhängig,
- EOD- und Echtzeitdaten dürfen semantisch nicht vermischt werden,
- Symbol-Mappings werden nur dort aktiviert, wo sie deterministisch sind,
- fehlende oder widersprüchliche Daten bleiben fail-closed.

Verifikation

- Unit-/Contract-Tests mit gemockten Providerantworten
- TypeScript-Check
- Production Build
- bestehendes SCA/CVE-Gate
- Deployment-Readiness-Gates

Referenzdokumentation

- CoinAPI REST API: <https://www.coinapi.io/products/market-data-api/docs/rest-api>
- EODHD Historical API: <https://eodhd.com/financial-apis/api-for-historical-data-and-volumes>
- Twelve Data API: <https://twelvedata.com/docs/advanced>

ADR-0022 — Evidence-gated Bond Scoring Architecture

Quelle: docs/adr/ADR-0022-bond-scoring-evidence-architecture.md

ADR-0022 — Evidence-gated Bond Scoring Architecture

Status: Proposed / implementation gate

Scope: CAPITAL-AI Bond Screening & Scoring

Depends on: ADR-0020, ADR-0021, PO Scoring Integrity

Context

CAPITAL-AI besitzt inzwischen einen realen EODHD-Evidence-Pfad für explizite `*.GBOND`-Instrumente sowie Macro-/Rate-Evidence aus FRED und der ECB Data API. Diese Daten reichen jedoch noch nicht aus, um einen belastbaren Enterprise-Bond-Score automatisch freizuschalten.

Ein Bond-Score darf insbesondere nicht aus AssetRegistry-Bootstrapwerten, Symbol-Hashes oder geschätzten Yield-/Duration-Werten entstehen.

Decision

Bond Scoring bleibt **deaktiviert**, bis ein Instrument den folgenden Evidence Contract erfüllt.

Mandatory instrument evidence

1. eindeutige Provider-Instrument-ID / ISIN oder expliziter Provider-Ticker;
2. reale Preis- oder Yield-Historie mit Observation Timestamps;
3. Maturity Date;
4. Coupon / Coupon Type;
5. Currency;
6. mindestens eine belastbare Duration- oder Cashflow-Basis, aus der Duration reproduzierbar berechnet werden kann;
7. Provider-Provenance und stabile Evidence IDs für alle verwendeten Faktoren.

Mandatory macro/rate context

Mindestens eine reale, zeitlich gültige Rate-/Curve-Quelle muss vorhanden sein. Zulässige Rollen sind beispielsweise:

- FRED Treasury-/Policy-Rate-Evidence;
- ECB offizielle Referenz-/Euro-Area-Rate-Evidence;
- weitere später governte Sovereign-Yield-/Reference-Rate-Provider.

Referenzdaten sind **keine Execution Prices**.

Proposed scoring dimensions

Erst nach Erfüllung des Evidence Contracts darf die Scoring Engine folgende Dimensionen berechnen:

Dimension	Beispiel-Evidence	Regel
Interest-rate sensitivity	Duration, maturity, yield curve	keine geschätzte Duration
Yield attractiveness	YTM / current yield + reference curve	nur belegte Yield-Werte
Credit quality	issuer/rating evidence	Rating-Quelle erforderlich
Liquidity	real volume/spread/trading evidence	fehlende Liquidität = Faktor ausgeschlossen
Price momentum	real historical prices	deterministisch aus History
Curve / regime context	FRED/ECB/official rates	reference-only context
Currency risk	bond currency + FX reference	keine ECB-Referenzrate als Execution Price

Gewichte dürfen erst mit einem separaten versionierten Scoring Contract festgelegt werden.

Fail-closed states

- BOND_EVIDENCE_INCOMPLETE
- SOURCE_UNAVAILABLE
- SOURCE_CONFLICT
- STALE_EVIDENCE
- SCORE_NOT_COMPUTABLE

Keiner dieser Zustände darf durch einen heuristischen Defaultscore ersetzt werden.

Activation gate

Ein produktiver Bond-Score benötigt vor Aktivierung:

1. versionierten Bond Feature Contract;
2. versionierten Bond Scoring Contract;
3. Golden Dataset / Regression Tests;
4. Mindest-Coverage je Instrument;
5. Source-Conflict-Regeln für kritische Felder;
6. Data Freshness Gate;
7. vollständige Lineage `request -> retrieval -> factor -> evidence -> score`;
8. Supervisor Provider Health;

- 9. Compliance-/Runtime-Evidence;
- 10. dokumentierte Lizenz-/Redistributionsfreigabe je Datenquelle.

Consequences

Der vorhandene EODHD-Bond-Pfad und FRED/ECB-Daten erhöhen die Evidence-Abdeckung, schalten aber **keinen Score automatisch frei**. Dies verhindert, dass unvollständige Fixed-Income-Daten als quantitativ belastbare Bewertung dargestellt werden.

UI / Visuals

Diese ADR verlangt keine grafischen Änderungen. UI-Änderungen bedürfen einer separaten Freigabe.

ADR-0023 — FRED & ECB Macro/Rate Evidence Sources

Quelle: docs/adr/ADR-0023-macro-rate-evidence-sources.md

ADR-0023 — FRED & ECB Macro/Rate Evidence Sources

Status: Accepted for evidence use

Scope: Financial Intelligence / Macro & Rate Evidence

Related: ADR-0020, ADR-0021, ADR-0022

Context

CAPITAL-AI benötigt für Fixed Income, Macro-Regime, FX-Plausibilisierung und spätere Cross-Asset-Risikomodelle belastbare Referenzdaten. Solche Quellen dürfen nicht mit Realtime-/Execution-Market-Data gleichgesetzt werden.

Decision

FRED

FRED wird als serverseitiger, API-key-geschützter Macro-/Rate-Evidence-Provider integriert.

Initiale Series-Allow-List:

- `DGS2` — U.S. Treasury 2Y Constant Maturity
- `DGS10` — U.S. Treasury 10Y Constant Maturity
- `FEDFUNDS` — Federal Funds Effective Rate
- `CPIAUCSL` — Consumer Price Index

Beliebige vom Client übergebene FRED-Series-IDs sind nicht zugelassen. Erweiterungen erfolgen über reviewed Code-/Governance-Änderungen.

ECB

Die ECB Data API wird als keyless `reference-only` Provider eingebunden. Initial wird die offizielle `EXR`-Dataflow für explizit zugelassene EUR-Referenzwährungen verwendet.

ECB-Referenzkurse werden niemals als handelbare oder ausführbare Preise markiert.

Common Evidence Contract

Jede Zeitreihe enthält:

- Provider;
- Series ID;
- Purpose;
- Unit;

- Observation Date;
- Retrieval Timestamp;
- secret-freien Source Path;
- stabile Evidence IDs;
- `executionPriceEligible: false`.

Runtime behavior

- Providerfehler sind fail-closed;
- fehlender `FRED_API_KEY` erzeugt `SOURCE_UNAVAILABLE`, keine Defaultdaten;
- Provider Health und adaptive Routing Telemetry werden erfasst;
- bestehendes `RUNTIME-01` Compliance Evidence verarbeitet beobachtete Providerzustände automatisch;
- der abgeleitete DGS2/DGS10-Macro-Regime-Contract besitzt zusätzlich ein Freshness-Gate und unterscheidet Provider-Erreichbarkeit von fachlicher Evidence-Aktualität.

Production handoff status — 2026-08-02

Der Betreiber hat `FRED_API_KEY` in der Render-Produktionsumgebung als serverseitige Environment Variable angelegt. Dieser ADR und der zugehörige PR enthalten **keinen Secret-Wert** und verändern Render nicht selbst. Nach Merge und erfolgreichem Deployment kann der implementierte FRED-Live-Pfad den Runtime-Key konsumieren.

API boundaries

- `GET /api/registry/macro/fred/:seriesId`
- `GET /api/registry/macro/ecb/fx/:currency`
- `GET /api/registry/macro/risk-regime`

Die Endpunkte unterstützen Request-/Response-Correlation und geben Evidence IDs aus. Der Risk-Regime-Endpunkt bleibt Evidence-/Context-only und kann bei veralteten Treasury-Daten `STALE_EVIDENCE` statt eines aktuellen Regimes ausgeben.

Security

`FRED_API_KEY` ist ausschließlich serverseitig und darf keinen `VITE_`-Prefix besitzen. Die ECB API benötigt keinen Secret-Key.

Consequences

Macro-/Rate-Evidence steht für nachgelagerte Analysen zur Verfügung, aktiviert jedoch weder Bond-Scoring noch Execution-/Trading-Funktionen automatisch. Eine Verwendung als Scoringfaktor benötigt einen separat versionierten Feature-/Scoring-Contract.

UI / Visuals

Keine grafischen Komponenten sind Bestandteil dieser Architekturentscheidung.

ADR-0024 — Cross-Asset Risk Context Contract

Quelle: docs/adr/ADR-0024-cross-asset-risk-context.md

ADR-0024 — Cross-Asset Risk Context Contract

Status: Accepted for evidence/context use

Scope: Financial Intelligence / Cross-Asset Risk Context

Related: ADR-0021, ADR-0022, ADR-0023

Context

CAPITAL-AI verfügt über verifizierte Asset-Scoring-Lineage sowie FRED-/ECB-Macro-Evidence. Makrodaten dürfen jedoch nicht stillschweigend als zusätzlicher Asset-Score-Faktor verwendet werden, solange dafür kein separat kalibrierter und regression-getesteter Scoring Contract existiert.

Decision

Ein eigenständiger, versionierter Cross-Asset Risk Context wird eingeführt.

Asset classes

- stock
- forex
- index
- crypto
- bond

Evidence inputs

Primäre Regime-Evidence:

- FRED DGS2
- FRED DGS10
- 10Y-2Y Treasury Spread

Supplemental Evidence:

- FRED FEDFUNDS
- FRED CPIAUCSL

Supplemental Evidence darf fehlende oder veraltete Yield-Curve-Evidence nicht zu **READY** hochstufen.

Contract rules

Der Contract führt Provider, Evidence IDs, Observation Dates und Macro-Sensitivity mit.

Fest verdrahtete Sicherheitsgrenzen:

- `executionPriceEligible: false`
- `scoreImpactEnabled: false`
- `recommendationEligible: false`

Ein zukünftiger Score-Impact erfordert eine neue ADR, einen versionierten Feature-/Scoring-Contract, Golden-Dataset-Regressionen und dokumentierte Kalibrierung.

Backend boundaries

- `GET /api/registry/macro/cross-asset/:assetClass`
- `GET /api/registry/assets/:symbol/verified-context`

Der zweite Endpunkt korreliert Asset-Score-Lineage und Macro-Evidence über eine gemeinsame Request-Korrelation, verrechnet beide Teilverträge aber nicht miteinander.

Runtime Governance

Source-Conflict-, Quorum- und Snapshot-Integrity-Beobachtungen werden separat in die Supervisor-/Compliance-Runtime-Evidence geschrieben. Damit bleibt unterscheidbar:

1. Provider Availability;
2. Market Data Integrity;
3. Asset Scoring Lineage;
4. Macro Context Evidence.

UI / Visuals

Keine grafischen Änderungen sind Bestandteil dieser ADR.

ADR-0025 — Verified Traditional Quotes & Screening Eligibility

Quelle: docs/adr/ADR-0025-verified-traditional-quotes-and-screening-eligibility.md

ADR-0025 — Verified Traditional Quotes & Screening Eligibility

- Status: Proposed / implementation in PR
- Date: 2026-08-02
- Scope: Stock, Forex, Index screening and price-alert boundaries

Context

After the P0 data-integrity hardening, registry bootstrap values are intentionally no longer exposed as verified market prices. Crypto already has an evidence-preserving spot-consensus path. Traditional assets require a separate quote boundary so screening and alerts do not regress to bootstrap or simulated prices.

At the same time, `screening-eligibility/1.0.0` exists as a reusable contract but must be applied directly at bulk screening boundaries rather than only inside clients.

Decision

1. Introduce `traditional-quote/1.0.0` as the verified quote contract for traditional assets.
2. Stock and Forex current quotes use Twelve Data because a server-side `TWELVEDATA_API_KEY` and a live quote endpoint are available.
3. Index current quotes use the existing FMP quote adapter and its approved index mapping.
4. EODHD remains EOD/historical/reference evidence. It MUST NOT masquerade as an intraday alert quote.
5. Quote responses contain provider, observation time, retrieval time, source path, evidence IDs and explicit alert eligibility.
6. `executionPriceEligible` remains false. These quotes are screening/alert reference evidence, not an execution venue feed.
7. Missing provider configuration or provider failure returns an explicit unavailable state with `price: null`; no bootstrap/default price is substituted.
8. The bulk verified-score endpoint evaluates every result through `screening-eligibility/1.0.0` and exposes both READY and ELIGIBLE counts.
9. Screening eligibility is separate from canonical score computation. It may exclude a score from a screening/ranking result but may not alter the score itself.

Consequences

- Traditional price alerts can be enabled only after their clients consume this contract.
- Provider-specific quote semantics remain visible.
- EOD and intraday data cannot be silently mixed.
- Bulk screening becomes auditable through a versioned eligibility decision.
- Hard market-integrity/SLA blocking remains disabled until runtime thresholds are calibrated and reviewed.

Security and operations

- API keys remain server-side only.
- No Render, Supabase or Stripe mutation is part of this ADR.
- Provider outcomes continue to feed the central runtime routing/health telemetry.
- Quote contract and eligibility behavior require regression tests and green CI before merge.

Architectural Decision Record (ADR-0026)

Quelle: docs/adr/ADR-0026-social-media-direct-publishing-real-integration.md

Architectural Decision Record (ADR-0026)

Social Media Direct Publishing — Reale OAuth-2.0- und Plattform-API-Integration

Status: ACCEPTED

Implementation-Status: COMPLETE (Account-/OAuth-/Publish-Backend) · IN PROGRESS (Content-Generation-Anbindung)

Date: 2026-08-01

Version: 0.6.0

Governance note: This ADR was formerly numbered ADR-0020. It was renumbered to ADR-0026 under AUD4-F-004 because ADR-0020 is canonically reserved for Multi-Provider Market Data Routing.

Kontext

Der ursprünglich übergebene Social-Media-Publishing-Prototyp enthielt Mock-Erfolgs-URLs, keinen echten OAuth-Code-Token-Exchange, keine State-Verifikation und globalen In-Memory-Zustand. Das widersprach der Zero-Breach-/No-Fake-Data-Policy.

Entscheidung

1. Persistenz über `social_media_accounts`, `social_media_oauth_states` und `social_media_publish_log` mit user-scoped Zugriff.
2. Echter OAuth-2.0-Handshake mit State-TTL/Einmalverwendung und plattformspezifischen Anforderungen.
3. Verschlüsselte Token-Ablage über die bestehende AES-256-GCM-Infrastruktur.
4. Reale Plattform-Publish-Calls für YouTube, TikTok, Instagram/Facebook und X.
5. Kein manuelles Fake-Connect; Verbinden ausschließlich über OAuth.
6. Authentifizierung über verifizierte Identität auf allen funktionalen Endpunkten.

Scope-Grenze

Video-/Audio-Publishing benötigt eine reale Rendering-/Hosting-Pipeline und eine erreichbare `mediaUrl`. Fehlt diese, muss der Publish-Pfad explizit fehlschlagen statt eine Erfolgs-URL zu erfinden. Text-Publishing auf X/Facebook kann unabhängig davon funktionieren.

Betroffene Bereiche

- `server/socialMedia/*`

- `src/routes/socialMediaRoutes.ts`
- `src/platform/SocialMediaEngine/*`
- `src/components/SocialAccountManager.tsx`
- `src/components/SocialDirectPublisherModal.tsx`
- `supabase/migrations/20260801150000_social_media_publishing.sql`
- `docs/runbooks/SOCIAL_MEDIA_OAUTH_SETUP.md`

Konsequenzen

- Keine synthetischen Publish-Erfolge.
- OAuth-/Token-Zustand ist persistent und user-scoped.
- Tokens verlassen den Server nicht.
- Externe Developer-Console-/Secret-Konfiguration bleibt Production-Handoff und wird nicht im Repository gespeichert.

Architectural Decision Record (ADR-0027)

Quelle: docs/adr/ADR-0027-social-media-access-restriction-owner-founder.md

Architectural Decision Record (ADR-0027)

Social Media Direct Publishing Hub — Zugriffsbeschränkung auf Owner-Rolle oder Founder-Abo

Status: ACCEPTED

Implementation-Status: COMPLETE

Date: 2026-08-01

Version: 0.6.0

Basis: ADR-0026 (Social Media Direct Publishing)

Governance note: This ADR was formerly numbered ADR-0021. It was renumbered to ADR-0027 under AUD4-F-004 because ADR-0021 is canonically reserved for External Market Data Provider Activation.

Kontext

Das Social Media Direct Publishing Hub ist kein allgemeines Feature für registrierte Nutzer, sondern ausschließlich für Owner oder Founder-Abonnenten vorgesehen.

Entscheidung

1. Serverseitige zentrale Zugriffskontrolle in `server/socialMedia/accessControl.ts`.
2. Zugriff nur für `profiles.iam_role === 'owner'` oder Subscription-Tier `Founder`.
3. Alle funktionalen Social-Media-Endpunkte werden serverseitig geschützt.
4. OAuth-Callbacks prüfen die Berechtigung erneut, bevor ein Code gegen Tokens getauscht wird.
5. Frontend-Gates sind ausschließlich UX und niemals die Sicherheitsgrenze.
6. Fehlerzustände bleiben fail-closed.

Konsequenzen

- Pro/Enterprise erhalten ohne Founder-Zugriff keine implizite Berechtigung.
- Kein neues Autorisierungskonzept; bestehende verifizierte Identity-/Subscription-Infrastruktur wird wiederverwendet.
- Änderungen an Stripe-/Supabase-Produktionskonfiguration bleiben Production-Handoff und sind nicht Teil dieser ADR-Neunummerierung.

ADR-0028 — Screening SLO, empirical confidence and horizon-exact validation

Quelle: docs/adr/ADR-0028-screening-slo-confidence-and-horizon-validation.md

ADR-0028 — Screening SLO, empirical confidence and horizon-exact validation

Status

Accepted for staged implementation.

Context

CAPITAL-AI already separates canonical financial scores from operational screening evidence. Screening eligibility, provider SLA and empirical score validation exist as independent contracts, but runtime confidence was not yet observable in the Supervisor and historical score validation still compares old snapshots with the current registry price instead of a verified price near the exact validation horizon.

Decision

1. Runtime score-confidence evidence is represented by `score-confidence-evidence/1.0.0` and is created only by an actually executed score-validation run.
2. `INSUFFICIENT_DATA` always carries `confidencePct: null`; no default confidence is permitted.
3. Confidence is observability/model-risk evidence only. It must not alter the canonical score or recommendation and must not be represented as execution probability.
4. Screening operational evidence remains separate from the score through `screening-operations/1.0.0`, `screening-sla/1.0.0` and `screening-slo-evidence/1.0.0`.
5. Future horizon-exact validation must use `horizon-validation-evidence/1.0.0`: only a verified provider observation inside an explicitly bounded target window around `snapshotDate + horizonDays` is admissible.
6. Interpolation, registry bootstrap values and synthetic price evidence are forbidden for horizon validation.
7. Persistent SLO/validation evidence may be added during production handoff, but this development change does not create or mutate Supabase/Render configuration.

Consequences

- Supervisor can expose real validation-confidence state without fabricating a metric.
- Current score validation remains explicitly limited until provider-backed historical retrieval is wired into the horizon contract.

- Missing historical evidence reduces validation sample size instead of silently substituting a value.
- Activating any score/ranking hard gate from SLA or confidence requires a separate reviewed ADR and calibrated production evidence.

Non-goals

- No score-weight modification.
- No trading/execution probability.
- No automatic hard-gate activation.
- No UI redesign.

ADR-0029 — Bond Scoring Weights Proposal (ADR-0022 Follow-up)

Quelle: docs/adr/ADR-0029-bond-scoring-weights-proposal.md

ADR-0029 — Bond Scoring Weights Proposal (ADR-0022 Follow-up)

- Status: Proposed / Pending Review — **kein produktiver Scoring-Pfad, keine Freigabe**
- Date: 2026-08-02
- Scope: Bonds (`src/lib/assetRegistry.ts` `type: 'bond'`)
- Depends on: ADR-0022 (Evidence-gated Bond Scoring Architecture)

Context

ADR-0022 sperrt Bond Scoring bewusst, bis (a) der Evidence Contract vollständig erfüllt ist (bereits umgesetzt: `src/services/bondFeatureContract.ts`, `src/types/bondScoringContract.ts`, `src/services/eodhdBondEvidence.ts`, alle mit Tests) und (b) "Gewichte ... mit einem separaten versionierten Scoring Contract festgelegt werden". Teil (b) fehlte bisher als konkretes Artefakt.

Diese ADR liefert **ausschließlich den Entwurf** dieses separaten Contracts (`src/services/bondScoringWeightsProposal.ts`, `bond-scoring-weights-proposal/0.1.0-draft`) für die fachliche Prüfung. Sie hebt die Sperre aus ADR-0022 nicht auf und ändert `src/types/bondScoringContract.ts` / `src/services/bondFeatureContract.ts` nicht — beide bleiben unverändert die alleinige Instanz, die `score: null` erzwingt.

Decision

1. `DRAFT_BOND_SCORING_WEIGHTS` schlägt Gewichte für die sieben in ADR-0022 genannten Scoring-Dimensionen vor (Summe = 1.0), jeweils mit Evidenzgrundlage und Begründung im Code selbst.
2. Der Proposal-Contract trägt ein eigenständiges `status`-Feld (`proposed` | `approved` | `rejected`), das **manuell** von einer reviewenden Person gesetzt werden muss — es gibt keinen automatisierten Übergang zu `approved`.
3. `previewHypotheticalBondScore()` existiert ausschließlich für die in ADR-0022 geforderte Golden-Dataset-/Backtesting-Validierung durch Reviewer:innen außerhalb von Produktionspfaden. Die Funktion wirft, solange `status !== 'approved'`.
4. Keine Route, kein Endpunkt und keine UI-Komponente importiert `bondScoringWeightsProposal.ts` (durch Regressionstest erzwungen, siehe unten). Das Proposal ist damit strukturell tot für den produktiven Pfad, bis es explizit verdrahtet wird — und diese Verdrahtung ist ausdrücklich **nicht** Teil dieser ADR.

5. `evaluateBondEvidenceGate()` (ADR-0022, `src/types/bondScoringContract.ts`) bleibt unverändert: `score: null` in jedem Fall, auch bei vollständiger Evidence.

Was für eine formale Freigabe noch fehlt

Diese ADR macht **keine** Aussage darüber, dass die vorgeschlagenen Gewichte korrekt oder ausreichend validiert sind. Vor einer Freigabe (`status: 'approved'`) sind mindestens erforderlich:

1. Fachliche Prüfung der sieben Dimensionen und ihrer Gewichte durch eine für Fixed-Income-Risikomodelle qualifizierte Person.
2. Ein Golden Dataset (reale historische Bond-Preise/Yields/Ratings mit bekanntem Ausgang) zum Backtesting von `previewHypotheticalBondScore()`.
3. Eine Ratingagentur-Anbindung für den Faktor „Credit quality“ — aktuell ohne Provider (`evidenceBasis` vermerkt dies explizit im Proposal).
4. Eine Bond-Spread-/Liquiditäts-Datenquelle für den Faktor „Liquidity“ — aktuell ebenfalls ohne Provider.
5. Nach Freigabe: eine eigene, neue ADR oder ein Nachtrag zu dieser ADR, die die tatsächliche Verdrahtung in eine Route beschreibt (analog zu ADR-0025 für traditionelle Assets), inklusive erneuter Regressionstests, die die dann gewollte Live-Nutzung erlauben.

Consequences

- Der ADR-0022-Auftrag „separater versionierter Scoring Contract“ hat jetzt ein konkretes, review-fähiges Artefakt statt nur einer Textanforderung.
- Es entsteht keine neue Angriffsfläche für fabrizierte Finanzaussagen: das Proposal kann nicht versehentlich live gehen (Typ-Sperre `scoringEnabled: false`, Laufzeit-Wurf bei `status !== 'approved'`, Regressionstest gegen jede Route-Einbindung).
- Zwei strukturelle Lücken (Rating-Provider, Liquiditäts-Provider) werden durch das Proposal selbst sichtbar gemacht statt verschwiegen.

Security and operations

- Keine neuen externen API-Calls, keine neuen Secrets.
- Keine Änderung an `src/types/scoringIntegrity.ts`, `src/types/bondScoringContract.ts` oder `src/services/bondFeatureContract.ts`.
- Regressionstests (`tests/unit/bondScoringWeightsProposal.test.ts`) müssen grün bleiben, inklusive des Tests, der jede Route/jeden Server-Einstiegspunkt auf Abwesenheit eines Imports von `bondScoringWeightsProposal.ts` prüft.

ADR-0031 — Supabase Free-Tier Auth Security Compensating Controls

Quelle: docs/adr/ADR-0031-supabase-free-tier-auth-security-compensating-controls.md

ADR-0031 — Supabase Free-Tier Auth Security Compensating Controls

- **Status:** Accepted
- **Date:** 2026-08-02
- **Scope:** CAPITAL-AI Production Authentication / Supabase Auth
- **Platform Version:** 0.6.0
- **Governance ID:** SEC-AUTH-SUPABASE-001
- **Related:** ADR-0003.5, ADR-0008, ADR-0030
- **Security Advisor Target:** SUPABASE_SECURITY_ZERO_ACTIONABLE_WARNINGS

1. Context

The production Supabase Security Advisor currently reports one remaining warning after remediation of the Stripe schema function `search_path` findings:

`auth_leaked_password_protection`

Supabase documents leaked-password protection as a Pro Plan and above feature. The active CAPITAL-AI Supabase project is intentionally operated on the Free Tier at the time of this decision. Therefore the warning cannot be remediated within the current plan by enabling the native HavelBeenPwned-based leaked-password protection feature.

The advisor additionally reports `rls_enabled_no_policy` for `public.screening_slo_evidence` at INFO level. That state is intentional: the table is server-write-only and deliberately has no `anon` or `authenticated` policies. It is not considered a security defect and must not be "fixed" by adding a client policy solely to silence the advisor.

2. Decision

CAPITAL-AI formally adopts the following security state for the Supabase Free Tier:

1. Native leaked-password protection remains unavailable while the project is on the Free Tier.
2. The advisor warning is classified as **PLAN-CONSTRAINED / ACCEPTED RISK**, not as a remediated control.
3. The target production status is `SUPABASE_SECURITY_ZERO_ACTIONABLE_WARNINGS`, not `ZERO_WARN`, because plan-constrained warnings and intentional INFO findings are distinguished from actionable security defects.

4. The risk acceptance is temporary and automatically expires as soon as the project is upgraded to Supabase Pro or above.
5. On any plan upgrade to Pro or above, enabling leaked-password protection becomes a mandatory Production Release Gate before the next release may be marked accepted.
6. No SQL workaround, custom database function, copied breach corpus, or client-side imitation may be represented as equivalent to Supabase native leaked-password protection.

3. Compensating controls on Free Tier

While the native leaked-password control is unavailable, CAPITAL-AI must maintain defense in depth through controls available independently of that feature.

3.1 Required controls

- Email confirmation remains enabled for password-based registration.
- Production auth emails use the configured custom SMTP path rather than relying on the shared development mail service.
- IAM authorization must not depend on user-editable `user_metadata`; privileged roles remain server-verified.
- Owner/admin-sensitive actions remain protected by the existing Step-Up/TOTP architecture and IAM controls.
- Auth endpoints and application boundaries continue to use rate limiting and fail-closed authorization.
- All exposed application tables remain protected by RLS according to their intended access model.
- Service-role/secret keys remain server-side only and must never be shipped to browser bundles.
- Password reset and signup redirect URLs must remain explicit production allowlist entries; wildcard production redirects are prohibited.

3.2 Password policy baseline

Where the current Supabase Free Tier UI permits configuration, the production password policy should use:

- minimum length: **12 characters preferred**, never below 8;
- require lowercase characters;
- require uppercase characters;
- require digits;
- require symbols.

These settings are compensating controls only. They do not make a claim that the password has been checked against a known-compromised-password corpus.

3.3 Additional controls requiring explicit implementation/configuration

CAPTCHA/bot protection may be introduced only together with the corresponding frontend token flow and provider configuration. It must not be enabled server-side without the application flow supplying valid CAPTCHA tokens.

Any change to session lifetime, inactivity timeout, or single-session enforcement must be separately evaluated against the active Supabase plan and the CAPITAL-AI device/session product requirements before activation.

4. Release lifecycle integration

ADR-0030 remains the governing version/release lifecycle. This ADR adds the following security gate to it.

Current Free Tier

A release may pass the Supabase security gate when all of the following are true:

- Security Advisor contains no unresolved actionable database/auth warnings available to the current plan.
- `auth_leaked_password_protection` is the only remaining WARN and is documented as plan-constrained.
- Intentional `rls_enabled_no_policy` findings are traceable to a server-only/fail-closed access decision.
- No new WARN is silently accepted without a dedicated security review.

Resulting lifecycle status:

```
SUPABASE_SECURITY_ZERO_ACTIONABLE_WARNINGS
```

After upgrade to Pro or above

The first release after a plan upgrade is blocked until:

1. leaked-password protection is enabled in Supabase Auth;
2. the Security Advisor no longer reports `auth_leaked_password_protection` ;
3. production password signup/change/reset flows are smoke-tested;
4. the security exception record is closed as `REMEDIATED` ;
5. the release acceptance record references the remediation evidence.

Required gate status after upgrade:

```
SUPABASE_LEAKED_PASSWORD_PROTECTION_ENABLED
```

5. Review triggers

This decision must be re-evaluated when any of these events occur:

- Supabase plan changes from Free to Pro/Team/Enterprise;

- Supabase makes leaked-password protection available on Free Tier;
- Supabase changes the Security Advisor semantics for this finding;
- CAPITAL-AI introduces a materially different authentication provider or password architecture;
- a security incident indicates credential-stuffing or compromised-password abuse;
- a compliance requirement mandates breached-password screening.

6. Evidence and verification

The current decision is grounded in:

- production Supabase Security Advisor output;
- Supabase documentation stating leaked-password protection is available on Pro Plan and above;
- production confirmation that the project is operated under the Free Tier;
- prior remediation of all actionable `function_search_path_mutable` warnings affecting the `stripe` schema.

The Security Advisor must be rerun after every relevant auth/database hardening change and before production acceptance when the release touches authentication, RLS, database functions, or privileged data paths.

7. Consequences

Positive

- The repository no longer treats an unremediable tariff limitation as an unexplained security defect.
- Free-Tier production state has a precise, auditable acceptance condition.
- A future Pro upgrade has an explicit mandatory remediation gate.
- The design avoids weakening `screening_slo_evidence` RLS merely to remove an informational advisor notice.

Negative / residual risk

- Supabase Auth cannot natively reject passwords solely because they are present in a known breached-password corpus while the project remains on Free Tier.
- Strong password composition, TOTP step-up, rate limiting and email verification reduce but do not eliminate credential-reuse/credential-stuffing risk.

8. Final decision

For the active Supabase Free Tier, the remaining leaked-password warning is accepted only as a documented plan constraint with compensating controls and a mandatory upgrade-triggered remediation gate.

Accepted security target: SUPABASE_SECURITY_ZERO_ACTIONABLE_WARNINGS

ADR-0034 — Central Subscription Entitlements and Warren Buffett Access

Quelle: docs/adr/ADR-0034-central-subscription-entitlements-and-buffett-access.md

ADR-0034 — Central Subscription Entitlements and Warren Buffett Access

Status

Accepted

Implementation-Status

IN PROGRESS — canonical entitlement contract, server-side quota engine, Buffett authorization API and repository migration are implemented on the feature branch. UI migration and verified-score route adoption remain separate follow-up PRs. The Supabase migration is intentionally **not** applied live by this PR.

Date

2026-08-03

Context

Subscription behavior had drifted across `Abonnements.tsx`, `SubscriptionModal.tsx`, client-side screening trackers and server-side quota code. The Warren Buffett Value Check was particularly inconsistent: one UI described it as Starter-accessible, another comparison table described it as Enterprise-only, while the component itself performed no authoritative server-side entitlement check.

The recently added evidence-backed index, commodity and sovereign-benchmark scoring paths also need the same subscription contract as the existing screening lifecycle rather than independent feature-specific assumptions.

Decision

Capital-AI introduces `subscription-entitlements/1.0.0` in `src/config/subscriptionEntitlements.ts` as the canonical code-level contract for feature access and usage windows.

Canonical plan matrix

Capability	Free	Starter	Pro	Enterprise
Monthly price	€0	€7	€29	€109
Devices	1	2	2	5
Verified screening	3 / rolling 5 days	5 / rolling day	20 / rolling day	Unlimited

Capability	Free	Starter	Pro	Enterprise
Backtest	No	No	Yes	Yes
Monte Carlo	No	No	1 / rolling day	Unlimited
Full AI analysis	Preview only	1 / rolling day	Unlimited	Unlimited
Realtime AI Newsfeed	No	No	Yes	Yes
Warren Buffett Value Check	1 asset / rolling 3 days	1 asset / rolling 3 days	Full	Full
PDF / Compliance export	No	No	No	Yes
Annual self-service	n/a	10% discount	10% discount	On request

Guest users are not a paid subscription tier. They may use only explicitly guest-enabled surfaces. **Warren Buffett Value Check is denied to guests.**

Warren Buffett semantics

“Available from Pro” means **full access** starts at Pro. Free and Starter retain a deliberately constrained sampling entitlement: one distinct asset per rolling three-day window. Reopening the same already-authorized asset within the active window does not consume another quota unit.

The authorization path is server-side:

```
POST /api/entitlements/warren-buffett/authorize
```

The endpoint requires a verified identity, resolves the persisted subscription tier server-side, validates the asset against the asset catalog and consumes the entitlement quota. Client-side visibility must never be treated as authorization.

Persistent quota contract

`public.user_quota` is extended with `buffett_value_check` and `subject_key`. `public.consume_user_quota(...)` performs atomic quota consumption under an advisory transaction lock. It is `SECURITY INVOKER` with `search_path = ''` and fully schema-qualified relation access.

The repository migration is:

```
supabase/migrations/20260802203000_extend_user_quota_entitlement_kinds.sql
```

Production application belongs to the controlled Supabase production handoff. This ADR does not authorize ad-hoc live DDL from the development workflow.

Fail-closed fallback

If the production quota RPC is temporarily unavailable, the backend falls back to the existing process-local rate limiter rather than failing open. This preserves the entitlement boundary while the deployment/migration state is being reconciled.

New component integration

The evidence-backed index, commodity and sovereign-benchmark scoring paths are screening capabilities and must consume the canonical `verified_screening` entitlement. A follow-up PR will add the shared gate to the verified-score/context/batch routes so the new provider/evidence architecture cannot bypass subscription limits.

Stripe consistency finding

The code contract continues the product rule “Starter and Pro annual self-service = 10% discount.” The live Stripe configuration observed during the ADR review is not fully consistent with that rule:

- Starter: €7/month → €75.60/year, consistent with 10% discount.
- Pro: €29/month → expected €313.20/year at 10%, while the currently active Stripe yearly price observed was €248/year.

No Stripe price is mutated by this ADR/PR. Billing-price correction requires an explicit production billing decision and controlled Stripe change.

Consequences

Positive

- One code-level source of truth for plan entitlements.
- Guest Buffett access is fail-closed.
- Free/Starter Buffett sampling is enforceable server-side and device-independent after migration.
- Pro/Enterprise full Buffett access is explicit rather than inferred from UI text.
- New evidence-scoring components receive a defined subscription integration target.
- Annual pricing drift becomes observable instead of silently encoded in multiple UI calculations.

Remaining work

1. Migrate `Abonnements.tsx` and `SubscriptionModal.tsx` to the canonical plan contract.
2. Update `BuffetValueCheck.tsx` to request server authorization before showing/switching evaluated assets.
3. Apply canonical screening gates to verified-score/context/batch endpoints for index, commodity and sovereign bond evidence scoring.
4. Migrate remaining feature-specific subscription checks to `canUseFeature()` / `getWindowed-FeatureLimit()`.
5. During production handoff, apply the Supabase migration and validate Security Advisor/RLS behavior.
6. Resolve the live Stripe Pro yearly-price discrepancy through the billing lifecycle; do not silently change billing objects from application code.

Acceptance

ADR-0034 may move to `resolved/` only when:

- backend tests for the plan matrix and rolling windows pass;
- Buffett UI consumes the authorization API;
- verified-score routes use the canonical screening entitlement;
- duplicated plan matrices are removed or reduced to presentation-only mappings from the canonical contract;
- the production Supabase migration is applied and verified;
- the Stripe annual-price decision is explicitly closed or documented as an accepted commercial exception.

Architectural Decision Record (ADR-0035)

Quelle: docs/adr/ADR-0035-protected-google-marketing-integration-strict-csp.md

Architectural Decision Record (ADR-0035)

Protected Google Marketing Integration, Strict CSP & OWNER-delegated Service Accounts

Status: ACCEPTED

Implementation-Status: IN PROGRESS

Date: 2026-08-03

Version: 0.6.0

Related ESS: ESS-0014 / ESS-0014-CONTRACTS

Related ADR: ADR-0003.5, ADR-0008, ADR-0009, ADR-0012, ADR-0030

Kontext

CAPITAL-AI bindet CookieHub, Google Analytics und Google AdSense in die Browser-Run-time ein. Der CookieHub-Banner war zuvor ausgefallen, weil die CSP Inline-JavaScript blockierte, während `window.cookiehub.load(...)` inline in `index.html` ausgeführt wurde. Der Fix externalisierte die Initialisierung nach `public/cookiehub-init.js`, wodurch die bestehende `script-src 'self'`-Policy wieder mit der Consent-Runtime kompatibel wurde.

Mit der zusätzlichen AdSense-Integration entsteht ein weiterer Architekturkonflikt: Google unterstützt AdSense unter CSP ausschließlich mit einer Strict-CSP-Strategie, da sich die intern genutzten AdSense-Domains ändern können. Gleichzeitig benötigt CookieHub explizite Script-/Style-/Connect-Ziele.

Zusätzlich muss verhindert werden, dass ein späterer Agent, Refactor, Drift-Reconciler oder automatisierter Rollback diesen funktionierenden Consent-/Security-Zustand stillschweigend zurückbaut.

Entscheidung

1. Protected Marketing Invariants

CookieHub, Consent-Gates, AdSense-/Analytics-Integrationspunkte, Strict CSP, Credential Boundaries sowie IAM-/Approval-Gates werden als geschützte Integrationsinvarianten behandelt.

Ein Rückbau ist nur nach einem expliziten Protected-Change-Prozess zulässig.

2. Mandatory Impact Confirmation

Vor jedem Rückbau muss ein interaktiver Agent stoppen und die konkreten Auswirkungen erläutern. Die Bestätigung muss sich auf den beschriebenen Diff beziehen.

Mindestens zu benennen:

- Privacy/Consent;
- Analytics/Messung;
- AdSense/Monetarisierung;
- CSP/XSS-Sicherheit;
- Compliance/Audit/Traceability;
- Runtime/SEO/AMP;
- betroffene Ressourcen/Fingerprints.

3. Human Authorization

Direkte protected changes dürfen nur von einem verifizierten Human User mit:

```
profiles.iam_role == owner
AND fresh TOTP step-up
AND explicit impact confirmation
AND dry-run diff
```

ausgeführt werden.

`admin`, `supervisor`, `user` und Subscription-Tiers erhalten keine implizite Berechtigung.

4. Service Accounts

Service Accounts werden nicht als zusätzliche Human-IAM-Rolle modelliert. Sie erhalten eine separate Capability Registry.

Nur Human OWNER + frischer TOTP Step-up darf Capabilities vergeben, ändern, verlängern oder widerrufen.

Ein Service Account darf seine eigenen Rechte weder erweitern noch delegieren.

Für Rollbacks gilt zusätzlich:

```
active google.marketing.rollback capability
AND one-time job-specific OWNER approval artifact
```

Ein dauerhafter Rollback-Grant allein ist nicht ausreichend.

5. Strict CSP

Production HTML erhält einen kryptographisch zufälligen Nonce pro Response. Derselbe Request-Nonce wird:

- in den CSP Header;
- in alle autorisierten `<script>`-Tags der HTML-Antwort

injiziert.

AdSense-kompatibler Kern gemäß Google-Dokumentation:

```
object-src 'none';  
base-uri 'none';  
script-src 'nonce-{REQUEST_NONCE}' 'unsafe-inline' 'unsafe-eval' 'strict-dynamic' https: http;;
```

Die Entscheidung übernimmt `unsafe-eval` ausschließlich als dokumentierten AdSense-Kompatibilitäts-Trade-off. Die Vertrauenswurzel bleibt der zufällige Nonce + `strict-dynamic`; ein statischer oder global wiederverwendeter Nonce ist verboten.

6. CookieHub CSP

CookieHub erhält mindestens die aktuell dokumentierten Ziele:

```
script-src: cookiehub.net cdn.cookiehub.eu  
style-src: cookiehub.net cdn.cookiehub.eu  
connect-src: ds.cookiehub.net consent.cookiehub.net region-eu.cookiehub.net consent-eu.cookiehub.net cookiehub.net cdn.cookiehub.eu
```

7. AMP Auto Ads

Die angeforderten `amp-auto-ads` Snippets werden im Markup hinterlegt. Sie gelten jedoch **nicht** als bestätigte AMP-Implementierung, solange das Dokument keine valide AMP-Dokumentstruktur besitzt.

Für die aktuelle React/Vite-SPA bleibt die normale AdSense-Integration der aktive Nicht-AMP-Pfad. Eine echte AMP-Auslieferung benötigt eine separate AMP-Route/-Dokumentvariante und Validierung.

8. Google Marketing MCP Separation

ESS-0014 ist die normative Architektur.

Claude wird ausschließlich als Implementierungs-/MCP-Host-Profil behandelt. Das gesonderte Dokument `docs/architecture/CAPITAL_AI_G00GLE_MARKETING_MCP_ARCHITECTURE.md` beschreibt die technische Umsetzung für Claude, ohne die ESS-Autorität zu ersetzen.

Offizielle read-only MCPs (Google Analytics, Google Ads) bilden die Read/Evidence Plane. Mutationen erfolgen nur über explizite CAPITAL-AI Google-API-Adapter hinter IAM, Policy, Approval und Audit.

Begründung

Warum kein einfacher Domain-Whitelist-Fix für AdSense?

Google weist darauf hin, dass die von AdSense genutzten Domains wechseln können und unterstützt deshalb Strict CSP mit Nonce. Eine statische Allowlist wäre drifthanfällig und könnte Anzeigen später erneut still blockieren.

Warum keine `owner` -Rolle für Service Accounts?

Human Identity und Machine Identity haben unterschiedliche Lebenszyklen, Credentials und Kontrollanforderungen. Eine Maschinenidentität als Human OWNER würde Auditbarkeit und Least Privilege untergraben.

Warum one-time OWNER approval zusätzlich zum Service-Account-Grant?

Ein stehender Rollback-Grant darf nicht zu einem dauerhaft autonomen Rückbaupfad werden. Das einmalige Approval bindet den Rückbau an einen konkreten Diff, Impact und Zeitpunkt.

Konsequenzen

Positive Konsequenzen

- CookieHub-Fix kann nicht normativ still zurückgebaut werden;
- AdSense erhält eine provider-kompatible CSP-Strategie;
- Human-/Machine-Identity bleiben getrennt;
- Claude und andere Agents bleiben außerhalb der Trust Root;
- Rollbacks werden nachvollziehbar und impact-basiert;
- Google-MCP-Read-Plane kann sicher mit API-Write-Plane kombiniert werden.

Trade-offs

- HTML muss in Produktion dynamisch ausgeliefert werden, damit ein neuer Nonce pro Response eingesetzt werden kann;
- rein statisches `sendFile(index.html)` reicht dafür nicht mehr aus;
- `unsafe-eval` bleibt als von AdSense dokumentierter Kompatibilitäts-Trade-off Bestandteil der Script Policy und muss bei Provider-Änderungen erneut bewertet werden;
- echte Service-Account-Delegation benötigt eine Capability Registry und Persistenzschicht;
- AMP Auto Ads benötigen für Wirksamkeit eine separate valide AMP-Auslieferung.

Geschützte Dateien / Ressourcen (Initial Scope)

```
index.html
public/cookiehub-init.js
public/google-analytics-consent.js
server.ts (CSP/HTML delivery)
.ai/skills/ESS-0014-*
docs/architecture/CAPITAL_AI_GOOGLE_MARKETING_MCP_ARCHITECTURE.md
CookieHub production configuration
Google Analytics production property/data stream
Google Tag Manager production container/version
Google AdSense publisher/site configuration
Google Ads production resources
```

Rollback Procedure

Ein zulässiger Rückbau benötigt:

1. Change Classification `protected=true` ;
2. Impact Disclosure;
3. explizite Bestätigung;
4. OWNER + Step-up ODER Service Account + exact capability + one-time OWNER Approval;
5. Dry-run;
6. Fingerprint Check;
7. Backup/Evidence;
8. Apply;
9. Post-change Verification;
10. Audit/Traceability Update.

Ein `git revert` , automatisierter Reconciler oder AI-Refactor ist kein Bypass.

Verifikation / Definition of Done

ADR-0035 darf erst nach `docs/adr/resolved/` verschoben werden, wenn mindestens folgende Evidence vorliegt:

- production response zeigt pro Request unterschiedlichen CSP Nonce;
- alle relevanten Script-Tags tragen denselben Request Nonce;
- CookieHub Banner funktioniert ohne CSP Violation;
- AdSense Smoke Test läuft ohne CSP-bedingten Block;
- Analytics bleibt consent-gated;
- protected rollback endpoint/tool erzwingt OWNER/Step-up;
- Service-Account Grants sind OWNER-only verwaltbar;
- Rollback benötigt one-time OWNER approval;
- CI/Tests decken die Denial Matrix ab;
- Production Evidence ist dokumentiert.

Bis dahin bleibt `Implementation-Status:` `IN PROGRESS` .

Provider References (Stand 2026-08-03)

- Google AdSense CSP: <https://support.google.com/adsense/answer/16283098>
- CookieHub CSP: <https://docs.cookiehub.com/advanced/csp>
- Google Analytics MCP: <https://developers.google.com/analytics/devguides/reporting/data/v1/mcp>

- Google Ads MCP: <https://developers.google.com/google-ads/api/docs/developer-toolkit/mcp-server>

Provider-Anforderungen sind bei jeder Änderung erneut zu prüfen.

ADR-0036 — Server Composition and Modularization

Quelle: docs/adr/ADR-0036-server-composition-and-modularization.md

ADR-0036 — Server Composition and Modularization

- **Status:** Accepted
- **Scope:** CAPITAL-AI backend composition
- **Decision type:** Structural / maintainability / operational safety

Context

The root `server.ts` accumulated application composition, infrastructure configuration, domain routes, market-data aggregation, scoring, AI provider usage, frontend serving, background jobs and startup diagnostics. This created a high-change, high-coupling module with more than two thousand lines and increased regression risk.

Decision

Adopt a modular Express composition architecture in which the root `server.ts` is only the composition root. Route families, services, middleware, security configuration and lifecycle behavior are separated by responsibility.

AI-dependent route modules receive provider instances through dependency injection. Market-data aggregation is isolated as a service so HTTP routing and background refresh use one implementation.

Consequences

Positive

- Smaller change surface in the application entry point.
- Clearer ownership and dependency boundaries.
- Easier unit/integration testing.
- Reduced accidental coupling between billing, IAM, scoring and market-data concerns.
- Provider-specific and domain-specific future refactors can be performed independently.

Trade-offs

- More source files and explicit imports.
- Integration tests become important to ensure route registration order remains stable.
- Market-data service remains intentionally broad until provider-level tests are available.

Guardrails

- Do not move Stripe raw-body webhook parsing behind `express.json()`.
- Do not weaken IAM, TOTP step-up, quota or rate-limit middleware while extracting routes.
- Do not introduce demo/generated market values to replace unavailable live data.
- Do not put API secrets in route modules or frontend code.
- Maintain Documentary/ADR traceability for subsequent architecture moves.

Verification

The refactor must pass TypeScript parsing/type checking in the complete repository, existing route tests, billing webhook tests, IAM tests and production build before merge.

Architectural Decision Record (ADR-0037)

Quelle: docs/adr/ADR-0037-render-production-configuration-governance.md

Architectural Decision Record (ADR-0037)

Render Production Configuration Governance, Stateless Runtime & Reproducible Deployment Contract

Status: ACCEPTED

Implementation-Status: HIGH PRIORITY / IN PROGRESS

Date: 2026-08-03

Version: 0.6.0

Priority: P0/P1

Related audit: docs/architecture/RENDER_PRODUCTION_CONFIGURATION_AUDIT.md

Related ADR: ADR-0009, ADR-0011, ADR-0017, ADR-0030, ADR-0035

Dependency: ADR-0036 is being developed independently in PR #75; ADR-0037 must not overwrite its PR-governance scope.

1. Kontext

CAPITAL-AI wird als Docker-basierter Render Web Service betrieben. Das Repository enthält mit `render.yaml`, `Dockerfile`, `/healthz`, GitHub CI und Deployment-Readiness bereits wesentliche Produktionsbausteine.

Der Render-Audit vom 03.08.2026 zeigt jedoch, dass die tatsächliche Production Configuration noch nicht vollständig als reproduzierbarer Architekturvertrag vorliegt:

- `render.yaml` definiert nur Service Type, Name, Docker Runtime, Health Check und Secret-Platzhalter;
- Region, Instance Plan, Scaling, Auto-Deploy-Policy, Environment Protection, Domains, Shutdown Delay und weitere Controls sind nicht repositoryseitig festgeschrieben;
- der Server bindet fest auf Port 3000 statt auf den Render-`PORT`-Vertrag;
- der öffentliche Health Check ist primär Liveness und bildet kritische Readiness nicht explizit ab;
- ein expliziter `SIGTERM`-/Graceful-Shutdown-Flow fehlt;
- Background Jobs laufen im Web-Prozess und verhindern ein sicheres horizontales Scale-out;
- lokale Subscription-/Credit-Fallbacks können auf das Render-Dateisystem geschrieben werden, das standardmäßig ephemeral ist;
- das aktuelle Env-Inventory-Gate kann dynamisch zusammengesetzte Environment-Schlüssel übersehen;

- das bestehende Rollback-Runbook beschreibt Teile der heutigen Render-Rollback-Semantik nicht mehr korrekt.

Ein direkter Render-Connector stand beim Audit nicht zur Verfügung. Dashboard-only Werte bleiben daher bis zur Production-Evidence-Erhebung unverifiziert und dürfen nicht geschätzt werden.

2. Problem Statement

Ein Enterprise-FinTech-System darf seine Produktionswahrheit nicht aus einer Mischung aus:

```
partial render.yaml
+ undocumented Dashboard settings
+ runtime fallbacks
+ implicit agent knowledge
```

ableiten.

Ohne einen verbindlichen Contract entstehen insbesondere folgende Fehlerklassen:

1. **Configuration Drift:** Dashboard und Repository entwickeln sich auseinander.
2. **Non-Reproducible Recovery:** ein neu erstellter Service erhält andere Region-/Plan-/Deploy-Einstellungen.
3. **Silent Data Loss:** lokaler Fallback wird bei Restart/Deploy verworfen.
4. **Unsafe Scaling:** zusätzliche Web-Instanzen duplizieren Scheduler/Background Jobs.
5. **Partial Readiness:** Render schaltet eine Instanz live, obwohl kritische Teilfunktionen nicht verfügbar sind.
6. **Shutdown Races:** Deploys beenden laufende Jobs/Requests unkontrolliert.
7. **Deployment Bypass:** Production deploys vor oder unabhängig von verpflichtenden CI Gates.
8. **Incident Ambiguity:** Rollback-/Environment-Wirkung ist nicht vollständig bekannt.

3. Entscheidung

3.1 Render Production Configuration wird ein geschützter Architekturvertrag

Die Render-Konfiguration wird in drei klar getrennte Klassen aufgeteilt:

```
A. IaC-managed
B. Secret-managed
C. Operational-managed
```

Jede Produktionseinstellung MUSS genau einer Klasse zugeordnet sein.

A. IaC-managed

Diese Werte sollen nach erfolgreicher Dashboard-Evidence im `render.yaml` bzw. einer daraus abgeleiteten Render-IaC-Struktur die Source of Truth bilden:

- Service type/name/runtime;
- repository + production branch;
- region;
- plan/compute class;
- autoDeployTrigger;
- healthCheckPath;
- maxShutdownDelaySeconds;
- manual scaling / autoscaling policy;
- custom domains;
- Render subdomain policy;
- Dockerfile path/context/command where applicable;
- build filters;
- Project/Environment topology where supported;
- Environment protection/network isolation where supported;
- preview policy.

B. Secret-managed

Secret-Werte bleiben außerhalb des Repositories in Render.

Repository speichert ausschließlich den Contract:

```
key
classification
required/optional
build/runtime scope
owner
rotation impact
dependent components
```

Keine Secret-Werte, Deploy Hook URLs oder Access Tokens werden dokumentiert.

C. Operational-managed

Nicht vollständig per Blueprint repräsentierbare Einstellungen werden mit Evidence dokumentiert:

- notification destinations;
- workspace member roles;
- GitHub App installation scope;
- operational webhooks;
- incident contacts.

3.2 Production Web Tier ist stateless

Der Render Web Service darf keinen authoritative Business State im lokalen Dateisystem halten.

Insbesondere verboten als durable Source of Truth:

- Subscription Status;
- Billing Entitlements;
- PDF/Credit Balances;
- IAM State;
- Approval State;
- Financial user state.

Zulässige lokale Daten:

- bounded temporary files;
- regenerierbare caches;
- non-authoritative diagnostics.

Persistenter Business State gehört in Supabase/Postgres oder einen explizit freigegebenen durable Service.

Konsequenz für Render Disk

Ein Render Persistent Disk wird **nicht** als Standardlösung für diese Business-Daten eingeführt.

Begründung:

- Disk bindet den Service an eine Instanz;
- Multi-Instance-Scaling ist nicht möglich;
- Zero-Downtime Deploys werden deaktiviert;
- Disk löst keine Multi-Service-/Transaction-Semantik.

Ein Disk benötigt künftig ein separates ADR mit exaktem Datentyp und Downtime-/Scaling-Folgen.

3.3 Web Request Plane und Background Execution Plane werden getrennt

Der aktuelle Web-Prozess enthält HTTP-Serving und wiederkehrende Background-Aktionen.

Zielarchitektur:

```
Render Web Service
- HTTP/API
- auth/billing endpoints
- stateless request processing
```

```
Render Worker / controlled scheduler
- scheduled market refresh coordination
- alert evaluation
- snapshot jobs
- non-request documentary jobs, where applicable

Supabase / lock store
- idempotency
- lease/leader coordination
- durable job state
```

Bis diese Trennung technisch abgeschlossen und getestet ist:

```
numInstances = 1
AUTOSCALING = FORBIDDEN
```

für den aktuellen kombinierten Web-Prozess.

Eine Skalierungsänderung vor Abschluss gilt als protected architectural change.

3.4 Render Port Contract ist verbindlich

Die Anwendung muss ihren HTTP-Port aus `process.env.PORT` lesen und weiterhin an `0.0.0.0` binden.

Normativer Contract:

```
const PORT = Number(process.env.PORT || 3000);
```

Ein hartcodierter Production-Port ist nicht mehr zulässig.

Docker-internes `EXPOSE` und lokaler Docker-Healthcheck müssen mit demselben Portmodell kompatibel sein.

3.5 Production Deployments dürfen CI nicht umgehen

Für den production-linked Render Service gilt als Ziel:

```
autoDeployTrigger: checksPass
```

Ein Production Deploy darf nicht allein durch einen Commit auf `main` ausgelöst werden, wenn verpflichtende CI Checks fehlen oder fehlschlagen.

Da Render `success`, `neutral` und `skipped` als bestanden interpretieren kann, gilt zusätzlich:

```
Kritische Required Checks dürfen nicht durch optionale Workflow-Bedingungen zu skip-
ped werden.
```

Die GitHub-Required-Check-Architektur wird durch ADR-0036 ergänzt; ADR-0037 definiert die Render-Seite dieses Vertrages.

3.6 Production Environment muss geschützt werden

Wenn der aktuelle Render Workspace/Plan diese Funktion unterstützt und der Service einem Project Environment zugeordnet ist, wird Production als `Protected Environment` betrieben.

Ziel:

- destruktive Dashboard-Aktionen nur Admins;
- Production Secrets nur autorisierte Admins;
- environment-scoped Environment Groups;
- Cross-Environment-Netzwerkzugriffe nach Architekturbedarf blockieren.

Wichtig: Render weist darauf hin, dass Blueprint-managed Ressourcen trotz Environment Protection weiterhin über Repository-/Blueprint-Änderungen verändert werden können. Daher sind GitHub CODEOWNERS/Branch Rules zwingender Teil des Gesamtschutzes.

3.7 Health wird in Liveness und Readiness getrennt

Liveness

Beantwortet ausschließlich:

Läuft der Prozess und kann HTTP bedienen?

Keine externen Provider-Netzwerkaufrufe pro Probe.

Readiness

Beantwortet:

Darf diese Instanz Production Traffic erhalten?

Readiness verwendet eine dokumentierte Dependency-Policy und bounded/cached checks.

Nicht jede optionale API darf die gesamte Anwendung unready machen.

Kritische Dependency-Gruppen werden explizit klassifiziert:

```
CORE_RUNTIME
AUTH_IAM
DURABLE_STATE
BILLING
AI_OPTIONAL
MARKET_DATA_DEGRADED_ALLOWED
SMTP_OPTIONAL
```

Render `healthCheckPath` wird erst nach implementiertem und getestetem Readiness Contract auf den dafür vorgesehenen Endpoint umgestellt.

3.8 Graceful Shutdown ist verpflichtend

CAPITAL-AI implementiert einen zentralen Shutdown Coordinator für `SIGTERM`.

Normative Reihenfolge:

```
SIGTERM
-> readiness=false
-> stop scheduling new jobs
-> stop watchers/intervals
-> stop accepting new HTTP work
-> drain in-flight requests
-> settle bounded operations
-> flush security/audit logs where possible
-> close provider/client resources
-> exit(0)
```

`maxShutdownDelaySeconds` wird nicht willkürlich gewählt, sondern anhand eines Tests festgelegt.

3.9 Environment Contract wird zentralisiert

Die Anwendung benötigt einen kanonischen Production Env Contract, der dynamisch genutzte Schlüssel ebenfalls kennt.

Der Contract umfasst mindestens:

- Supabase URLs/keys;
- Stripe secrets + webhook;
- alle aktiven monthly/yearly/one-time Price IDs;
- TOTP Encryption;
- SMTP;
- AI Provider Keys;
- Market Data Provider Keys;
- Metrics;
- Social OAuth Clients;
- public Vite build variables.

Der Contract muss Quelle für automatisierte Validierung und Dokumentation sein.

Reguläre Expressions über `getCleanEnv('literal')` allein reichen nicht als vollständiger Nachweis.

3.10 Rollback Knowledge muss provideraktuell sein

Das Runbook muss folgende Render-Eigenschaften korrekt widerspiegeln:

- Dashboard-Rollback kann vorhandene Build-Artefakte wiederverwenden;

- Dashboard-Rollback deaktiviert automatische Deploys;
- Code-Rollback ist kein Datenbank-Rollback;
- ein Rollback setzt nicht automatisch jede aktuelle Servicekonfiguration zurück;
- Environment Groups und Disks besitzen eigene Semantik;
- ein Git-Revert und ein Render Instant Rollback sind unterschiedliche Recovery-Mechanismen.

Providerverhalten wird bei relevanten Render-Änderungen erneut gegen aktuelle Dokumentation geprüft.

3.11 Production Region ist explizite Architekturentscheidung

Die tatsächliche Render-Region wird als Production Evidence erfasst.

Da Render die Region eines bestehenden Services nicht nachträglich ändern lässt, darf kein Agent blind `region: frankfurt` oder eine andere Region in den Blueprint schreiben.

Ein Regionswechsel erfordert:

- Latenzbewertung;
- Datenfluss-/Compliance-Bewertung;
- externe Provider-Topologie;
- DNS-/Domain-Cutover-Plan;
- neuer Service / Migration Plan;
- Rollback Plan.

3.12 Monitoring besteht aus internem und externem Signal

Mindestens:

```
Render health check
+ Render failure/unhealthy notifications
+ external HTTP probe
+ CAPITAL-AI metrics/log correlation
```

Ein internes Healthcheck-Signal allein gilt nicht als vollständiges Production Monitoring.

4. High-Priority Finding Matrix

Finding	Priority	Decision
Ephemeral entitlement fallback	P0	durable Business State only in Supabase/approved store
Web + Background mixed	P0	split execution planes; block autoscaling until done

Finding	Priority	Decision
Hardcoded port	P1	use Render <code>PORT</code> contract
Partial Blueprint	P1	complete IaC after evidence collection
Auto Deploy unknown	P1	verify and target <code>checksPass</code>
Graceful shutdown absent	P1	central SIGTERM coordinator
Readiness unclear	P1	define liveness/readiness contract
Dynamic env keys	P1	central env contract
Region/plan/scale unknown	P1	mandatory dashboard evidence + IaC
Rollback docs drift	P1	update runbook

5. Production Change Authorization

Render Production Settings sind Teil der Production Control Plane.

Änderungen an folgenden Bereichen erfolgen nicht still aus einer Entwicklungsumgebung oder durch einen Agenten:

- region;
- plan;
- scaling;
- disk;
- custom domains;
- subdomain policy;
- environment protection;
- network isolation;
- auto deploy;
- secrets/environment groups;
- deploy hooks/API credentials;
- maintenance mode;
- notifications affecting incident routing.

Jede Änderung benötigt:

1. Current-State Evidence;
2. geplanten Sollzustand;
3. Impact/Blast Radius;
4. Rollback Plan;
5. zuständige Human-Freigabe nach CAPITAL-AI Governance;

6. post-change verification;
 7. Dokumentations-/Traceability-Update.
-

6. Implementation Phases

Phase A — Production Evidence / no mutation

- Render Dashboard Werte vollständig erfassen;
- Secret-Key-Inventar nur als Namen erfassen;
- current deploy commit dokumentieren;
- Region/Plan/Scaling/AutoDeploy/Environment Protection dokumentieren;
- Disk-Status verifizieren.

Phase B — Runtime Safety

- PORT Contract;
- Graceful Shutdown;
- Readiness Contract;
- local authoritative state entfernen;
- secret diagnostic logging reduzieren.

Phase C — Execution Plane Separation

- Background Jobs aus Web-Service lösen;
- idempotency/lease semantics;
- Worker/Scheduler Deployment;
- danach Scaling neu entscheiden.

Phase D — IaC Completion

- verified Dashboard Config in Blueprint übernehmen;
- environment groups / project environment;
- `checksPass` ;
- domains/subdomain policy;
- shutdown/scaling;
- blueprint validation in CI.

Phase E — Resilience

- notifications;
- external probe;
- recovery drill;
- recreate-from-IaC drill;

- rollback drill.
-

7. Nicht gewählte Alternativen

Alternative A — Alles im Dashboard lassen

Abgelehnt.

Grund: nicht reproduzierbar, Agenten kennen den echten Zustand nicht, hoher Drift-/Recovery-Risk.

Alternative B — Alle Secret-Werte in `render.yaml`

Abgelehnt.

Grund: Credential Exposure / Repository Leakage.

Alternative C — Render Disk für lokale Subscription-Dateien

Abgelehnt als Standardarchitektur.

Grund: Single-Instance-Bindung, kein Zero-Downtime Deploy, falsche Persistence-Schicht für relationale Billing-Wahrheit.

Alternative D — Web Service einfach autoskalieren

Abgelehnt bis Worker Separation.

Grund: jeder Replica startet aktuell Background Jobs.

Alternative E — Healthcheck direkt gegen alle Provider

Abgelehnt.

Grund: externe transient failures würden Deployments/Instances unnötig als komplett un-ready markieren. Readiness benötigt Dependency Classification und bounded checks.

8. Konsequenzen

Positiv

- Render-Architektur wird reproduzierbar;
- weniger implizite Dashboard-Abhängigkeit;
- sichere Grundlage für Multi-Agent-Entwicklung;
- klare Ownership für Secrets und Provider;
- Scaling wird erst nach Statelessness freigegeben;
- geringeres Datenverlust- und Rollback-Risiko;
- bessere Incident Response;

- klarer Disaster-Recovery-Pfad.

Trade-offs

- mehr IaC- und Evidence-Aufwand;
- Dashboard-Konfiguration muss zunächst vollständig inventarisiert werden;
- Background Work benötigt zusätzliche Deployment-Komponenten;
- Production Readiness erfordert explizite Dependency-Klassifikation;
- Render-Config-Änderungen werden formaler und langsamer, dafür nachvollziehbar.

9. Protected Invariants

Nach Umsetzung gelten mindestens als geschützt:

```
render.yaml production service identity
production branch
autoDeployTrigger=checksPass
PORT binding contract
health/readiness contract
graceful shutdown coordinator
stateless web-tier rule
no authoritative business state on local Render FS
numInstances=1 until worker separation
production environment protection
production env contract
custom domain/subdomain decision
rollback runbook
```

Ein späterer Agent darf diese Controls nicht als vereinfachendes Refactoring entfernen.

10. Definition of Done

ADR-0037 darf erst nach `docs/adr/resolved/` verschoben werden, wenn:

1. Render Dashboard Evidence vollständig ist;
2. Region, Plan, Instance Count/Scaling und Project Environment dokumentiert sind;
3. Production Environment Protection verifiziert ist;
4. Auto Deploy `checksPass` verifiziert ist;
5. Render-linked production branch verifiziert ist;
6. App `process.env.PORT` nutzt;
7. SIGTERM-Graceful-Shutdown getestet ist;
8. Render Health Check den akzeptierten Readiness Contract nutzt;
9. Subscription/Credit authoritative fallbacks nicht mehr auf ephemerem FS liegen;
10. Background Jobs entweder separiert oder durch getestete Singleton-/Lease-Semantik geschützt sind;
11. horizontales Scaling erst danach getestet/freigegeben ist;
12. vollständiger Env Contract CI-validiert wird;

13. Rollback Runbook aktualisiert ist;
14. failure/unhealthy notifications verifiziert sind;
15. external uptime probe dokumentiert ist;
16. IaC Disaster-Recreation Test durchgeführt wurde;
17. Traceability Matrix auf ADR-0037 und betroffene Komponenten aktualisiert wurde.

Bis dahin bleibt:

Implementation-Status: HIGH PRIORITY / IN PROGRESS

11. Provider References — 2026-08-03

- Render Web Services / Port Binding: <https://render.com/docs/web-services>
- Render Deploys / CI / SIGTERM: <https://render.com/docs/deploys>
- Render Health Checks: <https://render.com/docs/health-checks>
- Render Blueprints: <https://render.com/docs/infrastructure-as-code>
- Render Blueprint Spec: <https://render.com/docs/blueprint-spec>
- Render Environment Variables: <https://render.com/docs/configure-environment-variables>
- Render Projects / Protected Environments: <https://render.com/docs/projects>
- Render Persistent Disks: <https://render.com/docs/disks>
- Render Scaling: <https://render.com/docs/scaling>
- Render Rollbacks: <https://render.com/docs/rollbacks>
- Render Notifications: <https://render.com/docs/notifications>
- Render Uptime Best Practices: <https://render.com/docs/uptime-best-practices>

Die Provider-Referenzen sind bei Implementierung und Re-Audit erneut zu validieren.

ADR-0038 — Binance Spot als Datenquelle der öffentlichen AI-Kurzanalyse

Quelle: docs/adr/ADR-0038-binance-landing-ai-quick-analysis.md

ADR-0038 — Binance Spot als Datenquelle der öffentlichen AI-Kurzanalyse

Status: ACCEPTED

Implementation-Status: IN PROGRESS

Datum: 2026-08-03

Scope: Landingpage, Market Data, AI Provider Routing, No Demo Data Policy

Kontext

Die öffentliche CAPITAL-AI Landingpage soll eine kurze, nachvollziehbare Krypto-Marktanalyse anbieten. Die Analyse darf gemäß der No-Demo-Data-Policy keine simulierten Kurse verwenden und soll gleichzeitig keinen privaten Börsen-Account oder Trading-Key benötigen.

Binance stellt öffentliche Spot-Marktdaten über einen dedizierten Market-Data-Endpoint bereit. Für Ticker- und Kline-Daten ist deshalb keine Speicherung eines Binance API Secrets erforderlich.

Entscheidung

1. Öffentliche Binance-Spot-Marktdaten werden ausschließlich serverseitig über `https://data-api.binance.vision` gelesen.
2. Die Landingpage ruft ausschließlich den CAPITAL-AI-Endpoint `POST /api/landing/quick-analysis` auf und kommuniziert nicht direkt mit Binance.
3. Der Endpoint akzeptiert kompakte Asset-Symbole wie `BTC`, `ETH` oder `SOL` und normalisiert diese standardmäßig auf das Binance-USDT-Spot-Paar.
4. Als Marktevidenz werden mindestens 24h-Ticker und 24 stündliche Schlusskurse verwendet.
5. Die bestehende Provider-Kette Anthropic → OpenAI → Gemini erzeugt die sprachliche Kurzanalyse ausschließlich aus den gelieferten Binance-Werten.
6. Falls kein AI-Provider verfügbar ist oder die Provider-Kette fehlschlägt, liefert der Endpoint eine deterministische quantitative Zusammenfassung statt Demo- oder erfundener Daten.
7. Der öffentliche Endpoint ist separat pro IP rate-limitiert, um AI-Kosten und Missbrauch zu begrenzen.
8. Die Antwort nennt die Quelle `Binance Spot`, den Datenzeitpunkt und enthält keine Kauf- oder Verkaufsempfehlung.

Sicherheits- und Compliance-Eigenschaften

- Kein Binance API Key im Browser oder Repository.
- Kein Trading-, Order- oder Account-Zugriff.
- Eingabesymbole werden per Allowlist-Regex normalisiert; freie URLs oder beliebige Upstream-Ziele sind ausgeschlossen.
- Upstream-Requests besitzen ein festes Timeout.
- Die AI erhält ausschließlich serialisierte Marktmetriken; Nutzerinput wird nicht als Systemanweisung übernommen.
- No Demo Data: Upstream-Ausfall führt zu einem Fehler und niemals zu erfundenen Marktdaten.
- Öffentliche AI-Nutzung wird zusätzlich zur globalen Serverbegrenzung endpoint-spezifisch limitiert.

Implementierung

- `server/binanceLandingQuickAnalysis.ts`
- `server/ai.ts`
- `src/components/LandingBinanceQuickAnalysis.tsx`
- `src/components/CapitalAiLogo.tsx`

Der bestehende 120px-Landing-Hero des `CapitalAiLogo` ist aktuell die einzige Verwendung dieser Variantenkombination und dient als eng begrenzter Integrationspunkt für das neue Landing-Widget. Eine spätere Zerlegung der großen `LandingPage.tsx` in eigenständige Hero-/Auth-Sektionen kann diesen Integrationspunkt expliziter machen, ohne den API-Vertrag zu ändern.

Folgen

Positiv

- Reale, aktuelle Krypto-Marktdaten bereits vor der Anmeldung.
- Keine zusätzlichen Secrets oder Binance-Kontoabhängigkeiten.
- Wiederverwendung des bestehenden Multi-LLM-Routings.
- Sauberer Fallback ohne Verstoß gegen die No-Demo-Data-Policy.

Risiken

- Binance-Verfügbarkeit beeinflusst die öffentliche Kurzanalyse.
- Öffentliche AI-Aufrufe erzeugen Providerkosten; Rate-Limits bleiben verpflichtend.
- USDT-Normalisierung deckt absichtlich nicht alle möglichen Binance-Handelspaare ab.

Nachtrag 2026-08-08 — Wiederverwendung im Enterprise Scorer

Die Kernlogik (Binance-Marktdaten laden + AI-Kurzanalyse erzeugen) wurde aus dem Route-Handler in eine geteilte Funktion `computeBinanceQuickAnalysis()` (`server/binanceLandingQuickAnalysis.ts`) extrahiert und zusätzlich zum bestehenden öffentlichen Endpunkt von einem zweiten, **authentifizierten** Endpunkt verwendet: `POST /api/registry/assets/:symbol/quick-analysis` (`src/features/registry/registryRoutes.ts`), konsumiert von der neuen Komponente `src/components/EnterpriseBinanceQuickAnalysis.tsx`. Sie ersetzt die vormals im Enterprise Scorer (`EnterpriseAnalysisPanels.tsx`) gezeigte, ausschließlich auf bereits verifizierten internen Scoring-Fakten basierende "AI Kurzanalyse" (kein Live-Marktdaten-Zugriff) durch eine Live-Binance-Marktdaten-Variante, angetrieben vom im Enterprise Scorer bereits ausgewählten Symbol (kein eigenes Suchfeld). Darstellungsreihenfolge im Enterprise Scorer von oben nach unten: Asset-Suche (mit Assetklassen-Filtern und Registry-Bestand) → Kopfbereich des aktuell ausgewählten Assets (Standard: BTC) → Binance-Kurzanalyse.

Diese Erweiterung ändert die ursprünglichen Entscheidungen 1–8 nicht, ergänzt sie aber:

- Der öffentliche Landing-Endpunkt (`/api/landing/quick-analysis`) bleibt unverändert bestehen, weiterhin ohne Login erreichbar, weiterhin separat per IP rate-limitiert (`landing-binance-ai:<ip>`, 6/min) und weiterhin die einzige Instanz im nicht angemeldeten Landing-Hero — die Verifikationsaussage "Landing-Widget erscheint ausschließlich im öffentlichen Landing-Hero" bezieht sich unverändert auf `LandingBinanceQuickAnalysis.tsx` und bleibt zutreffend.
- Der neue authentifizierte Endpunkt nutzt einen eigenen Rate-Limit-Namespace (`registry-binance-ai:<ip>`, ebenfalls 6/min) und eine eigene Prompt-Registry-ID (`enterprise-binance-quick-analysis`), damit Kosten/Nutzung beider Aufrufkontexte getrennt auswertbar bleiben und keiner der beiden Kontexte den Rate-Limit-Kontingent des anderen verbraucht.
- Fail-Closed-/No-Demo-Data-Eigenschaften (Punkte 5–8 der ursprünglichen Entscheidung) gelten identisch für beide Aufrufkontexte, da beide dieselbe Kernfunktion verwenden.

Verifikation vor `resolved/`

- TypeScript-/Vite-Build erfolgreich.
- Endpoint liefert für mindestens BTC, ETH und SOL reale Binance-Daten.
- Ungültige Symbole liefern 4xx ohne AI-Aufruf.
- Binance-Ausfall/Timeout erzeugt 502 und keine Demo-Daten.
- Rate-Limit erzeugt 429.
- Landing-Widget erscheint ausschließlich im öffentlichen Landing-Hero.
- CI vollständig grün.

Erst nach diesen Nachweisen darf `Implementation-Status` auf `COMPLETE` gesetzt und der ADR nach `docs/adr/resolved/` verschoben werden.

ADR-0039 — Human-Authorized Pull Request Creation and Advisory Process Governance

Quelle: docs/adr/ADR-0039-human-authorized-pr-creation-and-advisory-governance.md

ADR-0039 — Human-Authorized Pull Request Creation and Advisory Process Governance

Status: Proposed

Date: 2026-08-03

Decision owners: CAPITAL-AI Owner / Platform Director / Security & Compliance

Supersedes: ADR-0036 process-control rules only where explicitly stated below

Context

The PR-first multi-agent governance introduced hard CI gates for three process concerns: a 15-minute claim-to-PR deadline, production-baseline consistency, and exclusive work-claim/overlap enforcement. These controls were executed in the same GitHub Actions path as sandbox candidate validation.

This conflated two different questions:

1. **Can the candidate build and pass technical quality checks?**
2. **Should a Pull Request be created, sequenced, merged or released?**

A successful sandbox build proves technical feasibility only. It must not create authority to open or merge a Pull Request. Conversely, process drift or concurrent-work warnings must not make an otherwise valid technical build appear broken.

Decision

1. Explicit human authorization before every PR

ChatGPT and other interactive agents **MUST** request explicit user authorization before creating every new Pull Request.

The authorization is:

- single-use;
- specific to the described PR scope;
- not inherited from a previous PR, branch, build, test, merge or general instruction;
- invalidated by a material scope expansion.

Before requesting authorization, the agent **SHOULD** summarize the intended scope, known risks, available technical validation, production drift and concurrent PR overlap.

2. Remove the 15-minute PR SLA

The claim-to-PR 15-minute deadline is deleted. No elapsed-time rule forces PR creation, invalidates a branch, or fails CI.

3. Production baseline becomes advisory process evidence

Production-state inspection remains a useful diagnostic and release-safety input, but it is removed from mandatory sandbox/build authorization.

Production drift **MUST** be surfaced to the human operator. The human decides whether to rebase, rescope, sequence, defer or proceed.

4. Work-claim / single-writer overlap becomes advisory process evidence

Changed-file and optional work-claim overlap checks remain useful for multi-agent coordination, but they no longer fail technical CI merely because another PR lacks a claim or has overlapping scope.

Detected conflict **MUST** be disclosed before PR creation or merge. Resolution choices include rescoping, sequencing, waiting, superseding duplicates or explicitly accepting the coordination risk.

5. Technical CI is restricted to candidate integrity

The PR technical-validation pipeline may gate on:

- dependency installation;
- production dependency vulnerability audit;
- TypeScript/type checking;
- automated tests;
- production build;
- deployment-readiness verification;
- workflow-security verification;
- other directly technical security/compliance checks.

A green technical pipeline is necessary evidence but never constitutes PR-creation or merge approval.

6. Merge/release authority remains human where required

This ADR does not weaken human/CODEOWNER approval for architecture, security, billing, IAM or other protected/high-impact changes. It separates that decision from sandbox build mechanics.

Consequences

Positive

- Build failures represent actual technical failures instead of workflow/process disagreements.
- ChatGPT cannot silently turn a successful test into a new PR.
- Parallel-agent conflicts remain visible without blocking independent technical validation.
- Production drift remains available for release decisions without contaminating build status.
- The workflow is easier to reason about and less prone to deadlock when multiple agents operate concurrently.

Trade-offs

- Work claims become advisory rather than hard exclusive locks.
- Human operators must actively evaluate disclosed overlap and drift.
- Repository rulesets/CODEOWNERS remain important for final protected merges.

Implementation

- `.github/workflows/pr-governance.yml` becomes a technical-validation workflow only.
- `AGENTS.md` defines explicit user authorization before every PR.
- `scripts/pr/createWorkClaim.mjs` removes PR-deadline language.
- `scripts/pr/validateWorkClaim.mjs` becomes an advisory coordination report rather than a failing CI gate.
- `.github/pull_request_template.md` records explicit PR-creation authorization and states that build evidence is non-authorizing.
- `scripts/pr/productionPreflight.mjs` remains available as an on-demand diagnostic/release evidence tool.

Non-goals

This ADR does not authorize direct writes to protected production configuration, remove least-privilege workflow controls, remove CODEOWNER review, weaken secret handling, or allow models to self-approve protected changes.

Rollback

If this governance model is reverted, the replacement must preserve the distinction between technical validation and human authorization. Reintroducing a time-based automatic PR creation requirement is explicitly rejected unless a future ADR documents a new human-approved rationale.

ADR-0040 — CSP Runtime Remediation and Safe Strict-Policy Rollout

Quelle: docs/adr/ADR-0040-csp-runtime-remediation-safe-rollout.md

ADR-0040 — CSP Runtime Remediation and Safe Strict-Policy Rollout

- **Status:** Accepted
- **Implementation status:** In progress / remediation
- **Date:** 2026-08-04
- **Version:** 0.6.0
- **Owner:** Platform Director / Security & Compliance
- **Related:** ADR-0035, ESS-0014, ADR-0009, ADR-0012, ADR-0036
- **Protected change:** Yes

Context

After deployment of the ADR-0035 per-response nonce implementation, `capital-ai.online` could return a successful HTML response while the browser displayed only the dark application background. Render build and process startup were healthy, the custom domains and TLS certificate were verified, and the production service returned successful HTTP responses. The failure pattern was therefore consistent with a browser bootstrap failure rather than DNS or process unavailability.

ADR-0035 introduced all of the following at once:

- a nonce placeholder on every authorized script element;
- a cryptographically random nonce per response;
- `strict-dynamic` as an immediately enforced production policy;
- response-method interception for HTML nonce replacement;
- removal of HTML conditional-cache headers;
- AMP Auto Ads hooks inside the normal React/Vite SPA.

The accompanying unit test used a synthetic response object and a hand-written HTML string. It did not exercise the real chain:

```
Vite production build
-> dist/index.html
-> Express static delivery / SPA fallback
-> nonce body transformation
-> CSP headers
-> /assets/index-*.js availability
-> browser application bootstrap
```

The governance guard verified that required strings existed in source files, but did not prove that the built application bundle remained executable. This created a governance inversion: the repository protected the presence of an implementation more strongly than its production behavior.

ADR-0035 already stated that its implementation remained in progress until production evidence existed. This ADR preserves the security intent while correcting the rollout and validation model.

Decision

1. Three explicit production CSP modes

The server supports one of three values through the optional server-side environment variable `CSP_MODE`:

```
baseline
report-only
strict
```

An unknown or missing value fails safely to `report-only`.

`baseline`

- Enforce an availability-safe production CSP.
- Explicitly allow the first-party Vite/React bundle through `script-src 'self'`.
- Keep object execution disabled and base-tag injection blocked.
- Keep CookieHub, Google Tag Manager, the AdSense loader and Stripe script origins explicitly constrained.
- Do not emit the strict report-only policy.

This mode is the emergency recovery state.

`report-only` — production default

- Enforce the same availability-safe baseline policy.
- Emit the ADR-0035 nonce + `strict-dynamic` target through `Content-Security-Policy-Report-Only`.
- Inject the same cryptographic nonce into the HTML and both policy variants.
- Preserve `Cache-Control: no-store` for HTML.

This mode restores application availability while collecting browser evidence for strict promotion.

`strict`

- Enforce the ADR-0035 nonce + `strict-dynamic` target policy.
- Do not emit a parallel report-only header.
- Permit promotion only after the gate below is satisfied.

2. Report-Only Promotion Gate

`CSP_MODE=strict` must not be enabled in production until all of the following evidence exists against the actual production candidate:

1. `GET /` returns `200`, never a nonce-mismatched `304` HTML response.
2. The built HTML contains no `__CSP_NONCE__` placeholder.
3. All authorized script elements carry the same nonce as the strict CSP header.
4. The generated `/assets/index-*.js` entry asset returns `200` with a JavaScript content type.
5. The React root renders visible content in Chromium desktop and Android/mobile viewport tests.
6. No `securitypolicyviolation` blocks a first-party application asset.
7. CookieHub renders and remains the consent source of truth.
8. Analytics remains consent-gated.
9. AdSense behavior is tested separately from core application availability.
10. Production evidence and rollback readiness are attached to the protected change.

A green TypeScript compile or Vite build alone is insufficient promotion evidence.

3. First-party application availability is a protected invariant

The following invariant is added alongside the existing marketing/security invariants:

A Google Marketing or CSP change must never prevent the first-party CAPITAL-AI application bundle from bootstrapping without an explicit, tested maintenance response.

The governance guard must therefore verify both security controls and the built delivery path.

4. AMP Auto Ads are removed from the non-AMP SPA

ESS-0014 states that AMP Auto Ads are valid only on a separately validated AMP document. The ordinary React/Vite `index.html` is not an AMP document.

Therefore:

- the AMP extension script is removed from the SPA shell;
- the `<amp-auto-ads>` element is removed from the SPA body;
- the normal AdSense loader remains;
- any future AMP page requires a dedicated route/document, AMP validation and separate evidence.

The protected-change guard is inverted accordingly: it now rejects reintroduction of `amp-auto-ads` into the normal SPA.

5. Production-path integration test becomes mandatory

The Google Marketing protected workflow must build the real production bundle and then execute a test using:

```
requestContext
-> express.static(dist)
-> real dist/index.html
-> CSP/nonce transformation
-> real /assets/*.js request
```

The test must verify:

- enforced baseline versus strict report-only headers;
- nonce equality between report-only policy and generated HTML;
- absence of the placeholder;
- no-store HTML behavior;
- no conditional 304 response for HTML;
- successful retrieval of the built Vite JavaScript asset.

6. CSP authority and legacy server entry point

`server/securityResponse.ts` remains the authoritative CSP response boundary during this remediation. It normalizes the enforced and report-only CSP headers so the older setter in `server.ts` cannot silently replace the selected policy.

This is a transitional compatibility decision, not the desired final composition. The repository currently has an active runtime-remediation writer on `server.ts`; this PR deliberately avoids taking over that file.

During the next coordinated server-composition reconciliation:

- remove the legacy CSP construction from `server.ts`;
- mount one explicit CSP/HTML delivery component;
- avoid maintaining two independently editable policy definitions;
- retain ADR-0040 modes and promotion evidence.

That follow-up must be performed only after the active `server.ts` writer scope is merged, closed or explicitly superseded.

7. Workflow supply-chain hardening

Any modified protected-marketing workflow must:

- use explicit minimum permissions;
- use immutable full commit SHAs for third-party Actions;
- set `persist-credentials: false` on checkout;
- retain concurrency cancellation;
- build before the production-path integration test.

Security consequences

Positive

- A nonce or strict-policy regression no longer blanks the application by default.
- The strict policy remains measurable and promotable rather than being removed.
- The first-party bundle has an explicit availability contract.
- AMP and non-AMP responsibilities are corrected.
- The real Vite/Express delivery chain becomes test evidence.
- CSP mode is visible through a non-secret response header.

Trade-offs

- `report-only` temporarily enforces a less restrictive script policy than the ADR-0035 target.
- The baseline retains the documented `unsafe-eval` provider-compatibility trade-off.
- The existing HTML response transformation remains until the server composition is reconciled.
- Browser report-only evidence is useful only when operational collection/inspection is performed.

Privacy and consent impact

This ADR does not authorize analytics or advertising before consent. CookieHub remains the consent source of truth, and the existing first-party consent bridge remains protected.

Removing invalid AMP hooks does not remove normal AdSense integration. It prevents a non-AMP document from pretending to support AMP Auto Ads.

Production handoff

Merging this PR requires human/CODEOWNER review because it changes a protected CSP invariant.

No Render environment mutation is required to restore the default behavior: missing `CSP_MODE` resolves to `report-only` in code.

Future production changes follow this sequence:

```
report-only
-> collect and review production evidence
-> strict candidate test
-> explicit protected-change approval
-> set CSP_MODE=strict
-> post-deploy verification
```

Emergency rollback sequence:

```
strict
-> CSP_MODE=report-only
```

```
-> if required CSP_MODE=baseline  
-> preserve logs/evidence  
-> remediate through a reviewed PR
```

Environment changes on Render remain a production handoff task and must not be silently performed by development automation.

Validation / Definition of Done

This ADR is complete only when:

- unit tests cover all CSP modes and invalid-value fallback;
- the production-path test passes after a real Vite build;
- the protected invariant guard rejects AMP hooks in the SPA;
- workflow action references are immutable;
- the Draft PR records production baseline and risk evidence;
- human/CODEOWNER approval is obtained;
- the deployed application visibly boots on `capital-ai.online`;
- strict mode remains disabled until the Report-Only Promotion Gate is satisfied.

Rollback governance

A rollback that removes nonce, consent, audit or strict-policy promotion controls is still a protected change under ADR-0035 and ESS-0014.

Switching from `strict` to `report-only` or `baseline` during an availability incident is an approved recovery mechanism defined by this ADR, but it must be logged, evidenced and followed by remediation. It is not permission to permanently abandon the strict target.

Architectural Decision Record (ADR-0041)

Quelle: docs/adr/ADR-0041-enterprise-market-data-provider-and-mcp-architecture.md

Architectural Decision Record (ADR-0041)

Enterprise Market Data Provider & MCP Architecture

Status: ACCEPTED

Implementation-Status: PHASE 3C CONTRACT AND ADAPTER IMPLEMENTED / SCORING WIRING NOT AUTHORIZED / ALPACA PROMOTION NOT AUTHORIZED

Date: 2026-08-03

Version: 0.6.0

Priority: P1

Related ESS: ESS-0016

Related ADR: ADR-0009, ADR-0012, ADR-0013, ADR-0018, ADR-0035, ADR-0037, ADR-0040

1. Kontext

CAPITAL-AI verarbeitet externe Finanzmarktdaten für Screening, Scoring, Backtesting, Monte-Carlo-Simulationen, Klassifikation, Explainability und AI-gestützte Analysen.

Die bestehende Backend-Architektur besitzt bereits serverseitige Market-Data-Zugriffe, Caching, einen Request-Orchestrator, Asset-spezifische Scoring-Services und mehrere externe Provider. Gleichzeitig wird die Plattform um agentische MCP-Integrationen erweitert.

Mit dem in Render hinterlegten Alpaca-Credential entsteht die Möglichkeit, Alpaca als produktiven Market-Data-Provider einzubinden. Zusätzlich steht ein offizieller Alpaca MCP Server V2 zur Verfügung. Alpha Vantage stellt ebenfalls einen offiziellen Remote-MCP-Server und ein breites Finanzdatenangebot bereit.

Ohne eine explizite Architekturentscheidung entstehen jedoch folgende Risiken:

- direkte Kopplung von Scoring-Code an Provider-Schemas;
- Vendor Lock-in;
- inkonsistente Datenqualität und Zeitstempel;
- unkontrollierte MCP-Tool-Berechtigungen;
- versehentliche Freigabe von Trading-/Order-Funktionen;
- Provider-Ausfälle mit stillen Demo-/Synthetic-Fallbacks;
- uneinheitliche Rate-Limit- und Cache-Strategien;
- fehlende Provenance und Auditierbarkeit;
- ungeprüfte Datenlizenz-/Redistributionsannahmen;
- Alpha-Vantage-Quota als versteckter Single Point of Failure.

2. Problem Statement

Die Plattform benötigt zwei unterschiedliche Integrationsarten:

- A. deterministische Runtime Data Plane
- B. agentische MCP Tool Plane

Diese beiden Pfade haben unterschiedliche Sicherheits-, Latenz-, Determinismus- und Audit-Anforderungen.

Würde ein LLM/MCP direkt zum Kern der Scoring-Pipeline gemacht, könnten Toolauswahl, Agentenverhalten, Response-Transformationen oder Provider-Tooländerungen den deterministischen Berechnungspfad beeinflussen.

Würde dagegen jeder Business-Service direkt einen bestimmten Provider ansprechen, entstünden Duplikate für Authentication, Retry, Rate Limits, Caching, Normalisierung, Fallback, Data Quality und Observability.

3. Entscheidung

CAPITAL-AI führt einen **providerneutralen Market Data Provider Gateway** als verbindliche Integrationsgrenze ein.

3.1 Provider-Rollen

```
Alpaca
  = Primary Market Data Provider

Alpha Vantage
  = Secondary / Enrichment / Research Provider

Alpaca MCP
  = Governed Read-only Agent Market-Data Plane

Alpha Vantage MCP
  = Governed Research / Enrichment Agent Plane
```

3.2 MCP ist nicht die Runtime-Pipeline

Produktive Screening-, Scoring-, Backtest- und Monte-Carlo-Services beziehen Daten über typisierte Provider-Adapter und den Market Data Gateway.

```
Business Logic
  -> MarketDataGateway
  -> ProviderAdapter
  -> Provider API / Stream
```

Nicht zulässig als Standardpfad:

```
Business Logic
  -> LLM
  -> MCP
  -> Market Data
```


MCP darf ergänzende Research-/Evidence-Daten liefern, aber keine ungeprüfte agentische Ausgabe als authoritative Marktpreis oder deterministischen Score einschleusen.

3.3 P1 Phase-1 implementation boundary (2026-08-14)

Phase 1 introduces the provider-neutral canonical snapshot contract, data-quality assessment, explicit provider registry, fail-closed gateway and Alpaca REST adapter under `src/platform/MarketData/`.

Alpaca remains registered with role `shadow`. The gateway excludes shadow providers unless the caller explicitly sets `includeShadow: true`; no existing quote, screening or scoring call site is migrated by this phase. Therefore the architectural target "Alpaca primary" remains a future promotion decision subject to the documented 14-trading-day / 1,000-observation evidence gate, entitlement review and separate Owner authorization.

3.4 P2 Phase-2 gateway resilience boundary (2026-08-14)

Phase 2 adds explicit provider routing with bounded server-side caching, identical-request coalescing, provider/capability rate-limit budgets, circuit-breaker isolation and structured payload-free telemetry. Cached observations are revalidated against the caller's freshness contract before use; expired or unacceptable observations are evicted and never converted into synthetic data.

This phase remains outside the active scoring/screening composition root. Alpaca remains `shadow`; provider promotion, entitlement approval and pipeline migration require separate evidence and Owner authorization.

3.5 P3A traditional quote migration boundary (2026-08-15)

The existing Twelve Data stock/forex quote path now enters through the canonical `Market-DataGateway` and a typed `TwelveDataMarketDataProvider`. The legacy `traditional-quote/1.0.0` response remains a compatibility boundary for current consumers, while freshness, provenance, cache, coalescing, rate-limit and circuit-breaker controls are enforced centrally.

Alpaca continues to run as independent score-neutral shadow evidence. Index quotes, historical series, fundamentals and scoring-input migrations remain outside P3A and require later bounded work packages.

3.6 P3B index quote migration boundary (2026-08-15)

FMP index quotes now enter the canonical gateway through an injected read-only loader adapter. The platform provider does not import server internals; existing approved ticker mappings, FMP cooldown and cache remain in the server composition boundary. The legacy Traditional Quote contract remains compatible and continues to deny execution-price eligibility.

Index histories, stock fundamentals and scoring-input generation remain outside P3B. Alpaca remains score-neutral shadow evidence and is not promoted.

3.7 P3C index history contract boundary (2026-08-15)

P3C introduces the versioned `market-data-history/1.0.0` contract, an independent history provider registry, a fail-closed history gateway and an injected FMP index-history adapter. The platform layer receives no credentials and imports no server internals. Invalid, empty or mixed invalid series are denied; shadow providers remain excluded unless explicitly requested.

P3C intentionally does not wire the gateway into `server/fmpIndices.ts`, scoring, alerts, backtesting or other runtime consumers. That activation is a separate Class-R/scoring-impact change requiring explicit human authorization, compatibility evidence and rollback planning.

4. Alpaca wird Primary Provider

Alpaca wird für unterstützte Asset-Klassen als bevorzugte Quelle vorgesehen, insbesondere für US Equities, historische und aktuelle Bars, Quotes, Trades, Snapshots, Crypto Market Data, Options Market Data, Option Chains, Corporate Actions und später optional WebSocket Streaming.

Die tatsächliche Verfügbarkeit einzelner Feeds und Echtzeitqualitäten hängt vom Alpaca-Plan, den Entitlements und Datenlizenzen ab und wird nicht durch das Vorhandensein eines API-Keys implizit angenommen.

5. Alpha Vantage wird Secondary / Enrichment Provider

Alpha Vantage wird nicht als alleinige zentrale Runtime-Abhängigkeit festgelegt.

Primäre Anwendungsfälle sind Fundamentals, FX, Commodities, globale Equity-Ergänzung, technische Indikatoren als Referenz, Historical Research, Cross-Provider Validation und MCP-basierte Agent-Recherche.

Tarif- und Quota-Grenzen dürfen nicht still als ausreichende Kapazität für eine produktive Multi-User-SaaS-Runtime behandelt werden.

6. Provider Gateway

Zielstruktur:

```
src/platform/MarketData/  
  contracts/  
  providers/  
    ProviderRegistry.ts  
    ProviderRouter.ts  
    MarketDataGateway.ts
```

```
DataNormalizer.ts  
DataQualityService.ts
```

Der Gateway übernimmt mindestens Capability Routing, Provider Health, Timeout, Retry, Circuit Breaker, serverseitigen Cache, Request Coalescing, Rate-Limit Budgets, Fallback, Provenance, Freshness, Data Quality und Observability.

Business-Services dürfen keine Provider-spezifische Response-Struktur als Domain Contract verwenden.

7. Canonical Market Data

Jeder produktiv verwendete Datensatz erhält mindestens:

```
provider  
providerFeed  
symbol  
assetClass  
currency  
sourceTimestamp  
ingestedAt  
freshnessMs  
qualityState  
isRealtime  
isDelayed  
correlationId
```

Provider-spezifische Zusatzfelder dürfen gespeichert werden, aber Domain-Logik darf nicht davon abhängig sein, sofern sie nicht über einen expliziten Capability Contract modelliert sind.

8. No Demo Data

Bei Ausfall gilt ausschließlich:

```
fresh approved secondary provider  
OR explicit stale cache with STALE marker  
OR DEGRADED / UNAVAILABLE
```

Nicht zulässig:

```
random / synthetic / simulated market price  
-> presented as production market data  
-> score / decision evidence
```

Damit wird die No Demo Data Policy auf die Market-Data-Infrastruktur explizit ausgedehnt.

9. Alpaca MCP — Read-only Entscheidung

CAPITAL-AI genehmigt im Rahmen dieses ADR nur Marktdaten-/Research-Capabilities, zum Beispiel:

```
assets
stock-data
crypto-data
options-data
corporate-actions
news
```

Das Toolset `trading` sowie Order-, Account- und Portfolio-Mutationen werden **nicht** genehmigt. Eine spätere Trading-Funktion benötigt ein eigenes ADR und darf nicht als Erweiterung dieses ADR interpretiert werden.

10. Alpha Vantage MCP

Der Alpha-Vantage-MCP-Server darf als kontrollierter Agent-/Research-Provider eingesetzt werden. Die MCP-Verbindung wird logisch hinter Supervisor-/Policy-Gates betrieben. Tool-Nutzung muss Rate-Limits, Data Provenance und Audit Logging respektieren.

Die MCP-Integration begründet keine automatische Runtime-Abhängigkeit. Runtime-Nutzung erfolgt über einen expliziten `AlphaVantageProvider` Adapter.

11. Security Boundary

Provider-Secrets verbleiben serverseitig.

Erwartete Secret-Klassen:

```
ALPACA_API_KEY
ALPACA_SECRET_KEY
ALPHA_VANTAGE_API_KEY
```

Regeln:

1. keine Provider-Secrets im Frontend;
2. keine `VITE_*` Provider-Secrets;
3. keine Secrets in Logs, Metrics oder Audit-Evidence;
4. Credential-Rotation ohne Codeänderung;
5. fehlende Credentials führen zu Provider `UNAVAILABLE` ;
6. kein Fallback auf Demo-/Synthetic-Daten;
7. MCP-Hosts erhalten nur erforderliche Credentials und Toolsets;
8. API-Key-Konfiguration ersetzt keine IAM-, Entitlement- oder Lizenzprüfung.

Render bleibt Production Control Plane für die dort gespeicherten Secrets. Dieses ADR autorisiert keine Offenlegung oder Verschiebung bestehender Secret-Werte.

12. IAM-Entscheidung

MCP-Market-Data-Zugriffe werden als explizite Capabilities modelliert.

```
marketdata.alpaca.read  
marketdata.alphavantage.read  
marketdata.provider.health.read  
marketdata.provider.config.read  
marketdata.provider.config.plan
```

Nicht eingeführt werden Trading-, Order-Cancel- oder Portfolio-Mutations-Capabilities. Ein Agent darf keine eigene Capability erweitern.

13. Data Licensing / Entitlements

Vor Produktionsfreigabe eines Feeds werden Realtime-/Delayed-Status, Feed/Exchange, Speicherung, Display, Redistribution, Nutzerzahl, geografische Einschränkungen, Tariflimits und API-/WebSocket-Entitlements geprüft.

Die Anwendung darf den Marketingbegriff „real-time“ nur verwenden, wenn Datenquelle und Lizenz dies für den betreffenden Datensatz rechtfertigen.

14. Cross-Provider Validation

Provider-Fallback und Provider-Consensus sind getrennte Funktionen.

```
Primary unavailable  
-> Secondary provider  
-> provenance preserved
```

```
Primary value + reference value  
-> divergence calculation  
-> confidence / degraded signal
```

Eine Abweichung wird nicht still durch Durchschnittsbildung versteckt.

15. Streaming-Entscheidung

Alpaca WebSocket Streaming wird als spätere Erweiterungsphase zugelassen, aber nicht allein durch dieses ADR produktiv aktiviert.

Vor Aktivierung erforderlich sind Plan-/Entitlement-Nachweis, Reconnect-Strategie, Back-pressure, Connection Limits, bounded fan-out, Event Ordering, Duplicate Handling, Freshness Monitoring, Graceful Shutdown, Render Runtime Impact und Observability.

16. Observability

Mindestsignale:

```
market_data_requests_total
market_data_errors_total
market_data_cache_hit_total
market_data_rate_limit_total
market_data_fallback_total
market_data_stale_total
market_data_unavailable_total
market_data_provider_latency_ms
market_data_provider_divergence
```

Provider-Fehler werden strukturiert klassifiziert, etwa AUTH, ENTITLEMENT, RATE_LIMIT, TIMEOUT, UPSTREAM_4XX, UPSTREAM_5XX, MALFORMED_DATA, STALE_DATA, NETWORK und UNKNOWN.

17. Auswirkungen auf bestehende Architektur

Positive Auswirkungen sind geringere Provider-Kopplung, bessere Testbarkeit, zentrale Datenqualität, klare Provenance, kontrollierbare Fallbacks, MCP-Least-Privilege, bessere Explainability und No-Demo-Data-konformes Failure Handling.

Akzeptierte Kosten sind zusätzlicher Gateway-/Adapter-Code, Provider-Health- und Quota-Logik, komplexere Observability sowie Lizenz-/Entitlement-Evidence im Production Handoff.

18. Nicht gewählte Alternativen

- **Direkte Alpaca-Integration in jeden Scoring-Service:** abgelehnt wegen Vendor Lock-in und duplizierter Fehlerbehandlung.
- **Alpaca MCP als alleiniger Datenpfad:** abgelehnt, da agentische Tool-Nutzung kein deterministischer Kern für Screening, Backtesting und Scoring ist.
- **Alpha Vantage als einziger Primary Provider:** abgelehnt wegen unterschiedlicher Datenstärken und tarifabhängiger Limits.
- **Simulierte Kurse bei Provider-Ausfall:** abgelehnt wegen Verletzung der No Demo Data Policy.
- **Vorsorgliche Aktivierung von Trading Tools:** abgelehnt wegen unnötiger Capability-Ausweitung und regulatorischem Blast Radius.

19. Implementierungsreihenfolge

1. Market Data Contracts und Canonical DTO.
2. Alpaca Adapter und Environment-Validierung.
3. Gateway, Registry, Routing, Cache, Quota, Circuit Breaker und Metrics.

4. Migration direkter Market-Data-Aufrufe.
 5. Read-only Alpaca MCP hinter Supervisor-/Policy-Gate.
 6. Alpha-Vantage-Adapter und MCP Research Plane.
 7. Optionales Streaming nach eigenem Production Gate.
-

20. Production Gate

Vor der technischen Produktionsfreigabe müssen mindestens folgende Evidence-Artefakte vorhanden sein:

1. aktive Provider-Capabilities;
 2. verwendete Feed-/Entitlement-Klasse;
 3. Secret-Namen vollständig, Werte nicht dokumentiert;
 4. Health-/Failure-Test;
 5. Rate-Limit-Test;
 6. Cache-/Freshness-Test;
 7. Fallback-Test;
 8. No-Demo-Data-Negativtest;
 9. MCP Tool Inventory;
 10. Nachweis, dass Alpaca Trading Tools nicht exponiert sind;
 11. Alpha-Vantage-Quota-Konfiguration;
 12. Data Licensing/Redistribution Review;
 13. Traceability ESS-0016 ↔ ADR-0041 ↔ Implementation ↔ Tests.
-

21. Verifikation externer Provider-Eigenschaften

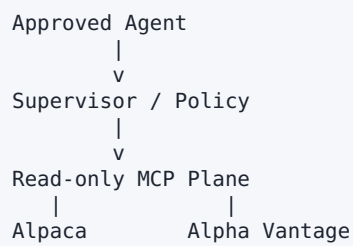
Provider-Funktionen, Preise, Entitlements und Limits sind externe Verträge und müssen vor produktionskritischen Änderungen erneut gegen offizielle Anbieterinformationen verifiziert werden.

22. Ergebnis

CAPITAL-AI entscheidet sich für eine **hybride, providerneutrale Market-Data-Architektur**:

```
Alpaca REST / Stream
  |
  v
Market Data Gateway
  |
  v
Deterministic FinTech Data Pipeline
```

plus getrennt:



Damit wird Alpaca zum bevorzugten Market-Data-Provider, ohne die Plattform an Alpaca zu koppeln. Alpha Vantage ergänzt Fundamentals, FX, Commodities, Research und Provider-Validation. MCP erweitert die Agentenfähigkeiten, ohne den deterministischen Kern der FinTech Data Pipeline oder die Security Boundary zu ersetzen.

Architectural Decision Record (ADR-0042)

Quelle: docs/adr/ADR-0042-basic-consent-mode-v2-google-tag-gating.md

Architectural Decision Record (ADR-0042)

Basic Consent Mode v2 and consent-gated Google tag delivery

Status: PROPOSED

Implementation-Status: PR VALIDATION

Date: 2026-08-04

Version: 0.6.0

Related ESS: ESS-0014 / ESS-0014-CONTRACTS

Related ADR: ADR-0035, ADR-0040

Trigger: WebscanRadar Security Audit `capital-ai.online`, 2026-08-04

1. Kontext

Der externe Webscan meldet zwei miteinander verbundene High-Findings:

1. kein erkanntes Cookie-Consent-Tool trotz aktiver Tracking-Skripte;
2. Google Analytics ohne Consent-Gate.

Die Repository-Prüfung zeigt zusätzlich zwei technische Driftpunkte:

- der normale AdSense-Loader wird in `index.html` unmittelbar und damit unabhängig von der CookieHub-Kategorie `marketing` geladen;
- `public/google-analytics-consent.js` registriert CookieHub-Custom-Events auf `window`, während die dokumentierte CookieHub-JavaScript-API diese Events auf `document` emittiert.

Damit ist die beabsichtigte Invariante aus ESS-0014 nicht hinreichend technisch abgesichert. Die Dokumentation in `docs/runbooks/GOOGLE_ANALYTICS_SETUP.md` widerspricht außerdem der bereits akzeptierten ESS-0014-Vorgabe, nach der **Basic Consent Mode v2** der CAPITAL-AI-Standard ist.

2. Entscheidung

2.1 Consent Source of Truth

CookieHub bleibt die einzige fachliche Quelle für die Kategorien:

- `necessary`;
- `analytics`;
- `marketing`;

- optional `preferences` .

Der Repository-Code ersetzt CookieHub nicht. Er bildet dessen Entscheidung deterministisch auf die Google-Runtime ab.

2.2 Basic Consent Mode v2

Vor jeder möglichen Google-Tag-Ausführung werden lokal, ohne Netzwerkanfrage, folgende Defaults in `dataLayer` gesetzt:

```
analytics_storage = denied
ad_storage        = denied
ad_user_data      = denied
ad_personalization = denied
functionality_storage = denied
personalization_storage = denied
security_storage  = granted
```

Zero Google network before opt-in ist die verbindliche Privacy-Invariante. CAPITAL-AI verwendet Basic Consent Mode v2: Google-Tags werden nicht vor Zustimmung geladen; es werden auch keine cookieless measurement pings als Advanced-Mode-Ersatz zugelassen.

2.3 Tag Delivery

`index.html` darf keine ausführbaren GA4- oder AdSense-Third-Party-Loader mehr enthalten. Öffentliche IDs werden ausschließlich als inert metadata bereitgestellt:

- `ga-measurement-id` ;
- `adsense-publisher-id` .

Der First-Party-Bridge `public/google-analytics-consent.js` lädt anschließend:

- GA4 nur bei `cookiehub.hasConsented('analytics') === true` ;
- AdSense nur bei `cookiehub.hasConsented('marketing') === true` .

Jeder Loader ist idempotent und trägt `data-consent-managed="true"` .

2.4 Event Contract

CookieHub-Ereignisse werden gemäß Provider-API auf `document` beobachtet:

```
cookiehub_onInitialise
cookiehub_onStatusChange
cookiehub_onAllow
cookiehub_onRevoke
```

Zusätzlich erfolgt nach `DOMContentLoaded` eine fail-closed Readiness-Prüfung. Ist CookieHub nicht verfügbar oder die Consent-Abfrage fehlerhaft, werden keine Google-Tags geladen.

2.5 Widerruf

Bei Widerruf von `analytics` :

- wird `ga-disable-<MEASUREMENT_ID>` gesetzt;

- werden erreichbare First-Party- `_ga` -/ `_ga_*` -Cookies gelöscht;
- werden Consent-Mode-Signale auf `denied` aktualisiert.

Da bereits ausgeführte Third-Party-Skripte nicht zuverlässig aus einem laufenden Dokument entfernt werden können, wird nach einem persistierten Widerruf einmalig neu geladen. Der neue Dokumentkontext startet wieder mit `denied` Defaults und ohne Google-Netzwerkzugriff.

2.6 Keine konkurrierende Consent-Implementierung

Während der First-Party-Bridge aktiv ist, darf weder ein zusätzlicher GTM-Consent-Tag noch eine zweite manuelle Consent-Mode-Implementierung dieselben Signale verwalten. Eine CookieHub-Dashboard- Option darf nur aktiviert werden, wenn sie nachweislich keine doppelte oder widersprüchliche Runtime erzeugt. Ein Wechsel auf die vollständig providerverwaltete CookieHub-Integration ist eine spätere geschützte Architekturänderung.

2.7 Protected Change

Die Änderung betrifft geschützte Google-Marketing-Invarianten aus ESS-0014 und ADR-0035. Merge/Production-Handoff benötigen daher Human-/CODEOWNER-Review sowie die dort definierten Approval- und Evidence-Gates. Dieser PR führt keine externe Google-, CookieHub- oder Render-Mutation aus.

3. Begründung

Warum Basic statt Advanced Consent Mode?

Basic Mode erfüllt die strengere CAPITAL-AI-Invariante, dass vor Opt-in keine Google-Verbindung entsteht. Advanced Mode kann bei `denied` Consent cookieless pings senden und würde deshalb eine neue Datenschutz- und Rechtsbewertung erfordern.

Warum sowohl GA4 als auch AdSense gaten?

Ein Analytics-Gate allein beseitigt das Webscan-Problem nicht, solange der AdSense-Loader bereits im HTML ausgeführt wird. Beide Google-Produkte müssen an getrennte, zweckbezogene Kategorien gebunden sein.

Warum `document` statt `window` ?

CookieHub dokumentiert die Custom Events ab Version 2.8.13 als `document.addEventListener(...)` - Events. Die bisherige `window` -Registrierung ist daher kein belastbarer Provider-Contract.

Warum Reload nach Widerruf?

Consent-Signale stoppen zukünftige erlaubte Verarbeitung, können aber bereits geladenen JavaScript- Code und bestehende Auto-Ad-Runtime nicht vollständig aus dem DOM/Execution Context entfernen. Ein Reload stellt den fail-closed Initialzustand technisch wieder her.

4. Konsequenzen

Positive Konsequenzen

- keine unbedingten Google-Third-Party-Skripte im HTML;
- vollständige Consent-Mode-v2-Signale für Analytics und Advertising;
- getrennte Zweckbindung von Analytics und Marketing;
- robuster Initialstatus für wiederkehrende Besucher;
- technisch einfacher Widerruf über den vorhandenen CookieHub-Settings-Link;
- statische Invarianten und verhaltensbasierte Tests verhindern Regressionen;
- bessere Nachweisbarkeit gegenüber Security-/DSGVO-Scannern.

Trade-offs

- GA4 und AdSense messen ohne Opt-in nichts;
- ein Widerruf löst einen Seitenreload aus;
- CookieHub bleibt eine externe Laufzeitabhängigkeit;
- Dashboard-/Domain-/Kategorie-Konfiguration kann nicht allein durch Repository-Code validiert werden;
- AdSense in EWR/UK/CH kann zusätzliche Google-CMP-/IAB-TCF-Anforderungen haben, die im externen Production Handoff geprüft werden müssen.

5. Externer Production Handoff

Folgende Schritte sind außerhalb des Entwicklungs-Repositorys durch den Repository-Owner zu prüfen:

1. CookieHub-Domain-Code `75f66920` gehört zur verifizierten Produktionsdomain `capital-ai.online`;
2. Kategorien `analytics` und `marketing` existieren exakt unter diesen IDs;
3. GA4 ist `analytics`, AdSense ist `marketing` zugeordnet;
4. Ablehnen ist ebenso leicht erreichbar wie Akzeptieren;
5. Consent-Protokollierung ist aktiv;
6. es existiert keine zweite GTM-/CookieHub-/Google-Consent-Mode-Implementierung;
7. für AdSense werden die aktuellen Google-CMP-/TCF-Anforderungen geprüft;
8. nach Merge wird ein neuer Render-Build ausgelöst, da die Vite-Measurement-ID build-time ist.

6. Verifikation / Definition of Done

Ohne Einwilligung

- CookieHub-Banner oder Preference Center ist sichtbar/erreichbar;
- kein Request an `googletagmanager.com`, `google-analytics.com`, `googlesyndication.com` oder `doubleclick.net`;
- keine `_ga` -/ `_ga_*` -Cookies;
- Consent Mode zeigt alle Analytics-/Ad-Felder `denied`.

Analytics akzeptiert, Marketing abgelehnt

- GA4-Loader wird genau einmal geladen;
- AdSense-Loader wird nicht geladen;
- `analytics_storage=granted`;
- alle Ad-Signale bleiben `denied`.

Marketing akzeptiert

- AdSense-Loader wird genau einmal geladen;
- `ad_storage`, `ad_user_data`, `ad_personalization` sind `granted`;
- Verhalten entspricht der im CookieHub-Banner beschriebenen Zweckbindung.

Widerruf

- Consent-Signale werden auf `denied` gesetzt;
- GA wird deaktiviert und erreichbare GA-Cookies werden gelöscht;
- der Seitenreload stellt einen Google-tag-freien Dokumentstart her.

Repository Gates

```
npx tsx scripts/security/verifyGoogleMarketingInvariants.ts
npx vitest run tests/unit/securityResponse.test.ts tests/unit/googleMarketingConsent.test.ts
npm run lint
npm run build
NODE_ENV=production CSP_MODE=report-only npx vitest run tests/unit/securityResponse.production.test.ts
```

Anschließend ist der Webscan mit leerem Browser-/Consent-Zustand zu wiederholen. Ein Scanner-Score allein ersetzt nicht die Browser-Netzwerk-Evidence.

7. Rollback

Ein Rollback, der GA4 oder AdSense wieder unconditionally lädt, Consent Mode v2 entfernt oder CookieHub-Events erneut falsch bindet, ist nach ESS-0014 / ADR-0035 ein protected change.

Zulässig ist ausschließlich:

1. Impact Disclosure;
 2. explizite Human-OWNER-Bestätigung;
 3. frischer Step-up bzw. gültiges Approval Artifact;
 4. Dry-run Diff;
 5. Post-change Browser-/Network-Evidence.
-

8. Provider References

- Google: <https://developers.google.com/tag-platform/security/guides/consent>
- Google Analytics: <https://support.google.com/analytics/answer/14275483>
- CookieHub Consent Mode v2: <https://docs.cookiehub.com/compliance/frameworks/google-consent-mode-v2>
- CookieHub JavaScript API: <https://docs.cookiehub.com/advanced/javascript-api>
- CookieHub Installation: <https://docs.cookiehub.com/installation/tag>
- TDDDG § 25: https://www.gesetze-im-internet.de/ttdsg/_25.html
- DSGVO Art. 7: <https://eur-lex.europa.eu/eli/reg/2016/679/oj>

ADR-0043 — Supabase Privilege Separation & Render Production Hardening

Quelle: docs/adr/ADR-0043-supabase-privilege-separation-render-production-hardening.md

ADR-0043 — Supabase Privilege Separation & Render Production Hardening

- **Status:** Accepted / Implementation in progress
- **Date:** 2026-08-04
- **Scope:** Production security, Supabase access model, billing/IAM persistence, Render deployment governance
- **Related:** ADR-0003.5, ADR-0008, ADR-0036, ADR-0040, Issue #92

Context

Issue #92 identified two production-hardening gaps:

1. `server/db.ts` selected one generic Supabase client from a credential fallback chain that mixed elevated `SUPABASE_SECRET_KEY / SUPABASE_SERVICE_ROLE_KEY` credentials with low-privilege publishable/anon credentials. This made server authorization and persistence semantics dependent on environment configuration.
2. The live Render `Finance` service did not match the repository `render.yaml` contract for deployment gating and the platform health check.

The security review also found legacy RLS policy patterns that targeted `PUBLIC` and then tested `auth.role() = 'service_role'`, a hard-coded owner e-mail in the `security_events` read policy, missing explicit client-deny policy on `screening_slo_evidence`, and unpinned actions in the general CI workflow.

Decision

1. Explicit Supabase client classes

The backend SHALL expose two distinct access contracts:

- **Privileged server client:** created only from `SUPABASE_SECRET_KEY` or legacy `SUPABASE_SERVICE_ROLE_KEY`; backend-only; elevated access; no publishable/anon fallback.
- **RLS-bound server client:** created only from publishable/anon credentials; intended for low-privilege/RLS semantics; no elevated-key fallback.

`getServerSupabase()` remains temporarily as a deprecated compatibility alias to the privileged client so existing IAM, billing, audit, quota and evidence call sites can be migrated without an unsafe all-at-once refactor. Its behavior is now unambiguously privileged.

2. Production persistence fails closed

Production subscription state SHALL NOT silently fall back to mutable container-local files.

- Privileged writes fail when privileged Supabase credentials are unavailable.
- Production subscription reads that cannot obtain authoritative database state fall back to the least-privileged `Free` tier, never a locally cached paid tier.
- Local subscription persistence remains development-only for compatibility.

3. RLS policy hardening

Service-only policies SHALL target `TO service_role` explicitly rather than granting a policy to `PUBLIC` and evaluating `auth.role()` per row.

Authenticated ownership policies SHALL use `(select auth.uid())` where semantically valid.

`security_events` owner visibility SHALL be determined by server-managed IAM state (`profiles.iam_role = 'owner'`) plus the existing `phone_verified` assurance gate. Authorization SHALL NOT depend on a hard-coded e-mail address.

`screening_slo_evidence` SHALL remain server-only and SHALL have an explicit restrictive deny policy for `anon` and `authenticated`, plus revoked direct table privileges for those roles.

4. Supply-chain and configuration governance

General CI actions SHALL be pinned to immutable commit SHAs, use read-only repository permissions, and avoid persisted checkout credentials.

A production configuration guard SHALL verify the repository invariants for:

- `autoDeployTrigger: checksPass`
- `healthCheckPath: /healthz`
- dedicated privileged Supabase secret declaration
- no publishable/anon fallback in privileged server access
- no production-local subscription authority
- pinned CI actions

`server/db.ts`, `render.yaml`, Supabase migrations, ADR-0043, and the relevant security tests SHALL be CODEOWNER-protected.

5. Render source-of-truth contract

`render.yaml` is the normative repository declaration for the `Finance` production service. The intended live state is:

- branch: `main`
- runtime: Docker
- auto deploy trigger: checks pass
- health check path: `/healthz`

If the Render management surface cannot mutate these service fields programmatically, the drift remains a production handoff item and Issue #92 must not be considered fully closed until live Render evidence matches the contract.

6. Supabase Auth leaked-password protection

Supabase Security Advisor findings are release evidence. Leaked-password protection SHOULD be enabled for password authentication. If the connected management API does not expose this Auth setting, it is an explicit dashboard handoff rather than a reason to bypass or suppress the advisor.

Consequences

Positive

- Privileged database operations can no longer silently downgrade to an RLS-bound key.
- Production billing state cannot be promoted from stale/mutable local container state.
- RLS intent is role-explicit and easier to review.
- Owner audit authorization no longer relies on an identity literal in SQL.
- CI supply-chain risk from mutable action tags is reduced.
- Repository drift for the Render/Supabase contract becomes fail-closed in CI.

Trade-offs

- Missing privileged Supabase credentials can now surface as production errors for privileged writes instead of silently persisting locally. This is intentional.
- Existing `getServerSupabase()` call sites still need gradual naming migration to `getPrivilegedServerSupabase()` for maximum code readability; security behavior is already enforced by the compatibility alias.
- Render live configuration and some Supabase Auth settings may require control-plane capabilities not exposed by the active connector.

Validation & Evidence

Required evidence before closing Issue #92:

1. Unit test proves publishable/anon credentials are never accepted as privileged configuration.
2. Unit test proves production subscription persistence fails closed without a privileged credential.
3. Supabase migration applies successfully.
4. Supabase Security Advisor no longer reports the `screening_slo_evidence` no-policy finding.
5. Supabase performance advisor no longer reports the remediated application RLS init-plan and public security FK findings.

6. GitHub CI, unit tests, lint, production build and predeploy checks pass.
7. Render deploy reaches `live` and `/healthz` remains operational.
8. Live Render service metadata is reconciled to `checksPass` and `/healthz`, or the unavailable control-plane mutation is explicitly recorded as the only remaining handoff.
9. Supabase leaked-password protection is enabled, or the unavailable Auth control-plane mutation is explicitly recorded as the only remaining Supabase handoff.

Rollback

- Reverting application code must not restore publishable/anon fallback for privileged operations.
- Database policy rollback requires an IAM/security review because broadening policies back to `PUBLIC` is a security regression.
- Render rollback may restore a prior application deploy, but must not weaken the declared health-check or checks-gated deployment contract.

ADR-0044 — Production Runtime Artifact Immutability

Quelle: docs/adr/ADR-0044-production-runtime-artifact-immutability.md

ADR-0044 — Production Runtime Artifact Immutability

- **Status:** Accepted
- **Implementation-Status:** COMPLETE — R-002 validated
- **Date:** 2026-08-08
- **Scope:** CAPITAL-AI production web runtime / Documentary / Version Manager / release governance
- **Platform Version:** 0.6.0
- **Roadmap:** R-002 — Make production documentary/release artifacts immutable
- **Related:** ADR-0030, ADR-0032, DC-005, DC-006

1. Context

The legacy production composition root still contains Document Hygiene and Version Manager implementation that historically could mutate repository-style documentation, local governance JSON state, generated release documents and runtime version state. It also starts a recursive Documentary watcher.

The legacy Version Manager embeds a 0.5.4 default state, while the governed package version is 0.6.0. Therefore blocking local persistence alone is insufficient: production release identity must be derived from immutable build evidence rather than mutable or fallback runtime state.

2. Decision

CAPITAL-AI establishes a fail-closed production runtime artifact and control-plane boundary.

The production web runtime MUST NOT:

1. mutate any path under docs/**;
2. mutate uploads/document_hygiene.json;
3. mutate uploads/version_manager.json;
4. establish an active filesystem watcher over docs/**;
5. execute legacy Documentary or release-governance mutation routes;
6. use local Version Manager JSON/default state as production release authority.

Protected production mutations return HTTP 409 with code:

READ_ONLY_CONTROL_PLANE_REQUIRED

Filesystem mutations on protected artifacts throw:

```
CAPITAL_AI_RUNTIME_ARTIFACT_READ_ONLY
```

The authoritative mutation owner is:

```
ci-or-authenticated-control-plane
```

3. Production enforcement

The production Docker image preloads `server/runtime/runtimeArtifactGuard.mjs` before `dist/server.cjs` when:

- `NODE_ENV=production`;
- `CAPITAL_AI_RUNTIME_ARTIFACT_MODE=readonly`.

The guard:

- blocks synchronous and promise-based filesystem mutations on protected paths;
- blocks protected rename/copy targets;
- suppresses `fs.watch()` / `fs.watchFile()` on `docs/**` with an inert watcher contract;
- rejects protected HTTP mutation routes before legacy Express handlers execute;
- serves read-only release identity for `GET /api/admin/version`;
- leaves ordinary application uploads and unrelated runtime routes untouched.

`/app/docs` is additionally permissioned `0555` for the non-root production user.

4. Immutable build evidence

Every production `npm run build` executes:

```
scripts/automation/buildRuntimeReleaseManifest.ts
```

and emits:

```
dist/control-plane/release-manifest.json
```

Contract:

```
capital-ai-runtime-release-manifest/1.0.0
```

The manifest records:

- governed `package.json.version`;
- source commit when available from controlled build/deploy metadata;
- SHA-256 of `package.json`;
- SHA-256 of `package-lock.json`;
- deterministic SHA-256 over the sorted Documentary tree under `docs/**` excluding `.history`;
- Documentary file count;

- deterministic `buildIdentity` derived from the release inputs;
- `authority: ci-or-controlled-build`;
- `mutable: false`.

If the manifest exists but violates its contract, production startup fails closed rather than silently trusting fallback release state.

`GET /api/admin/version` uses this manifest as the primary production release identity. Package/deploy metadata is retained only as a compatibility fallback when no manifest exists, for example in development probes or older deployment artifacts.

5. Existing controlled release workflow

The repository already contains the controlled `release:version` workflow in `scripts/automation/releaseVersion.ts`. It performs governed version planning/application, mandatory gates and versioned release-candidate evidence generation. It does not create a final immutable Git tag before production acceptance.

R-002 therefore does not introduce a competing runtime release engine. The ownership split is:

Capability	Production web runtime	Controlled build / CI / reviewed Git workflow
Repository documentation	read-only	authoritative writer
ADR / ESS / architecture changes	read-only	reviewed Git mutation
Documentary watcher	disabled/inert	offline/CI tooling only
Documentary mutation routes	rejected	control-plane/review workflow
Version bump	rejected	<code>release:version</code>
Release candidate evidence	consume only	<code>release:version</code>
Runtime release manifest	consume only	production build
Release identity	manifest-backed read-only	controlled build/release pipeline

6. Failure semantics

No protected mutation may silently fall back to another local path, JSON state file or mutation endpoint.

A corrupted or structurally invalid immutable release manifest is an integrity failure and must not be replaced by the legacy `0.5.4` state.

The production web process is an evidence consumer/emitter, not a repository or release writer.

7. Validation contract

Automated tests MUST prove:

1. `docs/**` writes are denied;
2. Document Hygiene local authority is denied;
3. Version Manager local JSON authority is denied;
4. ordinary application uploads remain writable;
5. production Documentary watchers are suppressed;
6. protected Documentary/version mutation routes return `409 READ_ONLY_CONTROL_PLANE_REQUIRED` before legacy handlers execute;
7. the controlled build emits a valid immutable release manifest;
8. the manifest contains version, source commit, dependency-lock hash and Documentary-tree hash evidence;
9. `GET /api/admin/version` prefers the immutable build manifest;
10. package/deploy metadata remains compatibility fallback only;
11. the Docker production runtime preloads the guard and keeps `/app/docs` OS-level read-only;
12. repository TypeScript, tests, production build and PR technical validation remain green.

8. Validation evidence

R-002 Phase 4 was validated on the branch synchronized with current `main` at merge-base `066d329558e77ffecc5064ce7c0daa037317ad4c`.

Successful gates before this status-only documentation commit:

- CI run `#482`: dependency audit, production invariants, TypeScript, Vitest, production build including immutable release-manifest generation, and deployment-readiness all succeeded;
- PR Technical Validation run `#133`: repository conventions, changed-workflow security, dependency vulnerability gate, TypeScript, tests, production build and deployment-readiness all succeeded;
- Google Marketing Protected Change Guard run `#63`: Docker-context contract, protected invariants, CSP/consent tests, TypeScript, production build and built-SPA CSP delivery all succeeded.

The final documentation-only head must retain these gates before merge.

9. Consequences

Positive consequences:

- deploy instances cannot rewrite Git-style documentation;
- rolling instances cannot race on Documentary/version files;

- the production watcher cannot trigger autonomous document mutation chains;
- legacy admin mutations fail explicitly;
- the embedded 0.5.4 fallback cannot override release 0.6.0 ;
- each build carries a content-addressable release/Documentary evidence record;
- runtime release identity can be correlated to source commit, dependency lock and Documentary state.

Trade-offs:

- legacy production admin mutation UI paths require migration to a control-plane workflow;
- legacy mutator implementation remains in the repository until consumer cleanup;
- older artifacts without the new manifest use the compatibility package/deploy fallback.

10. R-002 closure boundary

R-002 is technically complete. Production mutation, watcher and release-authority capabilities are fail-closed and release identity is build-evidence-backed.

Removal of now-inert legacy implementation and UI cleanup is follow-up technical debt rather than a prerequisite for the R-002 integrity invariant.

11. Decision

Accepted and validated. Production Documentary and release artifacts are immutable deployment inputs. The production web runtime is a read-only consumer and evidence emitter; controlled build/CI/review workflows are the authoritative mutation and release owners.

ADR-0045 — Stripe Event Ownership & Durable Inbox

Quelle: docs/adr/ADR-0045-stripe-event-ownership-durable-inbox.md

ADR-0045 — Stripe Event Ownership & Durable Inbox

- **Status:** Accepted
- **Implementation-Status:** COMPLETE — production migration applied, handler deployed, duplicate/replay verified against a real Stripe redelivery on 2026-08-10 (see docs/architecture/ROADMAP.md, R-003 row and "R-003 attempts-semantics clarification")
- **Date:** 2026-08-08
- **Scope:** CAPITAL-AI billing ingress / Stripe webhook / Supabase / application-owned checkout side effects
- **Platform Version:** 0.6.0
- **Roadmap:** R-003 — Single billing owner + Stripe event inbox
- **Related:** ADR-0017, ADR-0043, R-004, R-101

1. Context

CAPITAL-AI receives Stripe lifecycle data through more than one integration concern:

1. Supabase-managed Stripe synchronization projects subscription state into `stripe.subscriptions` and then `public.subscriptions`;
2. the application webhook handles `checkout.session.completed` for application-owned side effects such as PDF-credit grants and subscription confirmation e-mails.

The subscription projection is already the authoritative owner for subscription tier changes. Express no longer mutates the tier for `customer.subscription.updated` or `customer.subscription.deleted`.

The remaining integrity gap is Stripe's at-least-once webhook delivery model. Before this ADR, the application webhook executed checkout side effects without first durably claiming `event.id`. A retry could therefore re-enter the same application-side effect path.

The existing e-mail guard used the stable Checkout Session ID, but its Supabase sequence was `SELECT -> UPSERT`. Two concurrent requests could both observe an absent row and both proceed to SMTP before either reservation became authoritative.

2. Decision

CAPITAL-AI establishes one durable application-side Stripe ingress ledger:

`public.stripe_event_inbox`

Every signature-verified Stripe event that reaches the application webhook **MUST** be claimed by `event.id` before application-owned side effects execute.

The ownership split is:

Concern	Authoritative owner
Stripe signature verification	application webhook ingress
Stripe event-ID deduplication	<code>public.stripe_event_inbox</code>
subscription lifecycle/tier projection	Supabase Stripe synchronization -> <code>public.subscriptions</code>
Checkout Session confirmation mail	application side-effect consumer with atomic session reservation
PDF purchase side effect	application side-effect consumer; durable credit ledger deferred to R-004
durable worker/outbox/lease orchestration	deferred to R-101

Express **MUST NOT** regain subscription-tier ownership.

3. Durable inbox contract

The inbox stores at minimum:

- `event_id` as primary key;
- `event_type`;
- `livemode`;
- Stripe API version;
- logical ingress source;
- SHA-256 payload hash;
- processing status;
- attempt count;
- receive/processing/completion timestamps;
- last processing error.

Allowed states are:

- `processing`;
- `processed`;
- `failed`.

The service-role-only function `public.claim_stripe_event(...)` is the atomic claim boundary.

Claim semantics:

1. a new event is inserted as `processing`, attempt `1`;

2. a matching `processed` or `processing` event is a duplicate and MUST NOT re-enter side effects;
3. a matching `failed` event may be claimed again and increments the attempt counter;
4. the same `event_id` with a different event type, livemode, API version or payload hash is an integrity conflict and MUST fail closed;
5. application code does not automatically reclaim an existing `processing` event.

The final rule is deliberate. Until R-004 makes PDF-credit mutation transactional, automatically replaying a stale `processing` event could duplicate a side effect after a crash between side-effect completion and inbox finalization. A stuck `processing` row is therefore a visible reconciliation condition, not an invitation to guess.

4. Side-effect execution contract

`handleWebhookEvent()` becomes the canonical application-side Stripe processing authority.

Processing order:

1. Stripe signature is verified at the HTTP boundary;
2. `event.id` is durably claimed;
3. integrity conflict => fail closed, no side effects;
4. duplicate processing/processed event => return success without side effects;
5. claimed event => execute application-owned side effects;
6. if side effects fail before completion, mark the inbox event `failed` and allow a later re-try;
7. after side effects complete, mark the inbox event `processed`;
8. if finalization fails after side effects completed, leave the row `processing` and return an error. Do not convert it to `failed`, because that would enable an unsafe automatic replay while R-004 remains open.

Unsupported Stripe event types may be recorded and completed with no application-side effect. Subscription lifecycle events remain owned by the Supabase Stripe projection.

5. Atomic confirmation-mail reservation

`public.claim_subscription_confirmation(session_id)` replaces the race-prone application sequence:

```
SELECT -> UPSERT
```

with one database operation:

```
INSERT ... ON CONFLICT DO NOTHING
```

The boolean return value identifies the single caller that won the Checkout Session reservation.

Consequences:

- concurrent delivery for one Checkout Session has one winner;
- a later Stripe resend for the same Checkout Session is suppressed even when Stripe uses a different event ID;
- when privileged Supabase persistence is configured but the reservation RPC fails, production mail sending is blocked rather than falling back to Render-local files;
- a local file reservation remains development-only when Supabase is intentionally absent.

SMTP delivery itself remains best-effort. This ADR guarantees reservation/idempotency ownership, not guaranteed message delivery.

6. Security and database boundary

`stripe_event_inbox` is service-role-only:

- RLS enabled;
- no `anon` / `authenticated` table privileges;
- service-role `SELECT`, `INSERT`, `UPDATE` only;
- claim functions are `SECURITY DEFINER` with an explicit `search_path`;
- execute privilege is granted only to `service_role`.

The production application continues using the privileged server Supabase client defined by ADR-0043. No publishable/anon fallback is permitted for billing persistence.

7. Production handoff and deployment order

The Render production service currently deploys branch `main` automatically with:

- `autoDeploy = yes`;
- `autoDeployTrigger = checksPass`.

Therefore the database migration **MUST** be installed before the PR is merged. A merge-first procedure is unsafe because the successful merge checks can trigger the application deployment before the required RPC/table contract exists.

The safe handoff sequence is:

1. obtain human/code-owner approval for the reviewed PR, but keep the PR unmerged;
2. apply the additive migration `supabase/migrations/20260808013000_stripe_event_inbox.sql` to the production Supabase project through the controlled production handoff;
3. verify `stripe_event_inbox`, `claim_stripe_event(...)` and `claim_subscription_confirmation(...)` exist and remain service-role-only;
4. verify the currently deployed application remains healthy against the additive, unused database objects;
5. merge the reviewed PR;

6. allow Render `checksPass` auto-deploy from `main` to deploy the merged application;
7. verify `/healthz` and deployment identity after the Render deploy becomes live;
8. execute controlled Stripe webhook replay/duplicate tests;
9. verify duplicate delivery does not re-run application-owned side effects and inspect inbox state/evidence.

No live Supabase, Render or Stripe mutation is performed by the development branch or by this ADR. The pre-merge migration is a production-handoff operation requiring explicit protected-change authorization.

If the migration cannot be installed before merge, the PR MUST remain unmerged unless Render auto-deploy is deliberately disabled through an independently approved production change. Application deployment before the database migration is intentionally unsupported because the new production code fails closed when the durable claim contract is unavailable.

8. Explicit non-goals

R-003 does NOT:

- replace Supabase Stripe subscription projection;
- implement a generic queue, worker lease, outbox or dead-letter system;
- make PDF credits transactional or durable across Render instances;
- introduce new Stripe products/prices/coupons;
- mutate live Stripe, Supabase or Render configuration from the development branch.

Those concerns remain respectively under existing subscription ownership, R-101 and R-004.

9. Validation contract

Automated validation MUST prove:

1. a claimed Checkout event executes application-owned side effects once;
2. a duplicate event ID executes no side effects;
3. an event-ID/payload integrity conflict fails closed;
4. a pre-completion side-effect failure is recorded as `failed`;
5. a post-side-effect finalization failure is NOT converted to `failed` automatically;
6. subscription confirmation reservation has exactly one winner under concurrent calls;
7. a durable reservation failure blocks SMTP attempts rather than falling back to production-local state;
8. TypeScript, Vitest, production build and deployment-readiness remain green.

10. R-003 closure boundary

Repository implementation and CI are necessary but not sufficient to close R-003.

R-003 becomes **COMPLETE** only after:

- the migration is applied to production Supabase before the merge-triggered Render deploy;
- the application version containing this handler is deployed;
- a controlled duplicate/replay verification demonstrates durable event-ID deduplication;
- subscription projection remains owned by Supabase;
- no production-local fallback is used for event or confirmation reservation.

Until then the roadmap state is:

IMPLEMENTED IN PR -> PRODUCTION HANDOFF PENDING

Closure confirmed 2026-08-10: all five conditions above are met. Migration `20260808013000_stripe_event_inbox.sql` is applied to production (verified read-only: `stripe_event_inbox` RLS-enabled and `service_role`-only, both RPC functions present with the signatures specified in §3); the application version containing `handleWebhookEvent()` is deployed and live; a real Stripe redelivery of `evt_1TytdVpKr4joNbEcKgXL0mdK` was independently verified to be classified `duplicate_processed` with no re-entry into checkout side effects; subscription projection remains owned by the Supabase Stripe synchronization (unchanged); no production-local fallback was used (the row exists in `stripe_event_inbox`, not a local file). R-003 is **COMPLETE**.

11. Decision

Accepted. Stripe subscription state remains owned by the Supabase projection, while application-owned checkout side effects are gated by a durable service-role-only Stripe Event Inbox. Checkout confirmation mail uses an independent atomic Checkout Session reservation. Because Render auto-deploys passing `main` changes, the additive Supabase migration is a mandatory pre-merge production gate. Generic worker durability and transactional PDF credits remain separate roadmap items R-101 and R-004.

ADR-0046 — Modern Supabase Key Contracts and AI Admin Control Plane

Quelle: docs/adr/ADR-0046-modern-supabase-key-contracts-and-ai-admin-control-plane.md

ADR-0046 — Modern Supabase Key Contracts and AI Admin Control Plane

Status: PROPOSED / IMPLEMENTATION IN PROGRESS Date: 2026-08-10

Context

CAPITAL-AI historically accepted legacy Supabase JWT keys (`anon` , `service_role`) alongside modern publishable/secret keys. Supabase recommends modern `sb_publishable_*` and `sb_secret_*` keys for independent rotation and clearer privilege separation. CAPITAL-AI also requires AI-assisted system administration, diagnostics, evidence collection, reporting and controlled production mutations without granting unrestricted standing privilege to every agent.

Decision

Canonical Supabase key contract

Production clients SHALL use the following canonical variables:

- `SUPABASE_URL` — server project URL.
- `VITE_SUPABASE_URL` — build-time browser project URL.
- `SUPABASE_PUBLISHABLE_KEY` / `VITE_SUPABASE_PUBLISHABLE_KEY` — modern public client credential (`sb_publishable_*`).
- `SUPABASE_SECRET_KEY` — modern backend-only privileged credential (`sb_secret_*`).

Legacy `SUPABASE_ANON_KEY` , `VITE_SUPABASE_ANON_KEY` and `SUPABASE_SERVICE_ROLE_KEY` are compatibility-only during migration and MUST NOT be introduced into new runtime consumers.

`SUPABASE_SECRET_KEY` MUST never be exposed through Vite, logs, diagnostics, API responses, client bundles, PR bodies or Documentary artifacts.

Privilege planes

CAPITAL-AI separates five execution planes:

1. **Observe** — read-only health, logs, metrics, advisors, schema metadata and deployment state.
2. **Diagnose** — read-only queries and correlation across GitHub, Render, Supabase, Stripe and market-data providers.
3. **Operate** — bounded reversible mutations such as restart/redeploy, queue retry, cache invalidation or controlled configuration change.

4. **Administer** — privileged schema/configuration/credential changes guarded by IAM, step-up authentication and audit evidence.
5. **Break Glass** — time-limited emergency privilege with explicit owner authorization, reason, TTL and post-incident review.

No AI agent receives a raw long-lived secret merely because it can perform an administrative task. Tools/connectors SHALL hold credentials and enforce capability boundaries where possible.

Service identities

System-to-system operations SHALL use purpose-specific service identities rather than a shared owner identity. At minimum:

- `capital-ai-web-runtime` — application runtime, minimum required server DB access.
- `capital-ai-ci-release` — repository/release validation and deployment evidence.
- `capital-ai-diagnostics` — read-only diagnostics and reporting.
- `capital-ai-db-migrator` — migration-only privileged role/service identity.
- `capital-ai-billing-processor` — Stripe/Supabase billing side effects.
- `capital-ai-break-glass` — emergency identity, disabled by default.

Supabase secret keys are infrastructure credentials, not user IAM roles. Application authorization remains session/JWT + IAM-role + step-up based.

Mutation policy

Every production mutation must carry:

- authenticated principal/service identity;
- target environment/resource;
- capability/action;
- reason/change ticket or work item;
- precondition evidence;
- idempotency key where applicable;
- expected blast radius;
- rollback strategy;
- result and postcondition evidence;
- correlation/audit ID.

High-risk mutations require owner/step-up authorization. Read-only diagnostics do not require the same mutation approval but remain auditable.

Migration sequence

1. Set modern publishable keys in Render/build environment.

2. Provision a modern Supabase `sb_secret_*` key in the project API-key control plane.
3. Store it only as `SUPABASE_SECRET_KEY` in the Render secret-file/runtime secret contract.
4. Deploy and validate privileged and RLS-bound paths independently.
5. Add CI/readiness checks that reject new legacy-key consumers.
6. Remove runtime fallback to `SUPABASE_SERVICE_ROLE_KEY` and `*_ANON_KEY` after production evidence confirms the modern contract.
7. Disable/revoke legacy keys only after all consumers are proven migrated.

Consequences

Benefits: independent rotation, clearer privilege separation, reduced accidental browser exposure, auditable AI operations and narrower blast radius.

Tradeoffs: temporary dual-key compatibility during migration and additional IAM/control-plane complexity.

Acceptance criteria

- Browser bundle uses only `sb_publishable_*`.
- Privileged server path uses only `SUPABASE_SECRET_KEY ( sb_secret_*)`.
- No production consumer requires legacy `anon / service_role` keys.
- Diagnostics can operate read-only without privileged mutation credentials.
- All privileged mutations are capability-scoped and auditable.
- Break-glass is disabled by default and requires explicit owner approval/step-up.

ADR-0047 — GitHub Actions as authoritative Render production deploy gate

Quelle: docs/adr/ADR-0047-github-authoritative-pre-merge-ci-gate.md

ADR-0047 — GitHub Actions as authoritative Render production deploy gate

Status: Proposed Date: 2026-08-09

Retrospective correction

This ADR was originally merged by PR #144 under the identifier `ADR-0046`. That identifier was already assigned to Vocabulary Governance by PR #143. To restore deterministic ADR numbering, the deployment-gate decision is reassigned to `ADR-0047`. The architectural decision itself is unchanged.

Context

The Render `Finance` service previously used `After CI Checks Pass` (`checksPass`) as its production auto-deploy trigger. Controlled tests showed that GitHub CI could complete successfully while Render created no deploy or created it only after substantial delay.

A controlled A/B test switched Render to `On Commit`. Under this configuration Render successfully consumed PR-merge commits from `main` and created `trigger: new_commit` deploys, including PR #141 merge commit `ae35d4e57dbf06eadf5776c0e9d6353978f03f38`, deployed as `dep-d9sbnugu01pc73e2mq0` to `live`.

The A/B evidence isolates the unreliable path to Render's `checksPass` / GitHub Check Suite correlation rather than GitHub repository access, merge commits, Git credentials, or Render's normal commit event handling.

A repository inspection also found that `main` is not protected and required status-check enforcement is disabled. The current GitHub plan does not provide the required private-repository branch/ruleset enforcement without an upgrade, and no upgrade is planned at this time.

Therefore `Render: On Commit` cannot be the permanent production policy because it can deploy a `main` commit before GitHub CI has validated that commit.

Decision

GitHub Actions becomes the authoritative production deployment gate without relying on paid branch protection or Render `checksPass`.

Render Auto-Deploy SHALL be set to **Off**. GitHub Actions SHALL deploy only after the `build-and-test` job succeeds for a `push` to `main`.

Target flow:

PR / main update → GitHub CI → build-and-test success → deploy-production → Render Deploy Hook → exact verified commit → production

The deploy job **MUST NOT** run for `pull_request` events. It runs only for `push` events on `refs/heads/main` and depends on `build-and-test`.

The Render Deploy Hook URL is stored only as GitHub Actions repository secret `RENDER_DEPLOY_HOOK_URL`. The workflow appends `ref=${github.sha}` so Render deploys the exact commit that passed CI instead of an unverified later commit.

Required operational configuration

Before this ADR is considered implemented:

1. In Render `Finance → Settings → Auto-Deploy`, set Auto-Deploy to **Off**.
2. Copy the service's secret Deploy Hook URL from Render `Finance → Settings`.
3. In GitHub `Finance → Settings → Secrets and variables → Actions`, create repository secret `RENDER_DEPLOY_HOOK_URL` containing that URL.
4. Keep PR #144 in Draft state until steps 1–3 are complete.
5. Merge the workflow change only after steps 1–3 are complete.
6. Validate one documentation-only test merge end to end.

Security properties

- A failing `build-and-test` prevents the deploy job from running.
- The Render deploy secret is not stored in repository content or logs.
- The deployment request targets the exact `github.sha` that passed CI.
- Render no longer independently auto-deploys new commits.
- The workflow does not require `statuses: write` or a custom compatibility status.

Limitations

Because `main` is not protected on the current GitHub plan, GitHub cannot prevent a user with repository write permission from pushing directly to `main`. However, such a commit still must pass the push-triggered `build-and-test` job before the GitHub Actions deploy job can invoke Render.

This architecture therefore protects the production deployment boundary even when repository merge-policy enforcement is unavailable. It does not replace the governance value of branch protection if the GitHub plan changes in the future.

Consequences

Positive consequences:

- Production no longer depends on Render's unreliable `checksPass` correlation.

- No GitHub plan upgrade is required.
- Failed CI cannot invoke the Render deployment hook.
- The exact validated commit is selected for deployment.
- The legacy `capital-ai/ci-gate` status and `statuses: write` permission are unnecessary.

Trade-offs:

- Render Auto-Deploy must remain Off.
- The GitHub repository secret becomes production-critical and must be rotated if exposed.
- GitHub Actions availability becomes part of the production deployment control plane.
- Branch protection remains desirable if a future GitHub plan enables it.

Verification

The architecture is considered implemented only when all of the following are true:

- Render `Finance` reports Auto-Deploy Off;
- repository secret `RENDER_DEPLOY_HOOK_URL` exists;
- a pull-request CI run does not trigger a Render deploy;
- a deliberately failing `main` CI run does not trigger a Render deploy;
- a passing `main` CI run invokes `Deploy verified commit to Render`;
- Render deploys the same Git SHA that passed CI;
- the deployment reaches `live`;
- no Render `checksPass` or `On Commit` auto-deploy is required.

Evidence

See `docs/runbooks/RENDER_AUTODEPLOY_AB_TEST_2026-08-09.md` for the diagnostic record and exact deploy identifiers.

ADR-0048 — Render Docker Image Hardening

Quelle: docs/adr/ADR-0048-render-docker-image-hardening.md

ADR-0048 — Render Docker Image Hardening

Status: Proposed Date: 2026-08-09

Context

CAPITAL-AI runs on Render using `runtime: docker` and a repository-owned Dockerfile. The existing image already used an immutable Node base-image digest, a multi-stage build, a non-root runtime user, and a health check. The production deployment path is gated by GitHub Actions and triggers Render only after CI succeeds.

The remaining gaps were excessive runtime ownership changes, use of the npm process shim as PID 1, a broader-than-required Docker build context, and no CI control that proves the Dockerfile still satisfies the hardening invariants or that the resulting image metadata is non-root and health-checked.

Decision

The production Docker image SHALL be hardened as follows:

1. Continue pinning builder and runner base images by immutable SHA-256 digest.
2. Keep a multi-stage build and install only production dependencies in the runner using `npm ci --omit=dev`.
3. Clean npm cache from the runtime dependency layer.
4. Create the `capitalai` runtime identity before application artifacts are copied.
5. Use `COPY --chown=capitalai:capitalai` for runtime artifacts rather than broad recursive ownership changes.
6. Treat `/app/uploads` as the explicit writable application path and `/app/docs` as read-only governance content.
7. Run the application directly as `node dist/server.cjs`, making Node PID 1 instead of an npm wrapper.
8. Retain the `/healthz` container health check.
9. Extend `.dockerignore` to exclude environment files, private-key/certificate material, logs, coverage output, editor state, temporary files, secrets directories, and other non-runtime artifacts.
10. Add a fail-closed CI policy script that verifies Dockerfile and `.dockerignore` invariants and rejects secret-like Docker `ARG` declarations.
11. Build the production Docker image in CI and inspect the resulting image metadata before any production deploy hook can execute.

Secret handling

Docker `ARG` MUST NOT be used for secrets. Existing `VITE_*` arguments are permitted only for values intentionally exposed to browser clients, such as public/anon/publishable identifiers. Server credentials, service-role keys, passwords, tokens, private keys, and Stripe secret keys must remain Render runtime secrets or secret files.

Consequences

Positive:

- smaller and more deterministic runtime dependency surface;
- reduced filesystem authority for the runtime user;
- direct signal delivery to the Node process;
- reduced chance of local or secret material entering the Docker build context;
- Docker hardening becomes a CI-enforced invariant rather than documentation only;
- Render receives a deploy hook only after the hardened image definition has successfully built in GitHub CI.

Trade-offs:

- CI duration increases because the production Dockerfile is built in addition to the application build;
- the runner remains based on Node Alpine rather than a distroless image to avoid combining runtime-library compatibility changes with this hardening step;
- full read-only-root-filesystem enforcement is not available as a repository-only Dockerfile guarantee on the current Render service configuration, so writable-path minimization is used instead.

Verification

Implementation is accepted when:

- `node scripts/security/verifyDockerHardening.mjs` succeeds;
- GitHub CI successfully executes `docker build`;
- image metadata reports user `capitalai`;
- image command is `["node", "dist/server.cjs"]`;
- image healthcheck contains `/healthz`;
- application CI remains green;
- after merge, the GitHub deploy-hook job deploys the exact verified commit to Render and the service reaches `live`.

Numbering note

This decision uses ADR-0048 because ADR-0047 is already authoritative on `main` for the GitHub Actions / Render deploy-gate architecture after PR #147 corrected the earlier ADR-number collision.

ADR-0049 — Docker Runtime Filesystem Hardening

Quelle: docs/adr/ADR-0049-docker-runtime-filesystem-hardening.md

ADR-0049 — Docker Runtime Filesystem Hardening

Status: Proposed

Date: 2026-08-10

Context

ADR-0048 introduced the first production Docker hardening baseline for Render: immutable base-image digest, multi-stage build, production-only dependencies, non-root runtime user, explicit ownership, healthcheck, direct Node PID 1 execution and Docker-specific CI verification.

Render Web Services do not expose Docker runtime controls equivalent to a self-managed `docker run` configuration for `--read-only`, `--cap-drop`, `--security-opt no-new-privileges`, seccomp profiles or AppArmor policies. Therefore the image itself must reduce writable runtime surface as far as possible without relying on unavailable host-level controls.

The previous image copied production artifacts and dependencies as `capitalai:capitalai`. Although the process ran as non-root, this allowed the application identity to modify its own JavaScript bundles, server artifacts and installed dependencies at runtime if an application-level compromise occurred.

Decision

The production image SHALL apply an immutable-application-filesystem pattern inside the container:

1. Runtime application artifacts (`/app/dist` and `/app/server`) SHALL be owned by `root:root` and SHALL not be writable by the application user.
2. Runtime dependencies (`/app/node_modules`) and package manifests SHALL be owned by root and SHALL not be writable by the application user.
3. The runtime process SHALL continue to execute as the unprivileged `capitalai` user.
4. `/app/uploads` SHALL remain the explicit application-writable persistent/work path with restrictive permissions.
5. Runtime temporary/home writes SHALL be isolated to `/tmp/capitalai`, owned only by `capitalai`, with mode `0700`.
6. `HOME` and `TMPDIR` SHALL point to `/tmp/capitalai` so libraries that require temporary or home-directory writes do not fall back to application directories.
7. The Docker hardening verifier SHALL fail closed if root-owned/read-only application artifacts or isolated writable paths are removed.

8. The direct Node command, healthcheck, immutable image digest, production-only dependency install and secret-like Docker ARG prohibition remain mandatory.

Security Properties

This design reduces post-exploitation persistence opportunities inside the running container:

- application code cannot overwrite its own server bundle;
- dependencies cannot be modified by the runtime identity;
- the runtime artifact guard cannot be replaced by the application process;
- writable paths are explicit and narrow;
- temporary files are isolated from application artifacts and from other users;
- no additional Linux capabilities or privileged runtime mode are introduced.

The design does not claim to provide a fully read-only root filesystem. Render controls the host/container runtime and does not currently expose all corresponding Docker security flags for this service type.

Trade-offs

- Libraries that attempt to write into their package directory or application directory will fail rather than silently mutate production state.
- Any future feature requiring a new writable directory must be reviewed explicitly and added as a narrowly scoped writable path.
- Runtime temporary state is ephemeral and must not be treated as persistent storage.

Verification

CI SHALL validate both policy and built-image behavior. At minimum:

- `USER` is `capitalai`;
- `CMD` is direct Node PID 1;
- healthcheck targets `/healthz`;
- Dockerfile policy confirms root-owned/read-only artifacts and dependencies;
- only explicitly designated runtime directories are writable by `capitalai`.

Relationship to Render

Render Auto-Deploy remains Off. Production deployment continues through the GitHub Actions Deploy Hook after a successful `main` CI run. Pull-request preview generation should remain disabled unless explicitly required.

This ADR extends ADR-0048 and does not replace it.

ADR-0050 — Agent Tool & Capability IAM Foundation (Supabase, Phase 1)

Quelle: docs/adr/ADR-0050-agent-tool-capability-iam-foundation.md

ADR-0050 — Agent Tool & Capability IAM Foundation (Supabase, Phase 1)

- **Status:** Accepted / Phase 1 implemented, Phase 2 not started
- **Date:** 2026-08-09
- **Scope:** AI-agent read access to Supabase, new agent-tool layer, governance for a future capability/grant IAM extension
- **Related:** ADR-0043 (Supabase Privilege Separation & Render Production Hardening), ADR-0041 (Enterprise Market Data Provider & MCP Architecture — status reference for "specified, not implemented"), ESS-0014, ESS-0016, ESS-0018

Context

CAPITAL-AI's AI agents (`src/agents/*.ts`) previously had no way to read the application's own Supabase data — they only ever produced structured JSON/text via `src/services/agentModelRouting.ts`. A request emerged for two production capabilities: a Screening/Scoring explainability agent (read-only) and an Admin/Support/Diagnostics agent (read + controlled write) coupled to the existing `Compliance` and `Supervisor` components.

Investigation of the current codebase (documented in ESS-0018 §2) found that none of the prerequisites for the write-capable admin agent exist yet: no tool-calling layer for agents, no capability/grant IAM (only coarse role checks in `src/platform/Security/authMiddleware.ts`), no approval-artifact implementation, and `Compliance` / `Supervisor` explicitly do not gate live actions today. Building tool-calling, capability IAM, an approval workflow, real Compliance/Supervisor gating, and a write-capable agent in a single change would be an unreviewable diff for a FinTech application handling real financial data, and would violate the fail-closed, approval-gated mutation model CLAUDE.md mandates for provider mutations.

Decision

1. Phase 1 ships now: a read-only agent-tool foundation

- A new, additive `src/services/agentTools/` module hosts backend-only agent tools. The first tool, `getScoreSnapshotEvidence` (`supabaseScoreEvidenceTool.ts`), reads `score_snapshots` exclusively through `getPrivilegedServerSupabase()` (`server/db.ts` , unchanged, per ADR-0043) using the Supabase JS query builder with validated inputs — never raw SQL, never a write path.

- `src/services/rag/evidenceLayer.ts` gains an additive `buildRowEvidenceBundle` export that reuses the existing quality-verdict logic (previously inlined in `buildRagEvidenceBundle`, now extracted into a shared internal evaluator) so database-row evidence and vector-chunk evidence share one auditable, fail-closed quality model instead of two divergent ones. No existing exported function signature changes.
- A new `ScoreExplainabilityAgent` (`src/agents/scoreExplainabilityAgent.ts`) consumes the tool and the existing `generateTextWithFallback` router (`agentModelRouting.ts`, unchanged) to produce a cited explanation. It never fabricates an explanation when no evidence exists.
- A new admin-only endpoint (`server/scoreExplainability.ts`, `GET /api/scoring/explain/:symbol`) exposes this, gated by `checkAdminAccess(..., SUPERVISOR_ZONE_ROLES)` — the same pattern already used by `server/agentEvaluationRouter.ts`. No anonymous or browser-privileged-client access exists at any point.
- Every call is audited via the existing `logSystemEvent('ORCHESTRATOR', ...)` path (`server/systemEvents.ts`) with symbol/date-range/row-count/quality-verdict only — never row payloads.

This is purely additive. `authMiddleware.ts`, `Compliance/`, `Supervisor/`, `server/db.ts`, and all existing agents/routers are unmodified except for the one new `app.use(...)` mount line in `server.application.ts`.

2. Phase 2 (capability IAM, approval workflow, write-capable admin agent) is specified, not built

ESS-0018 §4.2 enumerates the target capability names and explicit non-goals (no raw-SQL tool ever, no schema/project/migration mutation via an agent, no agent-granted IAM roles) for the Admin/Support/Diagnostics agent. None of it ships in this ADR. Before any Phase 2 code:

- a capability/grant extension to `src/platform/Security/` must exist (new `Grant / Capability` types, storage, `checkCapability()`);
- an approval-artifact contract must be implemented (not just documented, as `Analytics-Approval` currently is);
- `Compliance` must gain a real policy-verdict surface and `Supervisor.executeSupervised()` must enforce Approval + Dry-run + Fingerprint before Apply;
- each write capability must be individually enumerated — never a general SQL-execution tool.

This requires its own ADR amendment before implementation, per CLAUDE.md's requirement that architecture-changing decisions get an ADR and that provider/database mutations follow the full Policy→IAM/Grant→Approval→Dry-run→Fingerprint→Apply→Verify→Audit chain.

3. No MCP protocol server in Phase 1

The only real MCP server in this repository (`ga4-analytics`, `.mcp.json`) is Claude Code developer tooling, not an in-app agent-capability pattern. With exactly one internal caller

(`ScoreExplainabilityAgent`), introducing MCP transport/schema machinery in-process adds complexity without a consumer that needs it. ESS-0018 §5 records the trigger condition for revisiting this (a second real, independent consumer — e.g. the Phase 2 write-capable admin agent, which may need its own process/trust boundary).

Consequences

Positive

- A real, working, auditable read path exists for score explainability without touching any protected/security-critical file.
- The eventual write-capable admin agent is designed now (ESS-0018 §4.2), not improvised later, and its non-goals (no raw SQL, no schema/project mutation, no self-granted IAM) are on record before any code exists.
- `score_snapshots` ' existing service-role-only RLS posture (ADR-0043) is reused unchanged, not weakened or bypassed.

Trade-offs

- The Admin/Support/Diagnostics agent requested by the user is **not** delivered by this ADR. It requires the Phase 2 capability-IAM and approval-workflow foundation first.
- `evidenceLayer.ts` 's internal quality-evaluation logic is refactored (extracted, not duplicated), which touches a file used by the existing chat-assistant RAG path (`server/ai.ts`); the exported `buildRagEvidenceBundle` signature and behavior are unchanged, and the existing `tests/unit/ragEvidenceLayer.test.ts` suite is the regression guard.

Validation & Evidence

1. Unit tests prove the row-evidence quality verdict is fail-closed (`NO_EVIDENCE` on empty rows) and shares thresholds with the existing RAG evidence path.
2. Unit tests prove `getScoreSnapshotEvidence` only ever calls the Supabase query builder with allowlisted filters (`.eq / .gte / .lte / .order / .limit`), never raw SQL, and only via `getPrivilegedServerSupabase()` .
3. `GET /api/scoring/explain/:symbol` returns 403 without a valid admin/supervisor bearer token.
4. `npm run lint` , `npm test` , `npm run build` , `npm run predeploy:check` pass.
5. `grep` confirms no write/upsert/delete export exists in `src/services/agentTools/` .

Rollback

Reverting Phase 1 removes only the new files and the one router mount line; it restores no prior behavior because none of this existed before. Rollback does not require IAM/security review since no protected invariant (CLAUDE.md list, ADR-0043's privilege separation, RLS policies) is touched or weakened.

ADR-0051 — Capability/Grant IAM, Approval-Artefakt & governter Admin-Agent (ESS-0018 Phase 2)

Quelle: docs/adr/ADR-0051-capability-grant-approval-workflow.md

ADR-0051 — Capability/Grant IAM, Approval-Artefakt & governter Admin-Agent (ESS-0018 Phase 2)

- **Status:** Accepted / Implemented (Foundation + eine Schreib-Capability)
- **Date:** 2026-08-10
- **Scope:** Neue Capability-/Grant-IAM-Schicht, echtes Approval-Artefakt, Compliance-Policy-Gate, Supervisor-Erweiterung, erster schreibender Admin/Support/Diagnose-Agent für Supabase.
- **Related:** ADR-0050 (Agent Tool & Capability IAM Foundation, Phase 1), ADR-0043 (Supabase Privilege Separation), ESS-0018 (Agentic Supabase Tool Governance), CLAUDE.md (Policy→IAM/Grant→Approval→Dry-run→Fingerprint→Apply→Verify→Audit-Kette)

Context

ADR-0050 (ESS-0018 Phase 1) lieferte einen rein lesenden Agenten. ESS-0018 §4.2 spezifizierte bereits vollständig, aber implementierte bewusst nicht, einen zweiten Agenten für CAPITAL-AI-Systemadministratoren mit Lese- und kontrollierten Schreibrechten. Voraussetzung dafür war laut ADR-0050: eine Capability-/Grant-IAM-Erweiterung, ein echtes Approval-Artefakt (statt nur dokumentiert wie `AnalyticsApproval`), eine echte Compliance/Supervisor-Kopplung und ein eng umrissenes Schreib-Capability-Register.

Erneute Verifikation am Implementierungstag (main `69d6315`) bestätigte: keiner dieser Bausteine existierte als Code. `Compliance` ist architektonisch ausschließlich ein Ganz-App-Auditor ohne Pro-Aktion-Hook (`src/platform/Compliance/store.ts:55`, `executeComplianceRun()`); der Router disclaint selbst, keine Berechtigungen vergeben zu dürfen. `Supervisor.executeSupervised()` hatte zum Zeitpunkt der Implementierung nur einen einzigen echten Aufrufer im Repository (`src/platform/Traceability/Services/runTraceability.ts`), also kein Pre-Execution-Gate und niedriges Risiko für eine additive Erweiterung.

Decision

1. Capability/Grant-IAM (`src/platform/Security/capabilities.ts`)

Additive Erweiterung des groben Rollenmodells (`owner|admin|supervisor|user`, `authMiddleware.ts`), nicht dessen Ersatz. Neue Tabelle `capability_grants` (Migration `20260810002100_capability_grants.sql`, Service-Role-only RLS wie `score_snapshots`). `checkCapability()` ist fail-closed: jeder DB-/Verbindungsfehler liefert `false`, nie stillschweigend

`true` . Nur drei benannte Capabilities existieren (`CAPABILITIES` -Konstante) — kein Wildcard, keine generische SQL-Capability. Ausstellung/Widerruf (`grantCapability()` / `revokeCapability()`) ist ausschließlich hinter Owner-Rolle + frischem TOTP-Step-up erreichbar (CLAUDE.md), durchgesetzt in `server/adminDiagnostics.ts` , nicht in der Funktion selbst (dieselbe Trennung wie bei `server/stepUp.ts`).

2. Approval-Artefakt (`src/platform/Security/approvals.ts`)

Neue Tabelle `agent_action_approvals` (Migration `20260810002200_agent_action_approvals.sql` , Service-Role-only RLS). Jede Genehmigung ist an `action` + `planHash` (deterministischer SHA-256 über die sortierten Plan-Parameter) gebunden und über denselben race-sicheren Single-Use-Mechanismus wie `step_up_tokens` konsumierbar (`UPDATE ... WHERE consumed_at IS NULL AND expires_at > now()`). 10 Minuten TTL. Eine Genehmigung für Plan A kann niemals Plan B autorisieren (`consumeApproval()` prüft beides vor dem Verbrauch).

3. Compliance Policy-Gate (`src/platform/Compliance/PolicyGate.ts`)

Rein additiv — `store.ts` , `scanners.ts` , `router.ts` bleiben unverändert. Zwei Allowlists (Lesen/Schreiben) gegen die exakten ESS-0018-§4.2-Capability-Namen. Jede Capability außerhalb der Allowlist — inklusive jeder Raw-SQL-förmigen — erhält `DENY` . Es gibt keinen Default-Allow-Pfad.

4. Supervisor-Erweiterung (`src/platform/Supervisor/supervisor.ts`)

Neue, separate Funktion `executeApprovedSupervisedAction()` statt Änderung von `executeSupervised()` selbst (dessen einziger bestehender Aufrufer bleibt unberührt). Erzwingt Policy-Gate → Approval-Konsum → `executeSupervised()` -Apply → EventMesh-Audit-Event in dieser Reihenfolge; ein `DENY` oder ein nicht konsumierbares Approval erreicht `fn` nie. Fingerprint-Prüfung gegen den aktuellen DB-Zustand ist bewusst Aufgabe des jeweiligen Schreib-Tools (Supervisor kennt keine Tabellenschemata) — ein Mismatch dort wirft einen Fehler, der hier nicht als stiller Erfolg maskiert wird.

5. Admin/Support/Diagnose-Agent (`src/agents/adminDiagnosticsAgent.ts` ,

`src/services/agentTools/supabaseAdminDiagnosticsTool.ts` , `server/adminDiagnostics.ts`)

Kein LLM-Aufruf — anders als beim Score-Explainability-Agenten (Phase 1) gibt es hier keine Freitext-Erklärung, die einen Modellaufruf rechtfertigt; der Wert liegt in der Governance-Kette. Jede Methode prüft die konkrete Capability zusätzlich zur groben Rollenprüfung der HTTP-Schicht (`checkAdminAccess`) — das ist die feingranulare Hälfte des Zwei-Ebenen-Autorisierungsmodells.

Erste und einzige real angebundene Schreib-Capability in diesem Schnitt: `supabase.admin.alert_subscription.disable` — Single-Row-Update `active=false` in `alert_subscriptions` . Bewusst gewählt, weil reversibel, ohne Geld-/Score-/Identitätsbezug, ohne Kaskadeneffekte. Weitere in ESS-0018 §4.2 genannte Schreib-Capabilities (z. B. `resend_confirmation`) folgen als eigene, kleine Folge-PRs nach demselben Muster.

Endpunkt-Kontrakt:

GET /api/admin/diagnostics	- lesend, Capability-Check
GET /api/admin/diagnostics/alert-subscriptions/:id/preview	- lesend, Capability-Check, kein Approval
POST /api/admin/capabilities/grant revoke	- Owner + Step-up
POST /api/admin/approvals	- Owner + Step-up
POST /api/admin/alert-subscriptions/:id/disable	- Capability + konsumiertes Approval

Consequences

Positive

- Der schreibende Admin-Agent aus ESS-0018 existiert jetzt real, nicht nur spezifiziert.
- Die vollständige, in CLAUDE.md vorgeschriebene Kette (Policy→IAM/Grant→Approval→Dry-run/Fingerprint→Apply→Verify/Audit) ist erstmals als Laufzeit-Code vorhanden und über eine reale Schreib-Capability bewiesen.
- Kein bestehender, sicherheitskritischer Code (`authMiddleware.ts` , `Compliance/store.ts` , `Supervisor.executeSupervised()` , RLS auf `alert_subscriptions`) wurde verändert — alles ist additiv.

Trade-offs

- Nur eine reale Schreib-Capability in diesem Schnitt; weitere folgen separat.
- Zwei neue Tabellen erfordern eine manuelle Produktions-Migration (`supabase db push` oder Dashboard) — CLAUDE.md untersagt automatische Produktions-DDL aus einem Entwicklungsschritt; dies bleibt ein offenes Handoff, bis ein Mensch die Migration gegen Production anwendet.
- Kein UI für Grant-/Approval-Verwaltung; reine API vorerst.

Validation & Evidence

1. Unit-Tests beweisen: `checkCapability` fail-closed bei DB-Fehler/fehlender Konfiguration; `consumeApproval` verweigert fehlende/abgelaufene/bereits verbrauchte/plan-fremde Genehmigungen; `evaluateWritePolicy` / `evaluateReadPolicy` lassen ausschließlich die dokumentierte Allowlist zu; `executeApprovedSupervisedAction` ruft `fn` niemals ohne konsumiertes, passendes Approval auf; `disableAlertSubscription` verweigert Apply bei Fingerprint-Mismatch und ist bei bereits inaktiver Zeile idempotent ohne redundanten Write.
2. `npm run lint` , `npm test` , `npm run build` , `npm run predeploy:check` grün.
3. Migrationsdateien folgen der bestehenden Namens-/RLS-Konvention (`supabase/migrations/20260801143614_score_snapshots.sql` als Vorbild).

Rollback

Entfernt ausschließlich neue Dateien/Router-Mount-Zeile/Migrationen; keine bestehende Invariante (CLAUDE.md-Liste, ADR-0043-Privilegientrennung, bestehende RLS-Policies) wird berührt oder geschwächt. Falls die Migrationen bereits gegen Production angewendet wurden, ist ein DROP-TABLE-Rollback separat zu entscheiden (Datenverlust bei bereits erteilten Grants/Approvals), nicht automatisch Teil dieses Rollbacks.

ADR-0052 — Transactional, Idempotent PDF-Credit Ledger

Quelle: docs/adr/ADR-0052-pdf-credit-ledger.md

ADR-0052 — Transactional, Idempotent PDF-Credit Ledger

- **Status:** Accepted
- **Implementation-Status:** IMPLEMENTED IN PR -> PRODUCTION HANDOFF PENDING
- **Date:** 2026-08-10
- **Scope:** CAPITAL-AI PDF export credit balance and grant accounting
- **Platform Version:** 0.6.0
- **Roadmap:** R-004 — Transactional PDF-credit ledger
- **Related:** ADR-0043, ADR-0045 (R-003), R-101

1. Context

ADR-0045 (R-003) established a durable Stripe event inbox for application-owned checkout side effects, but explicitly deferred one of those side effects: the PDF-export credit grant. Its ownership table lists the PDF purchase side effect as "application side-effect consumer; durable credit ledger deferred to R-004", and its non-goals section states R-003 does not "make PDF credits transactional or durable across Render instances."

Before this ADR, PDF credits were stored in `uploads/pdf_credits.json` (`server/db.ts`), a JSON file on Render's local disk, read and written by `getLocalPdfCredits` / `saveLocalPdfCredits`:

1. **Not durable.** Render's filesystem is ephemeral, and production now redeploys on every merge to `main` via a GitHub Actions `deploy_hook` step. Every redeploy silently re-sets every customer's credit balance back to the implicit default of 3, discarding purchased credits.
2. **Not atomic.** `POST /consume-pdf-credit` (`server/stripe.ts`) performed a read-then-write sequence — read the current balance, check it is positive, write `current - 1` — with no locking. Concurrent requests could both read the same balance and both succeed, allowing over-consumption.
3. **No independent idempotency guard on grants.** The webhook's PDF-credit grant (`processStripeEventSideEffects`, `server/stripe.ts`) was deduplicated only by ADR-0045's Stripe-event-ID inbox one layer above it. ADR-0045 §3 rule 5 states application code does not automatically reclaim a stuck `processing` event specifically because "automatic re-entry could duplicate a non-transactional side effect (notably the legacy PDF-credit file path until R-004)" — i.e. a manually reconciled stuck event had no second line of defense against a double grant.
4. **No production guard.** Unlike `saveSubscription` (`server/db.ts`) and `claimStripeEvent` (`server/stripeEventInbox.ts`), which both call `assertPrivilegedSupabaseConfigured()` and

fail closed in production when Supabase is not configured, `getLocalPdfCredits / saveLocalPdfCredits` silently used the local file even in production.

2. Decision

CAPITAL-AI establishes a durable, service-role-only PDF-credit ledger in Supabase, replacing the local JSON file:

`public.pdf_credits` — current balance per `user_identifier`. `public.pdf_credit_grants` — idempotency/audit ledger of grants, keyed by `grant_key`.

Two `SECURITY DEFINER` functions are the atomic boundary:

- `public.consume_pdf_credit(user_identifier)` — atomically decrements the balance by one, guarded by a single `UPDATE ... WHERE credits > 0` statement, replacing the read-then-write race.
- `public.grant_pdf_credits(grant_key, user_identifier, credits, source, reference)` — atomically grants credits, guarded by `grant_key (ON CONFLICT DO NOTHING on pdf_credit_grants)` as an independent idempotency check.

`grant_key` is the causing Stripe `event.id`. Because one Stripe checkout event corresponds to exactly one PDF-credit grant, this ties the grant's own idempotency directly to the same identity ADR-0045's inbox already uses, giving PDF-credit grants the second line of defense ADR-0045 §3 rule 5 flagged as missing.

The ownership split (extending ADR-0045 §2's table):

Concern	Authoritative owner
PDF-credit balance persistence	<code>public.pdf_credits</code>
PDF-credit grant idempotency	<code>public.pdf_credit_grants</code> via <code>grant_pdf_credits(...)</code>
PDF-credit consumption atomicity	<code>consume_pdf_credit(...)</code>
Stripe event-ID deduplication (one layer up)	<code>public.stripe_event_inbox</code> (ADR-0045, unchanged)
durable worker/outbox/lease orchestration	deferred to R-101

`server/pdfCreditLedger.ts` is the sole application-side module that touches these tables/functions, mirroring `server/stripeEventInbox.ts`'s structure: it normalizes the identifier, preserves the existing owner-identity unlimited-credits bypass (`isOwnerIdentifier`, unchanged), calls `assertPrivilegedSupabaseConfigured()` to fail closed in production when Supabase is not configured, and keeps a local-file fallback for local development only (weaker guarantees accepted there, same trade-off `stripeEventInbox.ts` already accepts for its `localDevelopmentClaims` fallback).

`server/stripe.ts`'s three call sites (`GET /pdf-credits`, `POST /consume-pdf-credit`, `processStripeEventSideEffects`'s PDF-grant branch) call this module exclusively; the HTTP response contracts consumed by `src/components/PdfExportModal.tsx` are unchanged.

3. Ledger contract

`pdf_credits`: `user_identifier` primary key, ``credits`` integer not null default 3 check (`credits = 0`), `updated_at``. A first-seen identifier is lazily initialized at the default of 3, matching the legacy default.

`pdf_credit_grants`: `grant_key` primary key (the Stripe `event.id`), `user_identifier`, `credits_granted`, `source`, `reference` (e.g. Checkout Session id), `granted_at`.

`consume_pdf_credit(p_user_identifier)`:

1. lazily inserts the balance row at 3 if absent (`ON CONFLICT DO NOTHING`);
2. atomically decrements via `UPDATE ... SET credits = credits - 1 WHERE user_identifier = $1 AND credits > 0 RETURNING credits` — the row lock this statement takes makes the check-then-write atomic;
3. returns `(success: true, credits: <new balance>)` on success, or `(success: false, credits: <unchanged balance>)` when the balance was already 0.

`grant_pdf_credits(p_grant_key, p_user_identifier, p_credits, p_source, p_reference)`:

1. inserts into `pdf_credit_grants` with `ON CONFLICT (grant_key) DO NOTHING`;
2. if the key already existed (no row inserted), returns `(granted: false, credits: <current balance, untouched>)`;
3. otherwise upserts `pdf_credits` — `INSERT ... VALUES (user_identifier, 3 + p_credits) ON CONFLICT (user_identifier) DO UPDATE SET credits = pdf_credits.credits + p_credits` — so a first-time purchaser starts from the implicit default of 3 before the purchased credits are added, matching legacy semantics exactly;
4. returns `(granted: true, credits: <new balance>)`.

4. Security and database boundary

Both tables follow the exact `stripe_event_inbox` pattern from ADR-0045 §6:

- RLS enabled on both tables;
- no `anon` / `authenticated` table privileges;
- service-role `SELECT`, `INSERT`, `UPDATE` only;
- both functions are `SECURITY DEFINER` with an explicit `search_path = public, pg_temp`;
- execute privilege is granted only to `service_role`.

`server/pdfCreditLedger.ts` uses the privileged server Supabase client defined by ADR-0043. No publishable/anon fallback is permitted for credit-balance persistence.

5. Production handoff and deployment order

Production currently deploys on merge to `main` via a GitHub Actions `deploy_hook` step (Render's own git-based `autoDeploy` / `checksPass` is disabled). The database migration MUST the-

refore be applied before the PR is merged, the same constraint ADR-0045 §7 already established for the Stripe inbox, adapted to the current deploy trigger:

1. obtain review approval for the PR, but keep it unmerged;
2. apply the additive migration `supabase/migrations/20260810110642_pdf_credit_ledger.sql` to the production Supabase project through the controlled production handoff;
3. verify `pdf_credits`, `pdf_credit_grants`, `consume_pdf_credit(...)` and `grant_pdf_credits(...)` exist and remain service-role-only;
4. verify the currently deployed application remains healthy against the additive, unused database objects;
5. merge the reviewed PR, which fires the `deploy_hook`;
6. verify `/healthz` and deployment identity after the deploy becomes live;
7. verify a real PDF-credit purchase and consumption round-trip against the new tables.

No live Supabase, Render or Stripe mutation is performed by the development branch or by this ADR. Application deployment before the database migration is intentionally unsupported because the new production code fails closed (`assertPrivilegedSupabaseConfigured`) when the durable ledger contract is unavailable.

6. Explicit non-goals

R-004 does NOT:

- change the `GET /pdf-credits` or `POST /consume-pdf-credit` HTTP response shape — the frontend (`src/components/PdfExportModal.tsx`) needs no changes;
- change how `isOwnerIdentifier` resolves ownership (`server/db.ts`) — reused as-is;
- implement a generic queue, worker lease, outbox or dead-letter system — that remains R-101;
- backfill `uploads/pdf_credits.json`'s existing contents. That file's balances are already unreliable (reset on every redeploy), so there is nothing trustworthy to migrate; new purchases and consumption become durable from this migration's rollout forward.

7. Validation contract

Automated validation MUST prove:

1. `consumePdfCredit` returns a decremented balance on success and leaves the balance unchanged with `success: false` when it was already 0;
2. `grantPdfCredits` credits the balance once for a first-seen `grant_key` and returns `granted: false` with the unchanged balance for a duplicate `grant_key`;
3. the owner identity bypasses the ledger entirely (unlimited credits, no RPC/table call);
4. production without a configured privileged Supabase client fails closed for all three exported functions rather than silently touching the local file;
5. TypeScript, Vitest, production build and deployment-readiness remain green.

8. Decision

Accepted. PDF-export credit balance and grant accounting move from an ephemeral, non-atomic local JSON file to a durable, service-role-only Supabase ledger with atomic consume/grant functions, ported directly from the pattern ADR-0045 already proved in production for the Stripe event inbox. Because production deploys on merge, the additive Supabase migration is a mandatory pre-merge production gate. Generic worker durability remains a separate roadmap item, R-101.

9. Nachtrag 2026-08-15 — authentifizierter Client und sichere Exportreihenfolge

Die nach ADR-0003.5 gehärteten Endpunkte `GET /api/stripe/pdf-credits` und `POST /api/stripe/consume-pdf-credit` akzeptieren ausschließlich die durch den Bearer-Token verifizierte Supabase-Identität. Der Exportdialog rief beide Endpunkte jedoch weiterhin über nacktes `fetch()` auf. Damit fehlte der Authorization-Header, jeder reale Exportversuch endete fail-closed mit HTTP 401 und selbst Enterprise-Konten konnten serverseitig nicht als unbegrenzt erkannt werden.

Der Client verwendet deshalb für beide Ledger-Endpunkte die zentrale Funktion `src/lib/authFetch.ts`. E-Mail-Adresse oder andere clientseitige Identifier werden nicht mehr an den Balance-Endpunkt übertragen; maßgeblich bleibt ausschließlich `resolveVerifiedIdentity(req).userId` auf dem Server.

Zusätzlich wird der Ablauf in `src/lib/pdfExportFlow.ts` festgelegt:

1. PDF vollständig im Speicher vorbereiten;
2. bei begrenzten Konten den Credit authentifiziert und atomar verbrauchen;
3. nur nach erfolgreicher Credit-Entscheidung den vorbereiteten Download auslösen;
4. Enterprise überspringt ausschließlich die Credit-Mutation, nicht die Identitätsprüfung beim initialen Ledger-/Entitlement-Lookup.

Schlägt die PDF-Vorbereitung fehl, wird kein Credit verbraucht. Schlägt die Autorisierung oder Credit-Mutation fehl, wird der Download nicht ausgelöst. Unit- und statische Contract-Tests sichern Reihenfolge, Enterprise-Bypass, 401-/Fehlerverhalten und die Verwendung von `authFetch()` ab. Die serverseitige Ledger-, RLS- und RPC-Grenze bleibt unverändert.

ADR-0053 — Node.js 24 LTS & Git 2.55 Toolchain Baseline

Quelle: docs/adr/ADR-0053-node24-lts-git255-toolchain.md

ADR-0053 — Node.js 24 LTS & Git 2.55 Toolchain Baseline

Status: Accepted

Date: 2026-08-10

Last updated: 2026-08-11 (M3 CI Hardening)

Owner: CAPITAL-AI

Deutsch

Kontext

CAPITAL-AI verwendete vor diesem ADR Node.js 22 in GitHub Actions und in beiden Docker-Stages. Die GitHub-hosted Ubuntu-Runner liefern eine andere Git-Version als die für CAPITAL-AI gewählte Upstream-Baseline Git 2.55.0. Node.js 24.18.0 LTS ist die gewählte Produktions-, CI- und lokale Entwicklungs-Linie.

Das Produktions-Docker-Image bleibt über einen SHA-256-Digest unveränderlich gepinnt. Git 2.55.0 wird für vollständige Validierung aus dem offiziellen Source-Tarball von kernel.org gebaut und unter `/opt/git-2.55.0` installiert. Der optionale Rust-Unterbau wird mit `NO_RUST=YesPlease` deaktiviert, weil CAPITAL-AI in CI ausschließlich die klassische Git-CLI benötigt.

Entscheidung

1. Produktions- und CI-Runtime: Node.js 24.18.0 LTS.
2. Docker: `node:24.18.0-alpine` mit immutable Multi-Platform-Digest.
3. Lokale Node-Authority: `.nvmrc` mit `24.18.0`; `package.json` begrenzt auf die Node-24-LTS-Linie.
4. Git CI Baseline: exakt 2.55.0 für vollständige Validierungsläufe.
5. Git-Source wird über HTTPS von kernel.org bezogen.
6. Der Git-2.55.0-Tarball MUSS vor `xz --test`, Extraction und Build fail-closed gegen den im Repository/Workflow gepinnten SHA-256-Wert `457fdb04d-c8728e007d4688695e6912e6f680727920f2a40bf11eacc17505357` geprüft werden.
7. `xz --test` bleibt als zusätzliche Strukturprüfung bestehen, ist aber nicht mehr der alleinige Integritätsnachweis.
8. Optionale Rust-Git-Subsysteme werden im CI-Build mit `NO_RUST=YesPlease` deaktiviert.
9. Nach Checkout ist für vollständige Validierung `git fsck --strict --no-dangling` ein fail-closed Repository-Integritätsgate.

10. Keine Node-26-Current-Runtime in Produktion vor einer separaten LTS-/Compatibility-Entscheidung.
11. Der stabile Required Check bleibt technisch `build-and-test`.

M3 Addendum — Risk-based CI & kryptografischer Source-Pin (2026-08-11)

Anlass

Die M0/M2G-Evidence zeigte, dass selbst reine Dokumentations-PRs den vollständigen Git-2.55.0-Source-Build, `npm ci`, Unit-Tests und den Production Build ausführen. Dies verursacht vermeidbare GitHub-Actions-Kosten, obwohl der Docker-Build bereits scope-basiert übersprungen wurde. Gleichzeitig dokumentierte dieser ADR selbst, dass `xz --test` keine kryptografische Integritätsprüfung darstellt.

Ergänzende Entscheidung

CAPITAL-AI führt einen fail-closed, pfadbasierten CI-Fast-Path ein:

- `push` auf `main`: immer vollständige Validierung.
- Pull Requests mit Änderungen außerhalb von `docs/**`, `.ai/**` oder Markdown-Dateien: immer vollständige Validierung.
- Reine Dokumentations-PRs: der Required Check `build-and-test` bleibt vorhanden und erfolgreich ausführbar, überspringt jedoch Git-Source-Kompilation, Node/npm-Setup, Tests, Production Build, Predeploy und Docker-Build.
- Die separaten Governance-Workflows bleiben für Dokumentations-PRs aktiv; Repository-Konventionen und Workflow-Security werden dadurch nicht umgangen.
- Die Scope-Klassifikation erfolgt unmittelbar nach einem minimalen, SHA-gepinnten `actions/checkout` mit `persist-credentials: false`.
- Dieser initiale Checkout dient nur der Ermittlung des Diffs und der Commit-Existenz. Bei Vollvalidierung wird vor Ausführung von Repository-Code, Dependency-Installation, Tests oder Builds die verifizierte Git-2.55.0-Toolchain installiert.
- Damit wird die frühere Formulierung „Git 2.55.0 wird vor `actions/checkout` gebaut“ für den M3-Stand ersetzt.

Trust Boundary

Der initiale Scope-Checkout besitzt keine persistierten Zugangsdaten und verwendet ausschließlich eine auf vollständigen 40-Zeichen-SHA gepinnte Checkout-Action. Er darf keine Repository-Skripte, Package-Manager oder Build-Schritte ausführen. Sobald `full_validation=true` ist, gilt Git 2.55.0 als Toolchain-Authority für die folgenden Git-Integritäts- und Software-Validierungsschritte.

Kosten-/Sicherheitsauswirkung

Die Optimierung reduziert teure Linux-Actions-Minuten nur bei nachweislich dokumentationsreinem PR-Scope. Sie reduziert keine Tests für Code, Workflow, Dependencies, Runtime,

Docker oder Deployment. `main` bleibt immer vollständig validiert, sodass vor dem bestehenden Produktions-Deploy-Pfad weiterhin die vollständige CI durchlaufen wird.

Risiken und Rollback

Risiko: Eine zu breite Docs-only-Klassifikation könnte eine relevante Änderung fälschlich in den Fast Path einordnen. Deshalb ist die Allowlist bewusst eng; jede nicht eindeutig dokumentarische Datei fällt automatisch auf vollständige Validierung zurück.

Rollback: Die Scope-Bedingungen können entfernt und alle Schritte wieder unconditional ausgeführt werden. Der SHA-256-Pin für den Git-Tarball darf bei einem Rollback der Kostenoptimierung nicht entfernt werden.

Verification

M3 ist erst abgeschlossen, wenn: - ein Workflow-ändernder PR die vollständige Validierung erfolgreich durchläuft; - die SHA-256-Prüfung des Git-Tarballs in diesem Lauf erfolgreich ist; - ein nachgelagerter reiner Dokumentations-PR den Fast Path nachweislich nutzt und trotzdem `build-and-test` + Governance erfolgreich meldet; - `main` nach Merge weiterhin vollständige Validierung ausführt.

ADR-ID-Integrität

Der Toolchain-ADR verwendet `ADR-0053`, da `ADR-0052` der Transactional PDF-Credit Ledger Authority ist.

English summary

CAPITAL-AI standardizes production/CI on Node.js 24.18.0 LTS and full-validation Git operations on Git 2.55.0. M3 adds a pinned SHA-256 verification for the upstream Git tarball before extraction/build and introduces a conservative documentation-only PR fast path. The required check remains `build-and-test`; every `main` push and every non-documentation change still receives full validation. The initial checkout used for scope classification is immutable-SHA pinned and does not persist credentials or execute repository code.

ADR-0054 — Durable Worker/Outbox/Lease

Quelle: docs/adr/ADR-0054-durable-worker-outbox-lease.md

ADR-0054 — Durable Worker/Outbox/Lease

- **Status:** Accepted
- **Implementation-Status:** VERIFIED PASS IN PRODUCTION
- **Date:** 2026-08-10
- **Scope:** CAPITAL-AI generic durable job retry infrastructure (first consumer: checkout-confirmation e-mail)
- **Platform Version:** 0.6.0
- **Roadmap:** R-101 — Durable worker/outbox/lease
- **Related:** ADR-0043, ADR-0045 (R-003), ADR-0052 (R-004)

1. Context

ADR-0045 (R-003) established a durable Stripe event inbox and explicitly deferred "a generic queue, worker lease, outbox or dead-letter system" to this workstream. ADR-0052 (R-004) repeated the same deferral for the credit ledger. Both ADRs list the concern in their ownership table as "durable worker/outbox/lease orchestration | deferred to R-101" and both name R-101 as the follow-up.

The concrete motivating failure is already recorded in production, not hypothetical. ROAD-MAP.md OPS-001 documents a real 2026-08-10 production log: the checkout-confirmation SMTP attempt (`server/mailer.ts -> sendMail`) failed with `535 Invalid login: Authentication credentials invalid`. The durable-inbox/reservation logic itself behaved correctly — `claim_subscription_confirmation(session_id)` (ADR-0045 §5) gave exactly one caller the reservation, so there was no duplicate attempt — but because the reservation is consumed at "attempted," not "delivered," the SMTP failure was terminal for that Checkout Session. No later automated retry occurs today; the customer confirmation mail can never be sent again through the direct-send code path. ADR-0045 §5 already named this trade-off explicitly: "SMTP delivery itself remains best-effort. This ADR guarantees reservation/idempotency ownership, not guaranteed message delivery."

R-101 generalizes a fix for this class of problem instead of hand-rolling a one-off retry for this one call site.

2. Decision

CAPITAL-AI establishes one generic, durable, service-role-only job outbox:

```
public.outbox_jobs
```

Any application code that wants a side effect retried with backoff and eventually dead-lettered enqueues a row instead of executing the side effect inline and discarding the

outcome on failure. Four `SECURITY DEFINER` functions are the atomic boundary, directly mirroring the shape ADR-0045 and ADR-0052 already proved in production:

- `public.enqueue_outbox_job(job_type, payload, idempotency_key, max_attempts, available_at)` — inserts a job, or returns the existing job's id unchanged when `idempotency_key` already exists.
- `public.claim_outbox_job(lease_owner, lease_seconds)` — atomically leases one eligible job (`FOR UPDATE SKIP LOCKED`), incrementing `attempts` .
- `public.complete_outbox_job(job_id, lease_owner)` — marks a held lease `succeeded` .
- `public.fail_outbox_job(job_id, lease_owner, error, backoff_seconds)` — below `max_attempts` , reschedules the job (`pending` , `available_at` pushed out by the backoff); at or beyond `max_attempts` , moves it to `dead_letter` .

`server/outbox.ts` is the sole application-side module that touches these functions, mirroring `server/stripeEventInbox.ts`'s and `server/pdfCreditLedger.ts`'s structure (production fail-closed via `assertPrivilegedSupabaseConfigured()` , a no-op skip — not a crash — when Supabase is legitimately absent in local development). `server/outboxWorker.ts` is a poll loop with a `job_type -> handler` registry: `claimOutboxJob()` -> dispatch to the registered handler -> `completeOutboxJob()` on success or `failOutboxJob()` on a thrown error. A `job_type` with no registered handler is itself recorded as a failure rather than silently dropped.

The ownership split (extending ADR-0045 §2 / ADR-0052 §2's table):

Concern	Authoritative owner
Generic durable job retry (enqueue/claim/complete/fail)	<code>public.outbox_jobs</code> via <code>server/outbox.ts</code>
Outbox job dispatch to a handler	<code>server/outboxWorker.ts</code>
Checkout-confirmation mail retry (first concrete consumer)	<code>subscription_confirmation_mail</code> job type, handled by <code>server/mailer.ts</code> 's <code>processSubscriptionConfirmationMailJob</code>
Stripe event-ID deduplication	<code>public.stripe_event_inbox</code> (ADR-0045, unchanged)
PDF-credit grant/consume atomicity	<code>public.pdf_credits</code> / <code>grant_pdf_credits</code> / <code>consume_pdf_credit</code> (ADR-0052, unchanged)

3. Idempotent-handler contract and the reclaim divergence from ADR-0045

ADR-0045 §3 rule 5 deliberately refuses to auto-reclaim a stuck `processing` Stripe event row: "automatically replaying a stale `processing` event could duplicate a side effect after a crash between side-effect completion and inbox finalization" — a real risk at the time, because the side effects it guarded (PDF-credit grants) were not yet transactional.

R-101 makes the opposite, and equally deliberate, choice: `claim_outbox_job` **does** automatically reclaim a job whose lease has expired (`status = 'processing'` and `lease_expires_at < now()`) and re-dispatches it. This is safe specifically because R-101's contract requires

every registered `job_type` handler to be idempotent — safely re-runnable with the same payload, including after a crash between handler completion and `complete_outbox_job()`. This is not a hypothetical constraint for the one handler this PR ships: `processSubscriptionConfirmationMailJob` calls `sendMail()` again, which is inherently at-least-once (SMTP itself gives no exactly-once guarantee, and neither did the pre-R-101 direct-send path). A future job type that is not safely re-runnable must not be registered against this worker without first making its side effect idempotent (e.g. the same `grant_pdf_credits`-style `ON CONFLICT` idempotency key pattern), the same discipline ADR-0052 applied to make the PDF-credit grant safe to sit behind ADR-0045's inbox.

4. Outbox contract

`outbox_jobs`: `id` (uuid, surrogate key), `job_type`, `idempotency_key` (nullable, unique when present — Postgres unique indexes treat `NULL` as distinct, so unkeyed jobs are never duplicated against each other), `payload` (jsonb), `status` (`pending` / `processing` / `succeeded` / `failed` / `dead_letter`), `attempts`, `max_attempts` (default 5), `available_at` (claim eligibility / backoff scheduling), `lease_owner`, `lease_expires_at`, `last_error`, `created_at`, `updated_at`.

`claim_outbox_job` selects one candidate — pending and due, or processing with an expired lease — ordered by `available_at`, via `FOR UPDATE SKIP LOCKED` so multiple worker instances (multiple Render dynos, if CAPITAL-AI scales beyond one) can poll concurrently without blocking each other. `complete_outbox_job` and `fail_outbox_job` both require the caller to hold the current `lease_owner`, so a worker that lost its lease to a reclaim cannot finalize a job it no longer owns. `fail_outbox_job` compares `attempts` to `max_attempts` to decide between rescheduling with backoff and moving to `dead_letter`.

`dead_letter` rows are not deleted or archived elsewhere. They remain directly queryable by any service-role Supabase access (`select * from outbox_jobs where status = 'dead_letter'`) — the same "visible reconciliation condition, not an invitation to guess" posture ADR-0045 established for stuck inbox rows, adapted from "manual reconciliation of a stuck claim" to "manual reconciliation of an exhausted retry budget."

5. First concrete consumer: checkout-confirmation mail retry

`server/mailer.ts`'s `sendSubscriptionConfirmation()` is unchanged up through the SMTP attempt. When the customer or owner leg's `sendMail()` call reports failure, it now additionally calls `scheduleConfirmationMailRetry(kind, sessionId, to, message)`, which enqueues a `subscription_confirmation_mail` job with `idempotency_key = "subscription_confirmation_mail:<sessionId>:<kind>"` — so even a hypothetical duplicate scheduling attempt for the same Checkout Session and recipient kind cannot enqueue two retry jobs. Scheduling is best-effort and never throws into the webhook response path, matching `sendMail()`'s own never-throws contract: a failure to enqueue is logged and the request completes exactly as it did before this ADR.

`processSubscriptionConfirmationMailJob` (the registered handler) re-attempts `sendMail()` from the stored payload and throws on failure so the worker records a backoff retry or

eventual dead-letter. It does not re-check the Checkout Session reservation, since that reservation was already durably won before the job was ever scheduled.

6. Security and database boundary

`outbox_jobs` follows the exact `stripe_event_inbox / pdf_credits` pattern:

- RLS enabled;
- no `anon / authenticated` table privileges;
- service-role `SELECT`, `INSERT`, `UPDATE` only;
- all four functions are `SECURITY DEFINER` with an explicit `search_path = public, pg_temp`;
- execute privilege is granted only to `service_role`.

`server/outbox.ts` uses the privileged server Supabase client defined by ADR-0043. No publishable/anon fallback is permitted.

7. Production handoff and deployment order

Production currently deploys on merge to `main` via a GitHub Actions `deploy_hook` step. The database migration MUST therefore be applied before the PR is merged, the same constraint ADR-0045 §7 and ADR-0052 §5 already established:

1. obtain review approval for the PR, but keep it unmerged;
2. apply the additive migration `supabase/migrations/20260810160000_outbox_jobs.sql` to the production Supabase project through the controlled production handoff;
3. verify `outbox_jobs`, `enqueue_outbox_job(...)`, `claim_outbox_job(...)`, `complete_outbox_job(...)` and `fail_outbox_job(...)` exist and remain service-role-only;
4. verify the currently deployed application remains healthy against the additive, unused database objects;
5. merge the reviewed PR, which fires the `deploy_hook`;
6. verify `/healthz` and deployment identity after the deploy becomes live;
7. verify the outbox worker poll loop is running (application logs) and that a real SMTP failure (or a manually enqueued test job) is retried with backoff and eventually either succeeds or reaches `dead_letter` after `max_attempts`.

No live Supabase, Render or Stripe mutation is performed by the development branch or by this ADR. Application deployment before the database migration is intentionally unsupported because `server/outbox.ts` fails closed (`assertPrivilegedSupabaseConfigured`) in production when the durable outbox contract is unavailable.

7.1 Production closure evidence (2026-08-14)

The Owner authorized the production migration explicitly after a read-only preflight. The exact repository migration `supabase/migrations/20260810160000_outbox_jobs.sql` was applied to the sole active production project `AIFINANCIAL` and registered as Supabase migration `20260814221830_outbox_jobs`.

Post-mutation verification proved:

- `public.outbox_jobs` exists with RLS enabled and zero pre-existing rows;
- all four RPCs exist as `SECURITY DEFINER` functions with explicit signatures;
- `anon` and `authenticated` have no execute privilege;
- `service_role` has execute privilege and the only table policy is `service_role_full_access`;
- an empty `claim_outbox_job` verification returned no job and created no test data;
- the repeating Render schema-cache error stopped immediately after the migration;
- the subsequent live deploy `dep-d9vpf3dbedkc73et92ag` at commit `ef711596f0020764441ff6ea9e1cf350b6ad2c1a` produced no Outbox or application errors.

Canonical evidence: `docs/evidence/r101/R101_OUTBOX_PRODUCTION_MIGRATION_EVIDENCE.md`.

8. Explicit non-goals

R-101 does NOT:

- migrate the Stripe-event inbox (ADR-0045) or the PDF-credit ledger (ADR-0052) onto the generic outbox — both remain on their existing, already-production-proven claim tables; only the confirmation-mail retry gap identified by OPS-001 is wired to the new outbox in this PR;
- introduce a distributed task scheduler, cron system, or priority queue — `claim_outbox_job`'s ordering is `available_at` only;
- change `claim_subscription_confirmation`'s reservation semantics (ADR-0045 §5) — the reservation is still won exactly once, before any SMTP attempt, exactly as before;
- guarantee eventual delivery — a job that keeps failing past `max_attempts` reaches `dead_letter` and requires manual reconciliation, the same bounded guarantee ADR-0045 gives for a stuck inbox row.

9. Validation contract

Automated validation MUST prove:

1. `enqueueOutboxJob` inserts a new job and returns its id, and returns `enqueued: false` with the existing job's id for a duplicate `idempotencyKey`;
2. `claimOutboxJob` returns the claimed job's fields when the RPC returns a row, and `null` when nothing is claimable;
3. `completeOutboxJob` / `failOutboxJob` pass through the RPC's boolean/outcome result;
4. the outbox worker completes a job whose handler succeeds, records a failure (never throwing) when a handler rejects, and records a failure for a `job_type` with no registered handler;
5. `sendSubscriptionConfirmation` schedules exactly one retry job per failed leg (keyed by session + recipient kind) and never throws when scheduling itself fails;

6. production without a configured privileged Supabase client fails closed for `enqueueOutboxJob / claimOutboxJob / completeOutboxJob / failOutboxJob` rather than silently degrading;
7. TypeScript, Vitest, production build and deployment-readiness remain green.

10. Decision

Accepted. CAPITAL-AI establishes a generic, service-role-only, lease-based job outbox — `outbox_jobs` plus `enqueue_outbox_job / claim_outbox_job / complete_outbox_job / fail_outbox_job` — ported directly from the claim-table pattern ADR-0045 and ADR-0052 already proved in production, with one deliberate divergence: because every registered job handler must be idempotent, a stale lease is safely auto-reclaimed instead of left for manual reconciliation. The first concrete consumer closes the OPS-001 gap: a failed checkout-confirmation SMTP send is now retried with backoff instead of permanently lost. Because production deploys on merge, the additive Supabase migration is a mandatory pre-merge production gate.

ADR-0055 — Event-Driven CAPITAL-AI Value Chain Governance Integration

Quelle: docs/adr/ADR-0055-event-driven-value-chain-governance-integration.md

ADR-0055 — Event-Driven CAPITAL-AI Value Chain Governance Integration

Renumbering note (2026-08-11): originally filed as `ADR-0045`, which collided with the already-established `ADR-0045-stripe-event-ownership-durable-inbox.md` (2026-08-08, heavily referenced by ADR-0052, ADR-0054, ROADMAP.md and multiple call sites). Renumbered to `ADR-0055` to resolve the collision; no content changed beyond the number.

Status: Proposed

Datum / Date: 2026-08-09

Authority: ESS-0001-CONTRACTS

Related: ESS-0003, ESS-0009, ESS-0010, ESS-0011, ESS-0012, ADR-0044

Deutsch

Kontext

CAPITAL-AI besitzt bereits Platform Director, Supervisor, Documentary, Knowledge, Traceability, Quality, Security, Compliance, Versioning und Release-Komponenten. Neue Ereignisse sollen diese Komponenten künftig autonom und deterministisch miteinander verzahnen, statt isolierte manuelle Folgearbeiten zu erzeugen.

Entscheidung

CAPITAL-AI verwendet ein standardisiertes Domain-Event-Modell für relevante Lifecycle-Änderungen. Ereignisse werden durch einen zentralen Event-Router an zuständige Plattformkomponenten verteilt.

Vorgesehene Ereignisklassen umfassen mindestens: - requirement.created - architecture.decision.accepted - vocabulary.updated - implementation.started - component.changed - contract.changed - documentation.updated - traceability.updated - quality.validated - compliance.validated - release.candidate.created - deployment.validated - release.published

Jedes Event enthält mindestens eventId, eventType, schemaVersion, occurredAt, source, aggregateType, aggregateId, actor, correlationId, optional causationId und Evidence-Referenzen.

Orchestrierungsregel

Automatische Folgeprozesse dürfen Knowledge, Documentary, Traceability, Quality, Compliance und Release-Evidence aktualisieren. Human-Approval-Gates für neue PRs, geschützte Architektur-/Security-Änderungen, produktionskritische Mutationen und Releases bleiben verbindlich.

Fehler- und Wiederholungsmodell

Event Handler müssen idempotent, nachvollziehbar und fehlertolerant implementiert werden. Ein Fehler in einem Consumer darf keine unkontrollierte Teilmutation auslösen. Wiederholungen müssen anhand eventId/correlationId erkennbar sein.

English

Context

CAPITAL-AI already contains Platform Director, Supervisor, Documentary, Knowledge, Traceability, Quality, Security, Compliance, Versioning, and Release capabilities. New events should connect these components autonomously and deterministically instead of creating isolated manual follow-up work.

Decision

CAPITAL-AI adopts a standardized domain-event model for relevant lifecycle changes. A central event router distributes events to responsible platform components.

The lifecycle includes requirement, architecture, vocabulary, implementation, component, contract, documentation, traceability, quality, compliance, release-candidate, deployment-validation, and release events.

Every event carries stable identity, version, timestamp, source, aggregate context, actor context, correlation, optional causation, and evidence references.

Orchestration rule

Automation may update Knowledge, Documentary, Traceability, Quality, Compliance, and release evidence. It must not bypass human approval gates for PR creation, protected architecture/security changes, production-critical mutations, or releases.

Reliability model

Event handlers must be idempotent, observable, and failure-tolerant. Consumer failures must not cause uncontrolled partial mutation, and replay must be detectable through event and correlation identifiers.

ADR-0056 — Observability & Telemetry Baseline

Quelle: docs/adr/ADR-0056-observability-telemetry-baseline.md

ADR-0056 — Observability & Telemetry Baseline

Renumbering note (2026-08-11): originally filed as `ADR-0050`, which collided with the already-established `ADR-0050-agent-tool-capability-iam-foundation.md` (2026-08-09, ESS-0018 Phase 1, referenced by ADR-0051 and multiple `server / src` call sites). Renumbered to `ADR-0056` to resolve the collision; no content changed beyond the number.

Status: Proposed

Date: 2026-08-10

Kontext

CAPITAL-AI besitzt bereits mehrere getrennte Evidence- und Betriebsbausteine: einen strukturierten JSON-Logger mit Request-ID, IAM-/Step-Up-Auditpfade, Screening-SLA-Logik, Provider Routing, Compliance Evidence, Supervisor, EventMesh und Deployment-Identität. Gleichzeitig existieren weiterhin zahlreiche `console.*`-Aufrufstellen und die Plattformkomponente `src/platform/Telemetry` war bislang nur ein als `unspecified` markierter Bootstrap-Platzhalter ohne Code.

Ein neues paralleles Logging-/Monitoring-System würde die bestehende Architektur fragmentieren. Für eine FinTech-Anwendung müssen zudem Operational Telemetry und revisionsrelevante Audit Evidence klar getrennt bleiben.

Entscheidung

`src/platform/Telemetry` wird zur vendor-neutralen Contract- und Governance-Schicht für Operational Telemetry ausgebaut. `server/logger.ts` bleibt die operative Logger-Authority und wird schrittweise an diesen Contract angebunden.

O1 führt ein:

- ein versioniertes `TelemetryRecord`-Schema;
- kanonische FinTech-Wertschöpfungsstufen;
- standardisierte Event-Namen, Outcomes und Durations;
- Request-/Trace-/Span-Kontext im Contract;
- zentrale Secret-/PII-Redaction;
- Service-, Environment-, Version- und Commit-Kontext im bestehenden Logger;
- standardisierte Request-Abschluss-Telemetrie;
- Contract- und Redaction-Tests.

OpenTelemetry wird als spätere vendor-neutrale Export-/Tracing-Schicht vorgesehen, aber in O1 noch nicht als externe Dependency oder Collector eingeführt.

FinTech-Wertschöpfungskette

Operational Telemetry wird auf folgende Stufen normalisiert:

request-intake → identity-access → entitlement-usage → orchestration → market-data-provider
→ data-validation → scoring-analysis → explainability → billing → output-delivery →
deployment-runtime

Diese Kette ermöglicht spätere End-to-End-SLIs, ohne Domain-Auditdaten in Logs zu replizieren.

Sicherheits- und Compliance-Grenzen

1. Secrets, Tokens und direkte PII dürfen nicht in Operational Telemetry persistiert werden.
2. Generische Request-Telemetry enthält weder Request-Body noch Query-String.
3. Audit Evidence für IAM, Break-Glass, Billing, Compliance oder Governance bleibt bei den zuständigen Authorities und wird höchstens per `auditReference` referenziert.
4. Telemetry ersetzt weder EventMesh noch Traceability noch Compliance Evidence.
5. Externe Observability-Vendor werden nicht in Domain-Code eingebettet.

Konsequenzen

Positiv:

- bestehende Logging-Infrastruktur wird konsolidiert statt dupliziert;
- FinTech-Wertschöpfung wird messbar;
- Datenminimierung und Redaction werden zentral erzwungen;
- spätere OpenTelemetry-/Collector-Integration bleibt vendor-neutral;
- Deployment-SHA kann mit Runtime-Telemetry korreliert werden.

Trade-offs:

- vorhandene `console.*`-Aufrufe bleiben zunächst technische Schuld und werden in O2 schrittweise migriert;
- O1 liefert noch keine externen Dashboards, Traces oder SLO-Alerts;
- Audit- und Operational-Evidence müssen bewusst getrennt gepflegt werden.

Folgephasen

O2 Structured Logging & Error Taxonomy.

O3 Metrics.

O4 OpenTelemetry / Distributed Tracing.

O5 FinTech Business Telemetry.

O6 SLI/SLO & Alerting.

O7 Production Evidence.

O8 Continuous Governance.

Verifikation

O1 ist erfüllt, wenn:

- der Telemetry-Contract typisiert und getestet ist;
- Secret-/PII-Redaction getestet ist;
- der zentrale Logger Redaction und Deployment-Kontext verwendet;
- Request-Abschlussmetrik Request-ID, Status und Dauer liefert;
- `/healthz` nicht als regulärer Business-/Request-Traffic gezählt wird;
- TypeScript, Unit Tests, Production Build und Docker-Hardening-CI grün sind.

ADR-0057 — Provider-Neutral Agent Control Plane and Plane Separation

Quelle: docs/adr/ADR-0057-provider-neutral-agent-control-plane-and-plane-separation.md

ADR-0057 — Provider-Neutral Agent Control Plane and Plane Separation

Status: ACCEPTED (provider-set clarified 2026-08-16) Date: 2026-08-11

Context

CAPITAL-AI is operated through multiple AI surfaces. Direct provider-specific privilege creates inconsistent authorization, audit and rollback semantics.

Owner clarification 2026-08-16: The active DEVELOPMENT Chain / AI value-chain agent providers are **ChatGPT, Claude and Grok**. Google AI Studio, NotebookLM and Gemini are **not** part of the active value chain (retired aliases in the control plane).

Decision

Adopt a provider-neutral Agent Control Plane as the only trust root for AI-assisted execution. Separate Research & Evidence, Agent Execution, Control and Production planes. Provider products are profiles/clients, never authority sources.

Canonical profiles (registry):

- `chatgpt-github-connector` — Research + controlled Execution
- `claude-code-cli` — controlled Execution
- `grok-xai-connector` — Research + controlled Execution

Retired aliases resolve to DENY/RETIRED: `google-ai-studio`, `notebooklm`, `gemini`.

Alternatives rejected

- Provider-specific direct admin access: rejected for privilege drift and audit fragmentation.
- Single-provider lock-in: rejected for resilience and portability.
- Model-prompt-only policy: rejected because prompts are not enforceable authorization controls.

Security

Deny-by-default, least privilege, no implicit privilege inheritance, no AI self-approval for HIGH/CRITICAL actions.

Consequences

Requires common identity/capability/audit contracts and provider adapters. Enables future provider replacement without rewriting production authorization.

Migration

Document-first M2; implementation via M8 provider profiles. Provider-set correction evidence: `docs/evidence/m8/M8_PROVIDER_SET_CORRECTION_2026-08-16.md`.

Rollback

Revert control-plane adoption before cutover; existing provider profiles remain capability-gated until full M8 evidence.

Verification

ESS-0019 plus traceability matrix must map every provider surface to one plane and capability set. Unit tests assert three canonical profiles and RETIRED for former Google surfaces.

ADR-0058 — Agent Identity, Capability and Risk Authorization

Quelle: docs/adr/ADR-0058-agent-identity-capability-and-risk-authorization.md

ADR-0058 — Agent Identity, Capability and Risk Authorization

Status: ACCEPTED — M4 CLOSURE HARDENING IN REVIEW Date: 2026-08-11

Context

Provider identities and model names are not sufficient authorization principals. CAPITAL-AI already has Supabase/tool-specific capability grants and single-use approvals under ADR-0050/0051 and ESS-0018. M4 adds a provider-neutral authorization layer above those existing controls without replacing them.

PR #198 merged the canonical M4 implementation into `main` at `69f719683b60ba6aad-c0022381c6cecc430f0ea5`. Parallel M4 drafts #200 and #201 were created from the older PR-#197 baseline and are superseded by this consolidated closure state.

Decision

Authorize attributable principals through explicit, non-inheriting `DevelopmentChain` capabilities and deterministic minimum risk classes.

Canonical agent capabilities are:

`READ`, `ANALYZE`, `PLAN`, `BRANCH`, `COMMIT`, `PR`, `CI_REQUEST`, `DEPLOY_REQUEST`, `PRODUCTION_MUTATION`.

`MERGE` is intentionally not an agent capability. It remains a Human/Owner-controlled repository transition.

Minimum risk classes are:

- LOW: `READ`, `ANALYZE`, `PLAN`;
- MEDIUM: `BRANCH`, `COMMIT`, `PR`, `CI_REQUEST`;
- HIGH: `DEPLOY_REQUEST`;
- CRITICAL: `PRODUCTION_MUTATION`.

Context may raise risk but may never lower these minimums.

Authorization is deny-by-default and binds at least:

- authenticated human actor;
- app/client identity;
- logical agent identity;
- session identity;

- request identity;
- credential-holder identity;
- target resource;
- environment;
- requested capability;
- applicable approval/step-up evidence.

Provider/model identifiers are metadata only and never grant authority.

HIGH actions require explicit, current Human Approval. CRITICAL actions require the same Human Approval plus verified Step-up. Approval evidence must be unexpired and bound to the same human actor, logical agent, capability and target resource. Agent/App/Credential principals may not self-approve.

Existing ADR-0050/0051 and ESS-0018 remain the product/tool implementation profile and are enforced in addition to this provider-neutral layer.

M4 implementation

`src/platform/Security/agentIam.ts` implements the provider-neutral decision surface with:

- complete principal attribution including request and credential holder;
- deterministic capability minimum-risk mapping;
- exact grants only; no privilege inheritance;
- environment boundary preventing development principals from `PRODUCTION_MUTATION`;
- approval actor/agent/target/capability binding and expiration validation;
- HIGH Human Approval and CRITICAL Human Approval + Step-up semantics;
- mutation kill switch;
- explicit denial of unknown/non-agent capabilities such as `MERGE`.

`src/platform/Compliance/PolicyGate.ts` exposes the generic Agent IAM check while retaining its existing ESS-0018 read/write allowlists as a second independent policy layer.

Alternatives

Provider-based allowlists, model-name trust, prompt-only constraints, wildcard capabilities and implicit capability inheritance are rejected.

Security

Retrieved/tool content cannot change authorization. `DEPLOY_REQUEST` never implies `PRODUCTION_MUTATION`. `COMMIT` never implies `PR`; `PR` never implies `CI_REQUEST`; `CI_REQUEST` never implies `DEPLOY_REQUEST`. Human/Owner merge governance remains outside the agent runtime authorization model.

Migration

M4 is additive and requires no production Supabase/Stripe/Render mutation and no new database migration. Existing Supabase capability grants and approval artifacts remain unchanged.

Rollback

Disable mutating agent capabilities and fall back to READ/ANALYZE. Removal of the provider-neutral layer does not modify the existing ADR-0050/0051 persistence model.

Verification

M4 closure requires green TypeScript/unit/build/governance validation plus negative tests for:

- incomplete principal attribution;
- missing exact grants;
- provider/model privilege non-inheritance;
- capability privilege non-inheritance;
- deterministic HIGH/CRITICAL minimum risk;
- missing/mismatched/expired approval evidence;
- approval subject-agent mismatch;
- missing Step-up for CRITICAL;
- self-approval;
- kill-switch enforcement;
- attempted `MERGE` delegation.

M5 remains blocked until the single M4 consolidation/closure PR is Human/Owner-reviewed and merged.

ADR-0059 — Agent Execution Audit and OpenTelemetry Correlation

Quelle: docs/adr/ADR-0059-agent-execution-audit-and-otel-correlation.md

ADR-0059 — Agent Execution Audit and OpenTelemetry Correlation

Status: ACCEPTED — CORRECTIVE APPLICATION REMEDIATION ACTIVE Date: 2026-08-11
Last updated: 2026-08-12

Decision

Every AI-assisted command is correlated end-to-end with `request_id`, `trace_id` and attributable actor/agent identity. OpenTelemetry and W3C Trace Context are the preferred neutral correlation standards. Security audit evidence is a separate durability/retention class from sampled operational telemetry.

Minimum audit fields: `human_actor_id`, `app_id`, `agent_id`, provider/model metadata, intent, scope, capability, `risk_class`, `policy_id/version`, authorization decision, approval/step-up reference, tool/command, repository/branch/commit/PR, CI run, artifact digest, deployment/runtime identity, result/error, rollback reference and timestamps.

M5 architecture decision

M5 extends the existing ADR-0056 / `src/platform/Telemetry` baseline and does not introduce a second logging stack.

Operational Telemetry: - remains vendor-neutral in `src/platform/Telemetry`; - may be sampled/exported later; - carries request/trace/span correlation and an optional audit reference; - MUST NOT become the durable security-audit store.

Security Audit Evidence: - uses dedicated durable append-only persistence; - MUST support reconstruction of an authorized AI-assisted change across actor, agent, policy, PR/CI and runtime evidence; - MUST be redacted before persistence; - MUST NOT store secrets, tokens, complete prompts, complete diffs or raw sensitive request bodies.

Supabase persistence decision

Read-only repository and production-schema assessment on 2026-08-12 found that existing `audit_logs_iam`, `iam_access_log` and `agent_action_approvals` did not carry the full ADR-0059 correlation contract. No dedicated agent execution audit table existed.

Therefore the M5 Supabase mutation was classified **REQUIRED** and was executed after Human/Owner approval of PR #206.

Production target: `public.agent_audit_events` in Supabase project `AIFINANCIAL` (`ryzywoktpmy-hwzxmstyu`).

Applied migrations: - `20260811230540_m5_agent_audit_events`; - `20260811230743_m5_agent_audit_events_least_privilege`.

The table is append-only, RLS-enabled and deny-by-default for `anon / authenticated`. `service_role` is explicitly limited to `SELECT + INSERT`.

The migration and actual production schema are the canonical persistence contract. Application code **MUST** adapt to that schema; production evidence **MUST NOT** be destructively reshaped to accommodate application-only aliases.

Application integration decision

The M5 application integration is implemented as a server-only adapter and requires **no additional Supabase schema mutation**:

- `server/agentAudit/agentAuditWriter.ts` is the single durable writer for `agent_audit_events`;
- the writer reuses `src/platform/Telemetry/redaction.ts` for secret/PII redaction;
- complete prompts, complete diffs and raw request/response bodies are omitted before persistence;
- `server/agentAudit/authorizedAgentExecution.ts` composes the existing provider-neutral PolicyGate with durable audit persistence;
- persistence returns an `auditReference` suitable for operational telemetry correlation;
- terminal `SUCCESS / ERROR` is persisted as a second append-only event correlated to the authorization evidence;
- persistence errors propagate and therefore fail closed instead of silently producing an unaudited authorization result.

The existing synchronous `evaluateAgentPolicy()` remains unchanged for compatibility. Server-side agent execution that requires ADR-0059 evidence must use the audited adapter.

Corrective addendum — runtime schema-contract verification

The real SA3B post-merge host probe on 2026-08-12 exposed an application/schema integration defect that mocked tests had not caught.

Canonical production migration fields include:

`human_actor_id, intent, scope, authorization_decision, approval_reference, step_up_reference, tool_name, branch, commit_sha, pull_request_number, ci_run_id, attributes`.

The pre-remediation application writer sent legacy/application-only aliases including:

`actor_id, decision, approval_id, tool_id, pr_number, workflow_run_id, metadata`

and omitted required `intent / scope`.

Decision:

1. **Do not mutate or roll back the production M5 schema to match the buggy writer.**
2. Correct the writer and audited callers to the canonical migration contract.
3. Add a migration↔writer CI contract test so this class of drift fails before deployment.
4. Never fabricate a Supabase UUID from a GitHub login or other external identity: - syntactically valid UUID may populate the database UUID field; - non-UUID external identifiers remain attributable in sanitized `attributes`, while the corresponding UUID column is `NULL`.
5. Apply the same non-coercion rule to approval and step-up UUID references.
6. Keep all persistence failures fail-closed.

Corrective repository branch: `fix/sa3b-m5-audit-schema-contract`.

This addendum changes the application mapping, **not** the accepted append-only persistence architecture and not the production schema.

Production credential boundary discovered by SA3B

The same live probe reached the production broker with valid GitHub OIDC but durable persistence returned `Unregistered API key`.

That credential failure is separate from the writer/schema defect. Production secret values are Owner-controlled and must not be written to repository evidence, PR text or logs. A valid backend-only Supabase secret/service-role credential must be restored and verified before the positive SA3B probe can complete.

Security

1. Redaction occurs before persistence/export.
2. Secrets/tokens/full prompts/full diffs are prohibited by default.
3. `anon` and `authenticated` have no direct access to the audit store.
4. UPDATE/DELETE of durable evidence fail closed via table privileges and append-only trigger.
5. Audit correlation IDs are metadata, not authorization credentials.
6. W3C `traceparent` / `tracestate` are treated as untrusted inbound context and validated at trust boundaries.
7. Operational telemetry retention/sampling MUST NOT delete or weaken required audit evidence.
8. Provider/model metadata is recorded for provenance only and never grants authority.
9. Application audit persistence failure is fail-closed for the audited server execution path.
10. Only syntactically valid `supabase:agent_audit_events:<id>` references may bypass generic authorization-key redaction; tokens/headers and malformed values remain redacted.

11. External identity strings **MUST NOT** be coerced into database UUID columns.
12. Canonical migration/application mapping is a CI-enforced contract.

Verified production persistence evidence

The production mutation passed: - RLS verification; - least-privilege verification; - synthetic redacted event write/read; - UPDATE negative test; - DELETE negative test; - Supabase Security Advisor review.

Evidence authority: `docs/evidence/m5/M5_SUPABASE_AGENT_AUDIT_MUTATION_EVIDENCE.md`.

The Advisor's `RLS Enabled No Policy INFO` is intentional for this deny-by-default table. Existing Auth warnings for leaked-password protection and MFA are independent of M5 and tracked under M5A.

Historical M5 application validation

PR #210 completed the original application integration and was merged on 2026-08-12.

- final PR head: `f feab08c218314edd5292fbaf5ccb77413cee80e` ;
- merge commit: `e39d5370d8b1498e84952535a38a339cc200082f` ;
- required CI run #892 / run id `31559124198` : PASS;
- Human-/Owner gate: PASS;
- TypeScript/lint: PASS;
- unit tests including audit-specific negative tests: PASS;
- production build/predeploy: PASS;
- Docker hardening and production image verification: PASS.

Those tests used a mocked Supabase client and therefore proved fail-closed adapter behavior but did **not** prove the exact production column vocabulary. The SA3B runtime probe supplied the missing integration evidence and revealed the contract drift.

Rollback

No destructive database rollback is authorized by this ADR.

For application integration, rollback is code-only: revert the application adapter change while preserving all previously persisted audit evidence. After productive audit evidence exists, writers are disabled first and evidence preserved before any future schema remediation.

Verification state

The append-only production table, RLS and least-privilege controls remain verified.

The application writer is under corrective verification until the schema-contract remediation passes final CI, Human merge, deployment and a real successful audit insert. SA3B and

later autonomous mutation stages **MUST** remain blocked while this corrective verification is incomplete.

ADR-0060 — Software Supply Chain Provenance and Attestation

Quelle: docs/adr/ADR-0060-software-supply-chain-provenance-and-attestation.md

ADR-0060 — Software Supply Chain Provenance and Attestation

Status: PROPOSED Date: 2026-08-11

Decision

All human- and agent-authored changes share one supply-chain path: source SHA -> dependency lock -> SBOM -> build -> artifact digest -> provenance -> attestation -> deployment -> runtime identity. M6 maps controls to SLSA, OpenSSF and NIST SSDF.

Actions and dependencies must be immutable/pinned where supported. Generated SBOM and provenance are CI evidence artifacts and must bind to the exact source SHA.

Security

Untrusted agent origin provides no trust elevation. Build isolation and evidence integrity are mandatory.

Rollback

Release previous attested artifact; never rebuild an old release from mutable inputs.

Verification

A runtime version must be traceable to one exact source SHA, dependency set and build evidence bundle.

ADR-0061 — Deployment Identity and Production Mutation

Quelle: docs/adr/ADR-0061-deployment-identity-and-production-mutation.md

ADR-0061 — Deployment Identity and Production Mutation

Status: PROPOSED Date: 2026-08-11

Context

PR #190 attempted to implement a production Environment before the DevelopmentChain documentation set was complete.

Decision

No deployment-path code/config change is authorized until M2G Documentation Freeze. Production deploy authority is separated from agent execution authority. GitHub `production` Environment, Render deployment credentials/bridge, rotation, revocation, branch restriction and break-glass are defined before implementation.

Where Render cannot accept GitHub OIDC directly, a narrowly-scoped secret bridge may be used, documented with owner, rotation, exposure scope and revocation procedure. OIDC/short-lived credentials remain preferred wherever the target supports them.

Security

Only `main` may request production deployment. Agents may request `DEPLOY_REQUEST` but cannot possess implicit `PRODUCTION_MUTATION`.

Migration

M7 implements after M3–M6 controls exist.

Rollback

Retain last known-good deployment path until protected replacement is proven.

Verification

Production deployment evidence must bind verified source SHA to Render deployment/runtime identity.

ADR-0062 — Agent Cutover and Provider Profile Contract

Quelle: docs/adr/ADR-0062-agent-cutover-and-provider-profile-contract.md

ADR-0062 — Agent Cutover and Provider Profile Contract

Status: PROPOSED Date: 2026-08-11

Decision

At M8 cutover, ChatGPT, Claude, Gemini/AI Studio and future AI applications access CAPITAL-AI only through provider profiles mapped to ESS-0019 capabilities. NotebookLM remains read-only Research & Evidence Plane.

No provider-specific direct GitHub/Supabase/Stripe/Render admin path is canonical after cutover. Provider adapters may differ in transport (MCP, connector, function call, SDK) but not in authorization semantics.

Cutover gates

- M3 CI hardening complete.
- M4 capability IAM enforced.
- M5 audit correlation complete.
- M6 provenance complete.
- M7 deployment identity complete.

Rollback

Disable provider profile and restore read-only access without changing underlying governance.

Verification

Same policy test suite must pass for at least ChatGPT and Claude execution profiles; research-only profiles must fail mutation tests.

ADR-0063 — Agent Assurance, Incident Response and Break-Glass

Quelle: docs/adr/ADR-0063-agent-assurance-incident-break-glass.md

ADR-0063 — Agent Assurance, Incident Response and Break-Glass

Status: PROPOSED Date: 2026-08-11

Decision

M9 closes the DevelopmentChain only after negative authorization tests, prompt/tool injection tests, replay/idempotency tests, secret-exfiltration tests, audit-completeness checks, kill-switch test, break-glass drill, rollback drill and independent evidence review.

Break-glass is disabled by default, attributable, time-limited, reason-bound and followed by mandatory review. Kill switch must revoke agent mutation while preserving operator read access.

Evidence

Each drill records actor, policy version, timestamps, affected capabilities, expected/actual result and remediation.

Closure

M9 may be COMPLETE only when the traceability matrix has no unowned CRITICAL control and all residual risks are explicitly accepted.

ADR-0064 — Supabase Native MFA / AAL2 Hardening

Quelle: docs/adr/ADR-0064-supabase-native-mfa-aal2-hardening.md

ADR-0064 — Supabase Native MFA / AAL2 Hardening

Status: PROPOSED — HUMAN/OWNER APPROVAL REQUIRED

Implementation-Status: IN PROGRESS — READ-ONLY BASELINE COMPLETE

Date: 2026-08-12

Roadmap Phase: M5A

Authority: ESS-0020, ADR-0003.5, ADR-0058

Context

M5A requires a code- and production-evidence-based review of TOTP MFA and AAL2 before M6 can begin.

The read-only baseline found a split-brain authentication model:

- CAPITAL-AI stores and verifies its own RFC-6238 TOTP secrets in `public.profiles` and issues its own short-lived `step_up_tokens` ;
- Supabase Auth is the primary session and identity authority;
- the production Supabase Auth state contains **no native MFA factors** and **no AAL2 sessions**;
- both Owner profiles currently report the legacy application flag `totp_enabled=true` ;
- privileged IAM code verifies user identity and role but does not enforce the Supabase `aal` claim;
- login step-up contains a fail-open path on factor/status lookup errors;
- current passkey-first logic may grant login step-up without the TOTP requirement stated in ADR-0003.5.

Therefore the old application-level TOTP proof cannot satisfy M5A's requirement for a provider-authenticated AAL2 session.

Decision

1. Supabase Auth becomes the MFA authority

CAPITAL-AI will use Supabase Native TOTP MFA as the authoritative second-factor mechanism.

The supported flow is:

`enroll → challenge → verify → validate AAL2`

A factor is not considered active for authorization until successful verification has produced a valid AAL2 session state.

2. AAL2 is mandatory for privileged Owner/Admin actions

The server must centrally enforce a trusted AAL2 decision in addition to the existing IAM role decision.

A privileged request must fail closed when:

- AAL is `aal1`;
- factor verification is incomplete;
- the session is stale/downgraded;
- Auth/AAL verification cannot be completed;
- a client/UI-only marker claims step-up without matching server evidence.

No browser-only control may substitute for server enforcement.

3. Existing CAPITAL-AI step-up becomes defense-in-depth

The purpose-bound, single-use `x-step-up-token` contract may remain for critical actions because it provides action binding and replay resistance beyond generic session MFA.

However:

- it may only be issued/accepted together with a valid AAL2 session;
- it never elevates an AAL1 session to AAL2;
- its user/purpose/expiry/single-use constraints remain mandatory;
- freshness requirements for critical owner actions must be demonstrated by tests rather than assumed from an AMR timestamp.

4. Passkey does not automatically satisfy M5A AAL2

Passkeys remain a supported authentication mechanism, but M5A authorization is based on the actual Supabase AAL evidence. A passkey login or confirmation is not treated as equivalent to a verified TOTP MFA factor unless Supabase produces the AAL state explicitly accepted by ESS-0020.

5. Legacy TOTP is migrated, not trusted indefinitely

The custom TOTP implementation in `src/platform/Security/totp.ts`, `server/stepUp.ts`, `TotpSettings.tsx` and related profile fields becomes migration-only legacy scope.

Existing encrypted secrets will not be copied directly into Supabase Auth by unsupported database manipulation. Both Owner identities must re-enroll via supported native MFA APIs.

Legacy data removal is deferred until native enrollment, AAL2 enforcement and recovery have reached `VERIFIED_PASS`, and requires a separate Human/Owner-approved mutation/cleanup step.

6. Recovery is separated from normal authentication

Native factor recovery must remain Owner-controlled and auditable.

The target design prefers a tested backup factor/recovery procedure. If CAPITAL-AI retains its offline break-glass codes, they may authorize a tightly scoped server-side recovery operation but may not mint AAL2 or grant roles. Native factor removal must use supported Supabase Admin MFA operations and must produce critical audit evidence.

Automatic deletion of unrelated passkeys as a side effect of TOTP recovery is not part of the target contract unless separately approved.

Production baseline

Read-only evidence gathered on 2026-08-12:

Signal	Result
Supabase project	AIFINANCIAL / ACTIVE_HEALTHY
Organization plan	Free
Native <code>auth.mfa_factors</code>	0
Active session AAL distribution	2 × <code>aal1</code> , 0 × <code>aal2</code>
Legacy Owner TOTP flags	2 Owner profiles with <code>totp_enabled=true</code>
Legacy recovery codes	20 unused rows
Active legacy step-up tokens	0
Security Advisor	<code>auth_insufficient_mfa_options</code> warning present
Leaked Password Protection	warning present; Pro+ plan dependency, separate/deferred

No MFA secret, code, user email, token or factor identifier was read or recorded.

Mutation decision

Required

- repository/application changes for native enrollment/challenge/verify and AAL2 enforcement;
- native TOTP enrollment for the two Owner identities after Human/Owner approval;
- verification tests that produce AAL2 evidence.

Conditional

- Supabase project MFA configuration mutation. The available connector does not expose the Dashboard's exact Challenge/Verify configuration, so a project-setting change is authorized only if a pre-mutation check proves it necessary.

Not required for core M5A

- new Postgres tables for Supabase Native MFA;
- Stripe changes;
- unrelated Render configuration changes.

Deferred

- deletion of legacy TOTP columns/secrets, recovery-code rows/tables and obsolete custom verifier code after successful cutover;
- Leaked Password Protection until the Supabase plan supports it.

Implementation sequence

1. Merge this baseline/architecture package after Human/Owner review.
2. Implement native MFA client flows and centralized server AAL2 verifier on a fresh branch.
3. Add positive/negative unit/integration tests.
4. Pass required CI.
5. Human/Owner approves production Auth-factor mutation.
6. Enroll and verify native TOTP for Owner identities through supported API/UI flows.
7. Prove AAL2 and negative AAL1/stale/error behavior.
8. Verify recovery/backup procedure.
9. Rerun Supabase Security Advisor.
10. Synchronize M5A evidence/roadmap and only then unblock M6.

Rollback

Before native factors are enrolled, rollback is code/documentation-only.

After enrollment begins:

- do not remove legacy recovery capability until native recovery is proven;
- if application enforcement causes lockout risk, roll back application enforcement while preserving enrolled factors and audit evidence;
- native factor deletion is a separate, explicit recovery operation and must never be used as an automatic rollback;
- do not delete legacy encrypted secrets/schema until a separate cleanup gate is approved.

Consequences

Positive

- MFA assurance becomes cryptographically bound to the Supabase session/JWT instead of a parallel application flag;

- APIs can enforce one canonical AAL2 security invariant;
- stale/downgraded sessions become detectable;
- the existing purpose-bound step-up remains available as additional defense-in-depth;
- Owner MFA status becomes consistent with Supabase Auth's native factor/session model.

Costs / Migration Risk

- both Owner identities must re-enroll TOTP;
- login and privileged API paths require coordinated changes;
- recovery logic must be redesigned before custom TOTP can be removed;
- transition must avoid a state where both legacy and native factor models can independently authorize a privileged action.

Acceptance

ADR-0064 is accepted only after Human/Owner review of this PR. Acceptance authorizes **implementation**, not production factor enrollment or Auth configuration mutation. Production mutation requires the later explicit M5A mutation gate.

ADR-0065 — Systemadmin Roadmap Execution Mandate

Quelle: docs/adr/ADR-0065-systemadmin-roadmap-execution-mandate.md

ADR-0065 — Systemadmin Roadmap Execution Mandate

Status: Proposed

Date: 2026-08-12

Decision owners: CAPITAL-AI Owner / Platform Director / Security & Compliance

Supersedes: ADR-0039 §1 only for the Systemadmin profile while a valid Owner-approved REM is active

Related: ESS-0019, ESS-0021, ADR-0058, ADR-0059, ADR-0063

Context

CAPITAL-AI currently requires explicit Human authorization before every Pull Request created by an interactive agent. This provides a strong authorization boundary, but it also creates repeated low-information approval steps when a Roadmap package already defines a larger coherent implementation sequence.

The desired operating model is to let one privileged Systemadmin Agent execute an Owner-approved Roadmap segment autonomously through analysis, branch creation, implementation, testing, commits and Pull Request creation while preserving the existing Human review and merge boundary.

The current Control Plane already defines provider-neutral principal attribution, explicit capabilities, risk classes, approvals, kill switch and append-only audit correlation. The missing element is a bounded standing authorization that is wider than one PR but narrower than unrestricted administrator authority.

External risk-management guidance supports this direction only when roles, scope, oversight and least privilege remain explicit. NIST AI RMF emphasizes defined human/AI roles and documented human oversight. OWASP's Excessive Agency guidance recommends minimizing extension privileges and requiring Human approval for high-impact actions.

Decision

1. Introduce the Systemadmin Roadmap Executor

CAPITAL-AI introduces the logical agent profile:

```
capital-ai-systemadmin-roadmap-executor
```

The profile is provider-neutral. ChatGPT, Claude or another execution client can host it only when actions remain attributable to the same logical agent and Control-Plane policy.

2. Introduce Roadmap Execution Mandates (REM)

A Human/Owner-approved **Roadmap Execution Mandate** is the standing authorization for one bounded Roadmap segment.

A REM binds at minimum:

- Owner actor;
- subject agent;
- repository/base branch;
- Roadmap/ESS/ADR scope;
- paths and target resources;
- allowed capabilities;
- maximum risk;
- allowed and prohibited mutation classes;
- validity/expiry;
- tests, rollback and Evidence;
- kill switch.

The REM is not transferable to another agent, repository, Roadmap item or target.

3. Partial exception to ADR-0039

ADR-0039's explicit per-PR creation authorization remains the default for all normal interactive agents.

For the Systemadmin Roadmap Executor only, a valid REM counts as standing authorization to create and update Pull Requests whose complete scope is covered by that mandate. No additional per-PR creation prompt is required.

A material scope expansion, mandate expiry/revocation or target/path mismatch immediately restores the default ADR-0039 behavior and requires new Human authorization.

4. Autonomous repository mutation scope

Under a valid REM the Systemadmin profile may autonomously use:

READ → ANALYZE → PLAN → BRANCH → COMMIT → PR → CI_REQUEST

This includes iterative remediation before the final Human/Owner PR review.

MERGE remains unavailable to agents.

5. External production mutation is separately constrained

This ADR does not convert policy text into immediate production-write authority.

PRODUCTION_MUTATION can only be delegated later when the Control Plane technically validates REM identity, scope, expiry, exact target, mutation class, risk, rollback and audit evidence.

Until that technical enforcement exists and is VERIFIED PASS, Systemadmin autonomy stops at repository/branch/PR mutation and non-production environments.

6. Reserved Human/Owner actions

The following remain non-delegable under this ADR:

- PR merge;
- weakening repository/security/Human-review controls;
- Owner/admin IAM elevation;
- Owner MFA reset/recovery/break-glass;
- secret disclosure and unrestricted credential rotation;
- destructive production database/data operations;
- live billing mutations affecting money/entitlement;
- production resource deletion;
- DNS/TLS/domain ownership changes;
- disabling RLS/audit/security/consent controls;
- expansion or extension of the REM by the subject agent.

7. Mandatory security preflight

Before each Roadmap work package the agent performs a read-only security preflight against current main, Roadmap state, open PR overlap, relevant ADR/ESS/Evidence, risk/check class, tests, rollback and CI scope.

Any security-critical ambiguity fails closed.

8. Human review and merge remain independent controls

Systemadmin PRs continue to use the existing Owner gate:

```
Files changed / Viewed → current-head review ( /okay) → Owner checkboxes last → build-and-test
```

Successful CI does not authorize merge. Merge remains a separate Human/Owner instruction.

9. Branch lifecycle becomes mandatory

Every Systemadmin work package uses a fresh branch from current main. After successful merge into `Finance`, that branch MUST be deleted. Closed/superseded work branches are also removed after necessary Evidence retention. A merged branch is never reused.

Consequences

Positive

- One meaningful Owner decision can authorize a complete Roadmap work package rather than repeated low-value per-PR prompts.

- The agent can autonomously iterate through implementation and technical remediation up to a review-ready PR.
- Existing current-head Owner review, CI and merge separation remain intact.
- Mandate scope, expiry, targets and kill switch provide a narrower trust boundary than a permanent administrator role.
- Audit correlation can bind every mutation to one approved Roadmap mandate.

Trade-offs

- Standing authority increases blast radius if mandate scope is too broad.
- REM creation/review therefore becomes a security-critical governance action.
- Control-Plane support is required before external production mutations can safely use standing authorization.
- Roadmap packages must be written with sufficiently concrete boundaries to be enforceable.

Threat model

Primary threats:

1. **Scope creep:** agent changes files/targets not covered by the mandate.
2. **Self-elevation:** agent edits its own grant, expiry, target set or policy.
3. **Prompt/tool injection:** retrieved content attempts to alter authority.
4. **Stale-roadmap execution:** agent acts on an obsolete baseline.
5. **Concurrent writer conflict:** another PR changes the same protected area.
6. **Repeated CI consumption:** autonomous retries exceed budget without changing pre-conditions.
7. **Production blast radius:** a broad mandate is interpreted as generic admin authority.

Required mitigations:

- exact mandate binding and deny-by-default;
- current-main/read-only preflight;
- path/target allowlists;
- explicit reserved Human actions;
- no self-approval/self-extension;
- kill switch and expiry;
- audit correlation;
- negative tests;
- one-run CI discipline;
- no delegated external production mutation until technical REM enforcement is verified.

Rollback

This governance package is repository-only. Rollback is a protected Git revert of ADR-0065, ESS-0021 and related policy updates.

If a Systemadmin REM has already been enabled, rollback additionally requires immediate mandate revocation/kill-switch activation before reverting repository policy.

No production platform mutation is authorized by accepting this ADR alone.

References

- NIST AI Risk Management Framework — human roles, oversight and continuous risk management.
- OWASP GenAI Security Project, LLM06:2025 Excessive Agency — least privilege and Human approval for high-impact actions.

ADR-0066 — Passkey-only Human/Owner PR Authorization

Quelle: docs/adr/ADR-0066-passkey-only-owner-pr-authorization.md

ADR-0066 — Passkey-only Human/Owner PR Authorization

- Status: PROPOSED
- Date: 2026-08-12
- Decision owner: SvenKulesa
- Scope: DevelopmentChain M10 / Pull-Request authorization before expensive CI
- Replaces after VERIFIED PASS: emoji/text current-head approval (/ okay) and PR-body Owner-checkbox authorization as security gates
- Preserves: Human file review, Files changed / Viewed workflow, one CI run per approved head, Human-only merge boundary

1. Context

The current DevelopmentChain uses a layered Human/Owner gate before expensive CI:

FILES CHANGED → VIEWED → CURRENT-HEAD REVIEW (/okay) → OWNER CHECKBOXES → build-and-test

This proves process intent but the final authorization signal is not cryptographically bound to the exact Pull Request state. GitHub can identify the review author and commit id, but the repository workflow cannot prove which physical authenticator or passkey was used for that specific review action.

The target state is a phishing-resistant, cryptographically verifiable Owner authorization bound to the exact Pull Request and current head.

2. Decision

After M10 reaches VERIFIED PASS , CAPITAL-AI SHALL replace the current emoji/text review and Owner-checkbox authorization with a dedicated WebAuthn/passkey approval ceremony.

The target flow is:

PR OPEN/UPDATE → OWNER FILE REVIEW → ALL FILES VIEWED → PASSKEY CHALLENGE → VERIFIED OWNER ASSERTION → ONE build-and-test → MERGE ELIGIBLE

Human merge remains a separate, non-agentic action.

2.1 Sole approval factor

For PR authorization after M10 cutover:

- the Human/Owner MUST review all changed files;

- the Human/Owner SHOULD continue to mark every changed file `Viewed` in the GitHub UI;
- the only authorization action accepted by the CI gate SHALL be a verified CAPITAL-AI WebAuthn assertion;
- `👍`, `👎`, `okay`, GitHub review state, PR-body task-list checkboxes, reactions, comments, labels or device-id claims SHALL NOT independently authorize expensive CI;
- a GitHub account passkey used only for GitHub sign-in SHALL NOT be treated as proof that a specific PR head was approved.

3. PR approval transaction binding

The WebAuthn ceremony MUST bind the approval to a server-generated canonical transaction context containing at least:

- Owner actor id: `SvenKulessa` ;
- repository: `SvenKulessa/Finance` ;
- PR number;
- base branch;
- base SHA;
- exact current head SHA;
- canonical sorted changed-file list hash;
- canonical PR diff/review digest;
- privileged action: `AUTHORIZE_PR_CI` ;
- challenge id;
- issued-at time;
- expiry time;
- nonce / replay identifier.

A recommended canonical digest is:

```
SHA-256(repo || prNumber || baseSha || headSha || changedFileSetHash || diffDigest || action || challengeId)
```

The digest is authorization context, not a replacement for the WebAuthn challenge. The challenge itself MUST be generated with cryptographically secure server-side randomness.

4. WebAuthn verification requirements

The approval verifier MUST fail closed unless all required conditions are true:

1. the challenge matches a live server-side challenge record;
2. challenge entropy and generation are server-controlled;
3. challenge is single-use and within its short validity window;
4. expected RP ID matches;

5. expected HTTPS origin matches;
6. credential belongs to the authorized Owner identity;
7. assertion signature verifies against the registered public key;
8. User Presence (`UP`) is present;
9. User Verification (`UV`) is required and present;
10. the transaction binding still matches the current repository/PR/head/diff state;
11. the credential is active and not revoked;
12. replay or previously consumed approval is rejected.

A new commit, force-push, base change or material diff change invalidates all prior approval evidence.

5. File-review assurance

GitHub's per-user `Viewed` UI state is retained as a Human review workflow but **MUST NOT** be falsely represented as independently machine-verifiable if the available GitHub API does not expose reliable per-user `Viewed` state.

Therefore the passkey approval screen **MUST** present the exact PR identity and current-head review digest to the Owner and require an explicit confirmation that the displayed file set/diff was reviewed before invoking `WebAuthn`.

The resulting immutable approval evidence **MUST** include the changed-file-set hash and diff digest that were displayed for authorization.

6. CI trigger architecture

The target CI trigger is approval-record driven, not PR-body-edit driven.

A trusted M10 approval service **SHALL**:

1. resolve current PR/head/diff state read-only;
2. create the approval transaction and `WebAuthn` challenge;
3. verify the Owner assertion;
4. atomically persist immutable approval evidence;
5. mark the approval as unused;
6. request exactly one CI validation for the approved head;
7. atomically consume or lock the approval for that CI request.

CI **MUST** verify that the approval record:

- is for the exact repository/PR/head;
- carries action `AUTHORIZE_PR_CI` ;
- is not expired/revoked/consumed for a different run;
- has not been invalidated by a newer commit;

- references the exact review digest.

Duplicate CI requests for the same approval/head **MUST** be rejected or deduplicated.

7. Audit evidence

The M5 append-only audit chain **SHOULD** record, without private-key or biometric material:

- approval id;
- challenge id or redacted/hash reference;
- Owner actor id;
- WebAuthn credential reference or privacy-preserving credential hash;
- RP ID;
- origin;
- UV/UP verification result;
- repository / PR / base SHA / head SHA;
- changed-file-set hash;
- diff digest;
- action;
- issued / verified / consumed timestamps;
- CI run id;
- final outcome.

Private keys, authenticator secrets, biometric data and reusable credentials **MUST NOT** be stored.

8. Threat model / fail-closed cases

M10 tests **MUST** cover at minimum:

- wrong Owner credential → DENY;
- credential valid for different RP ID/origin → DENY;
- `UV=false` → DENY;
- missing `UP` → DENY;
- stale/expired challenge → DENY;
- challenge replay → DENY;
- assertion replay → DENY;
- wrong repository → DENY;
- wrong PR → DENY;
- wrong base SHA → DENY;
- wrong/current-head mismatch → DENY;
- changed file-set or diff after challenge issue → DENY;

- forged GitHub comment/review/reaction/checkbox → no authorization;
- GitHub session authenticated by unknown method → no PR authorization;
- duplicate CI request for consumed approval → DENY/DEDUPE;
- verifier/audit persistence unavailable → DENY;
- credential revoked during ceremony → DENY;
- agent attempts to self-approve → DENY.

9. Recovery and break-glass

Passkey loss MUST NOT silently downgrade PR authorization to emoji, checkbox, TOTP-only or device-id approval.

Recovery requires a separately documented Owner recovery/break-glass flow with:

- strong identity re-verification;
- explicit scope and expiry;
- audit evidence;
- credential revocation/re-enrollment;
- post-recovery security review.

Any break-glass approval is exceptional evidence and must not become the normal CI path.

10. Migration / cutover

M10 SHALL use a staged transition:

Stage A — design and implementation

Current Human/Owner gate remains authoritative while WebAuthn is implemented and tested.

Stage B — shadow verification

Passkey assertions are generated and audited, but existing approval still controls CI. Results are compared for false allow/deny behavior.

Stage C — enforced passkey cutover

Only after all M10 positive/negative/replay/recovery tests are **VERIFIED PASS** :

- remove **OKAY** / **okay** from CI authorization logic;
- remove PR-body Owner checkboxes from CI authorization logic;
- retain file review / Viewed workflow;
- make a valid, current-head-bound WebAuthn approval the sole Owner authorization before expensive CI.

Stage D — legacy cleanup

Delete obsolete parser/checkbox authorization code and update templates, runbooks, DevelopmentChain docs and tests.

Rollback from Stage C returns to the last verified gate only through an explicit Human/Owner security rollback decision; it MUST NOT occur automatically on verifier failure.

11. Relationship to merge authority

This ADR changes authorization for expensive PR CI and merge eligibility. It does not grant autonomous merge capability.

`MERGE` remains Human/Owner-only and outside autonomous Agent IAM. A green CI run and a valid passkey approval are evidence, not autonomous merge authorization.

12. External references

- W3C Web Authentication Level 3: <https://www.w3.org/TR/webauthn-3/>
- GitHub Docs — About passkeys: <https://docs.github.com/en/authentication/authenticating-with-a-passkey/about-passkeys>

13. Consequences

Positive:

- phishing-resistant Owner authorization;
- cryptographic binding to exact PR/head/diff;
- removal of ambiguous emoji/comment/checkbox authorization;
- improved replay resistance and auditability;
- cleaner single-trigger CI model.

Trade-offs:

- requires a trusted CAPITAL-AI WebAuthn relying-party service;
- requires credential registration/recovery lifecycle;
- introduces an availability dependency for PR authorization;
- requires explicit fail-closed recovery and rollback procedures.

14. Exit gate

ADR-0066 is architecture authority only. It does not activate passkey-only PR authorization by itself.

M10 reaches `VERIFIED PASS` only after dedicated ESS/runbook/threat model, implementation, Owner passkey enrollment, positive/negative tests, shadow comparison, audit persistence verification, CI single-trigger proof and controlled cutover are complete.

ADR-0067 — S1 Security Hardening Interlock

Quelle: docs/adr/ADR-0067-s1-security-hardening-interlock.md

ADR-0067 — S1 Security Hardening Interlock

Status: PROPOSED Date: 2026-08-12 Repository baseline reviewed: `main@4767b86d0a183fe-f97ec812e77da3e5dcdbd9d813` Related:

`docs/security/SECURITY_REMEDIATION_BASELINE_2026-08-12.md` , `docs/roadmaps/S1_SECURITY_HARDENING_ROADMAP.md` , `docs/roadmaps/DEVELOPMENT_CHAIN_ROADMAP.md`

Context

The 2026-08-12 security audit identifies application, runtime, CI, proxy, secret-handling and browser-policy weaknesses. The current DEVELOPMENT Chain already owns several related controls in M5, M5A, M6, M7 and M9, but the five High findings cannot safely wait for later assurance phases without an explicit blocking interlock.

Repository reconciliation confirms at least two important facts:

1. the documentation read/write routes still lack an explicit route-level authorization boundary;
2. the runtime artifact guard can be configured `off` in production-capable execution and can report that disabled state as accepted.

The current docs-file path implementation is stronger than the audit's original path-sanitizer description because it resolves and bounds the target to the docs root. Therefore remediation must be evidence-driven rather than copying the historical finding text unchanged.

Decision

Introduce **S1 Security Hardening** as a bounded security interlock before M6 implementation.

S1 does not replace existing DEVELOPMENT Chain phases. It owns only the remediation that is not already canonically owned elsewhere, and it blocks progression while High-severity findings remain unresolved.

Trust-boundary decisions

1. Application authorization is primary

Authentication/authorization **MUST** be enforced at the route/service boundary for privileged documentation operations. Runtime artifact immutability remains defense-in-depth and **MUST NOT** be used as the sole authorization mechanism.

2. Production security controls fail closed

A production-capable runtime MUST NOT accept an explicit `off`, missing, or invalid state for a security control that protects production runtime immutability. Startup/preflight/readiness MUST expose and enforce the invariant before serving production traffic.

3. Proxy metadata is not identity by default

Raw forwarding headers are untrusted input. Client network identity may only be derived after the deployment proxy topology and trust boundary are explicitly configured. User/session identity remains separate from network evidence.

4. Client and server environment namespaces are disjoint

Server secrets MUST NOT resolve from browser-exposed `VITE_*` aliases. Client-visible configuration uses an explicit allowlist; secret-class server variables fail closed if only a public alias is present.

5. CI cannot authorize its own weakening

Changes to workflows, composite actions and CI policy MUST be checked against trusted invariants that the evaluated pull request cannot replace. Existing M3 Owner/head-SHA controls remain authoritative and must be preserved.

6. One HTTP security policy authority

CSP and dynamic CORS behavior MUST converge on one authoritative implementation each. CORS must vary caches by origin where origin-specific responses are emitted; CSP promotion must be integration-tested before strict enforcement.

Phase ownership

S1 owns:

- F-01/F-11/F-12: documentation API authorization/filesystem boundary;
- F-02: fail-closed production runtime artifact control;
- F-04/F-15: proxy/client-IP/rate-limit trust boundary;
- F-05: secret namespace isolation;
- F-03/F-09: residual CI self-protection/output integrity;
- F-06/F-07/F-08: CSP/CORS convergence.

Existing phases retain:

- M5 → F-16 audit correlation;
- M5A → F-14 native MFA/AAL2 and replay semantics;
- M6 → F-10 plus remaining F-17 supply-chain/runtime pinning;
- M7 → deployment identity and topology transition constraints;
- M9 → F-13 and independent adversarial re-verification;
- F-18 → accepted only with deny-by-default RLS evidence.

Gate decision

M6 implementation **MUST NOT** start until all High findings are **VERIFIED PASS** :

F-01 && F-02 && F-03 && F-04 && F-05

A roadmap assignment is not evidence of closure. **VERIFIED PASS** requires exact-code positive/negative tests and traceable evidence.

Implementation model

S1 is implemented in small, trust-boundary-specific pull requests. Each starts from then-current `main`; no remediation branch is reused for the next slice. Human/Owner review and merge remain required. After successful merge the remote work branch and temporary clone/worktree, if any, are deleted according to the DEVELOPMENT Chain branch lifecycle policy.

External Render, Supabase, Stripe, DNS or other production mutation is **NOT** authorized by this ADR. Repository code and handoff documentation may be prepared, but external mutation requires the separately applicable Owner approval and execution-host policy.

Consequences

Positive

- High-severity application/runtime risks become explicit blockers instead of deferred assurance tasks.
- Existing M5/M5A/M6/M7/M9 work is reused rather than duplicated.
- Security remediation becomes traceable finding → control → test → evidence → closure.
- Route authorization, runtime immutability and CI governance are separated into correct defense layers.

Cost / trade-off

- M6 may be delayed until High findings close.
- Some findings require deployment-topology evidence before an implementation choice is safe, particularly trusted proxy and distributed rate limiting.
- CSP tightening may need staged validation to avoid breaking legitimate payment/consent/analytics integrations.

Rejected alternatives

Defer every finding to M9 assurance

Rejected because M9 should verify a hardened system, not become the first implementation phase for known High vulnerabilities.

Treat runtime artifact guard as sufficient protection for docs-file routes

Rejected because environment/configuration state can fail and because authorization belongs at the application boundary.

Create a second independent security development chain

Rejected because it would duplicate authorities, create sequencing conflicts and weaken evidence traceability.

Close findings based only on roadmap intent

Rejected because planned controls are not implemented controls.

Verification

ADR acceptance is proven when:

- the S1 baseline and roadmap are merged after Human review;
- each High finding receives an isolated remediation PR and negative tests;
- all five High findings reach **VERIFIED PASS** before M6 implementation;
- later-phase findings retain explicit owners and blocking criteria;
- DEVELOPMENT Chain roadmap/traceability is synchronized to the merged S1 evidence.

ADR-0068 — First Bounded Autonomous Work Package

Quelle: docs/adr/ADR-0068-first-bounded-autonomous-work-package.md

ADR-0068 — First Bounded Autonomous Work Package

Status: PROPOSED / OWNER-DIRECTED SA4 BOOTSTRAP Date: 2026-08-12 Authority: ESS-0021, ADR-0065, ADR-0059, ADR-0067

Context

SA3B has proven the execution-host boundary with a real GitHub Actions OIDC workload identity, a durable M5 authorization event before the repository side effect, and a correlated terminal outcome event. A separate stale-base probe has proven zero side effects for an invalid execution request.

SA4 must now prove the first bounded autonomous repository work package without widening the agent into a general-purpose GitHub operator.

The first pilot intentionally has no production effect. It creates exactly one deterministic documentation evidence file and opens one draft pull request. Human review and Human-only merge remain mandatory.

Decision

The existing GitHub Actions execution-host pattern is extended with a dedicated SA4 workflow and exact Owner-approved REM.

Canonical chain:

OWNER SA4 ISSUE → TRUSTED main WORKFLOW → STRICT REQUEST VALIDATION → SA3B CLEANUP GATE → GITHUB OIDC → M5 BRANCH AUTH → BRANCH → M5 BRANCH OUTCOME → M5 COMMIT AUTH → EXACT DETERMINISTIC FILE COMMIT → TARGETED VALIDATION → M5 COMMIT OUTCOME → M5 PR AUTH → DRAFT PR → M5 PR OUTCOME → HUMAN REVIEW → HUMAN MERGE

The pilot does not delegate `CI_REQUEST`, deployment, production mutation or merge.

Exact pilot output

The only repository content path the pilot may create is:

docs/evidence/sa4/SA4_FIRST_AUTONOMOUS_WORK_PACKAGE.md

The Issue body cannot provide file contents, shell commands, arbitrary paths, PR titles or workflow definitions. File content and PR metadata are generated by trusted code from bounded execution metadata only.

SA3B cleanup prerequisite

SA4 is not executable while the successful SA3B probe branch exists:

```
agent/sa3b-host-probe-20260812b
```

The SA4 workflow checks this GitHub ref before OIDC acquisition and before any broker authorization. Presence of the branch causes fail-closed termination with zero SA4 repository side effects.

This allows the SA4 implementation to be reviewed and merged without falsely declaring the SA3B lifecycle complete.

REM

The sole first-pilot mandate is:

```
.ai/mandates/REM-SA4-PILOT-001.json
```

It is bound to:

- Owner: `SvenKulesa` ;
- Agent: `capital-ai-systemadmin-roadmap-executor` ;
- Repository: `SvenKulesa/Finance` ;
- Base: `main` ;
- Roadmap item: `SA4-FIRST-AUTONOMOUS-WORK-PACKAGE` ;
- capabilities: `BRANCH` , `COMMIT` , `PR` only;
- exact evidence path above;
- repository mutation class only;
- maximum risk: `MEDIUM` ;
- maximum open SA4/Systemadmin PRs: `1` ;
- expiry `<= 7` days;
- Owner kill switch.

All reserved mutation classes from ADR-0065 remain denied.

Per-side-effect audit rule

Each of `BRANCH` , `COMMIT` and `PR` requires its own authorization event and its own terminal outcome event.

An authorization response is usable only when all of the following are true:

- verdict `ALLOW` ;
- valid `supabase:agent_audit_events:<id>` reference;
- `auditBoundExecutionPermitted=true` ;
- capability equals the exact requested operation;

- mandate, path, branch and current-head binding match the trusted workflow state.

No earlier permit may be reused for a later capability.

Deterministic commit

The workflow itself creates the evidence document from trusted values:

- mandate ID;
- roadmap item;
- issue number;
- workflow run ID;
- base SHA;
- branch name;
- preceding audit references.

The issue cannot inject Markdown content. Before the COMMIT outcome is recorded, the workflow re-reads the committed file and verifies its SHA-256 digest against the locally generated trusted content.

Draft PR only

PR authority is limited to creating one draft PR from the exact SA4 branch to `main`. The generated PR body identifies the mandate and audit references and explicitly states that CI success is not merge authorization.

The host does not approve, mark ready, merge, deploy or alter repository protection.

Failure and rollback

Before a side effect: any validation/OIDC/policy/audit failure means no side effect.

After branch creation but before a successful commit: the workflow may delete the exact pilot branch as bounded rollback.

After commit creation but before PR creation: the exact pilot branch may be deleted as bounded rollback because no PR exists.

If a draft PR was created but its terminal audit outcome cannot be persisted, the workflow closes that exact draft PR and deletes the exact pilot branch as bounded rollback, then fails.

Rollback authority cannot target `main`, another branch or another PR.

Trust-root boundary

SA4 must not be capable of modifying its own workflow, OIDC verifier, broker, REM, parser, IAM/REM evaluator or audit adapter. The bootstrap therefore extends the Systemadmin self-authority protection before the pilot is activated.

CI budget

Opening the draft PR may trigger lightweight governance/Owner-gate workflows. The Systemadmin host does not request the expensive final CI itself. The existing Human current-head review/attestation gate remains responsible for the single final `build-and-test` until M10 Passkey-only authorization is actually cut over.

Human boundary

`MERGE` remains non-delegable. No model, workflow, REM, audit permit, GitHub token or CI result constitutes merge approval.

Exit criteria

SA4 reaches `VERIFIED PASS` only after:

1. SA3B successful probe branch is deleted;
2. SA4 bootstrap PR is Human-reviewed, CI PASS, Human-merged and bootstrap branch deleted;
3. a fresh Owner SA4 Issue runs from the exact current `main`;
4. separate durable authorization/outcome pairs exist for BRANCH, COMMIT and PR;
5. the committed file is the exact allowlisted deterministic artifact;
6. one draft PR is created and points to the exact audited commit;
7. no production mutation occurred;
8. Human merge boundary remains intact;
9. pilot branch is deleted after the eventual Human merge;
10. evidence, roadmap and traceability are synchronized.

Consequences

SA4 increases autonomous repository execution from branch-only proof to a bounded branch/commit/draft-PR work package. The increase is deliberately narrow and evidence-driven. External production mutations remain blocked for SA5+ and require their own architecture and Owner assurance.

ADR-0069 — Human/Owner PR Gate ohne selbstreferenziellen Bootstrap

Quelle: docs/adr/ADR-0069-human-owner-comment-gate-and-dispatched-pr-ci.md

ADR-0069 — Human/Owner PR Gate ohne selbstreferenziellen Bootstrap

- **Status:** ACCEPTED — RECOVERY REVISION 2026-08-13 · **NACHTRAG 2026-08-16 (Owner-Gate-Ritual retired)**
- **Datum:** 2026-08-13
- **Scope:** Pull-Request-Governance, CI-Trust-Root, Required Checks
- **Incident Evidence:** docs/evidence/ci/PR236_BOOTSTRAP_INCIDENT_2026-08-13.md

Kontext

Die ursprüngliche ADR-0069 führte ab PR #236 einen trusted-main Human/Owner-Kommentar-Gate-Pfad ein. Das Ziel war, Human-Evidence an den aktuellen PR-Head zu binden und Kandidatencode von schreibenden Jobs zu isolieren.

Die reale Bootstrapserie PR #236–#240 zeigte jedoch mehrere Fehlerklassen: API-Berechtigungsfehler, falsche Check-SHA-Bindung, Dispatch-/YAML-Fehler und schließlich einen fehlerhaften trusted Preflight. Weil das GitHub-Ruleset den Status `build-and-test` zwingend verlangte und dieser Status durch die neue Control-Plane selbst erzeugt wurde, konnte ein Defekt der Control-Plane ihre eigene Reparatur blockieren.

Diese Revision verwirft daher nicht Human-/Owner-Evidence oder fail-closed CI. Sie verwirft ausschließlich die selbstreferenzielle Bootstrap-Strategie.

Entscheidung

1. Bestehender funktionierender Required Check bleibt Trust Root

Während einer Governance-/CI-Migration bleibt der zuletzt verifizierte, funktionierende Required Check aktiv. Ein Kandidaten-PR darf den alleinigen aktiven Trust Root nicht durch eine neue, erst nach Merge ausführbare Control-Plane ersetzen.

2. Kein Bootstrap darf den Required Check selbst erzeugen

Ein neuer Governance-Workflow darf nicht gleichzeitig den aktuell erforderlichen `build-and-test` ersetzen, seinen eigenen Aktivierungszustand erst nach Merge erhalten, und den Merge seiner eigenen Reparatur von genau diesem neuen Pfad abhängig machen. Diese Kombination ist verboten.

3. Neue Gate-Architektur nur als Shadow/Observation

Neue Human-Gate-, Dispatch-, Reporter- oder Auto-Status-Mechanismen werden zuerst parallel und **nicht merge-blockierend** ausgeführt.

4. Required-Check-Umstellung ist eine eigene Change-Klasse

Die Änderung eines GitHub-Rulesets oder der Identität eines Required Checks ist nicht Bestandteil eines normalen Workflow-PRs.

5. Human-Evidence / Merge (historisch bis 2026-08-16)

Bis zur Owner-Entscheidung 2026-08-16 galt transitional:

- vollständiger `Files changed` Review;
- alle Dateien Viewed;
- current-head Review exakt oder `okay` ;
- Human/Owner-Attestation;
- neuer Commit invalidiert Head-Evidence;
- Merge bleibt separat Human/Owner-only.

6-10.

Unverändert: kein Kandidatencode mit privilegierten Schreibrechten; kein synthetischer PASS als alleinige Merge-Evidence; Recovery muss repository-normal möglich bleiben; Bypass ist kein normaler Recovery-Mechanismus; PR-Template-Automation darf keine Gate-Autorität tragen.

Nachtrag 2026-08-15 — Shadow-Phase abgeschlossen

Die zeitlich begrenzte Parallelphase von `capital-ai-ci` und `build-and-test` ist abgeschlossen. Die Recovery-Invarianten dieser ADR bleiben bestehen.

Nachtrag 2026-08-16 — Owner-Gate-Ritual retired

Owner-Entscheidung: Pre-CI-Autorisierung über PR-Body-Checkboxen und Review-Text / `okay` wird **nicht fortgeführt**.

Folgen:

1. `.github/workflows/ci.yml` startet PR-CI ohne Owner-Gate-Job (Events: opened/synchronize/reopened/ready_for_review).
2. PR-Template v1.4.0 und `HUMAN_OWNER_PR_APPROVAL_POLICY` dokumentieren den vereinfachten Ablauf.
3. **Merge bleibt Human/Owner-only** (Agenten mergen nicht).
4. Ab Development-Chain-Punkt **M10** wird die starke CI-Autorisierung über **Passkey/WebAuthn** (`AUTHORIZE_PR_CI`) gemäß `docs/runbooks/M10_PASSKEY_OWNER_PR_AUTHORIZATION.md` eingeführt.

5. Recovery-/Bootstrap-Invarianten (kein selbstreferenzieller Required-Check-Bootstrap, kein Kandidatencode mit Schreibrechten) bleiben **vollständig** in Kraft.
6. Ein Wieder-Einführen von Checkbox-/Emoji-CI-Gates ist nur über eigenen ADR + Shadow → Cutover zulässig und nicht der Default-Pfad.

Rollback

Ein späterer ADR-0069-Nachfolger darf Recovery-Invarianten nur durch einen eigenen Human/Owner-reviewten ADR ersetzen. Ein Rückfall auf selbstreferenzielle Bootstrap-Required-Checks ist nicht zulässig. Ein Rückfall auf Checkbox-/Emoji-CI-Gates ist nicht der intendierte Rollback-Pfad; Rollback vor M10-Cutover bedeutet den vereinfachten CI-Pfad beizubehalten.

ADR-0070 — GitGuardian und Snyk als externe GitHub-Sicherheitsdienste

Quelle: docs/adr/ADR-0070-gitguardian-snyk-external-app-integration.md

ADR-0070 — GitGuardian und Snyk als externe GitHub-Sicherheitsdienste

Status: ACCEPTED

Datum: 2026-08-14

Owner: SvenKulesa

Repository: SvenKulesa/Finance

Baseline: main@5e8471de10644a5432017d28d2f4ff0657d92dd5

Kontext

Nach dem PR-#236-Bootstrap-Incident dürfen neue Sicherheitskontrollen die aktive CI-Control-Plane nicht erneut selbstreferenziell ersetzen. GitGuardian ist bereits als GitHub App angebunden; ein Snyk-Token ist nach Owner-Angabe bereits gesetzt. Die Integration soll zunächst ohne neue GitHub-Actions-Workflows erfolgen.

Entscheidung

GitGuardian und Snyk werden als externe GitHub-Apps beziehungsweise anbieterseitige Repository-Integrationen betrieben.

- keine neuen Scanner-Workflows in `.github/workflows/**`;
- kein Tokenwert in Source, Dokumentation, PR-Body, Logs oder Evidence;
- der vorhandene Snyk-Token wird von diesem PR weder gelesen noch verändert;
- Anbieterzugriff wird auf `SvenKulesa/Finance` begrenzt;
- minimal erforderlicher read-only Zugriff auf Code, Metadaten und Pull Requests;
- keine Contents-Schreibrechte und keine automatischen Fix-PRs in der ersten Stufe;
- GitGuardian und Snyk starten als beobachtende externe Checks;
- kein Check wird vor zwei Head-gebundenen erfolgreichen PR-Läufen und separater Owner-Freigabe als Required Check promoviert;
- Anbieterfehler dürfen den in Promotion befindlichen Shadow-Check `capital-ai-ci` (Stand 2026-08-14: Promotion-Gate erfüllt, serverseitiges Ruleset noch nicht aktiviert — siehe `docs/evidence/ci/P0_MAIN_PROTECTION_RECOVERY_2026-08-14.md`) in der Beobachtungsphase nicht ersetzen;
- Merge bleibt Human/Owner-only; ab M10 gilt zusätzlich der kanonische Passkey-Verifier.

Trust Boundaries

Repository-Inhalte werden an die ausgewählten Anbieter zur Sicherheitsanalyse übertragen. Ergebnisse der Anbieter sind nicht vertrauenswürdige externe Eingaben, bis Repository, PR, Head-SHA, Scanneridentität und Zeitpunkt geprüft wurden.

Sicherheitsinvarianten

1. Least privilege und Repository-Scoping.
2. Keine Secret-Ausgabe oder Token-Inventarisierung im Repository.
3. Keine automatische Remediation mit Schreibrechten in Stufe 1.
4. Keine Promotion eines Anbieterchecks während desselben PRs, der seine Integration einführt.
5. Promotion und Rollback sind separate Owner-Mutationen.
6. Branch nach erfolgreichem Human-Merge löschen.

Konsequenzen

Vorteile:

- keine zusätzliche GitHub-Actions-Laufzeit für die Scanner;
- kein neuer PR-kontrollierter Workflow-Trust-Root;
- zentrale Anbieter-Dashboards und historische Analyse.

Nachteile:

- externe Plattformabhängigkeit;
- Checknamen und Verfügbarkeit liegen teilweise außerhalb des Repositories;
- vollständige Reproduzierbarkeit erfordert exportierte, redigierte Evidence.

Rollback

1. Anbietercheck aus Required Checks entfernen, sofern später promoviert.
2. Repository im jeweiligen Anbieter deaktivieren.
3. GitHub-App-Zugriff auf `SvenKulessa/Finance` entziehen.
4. Token nur anbieterseitig widerrufen/rotieren; niemals in Repository-Evidence kopieren.
5. Bestehenden `capital-ai-ci`-Pfad unverändert weiterverwenden.

Nachtrag 2026-08-14 — Owner-Freigabe der Promotion, CodeQL bewusst nicht gewählt

`GitGuardian Security Checks`, `security/snyk` (`svenkulessa`) und `code/snyk` (`svenkulessa`) erreichten mit PR #249, #250 und #251 drei statt der geforderten zwei unabhängigen, head-gebundenen PASS-Läufe (Detail-Evidence: `docs/evidence/security/GITGUARDIAN_SNYK_BASELINE_2026-08-14.md`). Der Owner hat die Promotion freigegeben; `security/snyk` und `code/snyk` sind als Required Checks im `main-production-protection`-Ruleset

angewendet — das ist der in Punkt 4/5 der Sicherheitsinvarianten vorausgesetzte separate Owner-Mutationsschritt. Ob `GitGuardian Security Checks` ebenfalls promoviert wurde, ist zum Zeitpunkt dieses Nachtrags noch nicht bestätigt (siehe offene Nachweise in der Detail-Evidence).

Parallel wurde GitHub Advanced Security CodeQL geprüft, weil der Owner es testweise dem Ruleset hinzugefügt hatte. `code/snyk` ist Snyk Code — ein SAST-Check, der dieselbe Bedrohungskategorie (semantische Quellcode-Schwachstellen) abdeckt wie CodeQL, ohne zusätzliche GitHub-Actions-Laufzeit zu benötigen. Der Owner hat sich gegen CodeQL als zusätzlichen Required Check entschieden, um keine zweite, redundante SAST-Engine parallel pflegen zu müssen. Diese Entscheidung ist keine Schwächung — sie ersetzt keinen bestehenden Kontrollpfad, sondern verzichtet auf eine Ergänzung, deren Kategorie bereits abgedeckt ist.

Nachtrag 2026-08-15 — Snyk-Required-Check-Demotion (Kontingent erschöpft)

`security/snyk (svenkulessa)` und `code/snyk (svenkulessa)` begannen, unabhängig vom tatsächlichen Scan-Ergebnis, dauerhaft mit `error` / „You have used your limit of private tests“ zu antworten (Snyk-seitig erschöpftes Kontingent für private Repository-Scans, keine Erkenntnis über den Code-Zustand). Das blockierte real mindestens 3 gleichzeitig offene Pull Requests (#295, #296,

297), da beide Kontexte als Required Checks im `main-production-protection`-Ruleset standen.

Der Owner hat am 2026-08-15 `security/snyk` und `code/snyk` aus dem Required-Checks-Set des Rulesets entfernt — das ist exakt der in Abschnitt „Rollback“ Punkt 1 dieses ADRs vorgesehene Schritt („Anbietercheck aus Required Checks entfernen, sofern später promoviert“), keine Abweichung vom hier festgelegten Vorgehen. `GitGuardian Security Checks` bleibt unverändert Required Check. Snyk selbst (App-Integration, Repository-Scanning) bleibt aktiv; nur die Merge-blockierende Required-Check-Bindung wurde entfernt. `.github/policies/main-production-protection.expected.json` wurde entsprechend aktualisiert (`decision_2026-08-15`), und die 3 betroffenen offenen Pull Requests wurden geschlossen und mit identischem Branch/Commit als neue Pull Requests (#298, #299,

300) neu eröffnet, um eine saubere Status-Check-Auswertung unter dem aktualisierten Ruleset zu

erhalten. Siehe `docs/evidence/security/SNYK_REQUIRED_CHECK_DEMOTION_2026-08-15.md`.

ADR-0071 — Konsolidierte Roadmap-Authority und angefragte Systemadministrator-Mutationen

Quelle: docs/adr/ADR-0071-consolidated-roadmap-and-requested-systemadmin-mutations.md

ADR-0071 — Konsolidierte Roadmap-Authority und angefragte Systemadministrator-Mutationen

Status: ACCEPTED FOR DOCUMENTATION / IMPLEMENTATION REQUIRES PHASE GATES

Datum: 2026-08-14

Owner: SvenKulesa

Bezug: PR #257, ESS-0023

Kontext

CAPITAL-AI besitzt mehrere parallele Roadmaps mit teilweise abweichenden Baselines und Statusangaben. Zusätzlich soll der Systemadministrator-Agent codebasierte Prototypen aus dokumentierten Arbeitspaketen erstellen und notwendige Mutationen entweder vom Owner erhalten oder selbst beim Owner anfragen können.

Die Begriffe Anfrage, Freigabe und Ausführung dürfen nicht vermischt werden. Eine selbst erzeugte Agentenanfrage darf niemals zur Selbstautorisierung werden.

Entscheidung

1. `ROADMAP_CONSOLIDATION_MASTER_INDEX.md` ist der kanonische Portfolio- und Ausführungsinde-
dex.
2. Die DEVELOPMENT Chain bleibt das sequenzielle M0-M10-Gate.
3. Systemadministrator-Prototypen werden in separaten, phasenbezogenen PRs umge-
setzt.
4. Der Owner oder der Systemadministrator-Agent darf eine Mutation initiieren.
5. Eine Agenteninitiative besteht ausschließlich aus Diagnose, Lösungsoptionen und
einem Mutation Proposal.
6. Produktions-, IAM-, Datenbank-, Billing-, Secret-, Deployment-, DNS- sowie HIGH/CRITI-
CAL-Mutationen erfordern immer eine ausdrückliche Owner-Freigabe für den exakten
Zielzustand.
7. Kein Modell, Provider, Agent, Workflow oder Dokument darf seine eigene Authority er-
höhen.
8. Native MFA wird aufgrund der Owner-Attestation als aktiviert, aber bis technischer
Evidence als `VERIFICATION_PENDING` geführt.
9. PR #257 bleibt Documentation-only; Runtime-Prototypen erhalten eigene Branches/
PRs.

10. Nach Merge wird der Remote-Arbeitsbranch gelöscht.
11. Jede an einen Administrator delegierte Mutationsfreigabe gilt ausschließlich für einen exakt benannten Roadmap-Abschnitt bzw. ein Work Package.
12. Die Freigabe ist einmalig: ihre `approvalId` bzw. ihr gebundener `idempotencyKey` wird beim ersten Ausführungsversuch atomar konsumiert und darf weder nach Erfolg noch nach Fehler, Abbruch oder Rollback wiederverwendet werden.
13. Die Abschnittsfreigabe muss an Administrator-Principal, aktuellen `main`-SHA, Branch, Zielressource, Allowlist der Operationen, Risikoklasse, Ablaufzeit, Pre-/Postcheck, Rollback und Auditpfad gebunden sein.
14. Scope-, Principal-, SHA-, Ziel-, Operations-, Risiko- oder Zeitfensterabweichungen führen fail-closed zu `DENY`; eine Erweiterung benötigt eine neue Owner-Freigabe.
15. Die Regel gilt einheitlich für jeden menschlichen oder agentischen Administrator. Sie delegiert niemals Merge, Owner-IAM/MFA, Secret-Offenlegung, Billing-/Entitlement-Mutation, destructive production data, DNS/TLS, Security-Control-Abschaltung oder Selbst-Erweiterung.

Mutation-Proposal-Zustandsmodell

```

DISCOVERED
→ PROPOSED_BY_OWNER | PROPOSED_BY_SYSTEMADMIN
→ OWNER_REVIEW_REQUIRED
→ OWNER_APPROVED | REJECTED | EXPIRED
→ PRECHECK_PASSED
→ EXECUTING
→ VERIFIED_PASS | VERIFIED_FAIL
→ ROLLED_BACK | CLOSED

```

Nur der Owner darf `OWNER_APPROVED` erzeugen. Ein Zustands-, Ziel-, SHA-, Versions- oder Risikowechsel invalidiert die Freigabe.

Für Abschnittsfreigaben gilt zusätzlich:

```

OWNER_APPROVED (unused)
→ atomic consume before first side effect
→ CONSUMED_EXECUTING
→ VERIFIED_PASS | VERIFIED_FAIL | ROLLED_BACK | CLOSED

```

Jeder erneute Verbrauchsversuch ergibt `DENY_APPROVAL_REPLAY`. Ein fehlgeschlagener Pre-check darf keinen Side Effect auslösen; wurde die Freigabe bereits konsumiert, ist für einen weiteren Versuch eine neue Owner-Freigabe erforderlich.

Sicherheitsinvarianten

- permit before side effect;
- deny on ambiguity;
- exact target and environment binding;
- no secrets in roadmap, PR oder Evidence;
- minimal mutation and bounded blast radius;

- precheck, postcheck and rollback are mandatory;
- append-only redacted audit evidence;
- no Human-approval emulation by an agent;
- no merge or production mutation by documentation authority alone.

Folgen

Positiv:

- offene Programme werden nachvollziehbar priorisiert;
- der Agent kann proaktiv notwendige Mutationen anfragen;
- Owner-Authority bleibt getrennt und fail-closed;
- Prototypen werden klein, testbar und reviewbar.

Kosten:

- zusätzliche Proposal-, Evidence- und Traceability-Artefakte;
- blockierte Phasen bleiben trotz fertiger Dokumentation blockiert;
- Owner-Attestations benötigen technische Nachverifikation.

Abgelehnte Alternativen

- autonome Agentenfreigabe: verletzt Separation of Duties;
- vollständige Roadmap und Runtimecode in PR #257: zu großer Review- und Risikoumfang;
- Native MFA allein aufgrund verbaler Aktivierung als VERIFIED PASS: Evidence-Lücke;
- alle Fachroadmaps löschen: Verlust historischer und fachlicher Details.

Verifikation

- ESS-0023 Contract Review;
- Roadmap enthält M0-M10, P0-P3 und Prototyp-IDs;
- Agenteninitiierte Anfrage wird ohne Owner-Approval DENY;
- geändertes Proposal invalidiert Approval;
- branch cleanup ist Exit-Kriterium.

ADR-0072: Alpaca Shadow-Integration und Stilllegung von Gemini

Quelle: docs/adr/ADR-0072-alpaca-shadow-and-gemini-retirement.md

ADR-0072: Alpaca Shadow-Integration und Stilllegung von Gemini

- Status: Angenommen
- Datum: 2026-08-14

Kontext

Aktienkurse werden kanonisch über die bestehende Providerkette bezogen. Fehlende oder veraltete Echtzeitdaten sollen messbar werden, ohne den produktiven Score unkontrolliert zu verändern. Gemini wird anwendungsweit nicht mehr benötigt.

Entscheidung

1. Gemini wird aus Runtime, Provider-Routing, Governance, Embeddings, UI, Konfiguration und Abhängigkeiten entfernt.
2. Alpaca wird zunächst ausschließlich als read-only Shadow-Provider für den jeweils letzten Aktien-Trade angebunden.
3. Der kanonische Twelve-Data-Wert bleibt unverändert. Alpaca darf in dieser Phase weder Score noch Bewertung überschreiben.
4. Pro Beobachtung werden Feed, Zeitstempel, Alter, Preisabweichung, Providerzustand und Evidenz-ID erfasst.
5. Fehlende Credentials, Stale-Daten, Abweichungen und API-Fehler sind explizite Zustände; synthetische Ersatzwerte sind verboten.
6. Render lädt die beiden Alpaca-Schlüssel aus der kanonischen Secret File `/etc/secrets/finance-secrets.env`. Schlüsselwerte dürfen nicht geloggt oder in Client-Bundles aufgenommen werden.

Schwellenwerte

- Frische: maximal 90 Sekunden
- Preisabweichung gegenüber dem kanonischen Provider: maximal 1 %
- Feed: standardmäßig IEX; SIP nur nach Kosten-/Lizenzfreigabe

Folgen

Die Datenqualität kann ohne Änderung des produktiven Scores beobachtet werden. Eine Beförderung von Alpaca zum Fallback oder Primärprovider benötigt eine weitere Entscheidung mit ausreichender Shadow-Stichprobe, Marktzeiten-Segmentierung und Kostenfreigabe.

be. Die frühere Gemini-Bildanalyse ist bewusst nicht mehr verfügbar; der Endpunkt antwortet fail-closed.

ADR-0073 — Konsolidierung der technischen PR-CI auf build-and-test

Quelle: docs/adr/ADR-0073-ci-consolidation-build-and-test.md

ADR-0073 — Konsolidierung der technischen PR-CI auf build-and-test

- **Status:** ACCEPTED — OWNER CUTOVER HANDOFF REQUIRED
- **Datum:** 2026-08-15
- **Scope:** GitHub Actions, Required Checks, CI-Kosten und Merge-Sicherheit
- **Autorität:** Owner-Auswahl „1“ nach dokumentierter Lösungsmatrix

Kontext

`capital-ai-ci-shadow.yml` wurde als unabhängiger Beobachtungspfad eingeführt, nachdem der frühere synthetische `build-and-test` -Bootstrap zu einer selbstblockierenden Trust-Root-Migration geführt hatte. Der heutige echte `build-and-test` -Job in `.github/workflows/ci.yml` prüft denselben technischen Kernumfang und bindet zusätzlich das current-head Human-/Owner-Gate ein.

Der Parallelbetrieb verdoppelt Dependency-Installation, Audit, TypeScript, Tests und Build und verbraucht damit unnötig GitHub-Actions-Minuten.

Evidence

- PR #308: `build-and-test` PASS und `capital-ai-ci` PASS.
- PR #309: `build-and-test` PASS und `capital-ai-ci` PASS.
- Beide Pfade checken den exakten PR-Head aus, nutzen `persist-credentials: false` und besitzen keine schreibende Check-Reporter-Funktion.
- `build-and-test` umfasst Integrität, npm ci/audit, TypeScript, Tests, Produktions-Build, CSP/Predeploy sowie scopeabhängige Docker-Prüfung.

Entscheidung

1. `build-and-test` bleibt einziger repository-hosted technischer Required Check.
2. Human-/Owner-Vorprüfung, Head-Bindung und beide Attestations bleiben unverändert.
3. `GitGuardian Security Checks` bleibt Required Check.
4. `capital-ai-ci-shadow.yml` wird nach dem Ruleset-Cutover stillgelegt: Der `pull_request` -Trigger entfällt, die Datei bleibt mit `workflow_dispatch` erhalten.
5. Kein synthetischer Reporter darf den Namen `build-and-test` erzeugen.
6. Die Ausführungsreihenfolge ist verbindlich: Live-Ruleset zuerst, Workflow-Merge danach.

7. Keine Bypass Actors, kein Direct-Main-Push und kein Force-Push werden zugelassen.

Stilllegung statt Löschung

Die ursprüngliche Umsetzung löschte `.github/workflows/capital-ai-ci-shadow.yml`. Der Required-Check `Sicherheit geänderter Workflows` (`pr-governance.yml` → `scripts/security/verifyChangedWorkflowSecurity.mjs`) behandelt jede Workflow-Löschung fail-closed als Richtlini-enverstoß und kennt keinen Ausnahmepfad. Die Richtlinie wird aus dem vertrauenswürdigen `main`-Stand geladen; ein PR kann sie für sich selbst nicht ändern.

Entschieden wurde deshalb die Stilllegung über die Trigger-Fläche:

- `on:` enthält nur noch `workflow_dispatch`;
- der Workflow erzeugt für Pull Requests keinen `capital-ai-ci`-Check mehr;
- das Kostenziel (ein statt zwei Node-/Build-Läufe pro PR) ist vollständig erreicht;
- die Workflow-Löschungs-Invariante bleibt unangetastet und muss nicht aufgeweicht werden;
- der Rollback aus ADR-0073 reduziert sich auf das Wiedereinsetzen des `pull_request`-Triggers.

Eine spätere physische Löschung der Datei bleibt möglich, erfordert aber einen eigenen, Human/Owner-reviewten Änderungspfad für `verifyChangedWorkflowSecurity.mjs` und ist nicht Bestandteil dieser Entscheidung.

Sicherheitsinvarianten

- Checkout exakt des aktuellen PR-Heads.
- Read-only Workflow-Permissions.
- `persist-credentials: false`.
- Fail-closed Owner-Gate vor kostenintensivem PR-Build.
- Neue Commits invalidieren die Head-Freigabe.
- Separate menschliche Merge-Anweisung bleibt erforderlich.
- Workflow-/Ruleset-Reparatur erfolgt über frischen Branch und normalen PR.

Cutover-Runbook

Vor dem Merge dieses ADR-/Workflow-PRs muss der Owner:

1. Repository → Settings → Rules → Rulesets → `main-production-protection`.
2. Unter Required status checks ausschließlich `capital-ai-ci` entfernen.
3. `build-and-test` und `GitGuardian Security Checks` beibehalten.
4. Strict status checks, PR-Pflicht, Non-Fast-Forward und Codeowner-Schutz unverändert lassen.
5. Keine Bypass Actors hinzufügen.

6. Ruleset speichern und den angezeigten Sollzustand gegen

`.github/policies/main-production-protection.expected.json` prüfen.

7. Erst anschließend den PR mergen.

Verifikation nach Merge

Der nächste reale Pull Request muss zeigen:

- `build-and-test` auf dem aktuellen Head PASS;
- GitGuardian PASS;
- kein erwarteter oder hängender `capital-ai-ci`-Kontext;
- Merge bleibt ohne Owner-Gate oder ohne `build-and-test` blockiert.

Rollback

Vor Merge: Ruleset-Cutover zurücknehmen und PR offen lassen. Nach Merge: frischen Recovery-Branch erstellen, in `capital-ai-ci-shadow.yml` den `pull_request`-Trigger (`branches: [main]`, `types: [opened, synchronize, reopened, edited]`) wieder eintragen, PASS abwarten und erst danach `capital-ai-ci` wieder als Required Check aktivieren.

ADR-0074 — Generalized Systemadmin Work-Package Catalog

Quelle: docs/adr/ADR-0074-generalized-systemadmin-work-package-catalog.md

ADR-0074 — Generalized Systemadmin Work-Package Catalog

Status: PROPOSED / OWNER-DIRECTED, DORMANT UNTIL ACTIVATION Date: 2026-08-15

Authority: ESS-0021, ADR-0065, ADR-0059, ADR-0067, ADR-0068

Context

SA3B proved the execution-host boundary (real GitHub Actions OIDC workload identity, durable M5 authorization before every repository side effect, correlated terminal outcome). SA4 proved the full BRANCH -> COMMIT -> PR chain, but only for exactly one hardcoded, already-merged target (docs/evidence/sa4/SA4_FIRST_AUTONOMOUS_WORK_PACKAGE.md) — scripts/systemadmin/runSa4Pilot.mjs is not a reusable engine; a second work package would need an entirely new workflow file, a new broker route mapping and a new bespoke runner script, each independently reviewed.

Owner instruction ("fahre mit der Umsetzung des Systemadministrators fort... über die Action Workflows"), refined via three rounds of clarification, settled on Model A from docs/architecture/GENERALIZED_SYSTEMADMIN_EXECUTION_HOST_DESIGN.md : generalize the machinery so that multiple pre-approved, already-reviewed work packages can run through the same Action infrastructure, without changing the fundamental trust property that has held since SA4 — the triggering Issue never supplies executable content, only a selection among choices that were already code-reviewed and merged to main . Model B (an LLM generating code at runtime) is explicitly out of scope for this ADR; that design document identified an unsolved blocker (no tool-call-level policy enforcement) that this change does not attempt to solve.

Decision

Add one new, generic execution host alongside SA3B and SA4 — never replacing or modifying either — that dispatches to a hardcoded catalog of pre-approved work packages instead of one hardcoded target.

Canonical chain (identical structure to SA4's, ADR-0068):

OWNER WORK-PACKAGE ISSUE → TRUSTED main WORKFLOW → STRICT REQUEST VALIDATION AGAINST THE CATALOG → GITHUB OIDC → M5 BRANCH AUTH → BRANCH → M5 BRANCH OUTCOME → M5 COMMIT AUTH → CATALOG-GENERATED DETERMINISTIC FILE COMMIT → TARGETED VALIDATION → M5 COMMIT OUTCOME → M5 PR AUTH → DRAFT PR → M5 PR OUTCOME → HUMAN REVIEW → HUMAN MERGE

The host does not delegate CI_REQUEST , deployment, production mutation or merge — identical to SA4.

What actually generalizes

Only the selection step generalizes. Everything downstream of that selection remains exactly as trust-constrained as SA4:

- **Trusted catalog, not trusted input.** `scripts/systemadmin/workPackages/registry.mjs` is a hardcoded `Map` from `workPackageId` to a catalog module. Each module (e.g. `generalizationProof.mjs`) is ordinary, human/interactively-reviewed, `main`-merged code that declares its own exact target path, exact branch namespace, exact mandate file and a pure `generate()` function producing deterministic Markdown from bounded execution metadata only. The Issue body supplies **only** `workPackageId` (validated against the catalog's known keys), `baseSha` and `branchName` — never file content, never a path, never a mandate.
- **One mandate per work package, `REM-WORKPACKAGE-*` prefix required.** The broker (`systemadminExecutionBrokerRouter.ts`) does not enumerate individual mandates for this host — it checks only that the OIDC-verified workflow is the one new generic runner AND that the presented `mandateId` carries the reserved `REM-WORKPACKAGE-` prefix. Per-work-package scoping (`allowedPaths`, `allowedCapabilities`) is enforced entirely by the existing SA1 `REM_SCOPE` layer (`roadmapExecutionMandate.ts`) against that specific mandate's own content — unchanged from how SA3B/SA4 already work. SA3B and SA4 keep their existing exact-match bindings; this is purely additive.
- **Per-side-effect audit rule unchanged.** Each of `BRANCH`, `COMMIT` and `PR` still requires its own authorization event and its own terminal outcome event, with the identical permit-binding checks (mandate, branch, current-head, requested-paths) proven correct by SA4.
- **Deterministic commit, re-verified after write.** Identical to SA4: the runner re-reads the committed file and checks its SHA-256 digest against the trusted, locally generated content before recording the `COMMIT` outcome.
- **Draft PR only, Human merge boundary unchanged.** Identical to SA4.
- **`runSa4Pilot.mjs` is untouched.** This ADR does not refactor or extract shared code from the already-`VERIFIED PASS` SA4 pilot — the new `runWorkPackage.mjs` duplicates the proven security pattern rather than risk regressing a verified security-critical path for the sake of DRY.

First catalog entry

`GENERALIZATION-PROOF` (`scripts/systemadmin/workPackages/generalizationProof.mjs`) writes exactly one file, `docs/evidence/systemadmin-work-packages/WORK_PACKAGE_GENERALIZATION_PROOF.md`, proving the catalog-driven runner executes correctly through the new machinery. Its mandate, `.ai/mandates/REM-WORKPACKAGE-GEN-PROOF-001.json`, is **status: DRAFT** — not Owner-approved. Per `src/platform/Security/roadmapExecutionMandate.ts` (`mandate.status !== 'OWNER_APPROVED' → deny`), this is independently, structurally unusable even if the dormant workflow trigger below were ever accidentally enabled. This session did not and cannot self-approve it (CLAUDE.md Authorized Principals: only a verified human OWNER with fresh TOTP step-up).

Dormant by default

Exactly like SA4 before its first pilot run, `.github/workflows/systemadmin-work-package-runner.yml` carried `if: false && ...` — merging this ADR's code did not create a live privileged execution surface. Activation (flipping the `if:` condition) is a separate, explicit, documented Human/Owner change, and even after that flip, running the `GENERALIZATION-PROOF` work package specifically still additionally requires the Owner to approve `REM-WORKPACKAGE-GEN-PROOF-001` (flip `status` to `OWNER_APPROVED` with a real `approvalEvidenceRef`) — two independent gates, not one.

Nachtrag 2026-08-15 — Schritt 1 von 2 aktiviert

Auf explizite Owner-Anweisung ("aktiviere den Work-Package-Runner") wurde `if: false && ...` aus `.github/workflows/systemadmin-work-package-runner.yml` entfernt. Der Host kann jetzt bei einem passenden Owner-Issue anlaufen. `tests/unit/systemadminWorkPackageCatalogContracts.test.ts` entsprechend aktualisiert (Test benannt in „... (activated 2026-08-15)“ umbenannt, prüft jetzt das Fehlen von `false && ...`).

Nachtrag 2026-08-15 — Schritt 2 von 2: Mandat freigegeben

Auf explizite Owner-Anweisung ("fahre mit den Punkten 2 bis 4 fort" — Fortsetzung der in PR #321 dokumentierten vier Aktivierungsschritte) wurde `REM-WORKPACKAGE-GEN-PROOF-001.json` von `status: DRAFT` auf `status: OWNER_APPROVED` gesetzt, mit `approvalEvidenceRef: "human-owner-chat-2026-08-15-workpackage-gen-proof"` — identisches Muster wie `REM-SA3B-PROBE-001 / REM-SA4-PILOT-001` (Chat-Anweisung als Approval-Evidenz für ein begrenztes, nicht-produktives MEDIUM-Risiko-Mandat, keine separate TOTP-Zeremonie für diese Mandat-Klasse). `tests/unit/systemadminWorkPackageCatalogContracts.test.ts` um einen echten End-to-End-Beweis über `evaluateSystemadminRoadmapAuthorization()` ergänzt: BRANCH auf dem exakt erlaubten Pfad liefert jetzt `ALLOW`; COMMIT außerhalb des erlaubten Pfads liefert weiterhin `DENY` — die Freigabe erweitert also nicht stillschweigend den Scope.

Beide Aktivierungsgates (Trigger + Mandat) sind damit erfüllt. Gate 3 (ein frisches Owner-Issue mit dem exakten Request-JSON) und Gate 4 (Review/Merge des resultierenden Draft-PRs) bleiben ausschließlich Owner-Aktionen — diese Sitzung kann kein Issue im Namen des Owners öffnen (`github.event.issue.user.login == 'SvenKulesa'` wird von GitHub selbst geprüft, nicht simulierbar).

Trust-root boundary

The new workflow, both new scripts, the new catalog module, the new mandate and this ADR itself are added to **both** existing Systemadmin self-authority rings (`SYSTEMADMIN_SELF_AUTHORITY_PATHS` in `roadmapExecutionMandate.ts`, `SYSTEMADMIN_SA3_SELF_AUTHORITY_PATHS` in `systemadminAuditedExecution.ts`) — no future `REM-WORKPACKAGE-*` mandate can ever authorize modifying this execution machinery itself, mirroring the protection SA4 already has.

What this ADR does not do

- It does not build Model B (runtime code generation) — see `docs/architecture/GENERALIZED_SYSTEMADMIN_EXECUTION_HOST_DESIGN.md` for why that remains unsolved.
- It does not activate the new workflow.
- It does not approve the `GENERALIZATION-PROOF` mandate.
- It does not add a second real (non-proof) work package to the catalog — that is a separate, future Owner-directed change, same as any new SA4-style pilot would have been.

Exit criteria

This generalization reaches `VERIFIED PASS` only after:

1. this PR is Human-reviewed, CI PASS, Human-merged;
2. the Owner separately activates the workflow trigger;
3. the Owner separately approves `REM-WORKPACKAGE-GEN-PROOF-001` with a real `approvalEvidenceRef` ;
4. a fresh Owner work-package Issue runs from the exact current `main` ;
5. separate durable authorization/outcome pairs exist for BRANCH, COMMIT and PR;
6. the committed file is the exact catalog-generated deterministic artifact;
7. one draft PR is created and points to the exact audited commit;
8. no production mutation occurred;
9. Human merge boundary remains intact;
10. evidence, roadmap and traceability are synchronized.

Nachtrag 2026-08-15 — VERIFIED PASS: alle 10 Exit-Kriterien real erfüllt

Alle vier Aktivierungsschritte wurden auf separate, ausdrückliche Owner-Anweisungen durchgeführt und real durchlaufen — nicht nur strukturell getestet:

1. PR #321 Human-reviewed, CI PASS, Human-merged (Merge-Commit `51af4c9e0902a07841ee4b5f84d8592a49427459` , push-to-main-CI-Lauf `31891824130` `success`).
2. Owner-Anweisung "aktiviere den Work-Package-Runner" — PR #322, `if: false` && ... entfernt (Merge-Commit `fc4606150c896cf8980b561b7c6f436021bbad74` , CI-Lauf `31893202838` `success`).
3. Owner-Anweisung "fahre mit den Punkten 2 bis 4 fort" — PR #323, `REM-WORKPACKAGE-GEN-PROOF-001` auf `OWNER_APPROVED` mit `approvalEvidenceRef: "human-owner-chat-2026-08-15-workpackage-gen-proof"` (Merge-Commit `9d55826a2ef41d1a933e9f843929eb5f51ea5c64` , CI-Lauf `31893623159` `success`).
4. Owner öffnete Issue #325 (`[SYSTEMADMIN-WORK-PACKAGE]` , korrektes JSON, `baseSha` exakt `9d55826a...`) von seinem eigenen GitHub-Account aus `main` .
5. Drei getrennte, durable Autorisierungs-/Outcome-Paare (BRANCH, COMMIT, PR) — sechs `supabase:agent_audit_events:*` -Referenzen, im Issue-Kommentar und im PR-Body identisch dokumentiert.

6. Committed file is exactly the catalog-generated deterministic content from `generalizationProof.mjs.generate()`, verified per direct read against the Branch `agent/systemadmin-work-package-gen-proof-1`, Commit `e1ce748d9a4125777adeddac857603aff0c77ab3`.
7. Exactly one Draft-PR (#326) creates, points to exactly this audited commit, Basis exactly `main@9d55826a...`.
8. No production mutation — Class D/Documentation, `docker / deploy / CI_REQUEST` nowhere requested.
9. Human-Merge-Boundary intact — PR #326 stayed `draft: true` until the Owner reviewed and merged it (Merge-Commit `cb2db699ca7a2baf6cd9b4446aa7a2655d86ccdb`, push-to-main-CI-Run `31894252190 success`).
10. Evidence/Roadmap/Traceability synchronized — this addendum, `docs/roadmaps/SYSTEMADMIN_AGENT_ROADMAP.md`.

Damit ist die Generalisierung (Modell A, `docs/architecture/GENERALI-`

ZED_SYSTEMADMIN_EXECUTION_HOST_DESIGN.md) **VERIFIED PASS**. Der Katalogeintrag

`GENERALIZATION-PROOF` selbst wird **nicht gelöscht** — er ist durch die eigene `target-path-not-already-on-main`-Prüfung in `runWorkPackage.mjs` bereits selbstsperrend (das Zielfile existiert jetzt auf `main`, ein erneuter Lauf mit derselben `workPackageId` schlägt automatisch fehl), dient als Referenzimplementierung für künftige Katalogeinträge und bleibt — anders als SA4 — Teil eines bewusst wiederverwendbaren, weiterhin aktiven Hosts. Der Workflow-Trigger bleibt daher aktiv (keine Rückkehr zu `if: false && ...`), im Unterschied zum einmalig genutzten SA4-Piloten.

Consequences

Positive: future bounded, documentation/repository-scoped work packages need only a new catalog module + a new Owner-approved REM, not a new workflow file and broker route each time. Negative: the catalog itself is now a place where a careless future entry could request an inappropriately broad `allowedPaths / allowedCapabilities` — mitigated by the unchanged SA1 REM_SCOPE enforcement and by each new mandate still needing individual Owner review and approval, same as SA4's.

ADR-0075 Phase 3.4.7 — Market Data Runtime Facade

Quelle: docs/adr/ADR-0075-phase-3-4-7-runtime-facade.md

ADR-0075 Phase 3.4.7 — Market Data Runtime Facade

Status: Proposed

Date: 2026-08-08

Supersedes-Number-Only: formerly filed as ADR-0014 (number collision with Documentation Governance; content unchanged)

Parent decision: [ADR-0083 — Server Runtime Architecture Consolidation](#)

Context

The remaining compatibility cutover in `server.application.ts` still owns cache TTL handling, request coalescing, stale-cache fallback and AssetRegistry synchronization around the already extracted market-data refresh pipeline.

Decision

Extract those runtime mechanics to `server/marketData/marketDataRuntimeFacade.ts` before changing the shared compatibility module.

The facade owns only:

- 60-second cache semantics,
- coalescing concurrent refresh requests,
- stale-cache fallback after a refresh failure,
- best-effort background refresh behavior,
- synchronization of successfully refreshed assets through an injected `syncAsset` callback.

Provider I/O, fallback completion, scoring/enrichment, score snapshots and alert evaluation remain outside this facade and behind the existing **market-data compatibility refresh boundary** documented in:

- `docs/architecture/PHASE-3.4.6-MARKET-DATA-COMPATIBILITY-FACADE.md`
- related Phase 3.4.x notes under `docs/architecture/`

Numbering note (ADR-0081 / ADR-0083): Architecture phase documents no longer claim the ADR-0014 identity. The server/market-data workstream parent is **ADR-0083**. Canonical `docs/adr/ADR-0014` remains the Documentation Governance Validator.

Protected invariants

- No Stripe webhook or billing behavior changes.

- No R-001 scoring/provenance behavior changes.
- No R-002 runtime filesystem mutation changes.
- No Render or runtime secret-validation changes.
- No `server.application.ts` change in this preparatory phase.

Verification

Unit contracts cover request coalescing, cache reuse, stale-cache fallback, background fail-open behavior and registry synchronization.

ADR-0076: Naming & Repository Convention Validator als Capability des Enterprise Version Managers

Quelle: docs/adr/ADR-0076-version-manager-repository-convention-validator.md

ADR-0076: Naming & Repository Convention Validator als Capability des Enterprise Version Managers

Status

Angenommen — 2026-08-03

Numbering note

Formerly filed as ADR-0020 (number collision with multi-provider market-data routing).
Content unchanged; number reassigned under ADR-0081.

Kontext

CAPITAL-AI besitzt mit ESS-0004 bereits einen Enterprise Version Manager. Naming-, Repository- und Exception-Regeln aus ESS-0001-CONTRACTS sowie ADR-0011 sollen maschinell prüfbar sein — getrennt vom technischen Deployment-Gate.

Entscheidung

Der Enterprise Version Manager erhält eine **Naming & Repository Convention Validator**-Capability unter `src/platform/VersionManager/repositoryConventionValidator.ts`, aufrufbar über `scripts/automation/validateRepositoryConventions.ts`.

Die Capability ist **read-only** und prüft u. a. Projektidentität `capital-ai`, SemVer, package/lock-Sync, ADR-/ESS-Dateinamenskonventionen, PascalCase, Case-Kollisionen und Exception-Registry.

Betriebsmodi

- Advisory: `npm run repository:validate:advisory` (nicht blockierend)
- Strict: `npm run repository:validate` (Governance-Check, **kein** Deployment-Gate)

`predeploy:check` darf diesen Validator nicht aufrufen.

Referenzen

ESS-0001-CONTRACTS, ESS-0004, ADR-0011, ADR-0019, ADR-0081

ADR-0077 — Canonical Naming, Bilingual Documentation & Vocabulary Governance

Quelle: docs/adr/ADR-0077-canonical-naming-bilingual-documentation-vocabulary-governance.md

ADR-0077 — Canonical Naming, Bilingual Documentation & Vocabulary Governance

Status: Proposed

Datum / Date: 2026-08-09

Authority: ESS-0001-CONTRACTS

Related: ESS-0012, ESS-0003, ESS-0009, ESS-0010, ESS-0011

Numbering note

Formerly filed as ADR-0044 (number collision with production-runtime-artifact-immutability). Content unchanged; number reassigned under ADR-0081.

Deutsch

Kontext

CAPITAL-AI verwendet derzeit teilweise gemischte deutsche und englische technische Bezeichnungen sowie dezentrale UI- und Dokumentationstexte. Eine autonome AI-native Wertschöpfungskette benötigt eindeutige, maschinenlesbare und nachvollziehbare Begriffe.

Entscheidung

1. Aktiver Code und technische Identifier werden ausschließlich in Englisch geführt.
2. Enterprise-Dokumentation wird in Deutsch und Englisch gepflegt.
3. Menschlich sichtbare Pull-Request-Informationen werden standardmäßig in deutscher Sprache geführt.
4. Eine zentrale Vocabulary Registry wird Single Source of Truth für kanonische Begriffe, Aliase, verbotene Varianten und Übersetzungen.
5. Naming- und Terminologievalidierung wird in Repository- und CI-Governance integriert.
6. Aktive Renames dürfen erst nach Safe-Rename-Impact-Analyse durchgeführt werden.
7. Regex-, Dateisystem-Casing-, Import-/Export-, Routing-, API-, Schema-, Test-, Build- und Deployment-Abhängigkeiten müssen vor einem Rename validiert werden.
8. Technische maschinenlesbare Marker und stabile Keys dürfen Englisch bleiben, auch wenn ihre sichtbare Beschreibung Deutsch ist.

Konsequenzen

- Bestehende Namen werden nicht pauschal umbenannt.

- Migrationen erfolgen inkrementell nach Risiko und Evidence.
- UI- und Dokumentationsbegriffe werden aus kanonischen Concepts abgeleitet.
- PR-Template und PR-Validator müssen atomar synchronisiert werden.

English

Context

CAPITAL-AI currently contains a mixture of German and English technical naming and decentralized wording. An autonomous AI-native value chain requires unambiguous, machine-readable, traceable terminology.

Decision

1. Active code and technical identifiers use English only.
2. Enterprise documentation is maintained in German and English.
3. Human-facing pull request information is written in German by default.
4. A central Vocabulary Registry becomes the source of truth for canonical terms, aliases, forbidden variants, and translations.
5. Naming and terminology validation becomes part of repository and CI governance.
6. Active renames require a Safe Rename impact analysis before implementation.
7. Regex, filesystem casing, dependency, routing, API, schema, test, build, and deployment effects must be validated before a rename.
8. Stable machine-readable markers and keys may remain English while their human-facing descriptions are German.

Consequences

Existing code is not mass-renamed. Migration is incremental and evidence-driven. PR template and validation logic must remain synchronized.

ADR-0078 — Vocabulary Governance Authority and Namespace

Quelle: docs/adr/ADR-0078-vocabulary-governance-authority-and-namespace.md

ADR-0078 — Vocabulary Governance Authority and Namespace

Status: Proposed

Date: 2026-08-09

Decision Owner: Platform Director

Authority: ESS-0001-CONTRACTS

Numbering note

Formerly filed as ADR-0046 (number collision with modern-supabase-key-contracts).
Content intent unchanged; number reassigned under ADR-0081.

Kontext / Context

Die Vocabulary-Governance-Roadmap referenzierte ESS-0012, ADR-0044 und ADR-0045 ursprünglich als eigene Vocabulary-Authorities. Der Repository-Zustand zeigte Kollisionen mit bereits belegten IDs.

Entscheidung / Decision

1. ESS-0012 bleibt Documentation Governance.
2. Vocabulary Governance unter ESS-0017 / ESS-0017-CONTRACTS.
3. Dieser ADR (ADR-0078) trennt Documentation- und Vocabulary-Authorities.
4. Keine sofortigen Code-Renames; Safe Rename Gate bleibt Voraussetzung.

Runtime

Keine unmittelbare Runtime-/Deploy-Auswirkung.

Referenzen

ESS-0012, ESS-0017, ADR-0081

ADR-0079 — Systemadmin GitHub Actions Execution Host Binding

Quelle: docs/adr/ADR-0079-systemadmin-github-actions-execution-host.md

ADR-0079 — Systemadmin GitHub Actions Execution Host Binding

- Status: PROPOSED / SA3B IMPLEMENTATION
- Date: 2026-08-12
- Owner: SvenKulesa
- Authority: ESS-0021, ADR-0065, ADR-0059, SA1-SA3A

Numbering note

Formerly filed as ADR-0067 (number collision with s1-security-hardening-interlock).
Content unchanged; number reassigned under ADR-0081.

Decision (summary)

Use a dedicated GitHub Actions workflow as the first enforceable Systemadmin execution host.

Canonical chain: Owner execution-request issue → trusted workflow from main → GitHub OIDC → CAPITAL-AI audit broker → SA1/SA2/SA3 authorization → durable M5 auditReference → exact GitHub side effect → append-only outcome evidence.

No side effect before ALLOW + valid M5 auditReference. MERGE / DEPLOY_REQUEST / PRODUCTION_MUTATION remain hard-deny on this host.

Full detail preserved from the former ADR-0067 systemadmin host text; this file is the renumbered canonical location.

Rollback

`git revert` of the introducing PR; disable workflow. No Supabase/Stripe/Render schema mutation by this ADR.

Referenzen

ADR-0081, ESS-0021, ADR-0065

ADR-0080 — Marketing Roadmap Executor and Controlled Content Automation Boundary

Quelle: docs/adr/ADR-0080-marketing-roadmap-executor-and-content-automation-boundary.md

ADR-0080 — Marketing Roadmap Executor and Controlled Content Automation Boundary

Status: PROPOSED Date: 2026-08-12 Decision Owners: Platform Director / Repository
Owner Related ESS: ESS-0014, ESS-0019, ESS-0021, ESS-0022

Numbering note

Formerly filed as ADR-0068 (number collision with first-bounded-autonomous-work-package). Content unchanged; number reassigned under ADR-0081.

Decision (summary)

Create separate logical agent `capital-ai-marketing-roadmap-executor` with own identity, mandate, capability ceiling and kill switch. Must not inherit Systemadmin authority.

Initial state: documentation/specification and read-only analysis only. No standing BRANCH/COMMIT/PR/CI/DEPLOY/PRODUCTION_MUTATION or public social-publishing authority from this ADR.

Public social publishing is an external business mutation distinct from repository mutation. MERGE remains Human-only.

Referenzen

ADR-0081, ESS-0022, docs/governance/AUTONOMOUS_AGENT_CONCEPT_GATE.md

ADR-0081: ADR-Nummernraum-Bereinigung, Shadow-Metadaten-Validator und wöchentlicher Governance-Score

Quelle: docs/adr/ADR-0081-adr-namespace-cleanup-shadow-validator-and-weekly-score.md

ADR-0081: ADR-Nummernraum-Bereinigung, Shadow-Metadaten-Validator und wöchentlicher Governance-Score

Status

Accepted

Implementation-Status

IN PROGRESS — Nummernraum-Bereinigung und Shadow-Tools in diesem PR; volle ESS-0012-Engine bleibt Folgearbeit.

Datum

2026-08-15

Verantwortlich

Platform Director / Repository Owner (Human-authorized)

Kontext

Im aktiven ADR-Baum docs/adr/ existierten **Doppelnummern** (0014, 0020, 0044, 0046, 0067, 0068). ESS-0001-CONTRACTS Chapter 16 und ADR-0019 fordern einen eindeutigen ADR-Nummernraum.

Entscheidung

1. Nummernraum-Bereinigung

Behalten: 0014 documentation-governance, 0020 multi-provider, 0044 production-runtime-immutability, 0046 modern-supabase, 0067 s1-security, 0068 first-bounded-autonomous. Neu: 0075–0080 für die jeweiligen Zweitdateien; dieser ADR = 0081.

2. Shadow-Validator (nicht merge-blocking)

scripts/governance/shadowAdrMetadataValidator.mjs — advisory, exit 0 standard.

3. Wöchentlicher executable Governance Score

scripts/governance/weeklyGovernanceScore.mjs — Teilmenge der 57 GOV-Regeln, Shadow-Score 0–100.

4. Bibliotheksbericht

`docs/governance/CAPITAL_AI_GOVERNANCE_LIBRARY_REPORT_2026-08-15.md`

Konsequenzen

Keine Runtime-/Deploy-Mutation. Human/Owner-Gate unverändert (ADR-0039, ADR-0069).

Referenzen

ADR-0014, ADR-0019, ESS-0012, GOVERNANCE_MATURITY_REPORT

ADR-0082 — SeoEngine Platform Module (S1 / WP-S1)

Quelle: docs/adr/ADR-0082-seo-engine-platform-module.md

ADR-0082 — SeoEngine Platform Module (S1 / WP-S1)

Status: Accepted

Implementation-Status: Applied on production (schema + grants + FK RESTRICT + ledger + store code on main)

Date: 2026-08-15 (numbered 2026-08-16)

Roadmap: SEO-GM-ROADMAP-0002 / WP-S1

Related: ESS-0014 (read plane later), ADR-0043 privilege separation patterns

Supersedes-Draft: formerly docs/adr/ADR-DRAFT-seo-engine-platform-module.md

Context

CAPITAL-AI introduced an in-app SeoEngine (keywords, rank snapshots, content inventory) with admin HTTP under `/api/seo` and an in-memory store. A draft Supabase schema exists on main but is not wired and diverges on `source` enum values (`estimated` in SQL vs API No-Demo-Data).

Decision

1. Keep domain code under `src/platform/SeoEngine/`.
2. Persist via Supabase tables `seo_keywords`, `seo_rank_snapshots`, `seo_content_inventory`.
3. Rank `source` **only** `search-console` | `manual-import` — never synthetic/estimated SERP data.
4. Access path: Express admin routes → `checkAdminAccess` → store → **privileged** server Supabase client only; RLS deny-by-default for anon/authenticated.
5. Abstract storage behind `ISeoEngineStore`; memory adapter for tests; production fails closed without privileged configuration.
6. UI dashboard (S3) and prerender (S2) remain separate decisions/WPs.
7. Production migration apply is an Owner-gated external mutation, not implied by merging repository SQL.

Consequences

- Positive: durable inventory; honest empty ranks; aligns with platform store patterns (`Compliance/store`).
- Negative: schema + RLS security review and Owner apply step required before production durability.
- Neutral: in-memory remains valid offline/test path until apply.

Security invariants

1. No publishable/anon fallback for writes.
2. No estimated ranks in schema or API.
3. Admin zone IAM unchanged.
4. Shared migration files limited to `seo_*` objects.

Numbering note

Collision check 2026-08-16 after merge of PR #345 (ADR-0075-0081 namespace cleanup): next free active number was **ADR-0082**. Draft file renamed and accepted under this number (AUD5-F-002).

ADR-0083 — Server Runtime Architecture Consolidation

Quelle: docs/adr/ADR-0083-server-runtime-architecture-consolidation.md

ADR-0083 — Server Runtime Architecture Consolidation

Status: Accepted

Implementation-Status: IN PROGRESS (Phase 3 domain decomposition ongoing)

Date: 2026-08-08 (decision); numbered into canonical registry 2026-08-16

Scope: CAPITAL-AI production server architecture

Numbering / location note

This decision was historically filed under docs/architecture/adr/ADR-0014-server-runtime-architecture-consolidation.md and misused the number **ADR-0014**, which is reserved for the Documentation Governance Validator (docs/adr/ADR-0014-documentation-governance-validator.md).

Per Owner decision 2026-08-16 (unified ADR location = docs/adr/ only):

- Canonical file is **this** document under docs/adr/ as **ADR-0083**.
- Architecture phase notes no longer claim ADR-0014 as primary ID; they point here as parent.
- Former path under docs/architecture/adr/ is retained only as a short redirect stub until cleaned.

Context

ADR-0013 (canonical ESS documentation responsibility) is unrelated. The server composition extraction under the architecture tree completed the extraction of the root server.ts entry point. During the follow-up review, a second server architecture was identified under src/server/ in addition to the active production modules under server/ and the compatibility composition module server.application.ts.

The duplicate src/server/ scaffold contained an inactive createApplication() composition root and partial route/configuration modules. Repository code search found no production caller of createApplication(). Its architecture documentation also described modules that were not present in the repository tree.

At the same time, roadmap work R-001, R-002 and R-003 as well as ADR-0037 and ADR-0040 have hardened the active root-level server/ runtime contracts. Maintaining two competing server composition trees created ambiguity over ownership, made future refactors prone to semantic drift and increased the risk that an inactive scaffold could be accidentally promoted over the production-hardened runtime.

Decision

The canonical production server architecture is:

```
server.ts           # stable thin process entry
server.application.ts # compatibility composition module during domain extraction
server/             # canonical infrastructure/runtime modules
src/routes/         # application/domain route factories already owned here
src/features/       # feature-specific HTTP/domain modules
src/services/       # domain services and scoring engines
src/platform/       # governance, IAM, compliance and platform services
```

`src/server/` is retired and MUST NOT become a second production composition root.

No new server infrastructure module may be added under `src/server/` unless this ADR is superseded. Useful behavior must live in the canonical `server/`, `src/routes/`, `src/features/`, `src/services/` or `src/platform/` boundaries.

Production invariants

The consolidation MUST preserve:

- R-001 / ADR-0032 scoring provenance semantics;
- R-002 / ADR-0044 runtime artifact immutability and preloaded Runtime Artifact Guard;
- R-003 / ADR-0045 Stripe event ownership, raw-body verification and durable inbox semantics;
- ADR-0037 Render runtime port resolution and graceful shutdown behavior;
- ADR-0040 CSP safe-rollout ownership through the active security-response path;
- process-wide quiet dotenv behavior;
- fail-closed IAM and metrics contracts;
- network-independent `/healthz` behavior.

Migration record

Phase 1 — authority reconciliation

Completed in PR #108:

1. `src/server/app.ts` was explicitly classified as an inactive legacy composition root before retirement.
2. `docs/architecture/SERVER_MODULARIZATION.md` was marked superseded by the server composition work and this consolidation decision.
3. Repository code search established that `createApplication()` had no caller outside the legacy scaffold.

Phase 2 — dead scaffold retirement

Completed in PR #108 after reference analysis. Twelve files under `src/server/**` formed a closed, inactive dependency island and were retired together.

Phase 3 — `server.application.ts` domain decomposition

Phase 3 proceeds only against the canonical runtime and is split into independently gated steps. Phase notes live under `docs/architecture/` as **phase documents** (not ADR numbers):

- Phase 3.1 — route composition boundary (`server/routes/registerApplicationRoutes.ts`)
- Phase 3.2 — documentation and history boundaries
- Phase 3.3 — AI analysis route extraction (`PHASE-3.3-ai-analysis-extraction-note.md`)
- Phase 3.4 — market-data adapters and stages (`PHASE-3.4-*` notes)
- Phase 3.4.6 — market-data compatibility facade
- Phase 3.4.7 — runtime facade (ADR-0075)

Each extraction must be behavior-preserving unless an existing behavior conflicts with an already accepted production invariant.

Consequences

There is now one source of server architecture truth. `server.application.ts` is treated strictly as a temporary compatibility composition module whose responsibilities shrink monotonically. New domain logic must not be added there when an existing canonical domain boundary is available.

References

- ADR-0036 (related modularization note in `docs/adr`)
- ADR-0037, ADR-0040, ADR-0044, ADR-0045
- ADR-0075 (Phase 3.4.7 runtime facade)
- Architecture phase notes under `docs/architecture/` (PHASE-3.x naming)

ADR-DRAFT: Prerender / SSG for public SPA routes

Quelle: docs/adr/ADR-DRAFT-prerender-public-routes.md

ADR-DRAFT: Prerender / SSG for public SPA routes

Status: DRAFT — not authorized for production cutover

Date: 2026-08-15

Roadmap: SEO-ROADMAP-0001 / S2

Related: D2 route meta, D3 soft-404, Q1 sitemap

Context

CAPITAL-AI is a Vite React SPA. Crawlers that do not execute JS see an empty `#root`. Public marketing/legal routes (`/` , `/impressum` , `/agb` , `/datenschutz`) need indexable HTML.

Decision (proposed)

1. **Build-time prerender** of the public path set only (not authenticated app routes).
2. Emit static HTML files under `dist/` mirroring the path structure: - `dist/index.html` - `dist/impressum/index.html` - `dist/agb/index.html` - `dist/datenschutz/index.html`
3. Keep the React SPA for interactivity; prerendered HTML is the first paint / crawler shell.
4. Production server continues to use `express.static(dist)` so prerendered files are served first.
5. Soft-404 (`registerProductionSpaFallback`) remains required for unknown paths.

Non-goals

- Full SSR for authenticated dashboards
- Next.js migration
- Prerender of every client-side route

Implementation sketch

- `scripts/seo/prerender-public-routes.mjs` — post-`vite build` step
- Input: built `dist/index.html` + route title/meta map (`src/lib/routeSeo.ts`)
- Output: path-specific HTML with filled `<title>`, meta description, and a minimal visible body mirroring legal/marketing copy (or a noscript summary)

Consequences

- Sitemap URLs become meaningfully indexable
- Slightly more complex CI build

- Content in prerendered legal pages must stay in sync with in-app legal components (manual or shared content module)

Acceptance

- `curl -s https://capital-ai.online/impressum | grep -i impressum` returns text without requiring JS
- Unknown paths still return HTTP 404 (D3)

SUPERSEDED — SeoEngine Platform Module Draft

Quelle: docs/adr/ADR-DRAFT-seo-engine-platform-module.md

SUPERSEDED — SeoEngine Platform Module Draft

Status: SUPERSEDED

Canonical ADR: <docs/adr/ADR-0082-seo-engine-platform-module.md>

Date numbered: 2026-08-16

Collision check: post PR #345 (ADR-0075-0081); assigned **ADR-0082**

This draft path is retained only as a stable redirect so historical links do not break. Do not edit decision content here.

anforderungskatalog

Quelle: docs/adr/anforderungskatalog.md



⊞ CAPITAL-AI DOCUMENTARY

AUTONOMOUS AI DOCUMENT HYGIENIST • VERSION 0.5.4

System-Metadaten	Spezifikation
Plattform-Identität	Capital-AI Documentary (V0.5.4)
Gründer & Inhaber	Sven Kulesa
Zentrale E-Mail	sven.kulesa@capital-ai.online
Echtheits-Emblem	⊞ CAPITAL-AI CORE
Status	Revisionssicher verifiziert & bereinigt

CAPITAL-AI Anforderungskatalog & System-Analyse

Dezentrale Agenten-Architektur, Orchestratoren & Compliance-Dokumentation (Version 0.5.4)

Dieses Dokument dient als offizieller Anforderungskatalog und Revisionsbericht für die dezentrale **CAPITAL-AI Plattform**. Es analysiert die System-Ursachen für die fehlerhafte Dokumenten-Generierung bei der Anbindung neuer Kompetenzen (Agenten) sowie die Schnittstellen-Schnittpunkte zwischen dem Master Supervisor, den Orchestratoren und den dezentralen Agenten. Zudem wird der implementierte Lösungsansatz detailliert dokumentiert.

1. Ursachenanalyse (Root-Cause Analysis)

Problem 1: Fehlende Dokumenten-Erstellung ("Capital-AI Documentary")

- **Symptom:** Bei der Anbindung einer neuen Kompetenz wurden keine Revisionsdokumente für Änderungen, Risiken oder Ableitungen (ADR - Architectural Decision Record) erzeugt.
- **Ursache:** 1. Der im Hintergrund laufende Dokumenten-Wächter (`documentHygiene.ts`) basierte auf einem passiven Datei-Überwachungssystem (`chokidar`) mit der Option `ignoreInitial: true` . Vorhandene oder stillschweigend erzeugte Dateien wurden beim

Systemstart nicht indiziert. 2. Es fehlte eine aktive Registrierungs-Schnittstelle (API) zur dezentralen Registrierung neuer Agenten. Der Prozess war rein dateiorientiert, wodurch neue Module nicht aktiv angemeldet oder verarbeitet wurden. 3. Es gab keine programmgesteuerte Generierungskette, die bei einer Komponenten-Registrierung zwingend die drei notwendigen Revisionsdokumente erzeugt hat.

Problem 2: Schnittstellen- & Telemetrie-Fehler zwischen Supervisor und Orchestratoren

- **Symptom:** Der Master Supervisor hatte im Dashboard keine Live-Sichtbarkeit über die Arbeitszustände, Abfragen (Queries) oder den Aktivitätsstatus der dezentralen Agenten während der tatsächlichen Krypto-, Memecoin- oder Rohstoff-Analysen.
- **Ursache:** 1. Die Orchestratoren (`CryptoOrchestrator`, `MemeCoinOrchestrator`, `RawMaterial-sOrchestrator`) operierten als isolierte Klassen-Instanzen im Express-Server. 2. Es fehlte eine Rückkopplungs-Schleife (Feedback Loop) zur Systemregistrierung (`uploads/agents_registry.json`). Bei Ausführung paralleler Agenten-Anfragen wurden deren Abfragezähler (Query Counts) und Aktivitäts-Zustände nicht aktualisiert.

2. Implementierte System-Korrekturen (Architektur-Integration)

Um die Datenintegrität und Revisionssicherheit gemäß der **CAPITAL-AI Mandate (Version 0.5.4)** zu garantieren, wurden folgende Anpassungen vorgenommen:

A. Aktive Registrierungs- & Dokumenten-Pipeline (`server/systemEvents.ts`)

- **Registrierungs-Endpunkt (`POST /api/admin/agents/register`):**
 - Ermöglicht die dezentrale Registrierung neuer Kompetenzen inklusive Name, Rolle und zugewiesenem LLM-Routing.
 - Schreibt den Agenten direkt in die dezentrale Registry (`uploads/agents_registry.json`).
 - Generiert **sofort** und **vollautomatisch** die drei gesetzlich geforderten Revisionsdokumente unter `docs/` :
 1. **Änderungsdokument** (`docs/CHANGES_OF_<id>.md`)
 2. **Risikoanalyse** (`docs/RISK_ANALYSIS_OF_<id>.md`)
 3. **Architektur-Entscheidung (ADR)** (`docs/ADR_OF_<id>.md`)
 - Triggert manuell und direkt das Dokumenten-Hygiesystem (`documentHygiene.ts`), um die erstellten Dokumente sofort einlesen und validieren zu lassen, ohne auf den trägen Dateisystem-Wächter angewiesen zu sein.
- **Toggle-Endpunkt (`POST /api/admin/agents/toggle`):**
 - Erlaubt dem Administrator, Agenten zur Laufzeit ein- oder auszuschalten, und persistiert diesen Zustand im System.

B. Dynamische Telemetrie-Schnittstelle (Orchestrator-Rückkopplung)

- **Aktive Live-Meldungen (`updateAgentActivity`):**

- Es wurde eine zentrale Synchronisations-Funktion eingeführt, die den Status eines Agenten (`ACTIVE` / `IDLE`), seine aktuelle Teilaufgabe und die kumulierten Abfragen (`QueriesCount`) in Echtzeit aktualisiert.
- Die drei Kern-Orchestratoren (`CryptoOrchestrator` , `MemeCoinOrchestrator` , `RawMaterialsOrchestrator`) wurden so erweitert, dass sie vor dem Start paralleler Agenten-Aufgaben deren Status auf `ACTIVE` setzen, die Abfrage-Zähler inkrementieren und nach Abschluss des Analyse-Prozesses wieder in den Zustand `IDLE` versetzen.
- Dies behebt den Schnittstellen-Fehler vollständig. Der Supervisor hat im Dashboard nun eine exakte Live-Ansicht aller Agenten-Aktivitäten.

C. Administrator-UI im Supervisor Dashboard (`SupervisorDashboard.tsx`)

- **Agent-Monitoring & Steuerung:** Die UI greift nun direkt auf den Server-Endpoint `/api/admin/agents` zu, zeigt Live-Zähler und Echtzeit-Tasks an und ermöglicht das sofortige manuelle Stoppen oder Starten von Agenten.
- **Formular "Neu Kompetenz anbinden":** Ein hochmodernes, nahtlos integriertes Formular im Agenten-Tab erlaubt Administratoren die sofortige Anbindung neuer Kompetenzen. Nach der Übermittlung werden die Revisionsdokumente erzeugt, validiert und der neue Agent erscheint betriebsbereit in der Live-Liste.

3. Anforderungskatalog für zukünftige Agenten-Anbindungen

Jede künftige Anbindung eines neuen Agenten oder einer neuen System-Kompetenz an die CAPITAL-AI Plattform MUSS zwingend die folgenden Anforderungen erfüllen:

CAPITAL-AI AGENTEN-ANBINDUNG: PRÜFPROTOKOLL	
1. []	API-REGISTRIERUNG Der Agent muss über die Supervisor-Schnittstelle registriert werden, um eine eindeutige System-ID zu erhalten.
2. []	REVISIONSSICHERE DOKUMENTATION Für jeden Agenten müssen im 'docs/' Verzeichnis drei Markdown-Dateien vorliegen : - <code>CHANGES_OF_<id>.md</code> (Änderungen & Zwecke) - <code>RISK_ANALYSIS_OF_<id>.md</code> (Compliance & DSGVO-Risiken) - <code>ADR_OF_<id>.md</code> (Architektur-Entscheidung & LLM-Auswahl)
3. []	COMPLIANCE-HYGIENE-PRÜFUNG Die Dokumente müssen die gesetzlichen Header-Daten enthalten. Das System 'documentHygiene.ts' muss die Dokumente validieren und den Status " COMPLIANT " vergeben.
4. []	TELEMETRIE-INTEGRATION Bei Aufruf des Agenten im Orchestrator muss 'updateAgentActivity' aufgerufen werden, um Status und Queries an das Master-Supervisor Dashboard zu melden (Daten-Echtzeit-Sicherheit).
5. []	NO MOCK DATA POLICY Sämtliche Auswertungen müssen auf echten Live-Daten basieren. Die Anzeige simulierter Daten im Dashboard ist untersagt.

Detaillierte Spezifikation der Revisionsdokumente:

1. **Änderungsdokument (CHANGES_OF_<id>.md)**: - Muss den genauen Zeitpunkt der Anbindung, den verantwortlichen Administrator und den funktionalen Scope (Zuständigkeitsbereich) festhalten.
 2. **Risikoanalyse (RISK_ANALYSIS_OF_<id>.md)**: - Muss mögliche Halluzinationsrisiken des Modells, Datenabfluss-Risiken gemäß DSGVO sowie Latenz-Auswirkungen klassifizieren (Kategorien: Gering, Mittel, Hoch) und Gegenmaßnahmen definieren.
 3. **Architektur-Entscheidung (ADR_OF_<id>.md)**: - Muss die technologische Begründung für das gewählte LLM-Routing (z.B. Gemini 2.5 Flash für hohe Geschwindigkeit vs. Claude 3.5 Sonnet für komplexe Codegenerierung) nachvollziehbar dokumentieren.
-

Dokumentenstatus: PENDING / Version: 0.5.4 (Beta-Phase)

Erstellt am: 10. Juli 2026 von: AI Studio Coding Agent für: **Sven Kulesa (sven.kulesa@gmail.com)

ADR-0017 Revalidation Evidence — 2026-08-02

Quelle: docs/adr/evidence/ADR-0017-REVALIDATION-2026-08-02.md

ADR-0017 Revalidation Evidence — 2026-08-02

- **ADR:** ADR-0017 — Produktions-Audit-Nachbesserungen
- **Verification date:** 2026-08-02
- **Result:** REOPEN / IN PROGRESS
- **Reason:** Supabase and Compliance decisions remain implemented, but the Stripe decision is not fully conformant with the current production state.

1. Supabase revalidation — PASS

Live Supabase migration history contains the ADR-0017 migrations:

- `user_quota`
- `add_missing_service_role_rls_policies`
- `harden_handle_new_user_search_path`
- `compliance_runs`

The current Security Advisor no longer reports the RLS/search-path findings addressed by ADR-0017. Remaining findings are outside the ADR-0017 remediation scope:

1. `screening_slo_evidence` — RLS enabled with no policy, intentionally fail-closed/service-only.
2. `auth_leaked_password_protection` — unavailable on the current Supabase Free tier and governed separately by ADR-0031.

Status: `PASS` .

2. Compliance backend revalidation — PASS

The ADR-0012 backend is implemented and has since moved from the historical `server/compliance/*` location to:

`src/platform/Compliance/*`

The component manifest is `implemented` , the router remains admin/IAM-gated and the scanner tests are registered. ADR-0012 is separately resolved.

Status: `PASS` .

3. Stripe yearly-price revalidation — FAIL / CONFIGURATION DRIFT

The production Stripe account contains active live recurring prices for monthly and yearly plans.

Observed live prices:

Plan	Monthly	Yearly	Annualized monthly	Effective yearly discount
Starter	EUR 7.00	EUR 75.60	EUR 84.00	10.0%
Pro	EUR 29.00	EUR 248.00	EUR 348.00	~28.7%
Enterprise	EUR 109.00	EUR 1,280.00	EUR 1,308.00	~2.1%

ADR-0017 delegates the annual discount to separate Stripe yearly Price objects rather than server-side calculation. The Starter price satisfies the documented 10% model. The active Pro yearly price does not.

The Enterprise yearly product is also a separate live product/price. Its commercial eligibility must remain governed by the current product policy and must not be inferred merely from its existence in Stripe.

Status: **FAIL** — PRICE GOVERNANCE DRIFT.

4. Coupon validation revalidation — FAIL / NO-DEMO-DATA VIOLATION

`server/stripe.ts` still contains a local `demoCoupons` fallback in `/validate-coupon`, including entries such as `WELCOME10`, `SAVE50` and `FREE100`.

This is inconsistent with the production billing decision in ADR-0017:

- discounts must be represented by real Stripe objects;
- production checkout must not rely on local demo/sandbox billing data;
- coupon validation must fail closed when Stripe cannot verify the promotion/coupon.

Required remediation:

1. remove all local demo/sandbox coupon acceptance from the production route;
2. validate only active Stripe Promotion Codes/Coupons;
3. retain `couponId` solely as a server-verified Stripe identifier;
4. add regression tests proving `FREE100` or any unknown local code cannot be accepted without a real active Stripe object.

Status: **FAIL** — ACTIONABLE CODE REMEDIATION.

5. ADR lifecycle decision

ADR-0017 MUST remain under `docs/adr/` and its effective implementation state is now:

IN PROGRESS — production revalidation 2026-08-02

It may return to `COMPLETE` only when:

- the live Pro yearly price matches the approved commercial policy or the policy is explicitly changed by a new decision;
- the local demo-coupon fallback is removed and regression-tested;
- the corrected Stripe Price IDs are confirmed in the production environment;
- CI and a final Stripe read-back verification pass.

This evidence record does not modify Stripe, Supabase or application runtime state.

ADR-0032 Revalidation — R-001 Crypto/Meme Provenance Boundary

Quelle: docs/adr/evidence/ADR-0032-REVALIDATION-2026-08-08-R001.md

ADR-0032 Revalidation — R-001 Crypto/Meme Provenance Boundary

- **Date:** 2026-08-08
- **ADR:** ADR-0032 — Asset Catalog and Market Evidence Separation
- **Roadmap item:** R-001 — Enforce No-Demo Data + provenance truth
- **Scope:** Crypto and Meme-Coin scoring inputs
- **Execution:** repository code change only; no Supabase, Render or Stripe mutation

Finding

The accepted ADR-0032 boundary was not fully enforced in the legacy Crypto/Meme scoring generators. `AssetRegistry` contains bootstrap numeric values used for catalog/compatibility behavior, but `generateCryptoInputsSync()` and `generateMemeCoinInputsSync()` converted selected registry fields into score inputs.

Affected fields included:

- `marketCap` + `volume24h` → Crypto `avg_daily_volume`;
- `circulatingSupply` + `maxSupply` → Crypto `supply_dynamics`;
- `caller-provided/unverified change24h` → Crypto `regime_bonus`;
- `marketCap` + `volume24h` → Meme-Coin `liquidity`.

Because the legacy registry values do not carry field-level provider provenance, these conversions violated the ADR-0032 invariant that catalog/bootstrap metadata must not become verified market evidence.

Remediation

The synchronous Crypto and Meme-Coin generators are now fail-closed:

- `CryptoScoringService.generateCryptoInputsSync()` returns only the normalized `coin` identity;
- `MemeCoinScoringService.generateMemeCoinInputsSync()` returns only the normalized `coin` identity;
- registry `marketCap`, `volume24h`, supply values and unverified `change24h` no longer generate score factors;
- asynchronous history-derived technical factors remain permitted only when `assetRegistry.getHistory()` reports `source === 'live'`;

- liquidity, supply and regime factors remain `undefined` until a dedicated provider-evidence contract supplies verifiable observations.

Regression contract

`tests/unit/r001CryptoProvenanceBoundary.test.ts` locks the following invariants:

1. BTC registry bootstrap values cannot create Crypto liquidity, supply or regime factors.
2. DOGE registry market cap/volume cannot create Meme-Coin liquidity.
3. A scoring request with no verified factors fails closed with score `0`, `data_quality=unknown`, and explicit missing-factor evidence.

ADR status

ADR-0032 remains the accepted architecture decision. This change repairs implementation drift against that decision; it does not introduce a conflicting architecture.

The Crypto/Meme boundary can be considered revalidated for bootstrap-to-score separation once repository CI confirms type checks, unit tests and production build on the PR head.

Follow-up

A future provider-neutral market observation contract may re-enable:

- Crypto/Meme liquidity from verified market-cap and volume observations;
- tokenomics/supply factors from verified supply observations;
- regime factors from verified current/24h price observations.

Such factors must carry provider identity, evidence ID, observed/retrieved timestamps and freshness state before entering canonical scoring.

IAM Implementation Log — ADR-0003.5 / ADR-0008

Quelle: docs/adr/IAM_IMPLEMENTATION_LOG.md

IAM Implementation Log — ADR-0003.5 / ADR-0008

Zweck: Übergabe-Grundlage von Prompt 1 (Entwicklung, kein DB-Zugriff) an Prompt 2 (Produktion, Capital-AI Systemadmin-Schnittstelle mit Supabase-Zugriff) und Prompt 3 (Nachweis).

Eintrag 1 — Entwicklungsumgebung (Prompt 1)

- **Zeitstempel:** 2026-07-11
- **Agent/Autor:** Claude (Entwicklungsumgebung, kein DB-Zugriff)
- **Bezug:** ADR-0003.5, ADR-0008

1. Migration vorbereitet (NICHT ausgeführt)

- Datei: `supabase/migrations/20260711000000_iam.sql`
- Inhalt: `profiles.role`-Spalte (check-constraint `owner|admin|supervisor|user`, Default `user`), Tabellen `audit_logs_iam` und `iam_access_log`, RLS aktiviert auf `profiles / audit_logs_iam / iam_access_log`, Policy `profiles_select_own` (Nutzer darf eigene Zeile lesen), bewusst keine Update/Delete-Policies auf den Log-Tabellen.
- **Status: UNGETESTET gegen echte Datenbank.** Ausführung inkl. Backup/Dry-Run erfolgt in Prompt 2.

2. Server-Code (fertiggestellt, gegen Mock-Supabase-Client konzipiert)

- Neu: `server/iam/types.ts` — zentrale `Role`-Definition (`owner`, `admin`, `supervisor`, `user`).
- Neu: `server/iam/authMiddleware.ts` — `checkAdminAccess()`, `requireStepUp()`, `logIamEvent()`.
- **Übergangs-Fallback aktiv:** solange `profiles.role / audit_logs_iam / iam_access_log` in der Ziel-DB nicht existieren, fällt die Middleware kontrolliert auf die alte `ADMIN_EMAILS`-Liste zurück (mit `console.warn`-Markierung `[IAM][LEGACY FALLBACK AKTIV]`). Das verhindert einen Admin-Ausfall zwischen Code-Deployment und Migrationsausführung.
- `requireStepUp()` liefert aktuell immer `false` (Ausstellungs-Endpunkt für Step-up-Tokens ist Teil von Prompt 2 und noch zu ergänzen) — step-up-pflichtige Aktionen sind bis dahin serverseitig blockiert, nicht offen.
- Migrierte Routen (`ADMIN_EMAILS` entfernt, durch `checkAdminAccess` ersetzt):
- `server/systemEvents.ts`: `/system-events` (GET/POST/Stream), `/doc-status`, `/agents`, `/agents/register`, `/agents/toggle`, `/orchestrators/status`.

- `server/versionManager.ts`: zentrale `requireAdmin`-Middleware (gilt für alle daran hängenden Routen) auf `SUPERVISOR_ZONE_ROLES` umgestellt.
- `server/documentHygiene.ts`: zentrale `requireAdmin`-Middleware auf `ADMIN_ZONE_ROLES` umgestellt.
- `server.ts`: `/api/registry/assets/:symbol` (POST) — **zusätzlicher Fund**: diese Route hatte zuvor **keinerlei** Zugriffsprüfung (nur E-Mail-Auslesung fürs Logging). Jetzt über `checkAdminAccess(..., SUPERVISOR_ZONE_ROLES)` abgesichert.
- `server/db.ts`: `getSubscription()` und `getLocalPdfCredits()` — Owner-Bypass von hartcodiertem E-Mail-Vergleich auf `profiles.role === 'owner'`-Prüfung umgestellt (Helper `isOwnerIdentifier()`, gleiches Übergangs-Fallback-Prinzip). `getLocalPdfCredits()` ist jetzt `async` — Aufrufer in `server/stripe.ts` (3 Stellen) auf `await` umgestellt.

3. Frontend-Code

- `src/components/AuditLog.tsx`: client-seitige `ADMIN_EMAILS` / `isAdmin`-Logik entfernt. Zugriff wird jetzt per Supabase-Session-Token (`Authorization: Bearer`) an den Server übergeben; UI reagiert auf 401/403 (`accessDenied`-State), trifft keine eigene Autorisierungsentcheidung mehr.
- `src/components/AdminPortal.tsx`: client-seitige `ADMIN_EMAILS`-Liste entfernt. Rollenanzeige jetzt über RLS-geschützte Supabase-Abfrage der eigenen `profiles`-Zeile (`auth.uid() = id`). Ausdrücklich nur UX-Hinweis — echte Autorisierung bleibt in jedem Kind-Panel serverseitig.
- `src/components/SicherheitsmanagementPoC.tsx`: Deprecation-Header ergänzt.
- `src/components/DocumentHygienePanel.tsx`: Warnbanner vor der eingebetteten PoC-Komponente ergänzt (Tab `sicherheit_poc` bleibt technisch bestehen, ist aber als PoC gekennzeichnet — **offener Punkt**, siehe unten).

4. Tests

- Keine automatisierten Tests in dieser Umgebung ausgeführt (kein Testrunner-Zugriff in dieser Session). **Offener Punkt für Prompt 2/Follow-up**: Unit-Tests für `checkAdminAccess()` (401/403/200-Fälle, Legacy-Fallback, Rate-Limiting) gemäß ursprünglichem Architektur-Prompt Abschnitt 2.2 fehlen noch.

5. Offene Punkte / Annahmen (zu verifizieren in Produktion)

1. **Tabellenname** `profiles` **angenommen** — im Code bereits mehrfach verwendet (`ProfilePage.tsx`, `AdminPanel.tsx` u.a.), aber nicht explizit als Supabase-Schema-Definition im Repo vorgefunden. In Prompt 2 vor der Migration verifizieren, dass `public.profiles` existiert und eine `id`-Spalte (UUID, = `auth.users.id`) sowie eine `email`-Spalte besitzt.
2. **Rate-Limiting** aus dem ursprünglichen Architektur-Prompt (Abschnitt 2.2) ist **noch nicht implementiert** — `authMiddleware.ts` protokolliert Zugriffe, drosselt aber noch keine wiederholten Fehlversuche.
3. **Step-up-Ausstellungs-Endpunkt** existiert noch nicht — `requireStepUp()` ist ein Platzhalter, der aktuell alles blockiert, was ihn benötigt (sicherer Standardzustand, aber funktional unvollständig).

4. **Zusätzliche, bisher nicht in ADR-0003.5 erfasste hartcodierte Owner-Vergleiche** gefunden in: `src/components/AdminPanel.tsx`, `src/components/Dashboard.tsx`, `src/components/ProfilePage.tsx`, `src/components/Abonnements.tsx`, `src/components/DocumentHygienePanel.tsx` (Formularvorbelegung), `src/components/VersionManagerPanel.tsx`, `src/App.tsx`. Diese waren **nicht Teil des vereinbarten Prompt-1-Scopes** und wurden bewusst nicht verändert, um den Umfang nicht unkontrolliert auszuweiten. **Empfehlung:** eigener Folge-Prompt nach Abschluss von Prompt 2/3, da einige davon (`AdminPanel.tsx`) vermutlich denselben unautorisierten Zugriffscharakter haben wie die migrierten Dateien.
5. **SicherheitsmanagementPoC.tsx** bleibt über den Tab `sicherheit_poc` in `DocumentHygienePanel.tsx` technisch erreichbar (jetzt mit Warnbanner). Vollständige Entfernung aus der Navigation war nicht Teil des Scopes und sollte im Folge-Prompt entschieden werden.

6. Sqlfluff/Syntax-Prüfung der Migration

- Manuelle Review durchgeführt: Syntax ist Standard-PostgreSQL/Supabase-kompatibel (idempotente `IF NOT EXISTS / DO`-Blöcke). Kein `sqlfluff` in dieser Umgebung verfügbar — Ergebnis daher als "manuell geprüft, nicht automatisiert validiert" zu werten.

7. TypeScript-Prüfung (Einschränkung)

- `tsc` ist in dieser Umgebung vorhanden, aber `node_modules` (express-, react-, supabase-Typen) fehlen und konnten ohne Netzwerkzugriff nicht installiert werden — ein vollständiger `npm run lint (tsc --noEmit)` war daher nicht möglich.
- Ersatzweise durchgeführt: Klammer-/Parenthesen-Bilanzprüfung aller geänderten Dateien (unauffällig) sowie manuelle Zeile-für-Zeile-Durchsicht jeder Änderung.
- **Offener Punkt für Prompt 2:** vor dem produktiven Deployment `npm install && npm run lint` in einer Umgebung mit Netzwerkzugriff ausführen und Ergebnis hier ergänzen.

Ende Eintrag 1. Nächster Eintrag wird von Prompt 2 (Produktion) ergänzt.

Eintrag 2 — Finaler Stand (Runde 1-6, Produktion, mit Supabase-DB-Zugriff)

Zeitstempel: 2026-07-30 **Agent/Autor:** Claude (Produktionsumgebung, mit Supabase-MCP-Zugriff) **Bezug:** ADR-0003.5, ADR-0008, ADR-0009

Alle in Eintrag 1 offenen Punkte wurden über mehrere Runden bearbeitet:

1. **profiles -Tabellenname/Schema verifiziert:** `public.profiles` existiert, hat **keine** `email`-Spalte (die liegt in `auth.users`) - das war Ursache eines eigenen, bis dahin unentdeckten Bugs (`isOwnerIdentifier()` in `server/db.ts` griff bei jeder E-Mail-basierten Prüfung fälschlich auf eine Legacy-Liste zurück). Behoben: Owner- Prüfung läuft ausschließlich über `profiles.id` gegen `profiles.iam_role`.
2. **Rate-Limiting implementiert:** dependency-freier In-Memory-Limiter (`server/iam/rateLimiter.ts`), zentral in `checkAdminAccess()` integriert - deckt dadurch alle

Admin-Zonen einheitlich ab. Bekannte Grenze: pro Prozess/Instanz, kein geteilter Zustand bei horizontaler Skalierung.

3. **Step-up-Ausstellungs-Endpunkt gebaut:** `server/stepUp.ts` - TOTP-Setup/Verifikation (RFC 6238, ohne externe Dependency), Step-Up-Token-Ausstellung (`step_up_tokens` , einmalig, 5 Min. gültig), Break-Glass-Recovery (`break_glass_codes` , an Supabases eigenen Passwort-Reset-Flow gekoppelt, nicht als eigenständiger Auth-Bypass).
4. **Zusätzliche hartcodierte Owner-Vergleiche behoben:** alle in Punkt 4 (Eintrag 1) gelisteten Dateien wurden bereinigt (`AdminPanel.tsx` , `Dashboard.tsx` , `ProfilePage.tsx` , `Abonnements.tsx` , `DocumentHygienePanel.tsx` , `VersionManagerPanel.tsx` , `App.tsx`). `App.tsx` enthielt zusätzlich einen automatischen Dev-Login, der über jede `*.run.app` -Domain oder eine fehlerkonfigurierte `NODE_ENV` ausgelöst werden konnte - jetzt auf exakten `local-host` -Hostnamen plus explizites Build-Flag beschränkt.
5. **SicherheitsmanagementPoC.tsx entfernt:** Tab `sicherheit_poc` und Komponente vollständig aus `DocumentHygienePanel.tsx` entfernt, ersetzt durch die echte Supabase-native Passkey-Integration in `src/components/ProfilePage.tsx` .
6. **TypeScript-Prüfung weiterhin eingeschränkt:** kein Netzwerkzugriff in dieser Sandbox für `npm install / npm run lint` - **weiterhin offener Punkt für dich**, vor jedem Merge lokal/in CI zu verifizieren.

Zusätzlich in diesen Runden gefunden und behoben (nicht in Eintrag 1 antizipiert): - `server/stripe.ts` : mehrere unauthentifizierte Endpunkte erlaubten IDOR (fremde Abo-/Credit-Daten abfragen/verändern, beliebige Stripe-Billing-Portal-Sessions). -

`server/orchestrator.ts` : hartcodierter Fallback-Admin-Token (`aif-admin-2026`), im Frontend sogar standardmäßig vorausgefüllt; unauthentifizierter Endpunkt zum Schreiben gefälschter "Compliance-Audit"-Dateien. - ADR-0009 (CORS-Hardening): Wildcard-artige Origin-Prüfung (`*.run.app`) ersetzt durch feste Produktions-Allowlist. - Supabase-Stripe-Sync-Engine-Integration: ein bereits vorhandener, fehlerhafter DB- Trigger (Type-Fehler + gefährlicher Pro-Default bei unbekannter Price-ID) behoben. - "Passwort vergessen": fehlende `PASSWORD_RECOVERY` -Event-Behandlung gefunden und behoben - Funktion sendete zuvor nur die E-Mail, ohne je ein Formular zum tatsächlichen Setzen eines neuen Passworts zu zeigen.

Ausdrücklich NICHT Teil dieser Runden: ADR-0004 bis ADR-0007 (Branding, Frontend-Modul-Integration, Plattform-Direktor, Compliance-Wertschöpfungskette) - siehe `docs/adr/README.md` für den Status dieser ADRs.

Ende Eintrag 2.

ADR-Ablage: Konvention für aktive vs. abgeschlossene Entscheidungen

Quelle: docs/adr/README.md

ADR-Ablage: Konvention für aktive vs. abgeschlossene Entscheidungen

Zweck: Verhindert, dass jeder neue Prüf-/Audit-Durchlauf (durch KI-Agenten oder Menschen) bereits vollständig umgesetzte und verifizierte ADRs erneut komplett gegenprüfen muss.

Fester Ort (Invariante ab 2026-08-16)

- **Formale ADRs** werden ausschließlich unter `docs/adr/` angelegt und nummeriert.
- Dateien unter `docs/architecture/` dürfen den Präfix `ADR-XXXX` **nicht** als primäre ID oder Dateiname für neue Entscheidungsdokumente verwenden.
- Phase-/Work-Package-Notizen: `PHASE-...` oder beschreibende Namen; Parent-Verweis auf ein ADR unter `docs/adr/`.
- Begründung: einheitliche Architektur, einheitliche Nummerierung, keine parallelen ADR-Räume (siehe ADR-0083, GOV-XREF-UNIFIED-LOC-2026-08-16).

Struktur

- `docs/adr/*.md` — aktive ADRs. Entscheidung getroffen (`Status: ACCEPTED`), aber Umsetzung noch nicht vollständig verifiziert, oder noch in Arbeit. **Diese sollten bei jedem Security-/Architektur-Review erneut geprüft werden.**
- `docs/adr/resolved/*.md` — ADRs, deren Umsetzung gegen den tatsächlichen Code **verifiziert** wurde (nicht nur behauptet). Diese müssen bei einem neuen Durchlauf **nicht automatisch erneut komplett auditiert werden** - außer es gibt einen konkreten Anlass (z.B. eine Änderung an der betroffenen Datei, ein neuer gemeldeter Vorfall, oder eine explizite Bitte um Re-Audit).

Zwei getrennte Status-Felder pro ADR

- **Status:** — die ursprüngliche ADR-Entscheidung (`ACCEPTED` / `REJECTED` / `SUPERSEDED`). Ändert sich in der Regel nicht rückwirkend.
- **Implementation-Status:** — ob die Entscheidung tatsächlich im Code umgesetzt UND verifiziert ist. Werte: `COMPLETE` (verifiziert <Datum>), `IN PROGRESS`, `NOT STARTED`. **Nur mit `COMPLETE` markierte ADRs gehören nach `resolved/`.**

Für KI-Agenten: Empfohlener Ablauf bei einem neuen Audit-/Review-Auftrag

1. Nur `docs/adr/*.md` (aktive ADRs, nicht `resolved/`) standardmäßig gegen den Code prüfen.
2. `docs/adr/resolved/*.md` nur lesen, wenn der Auftrag sich explizit auf eine dort dokumentierte Funktion bezieht, oder wenn eine Datei geändert wurde, die in einem `resolved`-ADR referenziert ist.
3. Wird eine Regression in einer als `resolved` markierten ADR gefunden: Datei zurück nach `docs/adr/` verschieben, `Implementation-Status` auf `IN PROGRESS` mit Begründung aktualisieren.

Historischer Stand (2026-07-30)

ADR	Ablage	Implementation-Status
ADR-0001, 0002, 0003	<code>docs/adr/</code> (nicht als eigene Datei vorhanden, nur in <code>adr_history.json</code>)	nicht durch mich verifiziert
ADR-0003.5 (Owner-IAM)	<code>resolved/</code>	COMPLETE
ADR-0004 (Branding)	<code>docs/adr/</code>	nicht verifiziert
ADR-0005 (Frontend-Modul-Integration)	<code>docs/adr/</code>	nicht verifiziert
ADR-0006 (Plattform-Direktor)	<code>docs/adr/</code>	nicht verifiziert
ADR-0007 (Compliance-Wertschöpfungskette)	<code>docs/adr/</code>	nicht verifiziert
ADR-0008 (Document-Hygiene-Lifecycle)	<code>resolved/</code>	COMPLETE
ADR-0009 (CORS-Hardening)	<code>resolved/</code>	COMPLETE

Dieser historische Snapshot bleibt erhalten, ist aber **nicht** der aktuelle Arbeitsstand.

Verifizierter Nachtrag (2026-08-02)

Die aktuelle ADR-Bearbeitung folgt strikt der obigen Ablagekonvention. Folgende Entscheidungen wurden code- und CI-basiert abgeschlossen:

ADR	Ergebnis	Evidence
ADR-0012 — SecurityComplianceAuditor Integration	<code>resolved/</code> / COMPLETE	Compliance UI + autorisierte <code>/api/compliance/*</code> -Routen, <code>src/platform/Compliance</code> , Scanner-/Persistenz-Tests
ADR-0018 — Enterprise Event Mesh	<code>resolved/</code> / COMPLETE	EventMesh Stufe 1–4, Manifest-Nachpflege, SystemAudit-Bridge, Event-Tests

ADR	Ergebnis	Evidence
ADR-0030 — Platform Version and Release Lifecycle	resolved/ / COMPLETE	release-version-gate/1.0.0 , PR #57, vollständige CI PASS
ADR-0032 — Asset Catalog / Market Evidence Separation	resolved/ / COMPLETE	PR #54, 6x100 Asset-Expansion, Catalog-Integrity-Tests, CI PASS
ADR-0033 — Index / Commodity / Sovereign Evidence Scoring	resolved/ / COMPLETE	PR #56, Provider-Mapping- und Evidence-Scoring-Tests, CI PASS

Bekannte Entscheidungen, die **bewusst nicht** als erledigt klassifiziert werden:

ADR	Aktueller Grund
ADR-0011	Time-Limited Exceptions <code>server/</code> und <code>server.ts</code> bestehen weiterhin; Zielmigration ist nicht abgeschlossen.
ADR-0015	Traceability Stufe 1-2 und Stufe 4 sind umgesetzt, Stufe 3 nur teilweise; vollständige Consume-/State-Transition-Integration sowie Teile der Knowledge-/Twin-Zielarchitektur bleiben offen.
ADR-0017	Revalidierung 2026-08-02: offen. Supabase-Migrationen <code>user_quota</code> , Service-Role-RLS-Härtung, <code>handle_new_user</code> -search_path und <code>compliance_runs</code> sind live angewendet; der Security Advisor zeigt für diesen Scope keine neue DB-Warnung. Der Stripe-Code nutzt korrekterweise separate <code>STRIPE_PRICE_ID*_YEARLY</code> ohne serverseitige Rabattberechnung. Die aktuelle Live-Stripe-Konfiguration widerspricht jedoch der dokumentierten 10%-Jahresregel für PRO: 29,00 EUR/Monat = 348,00 EUR/Jahr; 10% rabattiert = 313,20 EUR, der aktive PRO-Jahrespreis beträgt 248,00 EUR. Bis die Preis-/Business-Rule-Abweichung bewusst entschieden und dokumentiert ist, darf ADR-0017 nicht nach <code>resolved/</code> .
ADR-0022	Allgemeines Individual-Bond-Scoring bleibt evidence-gated und gesperrt.
ADR-0029	Bond-Scoring-Gewichte sind weiterhin <code>Proposed / Pending Review</code> ; Rating- und Liquidity/Spread-Evidence sowie fachliche Modellfreigabe fehlen.
ADR-0031	Supabase-Free-Tier-Leaked-Password-Protection ist eine aktive, planbedingte Risk Acceptance bis Pro+.

ADR-0017 Revalidation Evidence — 2026-08-02

Die erneute Prüfung trennt Code-, Datenbank- und Billing-Wahrheit:

- **Supabase:** Die produktive Migration History enthält `user_quota` , `add_mising_service_role_rls_policies` , `harden_handle_new_user_search_path` und `compliance_runs` . Der verbleibende Auth-WARN `auth_leaked_password_protection` gehört zur separaten Free-Tier-Risk-Acceptance ADR-0031; `screening_slo_evidence` ohne Policy ist ein späterer, absichtlich fail-closed Service-Only-Befund.

- **Stripe Runtime:** Aktive Live-Prices: STARTER 7,00 EUR monatlich / 75,60 EUR jährlich (exakt 10% Rabatt); PRO 29,00 EUR monatlich / 248,00 EUR jährlich (nicht 10%); zusätzlich existiert ein aktiver ENTERPRISE-Jahrespreis von 1.280,00 EUR.
- **Backend:** `server/stripe.ts` wählt für STARTER und PRO bei `billingPeriod === 'yearly'` ausschließlich die konfigurierten `STRIPE_PRICE_ID * YEARLY`. Der Rabatt wird bewusst nicht im Backend berechnet. Damit muss die Price-Konfiguration selbst der freigegebenen Business Rule entsprechen.

Diese Revalidierung ändert **keinen** Stripe-Preis. Eine Preisänderung ist ein finanziell wirksamer Production-Billing-Change und benötigt eine explizite Entscheidung im Billing-/Release-Lifecycle.

ADR-0033 aktiviert ausschließlich den engeren **Sovereign Benchmark Yield** Contract. Das ist kein Abschluss von ADR-0022/ADR-0029 für allgemeine Einzel- oder Corporate Bonds.

Altbestand ADR-0004 bis ADR-0007

Die Aussage aus dem Snapshot vom 2026-07-30 wird nicht automatisch überschrieben. Seitdem wurden zahlreiche Plattformmodule implementiert, aber die vier ursprünglichen Entscheidungen müssen jeweils gegen ihren exakten historischen Decision Scope aus `adr_history.json` neu geprüft werden, bevor sie nach `resolved/` verschoben oder als `superseded/rejected` klassifiziert werden. Insbesondere darf ADR-0005 (Module Federation/PostMessage/JWT-Sharing) nicht allein wegen eines inzwischen umfangreichen Frontends als umgesetzt gelten.

Für ADR-0006 ist inzwischen zusätzlich belegt: `src/platform/PlatformDirector` ist weiterhin ohne Funktionscode, während ESS-0003 einen vollständigen Decision-/Release-/Exception-Contract definiert. Damit ist ADR-0006 ein echter Implementierungs-Backlog und darf nicht als abgeschlossen markiert werden.

`IAM_IMPLEMENTATION_LOG.md` war vor der historischen Aktualisierung ausschließlich auf dem Stand von "Prompt 1" (Entwicklungsumgebung, vor Migration); für IAM gilt inzwischen die separate verifizierte Resolved-Evidence der entsprechenden IAM-ADRs.

ADR 0008 document hygiene lifecycle fix

Quelle: docs/adr/resolved/ADR-0008-document-hygiene-lifecycle-fix.md



+ CAPITAL-AI DOCUMENTARY

AUTONOMOUS AI DOCUMENT HYGIENIST • VERSION 0.5.4

System-Metadaten	Spezifikation
Plattform-Identität	Capital-AI Documentary (V0.5.4)
Gründer & Inhaber	Sven Kulesa
Zentrale E-Mail	sven.kulesa@capital-ai.online
Echtheits-Emblem	▣ CAPITAL-AI CORE
Status	Revisionssicher verifiziert & bereinigt

ADR-0008: Behebung des Reaktivitäts- & Lebenszyklus-Ausfalls im Document Hygiene Panel

- **Status:** ACCEPTED
- **Implementation-Status:** COMPLETE (verifiziert 2026-07-30) — `useEffect` mit `if (!currentUserEmail) return;` und `[currentUserEmail]`-Dependency-Array bestätigt im aktuellen `src/components/DocumentHygienePanel.tsx` vorhanden.
- **Datum:** 2026-07-10
- **Autor:** Sven Kulesa

Kontext

Im `DocumentHygienePanel` wurde beim Laden der Komponente ein permanentes Abfragen von Status- und Revisionsdateien gestartet, noch bevor die Identität des aktuell eingeloggten Benutzers (`currentUserEmail`) vollständig im Client-Kontext aufgelöst wurde. Da die Backend-Endpunkte der Compliance- und Hygiene-Engine (`/api/admin/hygiene/*`) restriktive administrative Zugriffsprüfungen (Prüfung gegen die `ADMIN_EMAILS` Whitelist) durchführen, führte diese verfrühte Abfrage zu unauthorisierten API-Abbrüchen und blockierte die Initialisierung des Document-Hygiene-Intervalls. Dies beeinträchtigte das lückenlose Monitoring der Dokumentenintegrität.

Entscheidung

Wir beheben dieses Verhalten durch eine präzise Anpassung des React-Lebenszyklus im `DocumentHygienePanel` (`src/components/DocumentHygienePanel.tsx`): 1. **Lebenszyklus-Absicherung**: Der primäre `useEffect`, welcher die Backend-Abfragen (`fetchStatus` und `fetchHistoryFiles`) sowie das 3-Sekunden-Polling initialisiert, wird mit einem Conditional Early-Return abgesichert. Das Laden und die Abfrageintervalle starten erst, sobald `currentUserEmail` einen validen, nicht-leeren String enthält. 2. **Reaktivitäts-Abhängigkeit**: Der Dependency-Array des `useEffect`-Hooks wird explizit von `[]` auf `[currentUserEmail]` angepasst. Dies stellt sicher, dass bei einem dynamischen Benutzerwechsel oder nach dem verzögerten Laden der Firebase/Mock-Authentifizierungs-Sitzung die Synchronisierung der Dokumentenpflege sofort und nahtlos neu gestartet wird.

Konsequenzen

- **Vorteile:**
- **Ausfallsichere Initialisierung**: Keine unauthorisierten API-Fehler (HTTP 401/403) mehr durch unvollständige Authentifizierungs-Kontexte im Client.
- **Echtzeit-Synchronisierung**: Sofortiger Start des Polling-Dienstes, sobald der globale Administrator angemeldet ist.
- **Erhöhte Robustheit**: Einhaltung der strengen PII- und DSGVO-Schutzrichtlinien der Plattform, da administrative Abfragen nur bei nachgewiesener Sitzungs-E-Mail übermittelt werden.
- **Herausforderungen:**
- Bei fehlerhaften Sitzungszuständen im Client, in denen `currentUserEmail` fälschlicherweise leer bleibt, ruht die Komponente im Wartezustand; dies ist jedoch ein erwünschtes Sicherheitsmerkmal (Security-by-Design).

ADR-0009: CORS Hardening für Entwicklungs- und Produktionsumgebung

Quelle: docs/adr/resolved/ADR-0009-cors-hardening.md

ADR-0009: CORS Hardening für Entwicklungs- und Produktionsumgebung

Status

Accepted

Implementation-Status

COMPLETE (verifiziert 2026-07-30) — feste Produktions-Allowlist (`capital-ai.online` , `www.capital-ai.online`), `AI_STUDIO_ORIGIN` nur außerhalb Produktion, Credentials nur für validierte Origins, blockierte Origins werden nach `security_events` geloggt. Umgesetzt in `server.ts` . Bewusste Abweichung vom ADR-Text: `x-orchestrator-admin-token` wurde NICHT als erlaubter Header übernommen, da der zugehörige Legacy-Token-Mechanismus in ADR-0003.5 entfernt wurde (server/orchestrator.ts nutzt jetzt JWT via `checkAdminAccess()`).

Datum

2026-07-14

Verantwortlich

Capital-AI Architecture Team

Kontext

Capital-AI verwendet eine moderne Webanwendungsarchitektur bestehend aus:

- React Frontend
- Node.js / Express Backend
- Supabase Authentication
- Stripe Billing Infrastruktur
- KI-Service Layer
- Orchestrator Architektur
- Entwicklungsumgebung mit Google AI Studio
- Produktionsumgebung unter <https://capital-ai.online>

Die Anwendung verarbeitet sensible Datenbereiche:

- Benutzerkonten

- Subscription Informationen
- Scoring Ergebnisse
- KI-Analysen
- Enterprise Funktionen
- interne Orchestrator Schnittstellen

Die bisherige CORS-Konfiguration erlaubt nicht ausreichend restriktiv definierte Origins.

Eine unzureichende CORS-Konfiguration kann dazu führen, dass:

- fremde Webseiten API-Anfragen initiieren können
- Benutzer-Sessions missbraucht werden
- Authentifizierungsinformationen übertragen werden
- interne API-Endpunkte unerwünscht erreichbar werden
- Manipulationsversuche gegen Backend-Funktionen erfolgen

CORS stellt zwar keine vollständige Zugriffskontrolle dar, ist aber ein wichtiger Bestandteil der Browser-basierten Sicherheitsarchitektur.

Problemstellung

Die Anwendung benötigt eine klare Trennung zwischen:

1. Entwicklungsumgebung

und

1. Produktionsumgebung

Dabei müssen beide Umgebungen gegen unautorisierte Fremdzugriffe geschützt werden.

Aktuelle Risiken:

- fehlende oder unvollständige Origin-Whitelist
- Nutzung von Wildcards (*)
- unkontrollierte lokale Entwicklungsfreigaben
- unkontrollierte Google AI Studio Origins
- fehlende Überwachung blockierter Origins

Ziel

Einführung eines zentral kontrollierten CORS Security Layers.

Der Security Layer soll:

- ausschließlich bekannte Origins akzeptieren

- unbekannte Origins blockieren
- Entwicklungs- und Produktionsumgebung getrennt behandeln
- keine Fremddmanipulation über Browser Requests ermöglichen
- zukünftige Erweiterungen kontrolliert ermöglichen

Entscheidung

Capital-AI implementiert eine restriktive CORS-Allowlist Architektur.

Es werden keine Wildcards verwendet.

Nicht erlaubt:

```
```http Access-Control-Allow-Origin: *
```

Erlaubt: Access-Control-Allow-Origin: https://capital-ai.online  
 Architekturentscheidung 1. Produktionsumgebung Produktive erlaubte Origin: https://capital-ai.online https://www.capital-ai.online Alle anderen Origins werden blockiert. Beispiel: Erlaubt: https://capital-ai.online Blockiert: https://evil-domain.com https://fake-capital-ai.com  
 2. Entwicklungsumgebung Die Entwicklungsumgebung erlaubt ausschließlich: Lokale Entwicklung http://localhost: http://127.0.0.1: Beispiele: http://localhost:3000 http://localhost:5173  
 Google AI Studio Entwicklung Google AI Studio darf nur über explizite Freigabe erfolgen. Die Freigabe erfolgt nicht hart codiert, sondern über Environment Variables. Beispiel: AI\_STUDIO\_ORIGIN=https://ai.studio  
 Dadurch bleibt die Produktionsumgebung frei von unnötigen Entwicklungsfreigaben. Technische Umsetzung Zentrale Origin Verwaltung Die erlaubten Origins werden abhängig von der Umgebung geladen. Beispiel: const productionOrigins = [ "https://capital-ai.online", "https://www.capital-ai.online" ];

```
const developmentOrigins = ["http://localhost", "http://127.0.0.1"];
Sicherheitsregeln
Regel 1 Keine offenen Origins: Nicht erlaubt: "*"
Regel 2 Keine dynamische Freigabe unbekannter Domains
Nicht erlaubt: res.setHeader("Access-Control-Allow-Origin", req.headers.origin); ohne Prüfung.
Regel 3 Jede Origin wird geprüft
Beispiel: if(originIsAllowed){
```

```
allow();
```

```
}else{
```

```
block();
```

```
}
Security Logging
Blockierte Origins werden protokolliert. Beispiel: [SECURITY] Blocked CORS Origin: https://unknown-domain.com
Diese Logs dienen: Angriffserkennung Security Monitoring Audit Nachweis
Zusätzliche Header
Folgende Header werden verpflichtend gesetzt:
Methoden GET POST PUT DELETE OPTIONS
Erlaubte Header Content-Type Authorization stripe-signature x-orchestrator-admin-token
Credentials Policy Cookies und Authentifizierungsdaten dürfen ausschließlich an bestätigte Origins übertragen werden.
Aktiv: Access-Control-Allow-Credentials:true
Voraussetzung: Origin muss vorher validiert sein.
Preflight Handling
OPTIONS Requests werden geprüft. Nur erlaubte Origins erhalten: HTTP 200 OK
Nicht erlaubte Requests werden verworfen.
Entwicklungsrichtlinien
Entwickler dürfen:
```

lokale Ports verwenden lokale API Tests durchführen definierte Testumgebungen nutzen Entwickler dürfen nicht: globale CORS Freigaben einführen Produktionsdomain erweitern ohne ADR Access-Control-Allow-Origin:\* verwenden Produktionsrichtlinien Änderungen an: erlaubten Domains CSP Regeln Security Headern benötigen: Code Review Security Prüfung ADR Aktualisierung Nicht gewählte Alternativen Alternative 1 Globale Freigabe: \* Ablehnung: Zu hohes Risiko für: Session Missbrauch Cross-Site Requests Datenzugriffe Alternative 2 Nur Frontend-Schutz Ablehnung: CORS ersetzt keine Backend-Sicherheit. Zusätzlich notwendig: IAM Supabase RLS API Authorization Rate Limiting Audit Logging Sicherheitsauswirkungen Positiv Erhöht Schutz gegen: Cross-Origin Angriffe API Missbrauch Session Manipulation unerlaubte Browser Requests Einschränkungen Neue Entwicklungsumgebungen müssen explizit registriert werden. Abhängigkeiten Diese Entscheidung betrifft: server.ts Express Middleware Deployment Environment Variables Google AI Studio Integration Render Production Deployment Validierung Nach Umsetzung müssen folgende Tests durchgeführt werden: Produktions-Test Erlaubt: <https://capital-ai.online> Blockiert: <https://test-attacker-domain.com> Entwicklungs-Test Erlaubt: localhost Blockiert: fremde Domains Rollback Bei Problemen kann die vorherige Middleware-Konfiguration wiederhergestellt werden. Ein Rollback darf nur temporär erfolgen und benötigt eine erneute Security Bewertung. Ergebnis Capital-AI verwendet eine kontrollierte CORS-Allowlist Architektur. Entwicklungs- und Produktionsumgebung sind logisch getrennt und gegen unautorisierte Browserbasierte Fremdzugriffe geschützt. Diese Architekturentscheidung ist Bestandteil der Capital-AI Security Baseline.

## Nachtrag 2026-08-11 — Abschaltung der Google-AI-Studio-Origin-Ausnahme (Q6)

Google AI Studio wird nicht mehr als Entwicklungsumgebung für CAPITAL-AI verwendet (siehe `.env.example` -Historie). Die in diesem ADR beschriebene, ausschließlich außerhalb der Produktionsumgebung wirksame `AI_STUDIO_ORIGIN`-Ausnahme (§ "Google AI Studio Entwicklung") ist damit obsolet und wurde vollständig entfernt, inklusive aller aktiven und noch nicht verdrahteten (ADR-0014-Dekompositions-)Duplikate der Origin-/CSP-Logik:

- `server/application.ts` — inline `isOriginAllowed()` (CORS) und `frameAncestors` (CSP) verlieren den `AI_STUDIO_ORIGIN`-Zweig; die Konstante entfällt vollständig.
- `server/middleware/cors.ts` — extrahierte, noch nicht aktiv verdrahtete Kopie von `isOriginAllowed()`, ebenso bereinigt, damit eine künftige Kompositions-Umstellung die Ausnahme nicht versehentlich wieder einführt.
- `server/middleware/securityHeaders.ts` — `buildFrameAncestors()`, dieselbe Bereinigung.
- `server/securityResponse.ts` — `buildDevelopmentCsp()` s `frame-ancestors` (aktiv über `attachSecurityResponseContext()` / `server/logger.ts` im Entwicklungsmodus wirksam).
- `.env.example`, `server/.env.example`, `render.yaml` — `AI_STUDIO_ORIGIN`-Variable und zugehörige Kommentare entfernt.

Die übrige CORS-Architektur (feste Produktions-Allowlist, `localhost / 127.0.0.1`-Ausnahme nur außerhalb Produktion, Security-Event-Logging blockierter Origins, keine Wildcards) bleibt unverändert bestehen und ist von diesem Nachtrag nicht betroffen. Referenz: `docs/seo/SEO_MANAGEMENT_ROADMAP.md`, Punkt Q6.

# ADR-0012: SecurityComplianceAuditor Integration

Quelle: docs/adr/resolved/ADR-0012-SecurityComplianceAuditor.md

## ADR-0012: SecurityComplianceAuditor Integration

- **Status:** Accepted
- **Datum:** 31.07.2026
- **Projekt:** Capital-AI / AIF-CORE
- **Version at decision:** 0.6.0
- **Autor:** Capital-AI Architecture Governance

### Implementation-Status

**COMPLETE** (verifiziert 2026-08-02; ursprüngliche Implementierungsfreigabe über ADR-0017)

Die ursprüngliche Architekturentscheidung ist vollständig umgesetzt. Seit der ersten Verifikation wurde das produktive Compliance-Backend im Zuge der Plattformstruktur-Härtung physisch von `server/compliance/` nach `src/platform/Compliance/` verschoben. Diese Pfadänderung verändert den Entscheidungsinhalt nicht; sie korrigiert lediglich die Repository-Zuordnung zur ESS-0006-Komponente.

### Verifizierte Umsetzung

- `src/components/SecurityComplianceAuditor.tsx` ist vorhanden und im Admin-Portal eingebunden.
- Die UI verwendet authentifizierte Backend-Aufrufe statt einer rein lokalen Mock-Oberfläche.
- Das Backend ist unter `src/platform/Compliance/router.ts` implementiert und stellt die benötigten `/api/compliance/*`-Routen bereit (`dashboard`, `risk`, `certificates`, `run`, `certify`, `report`).
- Sämtliche Compliance-Routen sind serverseitig über `checkAdminAccess()` und die zulässigen Admin-Zonenrollen geschützt.
- Die Scanner unter `src/platform/Compliance/scanners.ts` prüfen reale Repository-, Security-, Migrations- und Governance-Zustände und besitzen zusätzlich die dokumentierte interne ISO/IEC-27001:2022-Annex-A-Zuordnung.
- Persistenz erfolgt über die zweckgebundenen Compliance-Run-/Certificate-Strukturen statt über zweckfremde IAM-Audit-Tabellen.
- Die Compliance-Komponente ist im Manifest als `implemented` geführt und referenziert ADR-0012.
- TypeScript-, Unit-Test- und Production-Build-Gates wurden nach der Plattformmigration erfolgreich verifiziert.

## Bewusste Abweichung vom ursprünglichen ADR-Text

Der ursprüngliche Abschnitt Audit Logging nannte `audit_logs_iam` und `iam_access_log` als Datenquellen. Diese Tabellen bilden IAM-Aktionen ab und sind strukturell nicht für Compliance-Scan- Resultate ausgelegt. ADR-0017 führte deshalb zweckgebundene Compliance-Persistenz ein. Diese Korrektur verbessert die Datenintegrität und schwächt die Entscheidung nicht ab.

## Nicht Teil des Implementation-Status

- Eine unabhängige zweite Security-Review-Instanz bleibt als betriebliche Assurance-Maßnahme sinnvoll, ist aber kein fehlender Codepfad dieser ADR.
- Die internen Scanner ersetzen keinen externen SAST-/Dependency-/Zertifizierungsauditor.
- Compliance-Berichte und ISO-Zuordnungen sind interne Nachweise und keine behördliche oder rechtliche Zertifizierung.

## Enterprise-Referenzen

- **ESS-0001-CONTRACTS Chapter 11** — Security & Compliance Contracts
- **ESS-0006** — Security & Compliance Component Specification
- **ESS-0002** — Supervisor Architect
- **ESS-0011 / ESS-0011-CONTRACTS** — Enterprise Traceability
- **ESS-0012 / ESS-0012-CONTRACTS** — Documentation Governance
- **ADR-0003.5** — Owner-IAM
- **ADR-0017** — Backend-/Persistenz-Implementierung
- **Komponenten** — `src/components/SecurityComplianceAuditor.tsx`,  
`src/platform/Compliance/*`

---

## Kontext

Capital-AI besitzt eine sicherheitsorientierte Architektur mit zentralem IAM, Audit Logging und Produktionskontrollen.

Im Rahmen der Erweiterung des Admin-Portals wurde der SecurityComplianceAuditor als zentrale Komponente zur Überwachung und Darstellung sicherheitsrelevanter Prüfungen eingeführt.

Während einer Produktionspipeline-Ausführung trat ursprünglich ein Build-Fehler auf:

```
Could not resolve "../SecurityComplianceAuditor"
from "src/components/AdminPortal.tsx"
```

Ursache war eine fehlende bzw. nicht synchronisierte Frontend-Komponente. Nach Wiederherstellung der Datei konnte die Deployment-Pipeline erfolgreich durchlaufen werden.

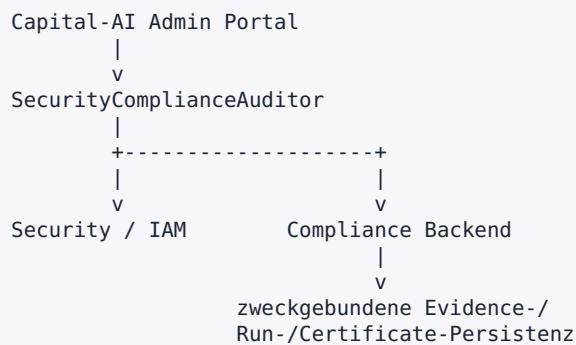
## Entscheidung

Der SecurityComplianceAuditor wird als fester Bestandteil der Capital-AI Governance-Architektur integriert.

Die Komponente bleibt Bestandteil des Admin-Portals und dient als zentrale Oberfläche für:

- Sicherheitsstatus
- Compliance-Prüfungen
- IAM-Validierungen
- Audit-Übersicht
- Produktionskonformität

## Architektur



## Verantwortlichkeiten

### SecurityComplianceAuditor

Verantwortlich für:

- Anzeige des Sicherheitszustands
- Darstellung von Compliance-Ergebnissen
- Verbindung zu Audit-/Compliance-Daten
- Reporting und Remediation-Übersicht

### IAM Integration

Der Auditor berücksichtigt:

- Rollenmodell
- Owner/Admin-Berechtigungen
- Step-Up-Authentifizierung
- TOTP-Anforderungen

- Zugriffskontrollen

### **Audit / Evidence**

IAM-Ereignisse und Compliance-Scan-Ergebnisse bleiben getrennte Datenklassen. Compliance-Resultate werden nicht in IAM-Tabellen gepresst, sondern über die dafür vorgesehenen Compliance-Strukturen geführt.

---

### **Sicherheitsanforderungen**

Der SecurityComplianceAuditor darf:

- keine Berechtigungen vergeben
- keine IAM-Regeln verändern
- keine Sicherheitsmechanismen umgehen
- keine Produktionsdaten außerhalb der freigegebenen Compliance-Workflows manipulieren

Seine Governance-Rolle bleibt Analyse-, Reporting- und kontrollierte Compliance-Ausführung innerhalb der serverseitig autorisierten Endpunkte.

---

### **Entwicklungs- und Produktionsrichtlinie**

#### **Entwicklungsumgebung**

Erlaubt:

- UI-Entwicklung
- Testdatenstrukturen, sofern eindeutig als Test gekennzeichnet
- Dokumentation
- Testintegration

Nicht erlaubt:

- direkte Änderungen an Produktions-IAM
- direkte Änderungen an Produktionsdatenbanken

#### **Produktionsumgebung**

Aktive Integration:

- Supabase IAM
  - Audit-/Security-Evidence
  - Produktionsrollen
  - Compliance Runs / Reports / Certificates
-



## Versionierung

Die Entscheidung wurde für Capital-AI 0.6.0 getroffen. Die aktuelle Plattformversion bleibt über ADR-0030 / `package.json#version` geregelt; diese ADR führt keine eigene Versionshoheit ein.

---

## Deployment-Anforderungen — Abschlussstand

- [x] SecurityComplianceAuditor vorhanden
- [x] Frontend Build erfolgreich
- [x] AdminPortal Import geprüft
- [x] Backend-Routen vorhanden und autorisiert
- [x] produktive Compliance-Evidence-Persistenz angebunden
- [x] Scanner und Reports mit Tests / Build-Gates verifiziert

Eine unabhängige Security Review bleibt als zusätzliche Assurance-Maßnahme empfehlenswert, ohne den bereits implementierten Architecture Contract wieder auf `IN PROGRESS` zu setzen.

---

## Konsequenzen

### Vorteile

- zentrale Security-/Compliance-Governance-Oberfläche
- bessere Auditierbarkeit
- automatisierte, repository-basierte Compliance-Prüfungen
- kontrollierte Remediation- und Reporting-Pfade
- zweckgebundene Evidence-Persistenz

### Nachteile / Grenzen

- zusätzlicher Wartungsaufwand
  - interne Scanner ersetzen keine externen regulatorischen oder unabhängigen Prüfungen
  - Aussagen müssen strikt als interne Governance-/Compliance-Nachweise gekennzeichnet bleiben
- 

## Implementation Evidence

- `src/components/SecurityComplianceAuditor.tsx`
- `src/components/AdminPortal.tsx`
- `src/platform/Compliance/router.ts`
- `src/platform/Compliance/scanners.ts`

- `src/platform/Compliance/store.ts`
- `src/platform/Compliance/manifest.json`
- `tests/unit/complianceScanners.test.ts`
- ADR-0017
- Commit `52fd5fb04756bba1f608e8e279597fc98230ee94` — physische Plattformmigration von Security/Compliance

**Entscheidung:** SecurityComplianceAuditor bleibt Bestandteil der Capital-AI Architektur; die Integration ist codebasiert umgesetzt und verifiziert.

# ADR-0018: Einführung der Enterprise Event Mesh als Plattformmodul

Quelle: docs/adr/resolved/ADR-0018-enterprise-event-mesh.md

## ADR-0018: Einführung der Enterprise Event Mesh als Plattformmodul

### Status

Accepted

### Implementation-Status

**COMPLETE** (verifiziert 2026-07-31) — Komponentenstruktur, vollständiger Metadaten-satz, Spezifikation (ESS-0013/ESS-0013-CONTRACTS) und alle vier vom Platform Director freigegebenen Folgeentscheidungen sind umgesetzt:

1. Implementierung Stufe 1–4 ( `src/platform/EventMesh/{Contracts,Interfaces,Models, Registry,Core,Validators,Reports,Policies,Discovery,Services,Events,Tests}/` ) — erste Komponente im gesamten `src/platform/`-Baum mit ausführbarem Code.
2. Manifest-Nachpflege in 12 Komponenten ( `events.produces` / `events.consumes` befüllt, Documentary-Manifest korrigiert).
3. `server/systemEvents.ts` additiv auf die Enterprise Event Mesh erweitert (bestehender Audit-Log-Mechanismus vollständig erhalten).
4. 7 neu identifizierte Events registriert.

Verifiziert: `npx tsx src/platform/EventMesh/Tests/eventBus.test.ts` (7/7 bestanden), `npx tsc --noEmit` unverändert 9 vorbestehende Fehler (keine neuen), `npx vite build` erfolgreich.

### Datum

2026-07-31

### Verantwortlich

Platform Director

### Betroffene Artefakte

`src/platform/EventMesh/` (neu, 12 Unterverzeichnisse)

`.ai/skills/ESS-0013-Enterprise-Event-Mesh.md` (neu)

`.ai/skills/ESS-0013-Contracts.md` (neu)

`.ai/skills/Enterprise-Event-Mesh.md` (neu)

`.ai/registry/ess-registry.json` (fortgeschrieben)

`docs/architecture/ESS_RESPONSIBILITY_MATRIX.md` (erweitert)

`docs/architecture/ENTERPRISE_EVENT_READINESS_REPORT.md` (neu, vor dieser Entscheidung erstellt)

`docs/architecture/ENTERPRISE_EVENT_MESH_READINESS_REPORT.md` (neu, nach dieser Entscheidung erstellt)

## Kontext

Die auslösende Anforderung forderte die Entwicklung einer neuen Kernkomponente „Enterprise Event Mesh“ unter den vorgeschlagenen Nummern ESS-0014 und ADR-0013.

Die vorab durchgeführte Analyse ( `ENTERPRISE_EVENT_READINESS_REPORT.md` ) ergab drei zentrale Befunde:

1. **Beide vorgeschlagenen Nummern waren belegt bzw. falsch.** `ess-registry.json` weist `freeNumberSpaceStartsAt: "ESS-0013"` aus; ADR-0013 existiert bereits (ESS Documentation Responsibility Consolidation). Korrekt sind **ESS-0013** und **ADR-0018**.
2. **Die Enterprise Event Mesh ist kein neues Konzept.** ESS-0001-CONTRACTS Chapter 8 („Enterprise Event & Messaging Contracts“) definiert Namen, Architektur-Grundprinzip, Event-Kategorien, Contract-Pflichtfelder, Registry-Konzept, Producer-/ Consumer-Regeln, Routing-Prinzip und eine zehnteilige Validation-Checkliste bereits vollständig. `REPOSITORY_STRUCTURE_ANALYSIS.md` führt zusätzlich bereits „Stufe 3 — Enterprise Event Bus“ mit `IEventBus` als eigene Umsetzungsstufe.
3. **Es fehlt die Komponentenspezifikation und der Implementierungsort** — exakt das Muster, das ADR-0016 bereits für ESS-0004 bis ESS-0009 etabliert hat: Chapter X legt die Regel fest, ESS-000Y legt die Komponente fest.

Zusätzlich zeigte der Abgleich der 15 in der Anforderung genannten Standardereignisse gegen Chapter 8/15/18, ESS-0011 und ESS-0012-CONTRACTS: 12 Namen sind bereits kanonisch, 3 ( `TraceabilityUpdatedEvent` , `KnowledgeGraphUpdatedEvent` , `DigitalTwinUpdatedEvent` ) hätten mit bereits existierenden, bedeutungsgleichen Events ( `TraceabilityBuildCompletedEvent` , `KnowledgeUpdatedEvent` , `TwinSynchronizedEvent` ) kollidiert.

## Entscheidung

### 1. Enterprise Event Mesh wird eigenständiges Plattformmodul

```
src/platform/EventMesh/
 Contracts/ Core/ Discovery/ Events/ Interfaces/ Models/
 Policies/ Registry/ Reports/ Services/ Tests/ Validators/
```

Zwölf Unterverzeichnisse, wie in der Anforderung vorgegeben. Jedes besitzt genau eine Verantwortung gemäß ESS-0001-CONTRACTS Chapter 3.

## 2. Nummernkorrektur statt ungeprüfter Übernahme

ESS-0013 und ADR-0018 werden vergeben, nicht die in der Anforderung genannten ESS-0014/ADR-0013. Dies folgt der bereits etablierten Praxis dieser Session (siehe ADR-0013, Abschnitt zur externen ADR-0012-Integration): Vorgeschlagene Nummern werden gegen `ess-registry.json` und `docs/adr/` geprüft, nicht ungeprüft übernommen.

## 3. ESS-0013 als Komponentenspezifikation — Chapter 8 bleibt unverändert

ESS-0013 und ESS-0013-CONTRACTS wiederholen **keine** der in Chapter 8 bereits verbindlichen Regeln. Sie spezifizieren ausschließlich die Komponente: fünf Core-Klassen ( `EventBus` , `EventDispatcher` , `EventPublisher` , `EventSubscriber` , `EventRouter` ), fünf Contract-Typen, vier Registry-Klassen, vier Validator-Klassen, vier Report-Typen — exakt wie in der Anforderung vorgegeben.

**Begründung, warum trotz Chapter 8 ein eigenes ESS-Dokument gerechtfertigt ist:** dasselbe Muster wie bei ESS-0004 bis ESS-0009 (ADR-0016) — ein Regelkapitel beschreibt nicht die Komponente, die es umsetzt. Ohne ESS-0013 gäbe es für die künftige Implementierung keinen spezifizierten Ort und keine spezifizierte Klassenstruktur.

## 4. Standard-Event-Katalog mit korrigierten Namen statt Duplikaten

Drei der 15 angeforderten Standardereignisse wurden auf bereits bestehende, kanonische Event-Namen abgebildet statt als neue, bedeutungsgleiche Events eingeführt:

Angefordert	Verwendet	Quelle
<code>TraceabilityUpdatedEvent</code>	<code>TraceabilityBuildCompletedEvent</code>	ESS-0011 ( <code>Traceability/manifest.json</code> )
<code>KnowledgeGraphUpdatedEvent</code>	<code>KnowledgeUpdatedEvent</code>	Chapter 8, Chapter 15
<code>DigitalTwinUpdatedEvent</code>	<code>TwinSynchronizedEvent</code>	Chapter 18, Twin Events

Ebenso wurden `ContractViolationDetectedEvent` und `LayerViolationDetectedEvent` (an anderer Stelle der Anforderung genannt) auf die bereits kanonischen `ContractViolationEvent` und `LayerViolationEvent` (Chapter 8) abgebildet.

**Begründung:** ESS-0013-CONTRACTS Abschnitt 1 (Event Catalog Contract) verbietet ausdrücklich, ein bereits registriertes Event unter neuem Namen zu duplizieren. Dieselbe Regel wurde hier auf die eigene Erstbefüllung angewendet.

## 5. Umgekehrte Abhängigkeitsregel

Anders als bei Traceability (die Zugriff auf mehrere Komponenten braucht) darf **keine** Fachkomponente die EventMesh-Implementierung direkt importieren — ausschließlich über `Interfaces/`. Diese Umkehrung ist notwendig, weil die Mesh sonst selbst zum Kopplungspunkt würde, den sie auflösen soll.

## 6. Kein Eingriff in bestehenden Produktivcode

Die Erstentscheidung änderte bestehenden Produktivcode noch nicht. Die später vom Platform Director freigegebene Folgeentscheidung erweiterte `server/systemEvents.ts` ausschließlich additiv: der bestehende Audit-Log-Mechanismus blieb erhalten, während zusätzlich ein Enterprise Event publiziert wird.

## 7. Vollständiger Metadatensatz als zweite Referenzimplementierung

`README.md`, `manifest.json`, `component.yaml`, `CHANGELOG.md` — nach `Traceability` (ADR-0015) die zweite Komponente im Repository mit vollständigem Metadatensatz.

---

## Alternativen

### A) Kein neues ESS-Dokument, ausschließlich Implementierung von Chapter 8.

Verworfen. Ohne Komponentenspezifikation gäbe es keinen definierten Ort und keine definierte Klassenstruktur für die Umsetzung — derselbe Fall, den ADR-0016 bereits für sechs andere Komponenten entschieden hat.

**B) Die drei kollidierenden Event-Namen unverändert aus der Anforderung übernehmen.** Verworfen. Hätte drei Paare bedeutungsgleicher, aber unterschiedlich benannter Events erzeugt und damit Zero Duplication verletzt.

**C) Vorgeschlagene Nummern ESS-0014/ADR-0013 unverändert übernehmen.** Verworfen. ADR-0013 ist bereits vergeben; ESS-0014 hätte eine Zwischennummer vor der tatsächlich freien ESS-0013 übersprungen — ausdrücklich durch die Registry-Regel „Zwischennummern sind nicht zulässig“ verboten.

**D) EventMesh als Unterkomponente von Documentary oder Traceability.** Verworfen. Die Mesh ist Kommunikationsinfrastruktur für **alle** Komponenten, einschließlich Documentary und Traceability selbst — eine Unterordnung unter eine der Komponenten, die sie bedient, wäre ein Zirkelbezug.

**E) Bestehende Audit-Log-Funktion ( `server/systemEvents.ts` ) durch die Mesh ersetzen.** Verworfen. Die spätere Integration wurde additiv umgesetzt; der bestehende Audit-Log blieb bestehen.

---

## Konsequenzen

### Positiv

- Die Enterprise Event Mesh besitzt einen eindeutigen Ort, eine definierte Klassenstruktur und einen vollständigen Standard-Event-Ausgangskatalog.
- Potenzielle Event-Namen-Duplikate wurden vor ihrer Entstehung erkannt und aufgelöst.
- Chapter 8 bleibt vollständig unverändert; keine Doppelregelung entstand.
- Event Bus, Registry, Validatoren, Reports, Discovery und additive System-Audit-Bridge sind real implementiert und getestet.

## Negativ / Aufwand

- Der in-memory Bus teilt seinen Zustand nicht automatisch zwischen mehreren Prozessinstanzen; dieser Betriebsaspekt bleibt bei horizontaler Skalierung gesondert zu behandeln.
- Events dürfen weiterhin nur dann publiziert werden, wenn ein realer Producer und ein realer Zustandsübergang existieren.

## Neutral

- Kein bestehender kanonischer Event-Name wurde umbenannt.
- EventMesh ersetzt keine fachlichen Audit-, Security- oder Business-Datenbanken.

---

## Folgeentscheidungen

Alle vier wurden am 2026-07-31 vom Platform Director zur Ausführung freigegeben und sind umgesetzt:

1. **Implementierung der Stufen 1 bis 4** — `src/platform/EventMesh/`.
2. **Nachpflege der `events`-Felder** in den betroffenen Plattformkomponenten.
3. `server/systemEvents.ts` **additiv erweitert** — bestehender Audit-Log-Mechanismus erhalten, EventMesh-Publikation ergänzt.
4. **7 neu identifizierte Events registriert** (`VersionApprovedEvent`, `RoadmapUpdatedEvent`, `ArchitectureDecisionApprovedEvent`, `CriticalArchitectureViolationEvent`, `DependencyMappedEvent`, `KnowledgeRelationCreatedEvent`, `KnowledgeValidationCompletedEvent`).

Details und Testergebnisse: `docs/architecture/ENTERPRISE_EVENT_MESH_READINESS_REPORT.md`.

---

## Referenzen

- ESS-0001-CONTRACTS Chapter 8 — Enterprise Event & Messaging Contracts
- ESS-0013 — Enterprise Event Mesh
- ESS-0013-CONTRACTS — Event-Katalog-, Kompatibilitäts-, Registry-, Routing-, Discovery-, Policy- und Report-Contracts
- ESS-0011 — Enterprise Traceability
- ESS-0012-CONTRACTS — Governance-Validator-Events
- ADR-0013 — ESS Documentation Responsibility Consolidation
- ADR-0014 — Documentation Governance Validator
- ADR-0015 — Enterprise Traceability Component
- ADR-0016 — Vergabe ESS-0004 bis ESS-0009
- `docs/architecture/ENTERPRISE_EVENT_READINESS_REPORT.md`
- `docs/architecture/ENTERPRISE_EVENT_MESH_READINESS_REPORT.md`

- `docs/architecture/REPOSITORY_STRUCTURE_ANALYSIS.md`
- `.ai/registry/ess-registry.json`



# ADR-0030 — Platform Version and Release Lifecycle

Quelle: docs/adr/resolved/ADR-0030-platform-version-release-lifecycle.md

## ADR-0030 — Platform Version and Release Lifecycle

- **Status:** Accepted
- **Implementation-Status:** COMPLETE (verifiziert 2026-08-02)
- **Date:** 2026-08-02
- **Scope:** CAPITAL-AI Web Application / Production Release Lifecycle
- **Current Platform Version at decision:** 0.6.0 (Beta)
- **Governance ID:** GOV-VER-002
- **Builds on:** GOV-VER-001 / tests/unit/platformVersionConsistency.test.ts
- **Single Source of Truth:** package.json#version
- **Implementation Contract:** release-version-gate/1.0.0

### 1. Context

CAPITAL-AI currently declares platform version 0.6.0. A prior governance correction (GOV-VER-001) aligned previously conflicting declarations and established package.json as the single source of truth, with metadata.json and user-/decision-maker-visible files required to match it.

What remained undefined was **when** the platform is allowed to advance to the next version, **which version component is incremented**, and **where that decision occurs in the development, integration, production and acceptance lifecycle**.

Without a release gate, version numbers can drift from the actual production capability set: individual features, migrations, documentation changes or acceptance records could cause an ad-hoc bump even though no coherent platform release has been approved.

This ADR defines the release/version contract.

### 2. Decision

CAPITAL-AI uses a controlled Semantic Versioning-inspired release model:

MAJOR.MINOR.PATCH

While the product remains in Beta (0.x.y), CAPITAL-AI applies stricter internal compatibility rules than SemVer requires. A version number represents a **released, identifiable production platform state**, not the number of completed tasks, commits, ADRs or migrations.

The version at the time of this decision remains:

0.6.0

The successful Production SLO Persistence acceptance does **not** by itself increment the platform version.

### 3. Version Increment Rules

#### 3.1 PATCH — example 0.6.0 → 0.6.1

Use PATCH for a production release that contains only backward-compatible corrections or hardening and does not introduce a material new product capability or public contract.

Typical PATCH triggers:

- bug fixes
- security hardening without a new externally visible capability
- reliability/observability fixes
- performance improvements preserving existing behavior
- dependency remediation without material product behavior change
- production configuration corrections that require a traceable release artifact

A documentation-only change or the closing of an acceptance record does not require a PATCH by itself.

#### 3.2 MINOR — example 0.6.x → 0.7.0

Use MINOR when a release introduces at least one approved material capability or contract change.

Typical MINOR triggers:

- new end-user or Enterprise capability
- new production asset class or materially expanded screening/scoring capability
- new public/internal API contract used by production boundaries
- major orchestration, governance or platform module becoming operational
- material authentication/authorization workflow expansion
- database/schema capability that enables a new product function
- a coherent release train containing multiple accepted work packages that together represent a new platform capability level

The next MINOR after the 0.6.x release line is 0.7.0.

#### 3.3 MAJOR — 1.0.0

1.0.0 is reserved for a formal General Availability decision and must not be reached merely by accumulating features.

Minimum GA gate:

- Beta exit explicitly approved
- stable production deployment and rollback procedure
- stable identity/access and security baseline
- production observability and evidence persistence accepted
- public/product API contracts governed and versioned
- data integrity and No-Demo-Data controls enforced
- required compliance/security acceptance records closed
- release and traceability evidence reproducible
- critical production migrations validated
- no unresolved release-blocking findings

A dedicated GA ADR is required before `1.0.0`.

## 4. What Does NOT Trigger a Version Increment

The following events are lifecycle evidence, not release decisions by themselves:

- completing a single ADR
- merging a documentation-only PR
- creating an acceptance record
- completing one production database migration
- adding tests without changing released behavior
- refactoring internal code with identical externally observable behavior
- correcting stale documentation to match the already released version

These changes may be included in the next PATCH or MINOR release, but they do not independently change the web application version.

## 5. Release Lifecycle Integration

Version advancement occurs at a dedicated **Release Gate**, not during normal feature development.

### Stage A — Development

Goal: implement and test individual work packages.

Rules:

- platform version remains the currently released version
- feature/architecture changes receive ADRs where required
- tests and documentary/traceability artifacts are updated with the work
- external production-system changes are not treated as completed merely because development code exists

Output: implementation-ready work packages, tests and documentation.

### Stage B — Integration / Staging

Goal: combine release candidates and resolve cross-module dependencies.

Required checks:

- build passes
- type/lint checks pass
- relevant unit/integration tests pass
- schema/API contracts are compatible
- security/compliance findings have no unresolved release blocker
- required migration/handoff documents exist
- traceability covers the candidate scope

At the end of this stage, the release scope is frozen and classified as PATCH or MINOR.

Output: **Release Candidate Scope.**

### Stage C — Release Version Gate

This is the only normal point where `package.json#version` is advanced.

The release owner selects exactly one target version based on Section 3.

Examples:

- fix-only release from `0.6.0` → `0.6.1`
- capability release from `0.6.x` → `0.7.0`

The version bump is performed in a dedicated release change/commit, not mixed invisibly into a feature commit.

Output: **Versioned Release Candidate.**

### Stage D — Production Deployment and Acceptance

The exact versioned release candidate is deployed to production.

Required evidence is scope-dependent and can include:

- production smoke tests
- deployment health
- security/IAM checks
- database migration acceptance
- API/provider health
- SLO/observability evidence
- business-critical path checks
- rollback readiness

Production-affecting migrations receive their own acceptance record when their risk warrants it.

If a critical check fails, the release is **not accepted**. A new corrective release candidate is created; acceptance evidence must never be falsified or retroactively rewritten.

Output: **Production Acceptance Decision**.

### Stage E — Release Closure

Only after production acceptance:

1. the exact accepted commit is identified;
2. the final release record is closed;
3. an immutable Git tag `vMAJOR.MINOR.PATCH` is created on the accepted commit;
4. release notes/changelog are finalized;
5. traceability links version → commit → ADRs → migrations → acceptance evidence;
6. the lifecycle moves to the next development cycle.

Output: **Released Platform Version**.

## 6. Version Source-of-Truth Contract

`package.json#version` is the authoritative platform version.

The release procedure synchronizes at least:

1. `package.json`
2. root package version in `package-lock.json`
3. `packages[""]#version` in `package-lock.json`
4. `metadata.json`
5. `README.md`
6. `AGENTS.md`
7. `docs/code-quality/CODE_QUALITY_STANDARDS.md`
8. `docs/ceo/EXECUTIVE_SUMMARY.md`
9. `docs/API.md`
10. `index.html`

`tests/unit/platformVersionConsistency.test.ts` remains a mandatory regression gate and must be extended whenever another file becomes an official platform-version declaration.

No component may invent a separate current platform version.

## 7. Required Version Bump Procedure

For every future platform version change:

## Step 1 — Determine target version

Create a release decision containing:

- current version
- target version
- PATCH/MINOR/MAJOR classification
- included work packages / PRs
- included ADRs
- included migrations
- known risks and rollback boundary

## Step 2 — Update the source of truth

Use the controlled Release component. The command is dry-run by default and requires explicit `--apply` before any governed version file is changed.

## Step 3 — Synchronize declared versions

The Release component updates all governed declarations from Section 6 atomically as one version-gate operation.

## Step 4 — Run version governance gate

The implementation runs at minimum:

- `npm run lint`
- `npx vitest run tests/unit/platformVersionConsistency.test.ts`
- `npm run build`
- `npm run predeploy:check`

A version mismatch or failed gate is release-blocking and restores the governed version files.

## Step 5 — Create release candidate evidence

Only after those checks pass, create the versioned candidate record containing:

- target version
- release scope
- ADR / migration scope
- test/build gate definition
- rollback boundary
- acceptance requirements
- explicit pending-production-acceptance state

## Step 6 — Production acceptance

Deploy and validate the exact release candidate. Do not tag failed candidates as final releases.

## Step 7 — Tag only the accepted commit

After acceptance, create:

```
v<package.json version>
```

The tag must point to the exact accepted production commit. The implemented release-version tool intentionally contains no final-tag operation.

## 8. Release Artifact and Evidence Model

Every released platform version must be reconstructable through this chain:

```
Platform Version → Git Tag → Accepted Commit SHA → Release/PR Scope → ADRs → Migrations / Contracts → Tests and Build Evidence → Production Acceptance Records → Traceability Matrix
```

This chain is the lifecycle definition of a CAPITAL-AI release.

## 9. Rollback Rules

A rollback does not silently reuse a version number for different code.

- If production is rolled back to the exact prior accepted artifact, production returns to that prior tagged version.
- If a corrected artifact is required, issue a new PATCH version.
- Never move an existing final version tag to another commit.
- Never overwrite a historical acceptance record to make a failed release appear successful.

## 10. Database Migration Relationship

Database migrations and application versions are related but not identical.

A migration receives:

- its own immutable migration identifier/timestamp;
- its own production validation evidence when required;
- a reference from the release that consumes or depends on it.

A successful migration such as Screening SLO Persistence can therefore be accepted inside `0.6.0` without automatically becoming `0.6.1` or `0.7.0`.

If a migration enables a material new product capability that is actually released to users, that release will normally qualify as a MINOR increment.

## 11. Automation Target — Implemented

CAPITAL-AI now provides `release-version-gate/1.0.0` through:

- `src/platform/Release/Services/releaseVersionGate.ts`
- `scripts/automation/releaseVersion.ts`
- `npm run release:version`

The automation:

1. validates the requested strict SemVer target;
2. verifies it is greater than and is the exact permitted next version;
3. validates PATCH/MINOR/MAJOR classification;
4. validates work-package, ADR, risk, rollback and acceptance metadata;
5. requires a dedicated GA ADR for MAJOR;
6. updates `package.json` and both governed `package-lock.json` version fields together;
7. updates governed mirror declarations;
8. runs the mandatory version/build/readiness gates;
9. refuses release on mismatch or failed gates and restores governed version files;
10. produces a release-candidate evidence file only after successful apply-stage gates;
11. does **not** create the final Git tag.

Dry-run is the default. File mutation requires explicit `--apply`.

## 12. Current Decision for the Next Version

As of implementation verification:

- current platform version remains `0.6.0`;
- no automatic increment is performed by implementation of this ADR;
- next fix-only candidate from `0.6.0`: `0.6.1`;
- next capability release candidate: `0.7.0`;
- `1.0.0`: blocked until a dedicated GA decision and its acceptance gates are complete.

## 13. Consequences

### Positive

- version numbers identify real production releases rather than arbitrary development milestones;
- `package.json` remains the authoritative source;
- release acceptance becomes part of version governance;
- database migrations and feature completion can be traced without causing version churn;
- tags, acceptance records and traceability form an auditable release chain;



- stale version declarations become release-blocking rather than cosmetic debt;
- a failed local release gate cannot leave a partial governed version bump behind.

### Trade-offs

- a release requires explicit closure evidence;
- version changes become deliberate release operations rather than casual edits;
- failed production candidates may require a corrective release cycle before tagging.

These costs are intentional for an Enterprise FinTech production lifecycle.

## 14. Acceptance Criteria for GOV-VER-002

Implementation is verified because:

- `package.json` remains the platform-version source of truth;
- strict transition classification is encoded and unit-tested;
- the package-lock root metadata is now independently covered by regression tests;
- a controlled version command exists and is dry-run by default;
- governed version files are synchronized by a single fail-closed operation;
- local release gates are executed after synchronization;
- failed gates restore the governed version files;
- a versioned release-candidate evidence artifact is generated only after successful gates;
- the implementation deliberately has no final Git-tag operation;
- repository CI on PR #57 passed dependency audit, TypeScript, Vitest, production build and H7 deployment readiness.

Actual future releases still require their own scope classification, Release Version Gate invocation, production deployment, acceptance record and immutable final tag.

## 15. Implementation Evidence

Verified on 2026-08-02 against the production repository codebase:

- `src/platform/Release/Services/releaseVersionGate.ts`
- `scripts/automation/releaseVersion.ts`
- `src/platform/Release/manifest.json`
- `src/platform/Release/README.md`
- `tests/unit/releaseVersionGate.test.ts`
- `tests/unit/platformVersionConsistency.test.ts`
- `package.json#scripts.release:version`
- GitHub PR #57 CI: PASS

No platform version bump and no final Git tag were performed as part of this implementation closure.

# ADR-0032 — Asset Catalog and Market Evidence Separation

---

Quelle: docs/adr/resolved/ADR-0032-asset-catalog-market-evidence-separation.md

## ADR-0032 — Asset Catalog and Market Evidence Separation

- **Status:** Accepted
- **Implementation-Status:** COMPLETE (verifiziert 2026-08-02)
- **Date:** 2026-08-02
- **Scope:** CAPITAL-AI Multi-Asset Registry / Search / Data Integrity
- **Platform Version:** 0.6.0
- **Governance ID:** DATA-ASSET-CATALOG-001
- **Related:** ADR-0025, ADR-0030, ADR-0033
- **Integrity Contract:** asset-catalog-integrity/1.0.0

### 1. Context

CAPITAL-AI maintains a multi-asset registry used by search, screening and supporting workflows. The legacy `AssetRegistry` combines instrument metadata with bootstrap numeric fields. Those bootstrap fields are intentionally suppressed by the public registry API because they do not carry field-level provider provenance and therefore must not be represented as verified market observations.

The asset universe was expanded by 100 additional instruments for each supported class:

- crypto;
- stock;
- forex;
- commodity;
- index;
- bond.

Growing the catalog by populating required legacy numeric fields with deterministic, estimated or otherwise non-provider-backed values would violate the CAPITAL-AI no-demo-data and provenance requirements. Catalog membership therefore has an explicit architectural boundary from financial observations and canonical scoring.

### 2. Decision

CAPITAL-AI separates **instrument catalog metadata** from **verified market evidence**.

## 2.1 Catalog layer

The search catalog may contain only descriptive instrument metadata required to discover and address an instrument, including:

- canonical symbol;
- display name;
- asset class;
- optional aliases;
- instrument kind;
- catalog source declaration;
- screening-contract classification;
- origin ( `legacy-registry` or `catalog-expansion` ).

Catalog presence does **not** imply that a current price, historical series, score, ranking, recommendation or screening eligibility exists.

## 2.2 Market-evidence layer

Financial values become usable only after provider, freshness, provenance and scoring-integrity controls accept them.

The following rules remain mandatory:

1. Catalog metadata must never be converted into price or score evidence.
2. Provider failures remain fail-closed.
3. Simulated/bootstrap history must not be accepted as verified scoring history.
4. Missing provider mappings produce `DATA_UNAVAILABLE` , `SOURCE_UNAVAILABLE` , `UNSUPPORTED_ASSET` or `SCORE_NOT_COMPUTABLE` rather than fallback financial values.
5. Catalog integrity has `scoreImpactEnabled: false` .

## 3. Expansion policy

The target was **100 additional unique canonical assets per supported asset class**. The implementation deliberately provides more candidates than the target where necessary so legacy symbols and aliases can be skipped without lowering the target.

A candidate counts toward the target only when:

- its canonical symbol is not already present in the merged catalog;
- symbol and name pass structural validation;
- its type is one of the six supported classes;
- it contains a catalog-source declaration;
- compatibility aliases are not counted as independent assets.

The expansion process does not overwrite an existing legacy instrument solely to reach a target count.

## 4. Screening contracts by class

### Crypto

New crypto catalog entries are classified `crypto-provenance`. A selected symbol may enter the existing crypto provider/scoring pipeline. A score is emitted only if the provider-backed technical and integrity gates succeed.

### Stocks and Forex

New stock and forex entries are classified `traditional-provenance`. They may enter the existing verified history/fundamentals/quote paths. An approved provider must supply the required evidence. Unsupported or unavailable symbols fail closed.

### Indices

Index entries are classified `traditional-provenance`. ADR-0033 subsequently introduced the approved `index-provider-mapping/1.0.0` contract with FMP priority and TwelveData runtime identity verification. This is an implementation evolution of the boundary established here: a catalog entry still does not become verified evidence merely because it exists.

### Commodities and Bonds

At the time of this ADR, expanded commodity and bond entries were intentionally `catalog-only` pending dedicated contracts. ADR-0033 subsequently approved `commodity-evidence-scoring/1.0.0` and the narrowly scoped `sovereign-benchmark-yield-scoring/1.0.0`. That later activation does not supersede the catalog/evidence separation: READY still requires approved provider evidence, identity, freshness and scoring gates. General individual/corporate bond scoring remains locked under ADR-0022/ADR-0029.

## 5. Asset search inventory

The Market Screener displays the **current loaded catalog count** directly below the asset search field for:

- Krypto;
- Aktien;
- Forex;
- Rohstoffe;
- Indizes;
- Anleihen;
- Gesamt.

These values are calculated from the catalog response received by the UI and are not hardcoded to planned targets. Aliases may participate in search matching but do not increase the asset count.

## 6. Data-integrity integration

The contract `asset-catalog-integrity/1.0.0` validates the merged catalog independently of financial scoring.

It checks at minimum:

- global canonical-symbol uniqueness;
- symbol syntax;
- non-empty names;
- supported asset classes;
- catalog-source presence;
- expansion count against target per class;
- separation of catalog-only and provider-verifiable assets.

The registry exposes a read-only catalog-integrity endpoint for operational inspection. A degraded catalog-integrity result is not allowed to silently become a market-data or scoring fallback.

Automated tests enforce exactly 100 successful additions per class, 600 total expansion entries, uniqueness, real class counts, absence of financial observation fields on catalog entries and valid screening-contract classification.

## 7. Manual mutation boundary

Existing legacy assets retain the existing privileged admin update path subject to IAM controls.

A newly introduced catalog-only entry that has no legacy `RegistryAsset` record must not accept manual price, volatility, expected-return, market-cap or score-like bootstrap enrichment through that legacy path. Such a request is rejected with `CATALOG_ONLY_ASSET`.

Future authoritative catalog-management writes require a separately reviewed contract rather than reusing the legacy market-parameter mutation endpoint.

## 8. Operational consequences

### Positive

- The searchable asset universe can grow without inventing financial data.
- Asset count is transparent in the UI and derived from runtime state.
- Existing scoring-integrity and provider-provenance controls remain authoritative.
- Unsupported newly cataloged assets fail closed instead of receiving placeholder scores.
- The catalog has a dedicated versioned integrity contract and regression tests.

### Trade-offs

- Catalog coverage can exceed immediately scoreable coverage.

- Provider rate limits, entitlements and symbol coverage determine runtime evidence availability and must not be hidden by catalog metadata.

## 9. Release lifecycle

ADR-0030 remains authoritative for version changes. This implementation did **not** by itself modify `package.json#version`; the platform remained `0.6.0` during implementation.

Because the change materially expands an end-user multi-asset capability, it is eligible for a future **MINOR** release classification when promoted as part of an accepted production release train. The actual version decision occurs only at the ADR-0030 Release Version Gate.

## 10. Acceptance criteria — verified

Verified on 2026-08-02:

1. all six classes receive exactly 100 new unique catalog entries;
2. the catalog-integrity contract reports `READY` under tests;
3. dependency audit, TypeScript, Vitest, production build and H7 deployment-readiness passed;
4. the search UI derives and displays class counts from the returned catalog;
5. expansion entries contain no invented financial observation fields;
6. unsupported provider paths remain fail-closed;
7. existing scoring contracts were not weakened to increase apparent asset coverage.

Implementation evidence:

- PR #54, merged as `cd34febd13213635b97322c398bb10c83c188a58` ;
- `src/data/assetCatalogExpansion.ts` ;
- `src/lib/assetSearchCatalog.ts` ;
- `src/services/assetCatalogIntegrity.ts` ;
- `tests/unit/assetSearchCatalog.test.ts` ;
- `src/components/MarketScreener.tsx` .

**Decision:** Accepted and implementation verified.

# ADR-0033 — Index Provider Mapping and Evidence-Gated Commodity / Sovereign Benchmark Scoring

Quelle: docs/adr/resolved/ADR-0033-index-provider-mapping-commodity-sovereign-evidence-scoring.md

## ADR-0033 — Index Provider Mapping and Evidence-Gated Commodity / Sovereign Benchmark Scoring

- **Status:** Accepted
- **Implementation-Status:** COMPLETE (verifiziert 2026-08-02)
- **Date:** 2026-08-02
- **Scope:** CAPITAL-AI Multi-Asset Screening / Registry / Market Evidence
- **Platform Version:** 0.6.0
- **Governance ID:** GOV-MARKET-EVIDENCE-0033
- **Depends on:** ADR-0022, ADR-0025, ADR-0029, ADR-0030, ADR-0032

### 1. Context

ADR-0032 separated catalog membership from verified market evidence. The registry contains substantially more index, commodity and bond catalog entries, but catalog presence alone must never create a price, a score or screening eligibility.

Three gaps remained:

1. index screening only accepted the small static INDEX\_FMP\_TICKERS mapping;
2. commodity scoring still had a legacy structural engine that could fill missing structural inputs with neutral fallback values and therefore was not suitable as a canonical market-evidence score;
3. bond catalog expansion consists primarily of sovereign benchmark yields, while ADR-0022/ADR-0029 correctly keep general individual-bond scoring locked until coupon, maturity, duration/cashflow, rating, liquidity and other mandatory evidence is available.

This ADR approves narrow, versioned runtime contracts that close those gaps without weakening the existing fail-closed architecture.

### 2. Decision

#### 2.1 Index provider mapping

CAPITAL-AI adopts index-provider-mapping/1.0.0.



Provider order:

1. **FMP** remains first priority wherever an existing explicitly approved static `IN-DEX_FMP_TICKERS` mapping exists.
2. **TwelveData** may provide index history for the remaining catalog indices only after runtime provider-identity verification.

A TwelveData candidate is not sufficient by itself. The returned provider metadata must confirm the expected symbol; if provider type metadata is present it must identify an index. Ambiguous or contradictory provider identity fails closed.

The resulting history is represented by `index-market-evidence/1.0.0`. Technical factors are derived from real closes and carry the actual provider, source path, observed/retrieved times and lineage into the existing traditional scoring pipeline.

## 2.2 Commodity market-evidence scoring

CAPITAL-AI approves `commodity-evidence-scoring/1.0.0` for canonical commodity **market-price technical evidence**.

This contract is deliberately narrower than the legacy raw-material structural model.

Approved factors:

Factor	Weight	Evidence
Trend	30%	real daily closes
Momentum	25%	real daily closes
Breakout quality	20%	real high/low position derived from daily closes
Volatility quality	25%	realised daily return volatility

Mandatory gates:

- provider identity present;
- provider evidence IDs present;
- at least 20 real daily observations;
- 100% factor coverage for the approved factor set;
- evidence not older than 7 days;
- no registry/bootstrap values participate in the canonical score.

TwelveData is the initial approved provider for this contract. Commodity identity is resolved against the provider's commodity catalog. Only explicit static mappings or unambiguous provider-catalog matches may be used. Ambiguous mappings remain `SOURCE_UNAVAILABLE` / `SCORE_NOT_COMPUTABLE`.

The contract does **not** claim to score ore grade, reserves, processing complexity, ESG, geopolitical concentration, supply-chain resilience or physical-market fundamentals. Those

remain separate structural/research dimensions until equivalent field-level evidence contracts exist.

2.3 Sovereign benchmark yield scoring

CAPITAL-AI approves `sovereign-benchmark-yield-scoring/1.0.0` only for catalog instruments representing **government benchmark yields**.

This is not an activation of general individual-bond scoring.

Approved factors:

Factor	Weight	Meaning
Yield-level percentile	45%	current benchmark yield relative to its own verified history window
Yield trend	30%	standardized current yield position relative to the history distribution
Yield stability	25%	realised volatility of daily yield changes in basis points

Mandatory gates:

- an explicit approved or provider-catalog-confirmed EODHD `*.GBOND` mapping;
- at least 20 real observations;
- provider and evidence IDs;
- full approved-factor coverage;
- evidence not older than 7 days;
- finite yields, including historically valid negative yields;
- no bootstrap yield, duration, rating or liquidity defaults.

The score semantics are **yield state**, not investment quality or expected return. The response must explicitly keep `individualBondScoringEligible: false`.

2.4 General individual-bond scoring remains locked

ADR-0022 and ADR-0029 remain authoritative for individual bonds and corporate bonds.

This ADR does not approve:

- the draft general bond weight proposal;
- guessed duration;
- guessed coupon/maturity;
- inferred credit ratings;
- inferred liquidity/spreads;
- a corporate-bond score such as `AAA-CORP` without the required evidence.

`bond-scoring/locked-1.0.0` remains in force for that scope.

### 3. Provider governance

#### FMP

- role: existing approved static index mappings;
- priority: first when a mapping exists;
- evidence use: real index history/quotes only;
- licensing/redistribution remains environment/plan dependent.

#### TwelveData

- role: index fallback/coverage extension and commodity history;
- secret: server-side `TWELVEDATA_API_KEY` only;
- identity rule: runtime metadata/catalog confirmation required;
- ambiguous symbol/name resolution fails closed;
- rate-limit or plan restrictions produce unavailable evidence, never a synthetic fallback score.

#### EODHD

- role: sovereign government-bond benchmark history through `GBOND` ;
- secret: server-side `EODHD_API_KEY` only;
- mapping rule: approved static exact ticker or exact match in the provider `GBOND` symbol-list endpoint;
- ticker guessing is prohibited;
- `GBOND` observations are research/yield evidence, not execution prices.

### 4. Data integrity and lineage

Every READY score under this ADR carries:

- catalog symbol;
- provider identity;
- provider symbol;
- source path;
- observed/retrieved times;
- evidence IDs;
- contract version;
- used/missing factors;
- canonical integrity status;
- correlation ID at the route boundary.

Catalog metadata remains non-financial metadata and does not become evidence by being listed in the registry.

The existing Screening Eligibility / Operations / SLO evidence pipeline receives these results through the same registry evaluation path. A non-READY score remains ineligible.

## 5. Legacy raw-material structural model

`RawMaterialsScoringService` remains available for explicitly labelled structural/sandbox research use.

Its output is marked:

- `scoreSemantic: legacy-structural-research;`
- `canonical: false;`
- `marketEvidenceVerified: false.`

The canonical commodity market score is available only through the approved evidence contract.

## 6. Fail-closed states

The following conditions must not produce a numerical canonical score:

- provider key unavailable;
- provider rate limit / network failure;
- unknown or ambiguous provider mapping;
- provider identity mismatch;
- insufficient history;
- stale evidence;
- missing evidence IDs or provider identity;
- missing approved factors;
- unsupported individual/corporate bond.

There is no symbol-hash, neutral-50, bootstrap-price or simulated-history fallback in the approved contracts.

## 7. Runtime surfaces

Approved surfaces include:

- `/api/registry/assets/index-provider-mappings` — read-only mapping governance view;
- `/api/registry/assets/:symbol/verified-score` — canonical catalog score evaluation;
- `/api/registry/assets/:symbol/verified-context` — score plus separated macro context;
- `/api/registry/assets/verified-scores` — governed batch path with Screening SLO evidence;
- `/api/raw-materials/verified-score/:symbol` — commodity canonical evidence score.

Commodity and sovereign benchmark evidence do not create an execution-quote contract. `/verified-quote` remains limited to asset classes with an approved quote path.

## 8. Acceptance criteria — verified

The implementation was verified to satisfy:

1. every index catalog entry has a versioned provider mapping candidate;
2. existing FMP mappings retain priority;
3. TwelveData identity mismatches fail closed;
4. commodity scores require real provider evidence and freshness;
5. stale commodity evidence cannot return READY;
6. sovereign benchmark scores require exact/verified GBOND mapping;
7. stale sovereign evidence cannot return READY;
8. negative sovereign yield observations are preserved as valid evidence;
9. individual/corporate bond scoring remains locked;
10. the expanded asset catalog remains unique and complete;
11. dependency audit, TypeScript, Vitest, production build and H7 deployment-readiness all pass.

Implementation evidence:

- PR #56, merged as `4cb1dd6467ad36779f26049d0cbfc180774059c1` ;
- `src/services/indexProviderMapping.ts` ;
- `src/services/indexMarketEvidence.ts` ;
- `src/services/commodityMarketEvidence.ts` ;
- `src/services/commodityEvidenceScoring.ts` ;
- `src/services/sovereignBondProviderMapping.ts` ;
- `src/services/sovereignBenchmarkEvidenceScoring.ts` ;
- `src/services/eodhdBondEvidence.ts` ;
- registry and MarketScreener integration;
- dedicated unit tests for index mappings, commodity evidence, sovereign mappings/scoring and negative yields.

## 9. Version lifecycle

This ADR does not independently change the application version. ADR-0030 remains authoritative.

The platform stayed `0.6.0` during implementation. This capability is eligible for a future MINOR release classification because it materially expands production screening capability, but implementation merge alone is not a version bump.

## 10. Final decision

The three runtime contracts are approved and implementation-verified with strict semantic boundaries:

- `index-provider-mapping/1.0.0`
- `commodity-evidence-scoring/1.0.0`
- `sovereign-benchmark-yield-scoring/1.0.0`

General individual-bond scoring remains locked under ADR-0022/ADR-0029.